economics-finance-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `economics_finance_ai999x.about()` for the same answer.

Every capability economics-finance-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import economics_finance_ai999x

result = economics_finance_ai999x.solve('simple_interest', principal=1000, rate=0.05, years=3)
result.show()
# interest = 150, amount = 1150
```

By topic

topic	capabilities
Actuarial mathematics / life annuities	Whole life annuity-due, Temporary life annuity-due
Actuarial mathematics / life insurance	Whole life insurance (net single premium), Term life insurance (net single premium), Endowment insurance
Actuarial mathematics / life tables	Survival and death probabilities from a life table, Life expectancy from a life table
Actuarial mathematics / premiums	Net annual premium for whole life insurance
Bonds / interest-rate risk	Macaulay and modified duration of a bond, Convexity of a bond, Bond price change from duration and convexity
Bonds / pricing	Bond price from its yield, Price of a zero-coupon bond
Bonds / term structure	Implied forward rate from spot rates
Bonds / yields	Yield to maturity of a bond
Capital budgeting / discounted cash flow	Net present value, Internal rate of return, Modified internal rate of return, Profitability index, Equivalent annual annuity
Capital budgeting / payback	Payback period, Discounted payback period
Capital budgeting / project comparison	NPV crossover rate of two projects
Econometrics / association	Correlation and covariance of two series
Econometrics / demand estimation	Constant elasticity from a log-log regression
Econometrics / diagnostics	Durbin-Watson statistic

topic	capabilities
Econometrics / regression	Simple linear regression (OLS), Multiple linear regression (OLS)
Loans and annuities / amortization	Loan payment (amortizing loan), Number of payments to repay a loan, Remaining balance of a loan, Interest and principal in a loan payment, Cumulative interest of a loan
Macroeconomics / aggregate demand	Keynesian spending multiplier
Macroeconomics / growth	Solow model steady state
Macroeconomics / interest and inflation	Real interest rate (Fisher equation), Inflation from a price index series
Macroeconomics / money and banking	Money multiplier
Macroeconomics / national accounts	Nominal GDP, real GDP and the GDP deflator
Macroeconomics / price indices	Laspeyres, Paasche and Fisher price indices
Microeconomics / consumer choice	Utility-maximising demand (Cobb-Douglas)
Microeconomics / cost-volume-profit	Break-even and target-profit volume
Microeconomics / elasticity	Price elasticity of demand (midpoint), Income elasticity of demand (midpoint), Cross-price elasticity of demand (midpoint), Point elasticity of a linear demand curve
Microeconomics / inequality	Gini coefficient
Microeconomics / market structure	Monopoly profit maximization, Herfindahl-Hirschman index and concentration ratio
Microeconomics / oligopoly	Cournot duopoly equilibrium, Stackelberg duopoly equilibrium
Microeconomics / supply and demand	Market equilibrium and surplus (linear curves)
Microeconomics / taxation	Per-unit tax incidence (linear curves)
Portfolio theory / mean-variance	Portfolio expected return and risk, Two-asset portfolio return and risk, Minimum-variance portfolio of two assets, Global minimum-variance portfolio, Tangency (maximum Sharpe ratio) portfolio
Portfolio theory / performance measures	Sharpe ratio, Sortino ratio, Treynor ratio and Jensen's alpha, Information ratio and tracking error
Portfolio theory / returns	Arithmetic and geometric mean return, Expected return and risk from scenarios
Portfolio theory / systematic risk	Beta of an asset from return series
Risk measures / drawdown	Maximum drawdown
Risk measures / value at risk	Historical value at risk and expected shortfall, Parametric (normal) value at risk and expected shortfall, Portfolio value at risk (variance-covariance)
Stocks / asset pricing	Capital asset pricing model
Stocks / cost of capital	Weighted average cost of capital
Stocks / cost of equity	Required return implied by the Gordon model
Stocks / equity valuation	Dividend discount model (Gordon growth), Two-stage dividend discount model
Stocks and options / arbitrage	Put-call parity

topic	capabilities
Stocks and options / forwards	Forward price with cost of carry, Forward price with known dividends
Stocks and options / option pricing	Black-Scholes option prices, Implied volatility, Binomial option price (Cox-Ross-Rubinstein)
Stocks and options / option risk	Black-Scholes Greeks
Time value of money / annuities	Future value of an annuity, Present value of an annuity, Sinking fund payment, Interest rate of an annuity, Present and future value of a growing annuity
Time value of money / discounting	Present value of a single sum
Time value of money / growth rates	Compound annual growth rate
Time value of money / interest	Simple interest, Compound interest, Continuous compounding, Doubling time
Time value of money / interest rates	Effective annual rate, Nominal rate from an effective rate
Time value of money / perpetuities	Present value of a perpetuity, Present value of a growing perpetuity

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on

field	meaning
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
simple_interest	Simple interest	principal, rate, years	interest, amount
compound_interest	Compound interest	principal, rate, years, periods_per_year	amount, interest
continuous_compounding	Continuous compounding	principal, rate, years	amount, interest
effective_annual_rate	Effective annual rate	rate, periods_per_year	effective_rate, periodic_rate
nominal_rate	Nominal rate from an effective rate	effective_rate, periods_per_year	nominal_rate, periodic_rate
present_value	Present value of a single sum	future_value, rate, periods	present_value, discount_factor
future_value_annuity	Future value of an annuity	payment, rate, periods, timing	future_value
present_value_annuity	Present value of an annuity	payment, rate, periods, timing	present_value
loan_payment	Loan payment (amortizing loan)	principal, annual_rate, years, payments_per_year	payment, number_of_payments, total_paid, total_interest
sinking_fund	Sinking fund payment	future_value, rate, periods, timing	payment
number_of_periods	Number of payments to repay a loan	present_value, payment, rate, timing	periods
annuity_rate	Interest rate of an annuity	present_value, payment, periods, timing	rate
perpetuity	Present value of a perpetuity	payment, rate, timing	present_value

id	operation	inputs	outputs
growing_perpetuity	Present value of a growing perpetuity	first_payment, rate, growth	present_value
growing_annuity	Present and future value of a growing annuity	first_payment, rate, growth, periods	present_value, future_value
doubling_time	Doubling time	rate	periods, rule_of_72
cagr	Compound annual growth rate	begin_value, end_value, years	cagr, total_growth
loan_balance	Remaining balance of a loan	principal, annual_rate, years, payments_made, payments_per_year	balance, payment
loan_payment_split	Interest and principal in a loan payment	principal, annual_rate, years, payment_number, payments_per_year	interest_part, principal_part, payment
loan_cumulative_interest	Cumulative interest of a loan	principal, annual_rate, years, payments_made, payments_per_year	cumulative_interest, principal_repaid
npv	Net present value	rate, cash_flows	npv
irr	Internal rate of return	cash_flows	irr
mirr	Modified internal rate of return	cash_flows, finance_rate, reinvest_rate	mirr, terminal_value, pv_outflows
payback_period	Payback period	cash_flows	payback
discounted_payback	Discounted payback period	rate, cash_flows	discounted_payback
profitability_index	Profitability index	rate, cash_flows	profitability_index, pv_future, npv
equivalent_annual_annuity	Equivalent annual annuity	rate, cash_flows	eaa, npv
crossover_rate	NPV crossover rate of two projects	cash_flows_a, cash_flows_b	crossover_rate, npv_at_crossover
bond_price	Bond price from its yield	face, coupon_rate, years, ytm, frequency	price, current_yield, coupon
bond_ytm	Yield to maturity of a bond	price, face, coupon_rate, years, frequency	ytm, current_yield
bond_duration	Macaulay and modified duration of a bond	face, coupon_rate, years, ytm, frequency	macaulay_duration, modified_duration, price
bond_convexity	Convexity of a bond	face, coupon_rate, years, ytm, frequency	convexity, price
zero_coupon_bond	Price of a zero-coupon bond	face, ytm, years, compounding	price, discount_factor
bond_price_change	Bond price change from duration and convexity	face, coupon_rate, years, ytm, yield_change, frequency	duration_estimate, duration_convexity_estimate, actual_change, new_price, price
forward_rate	Implied forward rate from spot rates	spot_rate_1, maturity_1, spot_rate_2, maturity_2	forward_rate

id	operation	inputs	outputs
gordon_growth	Dividend discount model (Gordon growth)	required_return, growth, dividend, next_dividend	price, next_dividend, dividend_yield
two_stage_ddm	Two-stage dividend discount model	dividend, high_growth, high_growth_years, stable_growth, required_return	price, terminal_value, pv_terminal, pv_high_growth_dividends
gordon_implied_return	Required return implied by the Gordon model	price, growth, dividend, next_dividend	required_return, dividend_yield, next_dividend
capm	Capital asset pricing model	risk_free, beta, market_return	expected_return, market_risk_premium, risk_premium
wacc	Weighted average cost of capital	equity_value, debt_value, cost_of_equity, cost_of_debt, tax_rate	wacc, after_tax_cost_of_debt, equity_weight, debt_weight
beta_regression	Beta of an asset from return series	asset_returns, market_returns	beta, alpha, correlation, r_squared
sharpe_ratio	Sharpe ratio	returns, risk_free, periods_per_year	sharpe, annualized_sharpe, mean_excess_return, std_dev
sortino_ratio	Sortino ratio	returns, target	sortino, downside_deviation, mean_return
treynor_jensen	Treynor ratio and Jensen's alpha	portfolio_returns, market_returns, risk_free	beta, treynor, jensen_alpha, mean_portfolio_return, mean_market_return
information_ratio	Information ratio and tracking error	portfolio_returns, benchmark_returns	information_ratio, tracking_error, mean_active_return
portfolio_return_risk	Portfolio expected return and risk	weights, expected_returns, covariance	expected_return, variance, std_dev
two_asset_portfolio	Two-asset portfolio return and risk	weight_1, return_1, return_2, sd_1, sd_2, correlation	expected_return, std_dev, variance, weight_2
min_variance_two_asset	Minimum-variance portfolio of two assets	sd_1, sd_2, correlation	weight_1, weight_2, std_dev
global_min_variance	Global minimum-variance portfolio	covariance	weights, variance, std_dev
tangency_portfolio	Tangency (maximum Sharpe ratio) portfolio	expected_returns, covariance, risk_free	weights, expected_return, std_dev, sharpe
mean_returns	Arithmetic and geometric mean return	returns	arithmetic_mean, geometric_mean, cumulative_return
scenario_return	Expected return and risk from scenarios	probabilities, returns	expected_return, variance, std_dev
historical_var	Historical value at risk and expected shortfall	returns, confidence, portfolio_value	var, cvar, tail_observations
parametric_var	Parametric (normal) value at risk and expected shortfall	mean, std_dev, confidence, horizon, portfolio_value	var, cvar, z

id	operation	inputs	outputs
max_drawdown	Maximum drawdown	prices	max_drawdown
portfolio_var	Portfolio value at risk (variance-covariance)	weights, covariance, confidence, portfolio_value, horizon	var, portfolio_sd, undiversified_var, diversification_benefit
black_scholes	Black-Scholes option prices	spot, strike, time, rate, volatility, dividend_yield	call, put, d1, d2
bs_greeks	Black-Scholes Greeks	spot, strike, time, rate, volatility, dividend_yield	call_delta, put_delta, gamma, vega, call_theta, put_theta, call_rho, put_rho
implied_volatility	Implied volatility	option_price, spot, strike, time, rate, dividend_yield, option_type	volatility, vega
put_call_parity	Put-call parity	call_price, spot, strike, time, rate, dividend_yield	put_price, pv_strike, pv_spot
binomial_option	Binomial option price (Cox-Ross-Rubinstein)	spot, strike, time, rate, volatility, steps, option_type, dividend_yield	price, delta, up_factor, probability_up
forward_price	Forward price with cost of carry	spot, rate, time, dividend_yield, storage_cost	forward, cost_of_carry
forward_price_dividends	Forward price with known dividends	spot, rate, time, dividends, dividend_times	forward, pv_dividends
survival_probability	Survival and death probabilities from a life table	mortality_rates, years, deferred_years	survival, death_probability, deferred_death_probability
life_expectancy	Life expectancy from a life table	mortality_rates	curtate, complete
whole_life_insurance	Whole life insurance (net single premium)	mortality_rates, interest_rate, benefit	apv
term_insurance	Term life insurance (net single premium)	mortality_rates, interest_rate, term, benefit	apv
life_annuity_due	Whole life annuity-due	mortality_rates, interest_rate, payment	apv
temporary_annuity_due	Temporary life annuity-due	mortality_rates, interest_rate, term, payment	apv
net_premium_whole_life	Net annual premium for whole life insurance	mortality_rates, interest_rate, benefit	premium, insurance_apv, annuity_due
endowment_insurance	Endowment insurance	mortality_rates, interest_rate, term, benefit	apv, term_part, pure_endowment
arc_price_elasticity	Price elasticity of demand (midpoint)	price_1, quantity_1, price_2, quantity_2	elasticity, abs_elasticity, classification
income_elasticity	Income elasticity of demand (midpoint)	income_1, quantity_1, income_2, quantity_2	elasticity, classification
cross_price_elasticity	Cross-price elasticity of demand (midpoint)	price_other_1, quantity_1, price_other_2, quantity_2	elasticity, relationship
point_elasticity	Point elasticity of a linear demand curve	demand_intercept, demand_slope, price	quantity, elasticity, revenue, revenue_maximizing_price

id	operation	inputs	outputs
market_equilibrium	Market equilibrium and surplus (linear curves)	demand_intercept, demand_slope, supply_intercept, supply_slope	price, quantity, consumer_surplus, producer_surplus, total_surplus
per_unit_tax	Per-unit tax incidence (linear curves)	demand_intercept, demand_slope, supply_intercept, supply_slope, tax	quantity, buyer_price, seller_price, tax_revenue, deadweight_loss, buyer_share
monopoly	Monopoly profit maximization	price_intercept, price_slope, marginal_cost, fixed_cost, cost_quadratic	quantity, price, profit, lerner_index, deadweight_loss
cournot	Cournot duopoly equilibrium	price_intercept, price_slope, cost_1, cost_2	q1, q2, price, profit_1, profit_2
stackelberg	Stackelberg duopoly equilibrium	price_intercept, price_slope, cost_leader, cost_follower	q_leader, q_follower, price, profit_leader, profit_follower
break_even	Break-even and target-profit volume	fixed_costs, price, variable_cost, target_profit	quantity, revenue, contribution_margin, contribution_margin_ratio
cobb_douglas_demand	Utility-maximising demand (Cobb-Douglas)	alpha, beta, income, price_x, price_y	x, y, utility, share_x
gdp_deflator	Nominal GDP, real GDP and the GDP deflator	base_prices, base_quantities, current_prices, current_quantities	nominal_gdp, real_gdp, deflator, inflation, real_growth
price_indices	Laspeyres, Paasche and Fisher price indices	base_prices, base_quantities, current_prices, current_quantities	laspeyres, paasche, fisher
fisher_real_rate	Real interest rate (Fisher equation)	nominal_rate, inflation_rate	real_rate, approximate_real_rate
inflation_adjustment	Inflation from a price index series	price_levels, amount	total_inflation, average_inflation, real_value
money_multiplier	Money multiplier	reserve_ratio, currency_ratio, monetary_base	simple_multiplier, money_multiplier, money_supply
spending_multiplier	Keynesian spending multiplier	mpc, tax_rate, import_propensity, spending_change	spending_multiplier, tax_multiplier, output_change
solow_steady_state	Solow model steady state	saving_rate, population_growth, depreciation, alpha, technology_growth, productivity	capital, output, consumption, golden_rule_saving, golden_rule_capital
ols_simple	Simple linear regression (OLS)	x, y	slope, intercept, r_squared, slope_std_error, n
multiple_regression	Multiple linear regression (OLS)	X, y	coefficients, r_squared, adj_r_squared
log_log_elasticity	Constant elasticity from a log-log regression	prices, quantities	elasticity, intercept, r_squared
gini_coefficient	Gini coefficient	incomes	gini

id	operation	inputs	outputs
herfindahl_index	Herfindahl-Hirschman index and concentration ratio	market_shares	hhi, hhi_fraction, cr4, equivalent_firms
durbin_watson	Durbin-Watson statistic	residuals	durbin_watson
correlation	Correlation and covariance of two series	x, y	correlation, covariance, r_squared

Simple interest — `simple_interest`

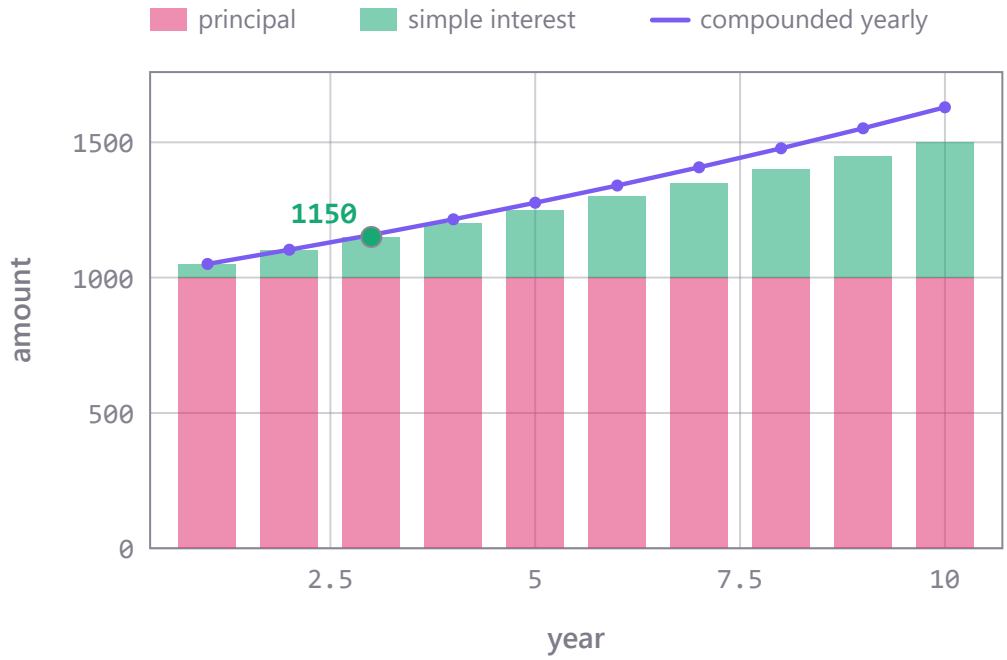
Interest and final amount when interest is earned on the principal only: $I = P$ r t.

$$I = Prt, \quad A = P(1 + rt)$$

input	meaning	condition
`principal`	amount invested or borrowed	
`rate`	annual simple interest rate (decimal; 5% = 0.05)	
`years`	time in years	

output	meaning
`interest`	simple interest earned P r t
`amount`	principal plus interest

- **Method:** $I = P$ r t, $A = P + I$
- **Assumes:** principal > 0; 0 <= rate < 1; years >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



1000 at 5 % simple interest: principal plus the interest returned by the package for each year 1–10 (stacked), with yearly compounding (line, compound-interest runs) for comparison. Marked: 3 years, amount 1150, interest 150.

```
economics_finance_ai999x.solve('simple_interest', principal=1000, rate=0.05, years=3)
# -> interest = 150, amount = 1150   (SOLVED)
```

```
economics_finance_ai999x.solve('simple_interest', principal=2500, rate=0.04, years=0.5)
# -> interest = 50, amount = 2550   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the simple interest with principal = 1000, rate = 5%, years = 3.	SOLVED	interest = 150, amount = 1150
What is the simple interest with principal = $2500, rate = 4%, time = 0.5?	SOLVED	interest = 50, amount = 2550
Find the simple interest with principal = 1000, rate = 5%.	NEEDS_INPUT	simple_interest needs years (time in years). Example: solve('simple_interest', principal=1000, rate=0.05, years=3).
Find the simple interest with principal = 1000, rate = 5, years = 3.	INVALID_INPUT	Simple interest: rate must be a decimal rate between -1e-300 and 1 (write 5% or 0.05).
Find the simple interest on 1000 at 5% for 3 years.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: economics_finance_ai999x.template

Compound interest — `compound_interest`

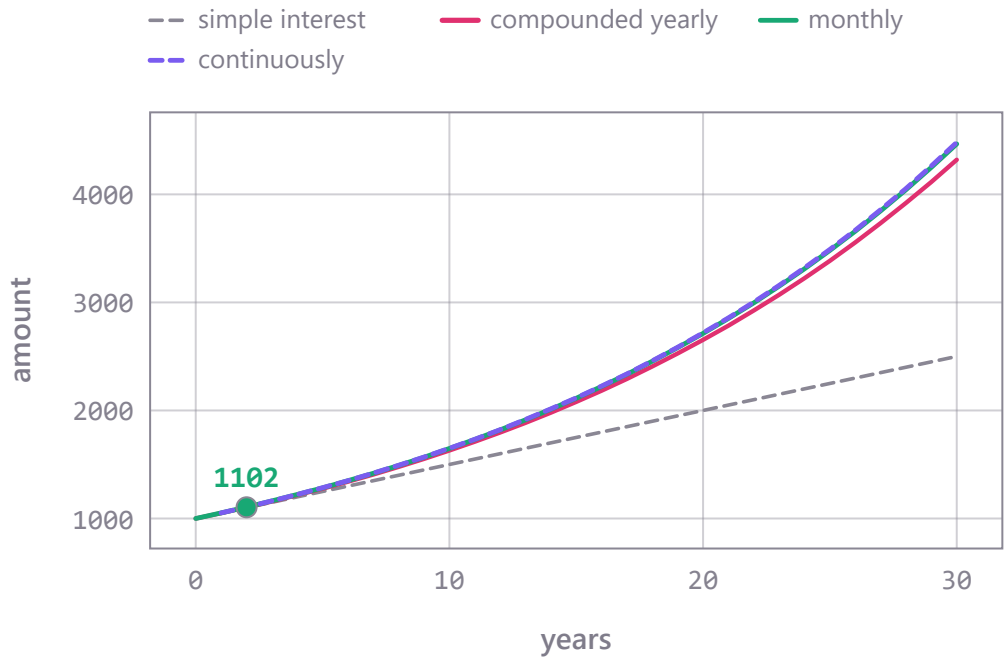
Future amount of a sum invested at a nominal annual rate compounded m times a year: $A = P$ (1 + r/m)^(m t).

$$A = P\left(1 + \frac{r}{m}\right)^{mt}$$

input	meaning	condition
`principal`	amount invested now	
`rate`	nominal annual interest rate (decimal; 5% = 0.05)	
`years`	time in years	
`periods_per_year`	compounding periods per year (1 annual, 12 monthly, 365 daily)	

output	meaning
`amount`	future amount A
`interest`	compound interest earned A - P

- **Method:** $A = P$ (1 + r/m)^(m t)
- **Assumes:** principal > 0; -1 < rate < 1; years >= 0; periods_per_year a whole number >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



1000 at 5 % a year over 30 years: simple interest, compounding once and 12 times a year and continuously, every point a run of the package (simple-interest, compound-interest and continuous-compounding capabilities). Marked: the worked example, 2 years compounded 1 time(s) a year, amount 1102.5.

```
economics_finance_ai999x.solve('compound_interest', principal=1000, rate=0.05, years=2,
periods_per_year=1)
# -> amount = 1102.5, interest = 102.5   (SOLVED)
```

```
economics_finance_ai999x.solve('compound_interest', principal=1000, rate=0.05, years=10,
periods_per_year=12)
# -> amount = 1647.01, interest = 647.009   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the compound interest with principal = 1000, rate = 5%, years = 2.	SOLVED	amount = 1102.5, interest = 102.5

question	outcome	result
Find the compound interest with principal = 1000, rate = 5%, years = 10, periods per year = 12.	SOLVED	amount = 1647.01, interest = 647.009
Find the compound interest with principal = 1000, rate = 5%, years = 10, periods per year = 2.5.	INVALID_INPUT	periods_per_year: expected a whole number written exactly, got 2.5.
Find the compound interest with principal = 1000, years = 10.	NEEDS_INPUT	compound_interest needs rate (nominal annual interest rate (decimal; 5% = 0.05)). Example: solve('compound_interest', pr

Continuous compounding — `continuous_compounding`

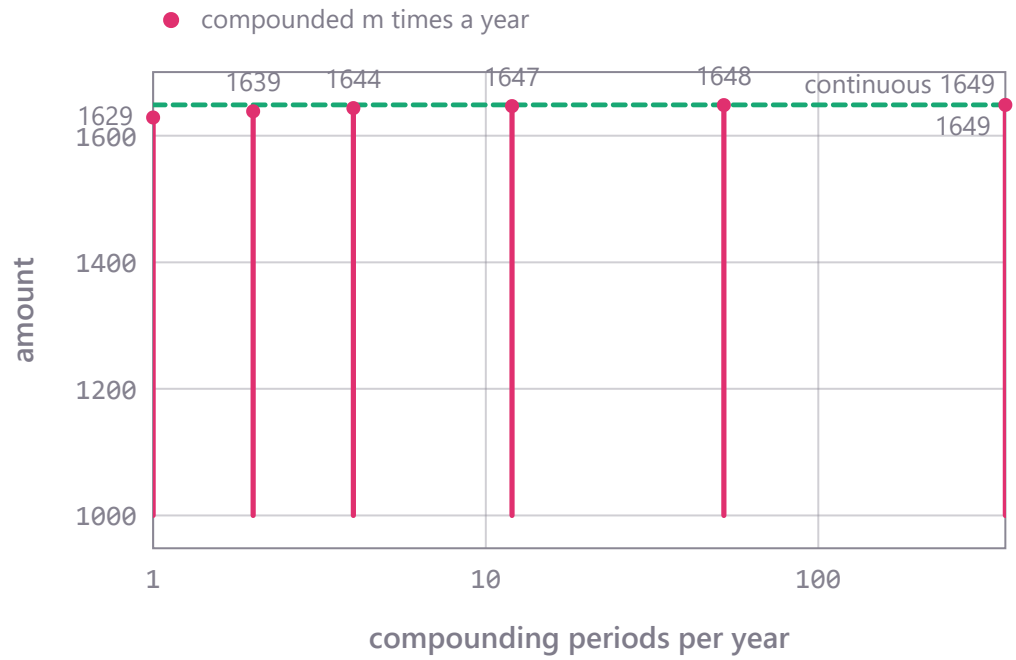
Future amount of a sum growing at a continuously compounded annual rate: $A = P$ e^(r t).

$$A = Pe^{rt}$$

input	meaning	condition
`principal`	amount invested now	
`rate`	continuously compounded annual rate (decimal)	
`years`	time in years	

output	meaning
`amount`	future amount P e^(r t)
`interest`	interest earned

- **Method:** $A = P$ exp(r t)
- **Assumes:** principal > 0; -1 < rate < 1; years >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



1000 at 5 % for 10 years compounded 1, 2, 4, 12, 52 and 365 times a year (stems from the principal, each a compound-interest run of the package) approach the continuously compounded amount the package returns, 1648.72 (dashed).

```
economics_finance_ai999x.solve('continuous_compounding', principal=1000, rate=0.05, years=10)
# -> amount = 1648.72, interest = 648.721   (SOLVED)
```

```
economics_finance_ai999x.solve('continuous_compounding', principal=500, rate=0.08, years=2.5)
# -> amount = 610.701, interest = 110.701   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the amount with continuous compounding with principal = 1000, rate = 5%, years = 10.	SOLVED	amount = 1648.72, interest = 648.721
What is the amount compounded continuously with principal = 500, rate = 8%, years = 2.5?	SOLVED	amount = 610.701, interest = 110.701
Find the amount with continuous compounding with principal = 0, rate = 5%, years = 10.	INVALID_INPUT	Continuous compounding: principal must be positive.

Effective annual rate — `effective_annual_rate`

The effective annual rate (annual percentage yield) of a nominal rate compounded m times a year.

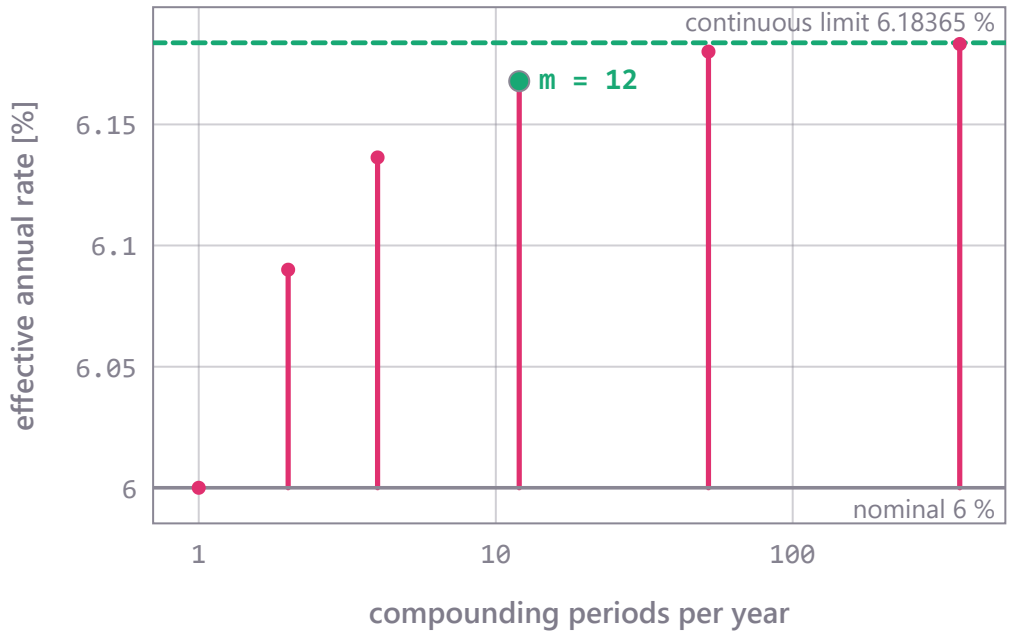
$$\mathrm{EAR} = \left(1 + \frac{r}{m}\right)^{m} - 1$$

input	meaning	condition
`rate`	nominal annual rate (decimal)	
`periods_per_year`	compounding periods per year	

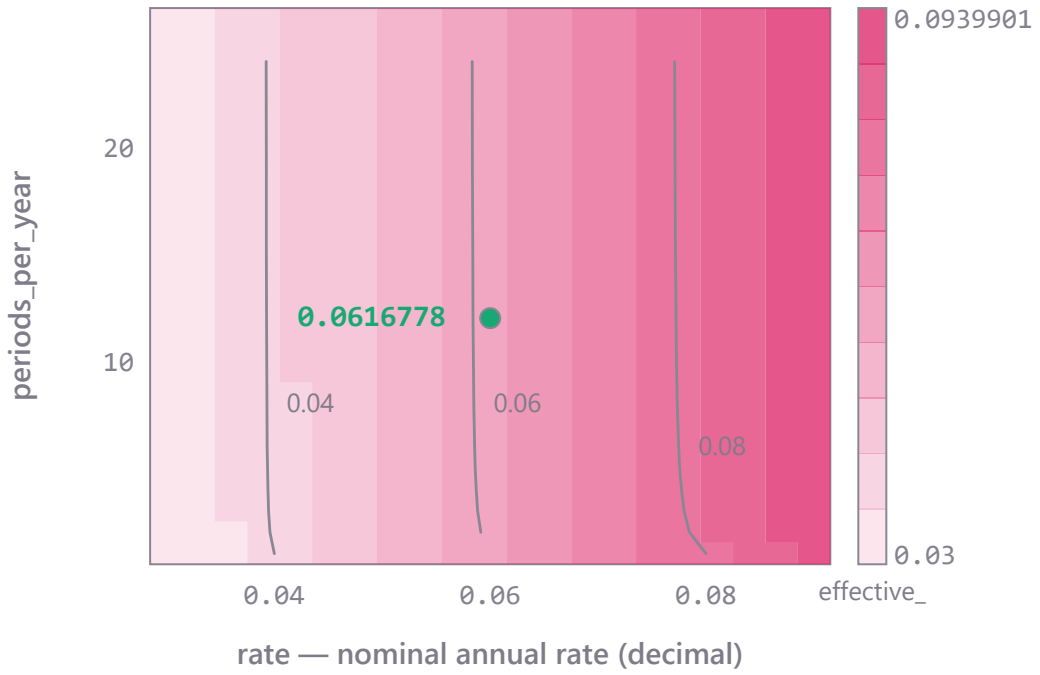
output	meaning
`effective_rate`	effective annual rate $(1 + r/m)^{m} - 1$

output	meaning
`periodic_rate`	rate per period r/m

- **Method:** EAR = (1 + r/m)^m - 1
- **Assumes:** -1 < rate < 1; periods_per_year a whole number >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The effective annual rate of a nominal 6 % compounded 1, 2, 4, 12, 52 and 365 times a year (each a run of the package), rising from the nominal rate towards the continuous limit (the package's continuous compounding of 1 for a year: 6.18365 %). Marked: the worked example, m = 12.



effective_rate (effective annual rate (1 + r/m)^m - 1) over rate (nominal annual rate (decimal)) from 0.03 to 0.09 and periods_per_year (compounding periods per year) from 1 to 24: each of the 21 × 12 cells is one run of economics-finance-ai999x, shaded by its value (light 0.03 to dark 0.0939901); iso-lines at effective_rate = 0.04, 0.06, 0.08. Marked: the worked example, rate = 0.06, periods_per_year = 12, effective_rate = 0.0616778.

```
economics_finance_ai999x.solve('effective_annual_rate', rate=0.06, periods_per_year=12)
# -> effective_rate = 0.0616778, periodic_rate = 0.005   (SOLVED)
```

```
economics_finance_ai999x.solve('effective_annual_rate', rate=0.1, periods_per_year=2)
# -> effective_rate = 0.1025, periodic_rate = 0.05   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the effective annual rate with rate = 6%, periods per year = 12.	SOLVED	effective_rate = 0.0616778, periodic_rate = 0.005
What is the annual percentage yield with nominal rate = 10%, compounding frequency = 2?	SOLVED	effective_rate = 0.1025, periodic_rate = 0.05
Find the effective annual rate with rate = 6%, periods per year = 0.	INVALID_INPUT	periods_per_year: must be at least 1, got 0.

Nominal rate from an effective rate — `nominal_rate`

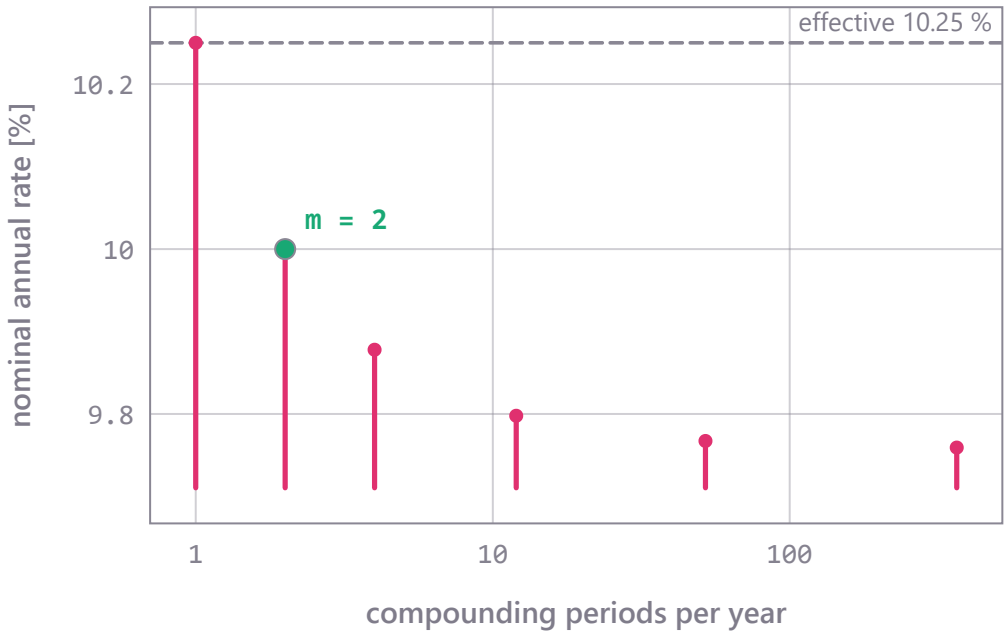
The nominal annual rate (APR) compounded m times a year that gives a stated effective annual rate.

$$r = m\left[(1+\mathrm{EAR})^{1/m} - 1\right]$$

input	meaning	condition
`effective_rate`	effective annual rate (decimal)	
`periods_per_year`	compounding periods per year	

output	meaning
`nominal_rate`	nominal annual rate m((1 + EAR)^(1/m) - 1)
`periodic_rate`	rate per period

- **Method:** $r = m$ ((1 + EAR)^(1/m) - 1)
- **Assumes:** -1 < effective_rate < 1; periods_per_year a whole number >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The nominal annual rate that gives an effective rate of 10.25 % when compounded 1, 2, 4, 12, 52 and 365 times a year, each a run of the package: the more often interest compounds, the lower the quoted rate. Marked: the worked example, m = 2.

```
economics_finance_ai999x.solve('nominal_rate', effective_rate=0.1025, periods_per_year=2)
# -> nominal_rate = 0.1, periodic_rate = 0.05   (SOLVED)
```

```
economics_finance_ai999x.solve('nominal_rate', effective_rate=0.08, periods_per_year=12)
# -> nominal_rate = 0.0772084, periodic_rate = 0.00643403   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the nominal annual rate with effective rate = 8%, periods per year = 12.	SOLVED	nominal_rate = 0.0772084, periodic_rate = 0.00643403
What APR with ear = 10.25%, m = 2?	SOLVED	nominal_rate = 0.1, periodic_rate = 0.05

Present value of a single sum — `present_value`

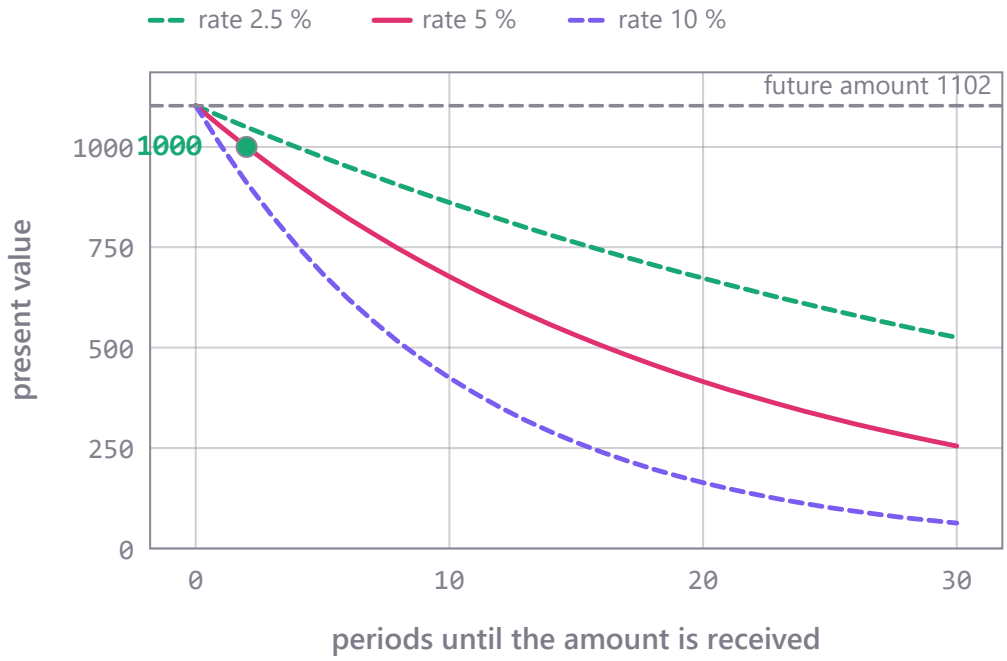
What a single amount received after n periods is worth today at a rate per period: $PV = FV/(1+r)^n$.

$$PV = \frac{FV}{(1+r)^n}$$

input	meaning	condition
`future_value`	amount received in the future	
`rate`	discount rate per period (decimal)	
`periods`	number of periods until the amount is received	

output	meaning
`present_value`	value today FV $(1+r)^{-n}$
`discount_factor`	$(1+r)^{-n}$

- **Method:** $PV = FV\,(1+r)^{-n}$
- **Assumes:** -1 < rate < 1; periods >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



What 1102.5 received in the future is worth today, against the number of periods until it arrives, at rates 2.5 %, 5 % and 10 % (every point a run of the package). Marked: the worked example, 2 periods at 5 %: 1000 (discount factor 0.907029).

```
economics_finance_ai999x.solve('present_value', future_value=1102.5, rate=0.05, periods=2)
# -> present_value = 1000, discount_factor = 0.907029   (SOLVED)
```

```
economics_finance_ai999x.solve('present_value', future_value=1000, rate=0.05, periods=10)
# -> present_value = 613.913, discount_factor = 0.613913   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the present value with future value = 1000, rate = 5%, periods = 10.	SOLVED	present_value = 613.913, discount_factor = 0.613913
What is the present value with fv = 1102.5, discount rate = 5%, years = 2?	SOLVED	present_value = 1000, discount_factor = 0.907029

question	outcome	result
Find the present value with future value = 1000, rate = 5%, periods = -2.	INVALID_INPUT	Present value of a single sum: periods must not be negative.

Future value of an annuity — `future_value_annuity`

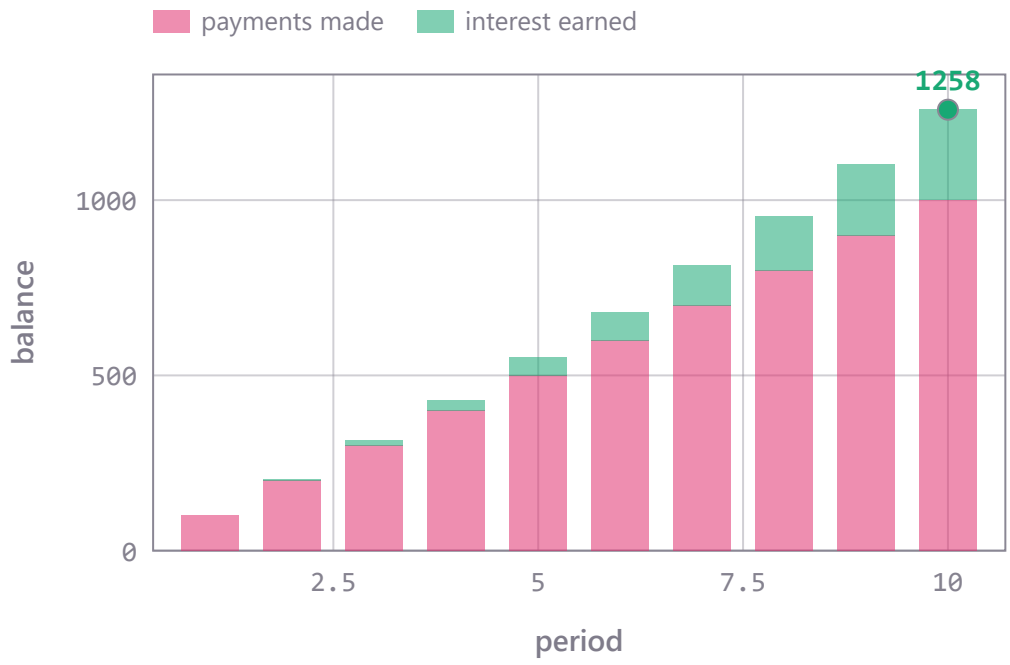
The accumulated value after n periods of equal payments, ordinary (end of period) or due (beginning).

$$FV = PMT\,\frac{(1+r)^n - 1}{r}\ (\times(1+r)\ \text{if due})$$

input	meaning	condition
`payment`	payment per period	
`rate`	interest rate per period (decimal)	
`periods`	number of payments	
`timing`	'ordinary' (end of each period, default) or 'due' (beginning)	

output	meaning
`future_value`	accumulated value at the end of period n

- **Method:** FV = PMT ((1 + r)^n - 1)/r, times $(1+r)$ for an annuity due (PMT n when r = 0)
- **Assumes:** -1 < rate < 1; periods a whole number >= 1; timing ordinary or due
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The balance after each of 10 payments of 100 at 5 % per period: the future value after k payments is a run of the package; it is split into the payments made (k × 100) and the interest (the remainder, computed here). Marked: after 10 payments, 1257.79.

```
economics_finance_ai999x.solve('future_value_annuity', payment=100, rate=0.05, periods=10)
# -> future_value = 1257.79   (SOLVED)
```

```
economics_finance_ai999x.solve('future_value_annuity', payment=100, rate=0.05, periods=10,
timing='due')
# -> future_value = 1320.68   (SOLVED)
```

```
economics_finance_ai999x.solve('future_value_annuity', payment=100, rate=0.0, periods=12)
# -> future_value = 1200   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the future value of an annuity with payment = 100, rate = 5%, periods = 10.	SOLVED	future_value = 1257.79
Find the future value of an annuity due with payment = 100, rate = 5%, periods = 10, timing = due.	SOLVED	future_value = 1320.68
Find the future value of an annuity with payment = 100, rate = 5%, periods = 10, timing = sometimes.	INVALID_INPUT	timing: one of ordinary, due; got 'sometimes'.
Find the future value of an annuity with payment = 100, periods = 10.	NEEDS_INPUT	future_value_annuity needs rate (interest rate per period (decimal)). Example: solve('future_value_annuity', payment=100

Present value of an annuity — `present_value_annuity`

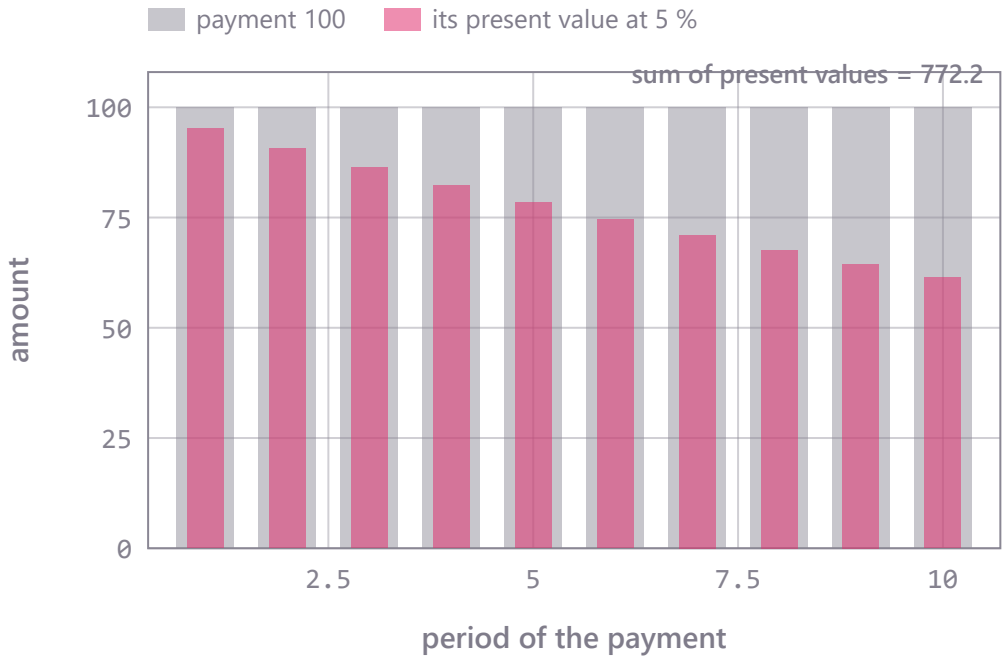
The value today of n equal payments, ordinary (end of period) or due (beginning).

$$PV = PMT\,\frac{1-(1+r)^{-n}}{r}\ (\times(1+r)\ \text{if due})$$

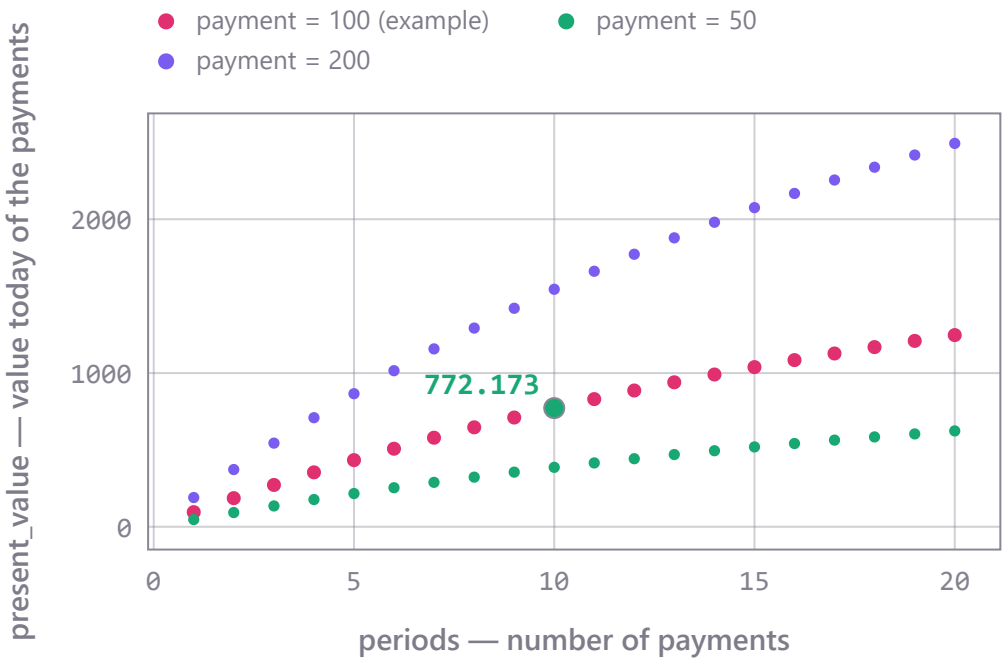
input	meaning	condition
`payment`	payment per period	
`rate`	interest rate per period (decimal)	
`periods`	number of payments	
`timing`	'ordinary' (end of each period, default) or 'due' (beginning)	

output	meaning
`present_value`	value today of the payments

- **Method:** PV = PMT $(1-(1+r)^{-n})/r$, times $(1+r)$ for an annuity due (PMT n when r = 0)
- **Assumes:** -1 < rate < 1; periods a whole number >= 1; timing ordinary or due
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



10 payments of 100 at the end of each period and the present value of each at 5 % (runs of the package's present value); they sum to the annuity's present value the package returns, 772.173.



present_value (value today of the payments) against periods (number of payments) from 1 to 20. One curve for each value of payment (payment per period): 100, 50, 200. Other inputs as in the worked example: rate = 0.05. 20 points per curve computed by economics-finance-ai999x; marked: the worked example, periods = 10, present_value = 772.173.

```
economics_finance_ai999x.solve('present_value_annuity', payment=100, rate=0.05, periods=10)
# -> present_value = 772.173   (SOLVED)
```

```
economics_finance_ai999x.solve('present_value_annuity', payment=100, rate=0.05, periods=10,
timing='due')
# -> present_value = 810.782   (SOLVED)
```

```
economics_finance_ai999x.solve('present_value_annuity', payment=250, rate=0.0, periods=4)
# -> present_value = 1000   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the present value of an annuity with payment = 100, rate = 5%, periods = 10.	SOLVED	present_value = 772.173
Find the present value of an annuity due with payment = 100, rate = 5%, periods = 10, timing = beginning.	SOLVED	present_value = 810.782
Find the present value of an annuity with payment = 100, rate = 5%, periods = 10.5.	INVALID_INPUT	Present value of an annuity: periods must be a whole number.

Loan payment (amortizing loan) — `loan_payment`

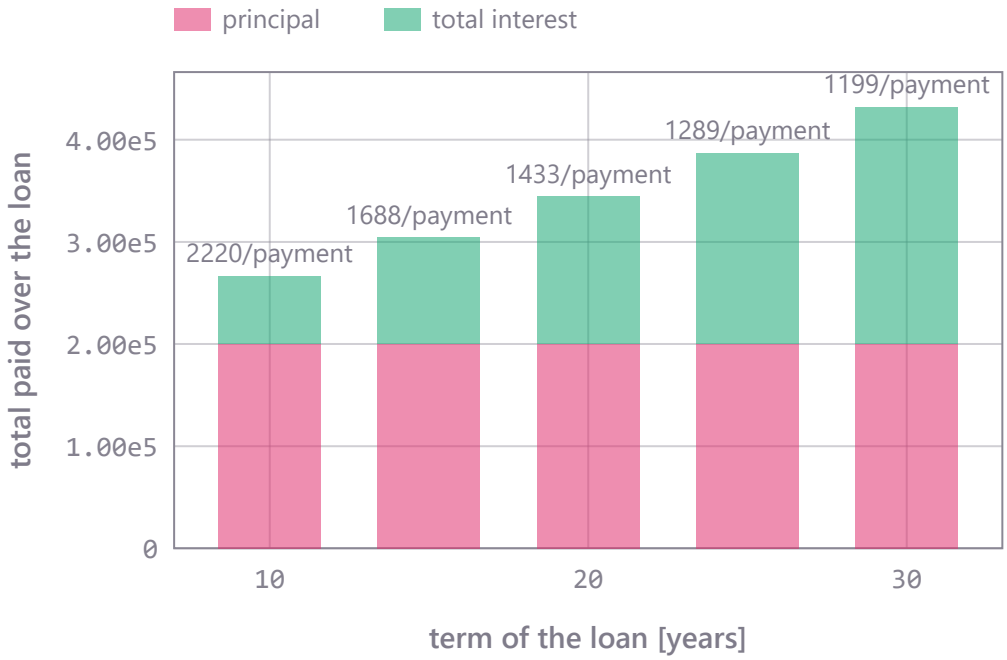
The level payment that repays a loan over its term, with the total paid and the total interest.

$$PMT = \frac{P\,i}{1-(1+i)^{-n}}$$

input	meaning	condition
`principal`	amount borrowed	
`annual_rate`	nominal annual interest rate (decimal), compounded at the payment frequency	
`years`	term of the loan in years	
`payments_per_year`	payments per year (12 = monthly, default)	

output	meaning
`payment`	level payment per period
`number_of_payments`	number of payments n = years x m
`total_paid`	payment x n
`total_interest`	total paid - principal

- **Method:** PMT = P $i/(1-(1+i)^{-n})$, i = annual rate / m, n = years x m
- **Assumes:** principal > 0; 0 <= annual_rate < 1; years x payments_per_year a whole number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A loan of 200000 at 6 % repaid over 10 to 30 years: the payment and the total interest returned by the package for each term (stacked on the principal). Longer terms lower the payment and raise the interest; the worked example, 30 years, pays 1199.1 a period and 231676 interest in all.

```
economics_finance_ai999x.solve('loan_payment', principal=200000, annual_rate=0.06, years=30)
# -> payment = 1199.1, number_of_payments = 360, total_paid = 431676, total_interest = 231676
(SOLVED)
```

```
economics_finance_ai999x.solve('loan_payment', principal=10000, annual_rate=0.08, years=5,
payments_per_year=4)
# -> payment = 611.567, number_of_payments = 20, total_paid = 12231.3, total_interest = 2231.34
(SOLVED)
```

```
economics_finance_ai999x.solve('loan_payment', principal=12000, annual_rate=0.0, years=1)
# -> payment = 1000, number_of_payments = 12, total_paid = 12000, total_interest = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the monthly payment on a loan with principal = 200000, annual rate = 6%, years = 30.	SOLVED	payment = 1199.1, number_of_payments = 360, total_paid = 431676, total_interest = 231676
Find the loan payment with principal = 10000, annual rate = 8%, years = 5, payments per year = 4.	SOLVED	payment = 611.567, number_of_payments = 20, total_paid = 12231.3, total_interest = 2231.34
Find the mortgage payment with loan amount = 300000, interest rate = 7%, term = 15.	SOLVED	payment = 2696.48, number_of_payments = 180, total_paid = 485367, total_interest = 185367
Find the loan payment with principal = 10000, annual rate = 8%, years = 2.55.	INVALID_INPUT	Loan payment (amortizing loan): years times the number of payments per year must be a whole number of periods (at least
Find the loan payment with principal = 10000, years = 5.	NEEDS_INPUT	loan_payment needs annual_rate (nominal annual interest rate (decimal), compounded at the payment frequency).

question	outcome	result
		Example: s

Sinking fund payment — `sinking_fund`

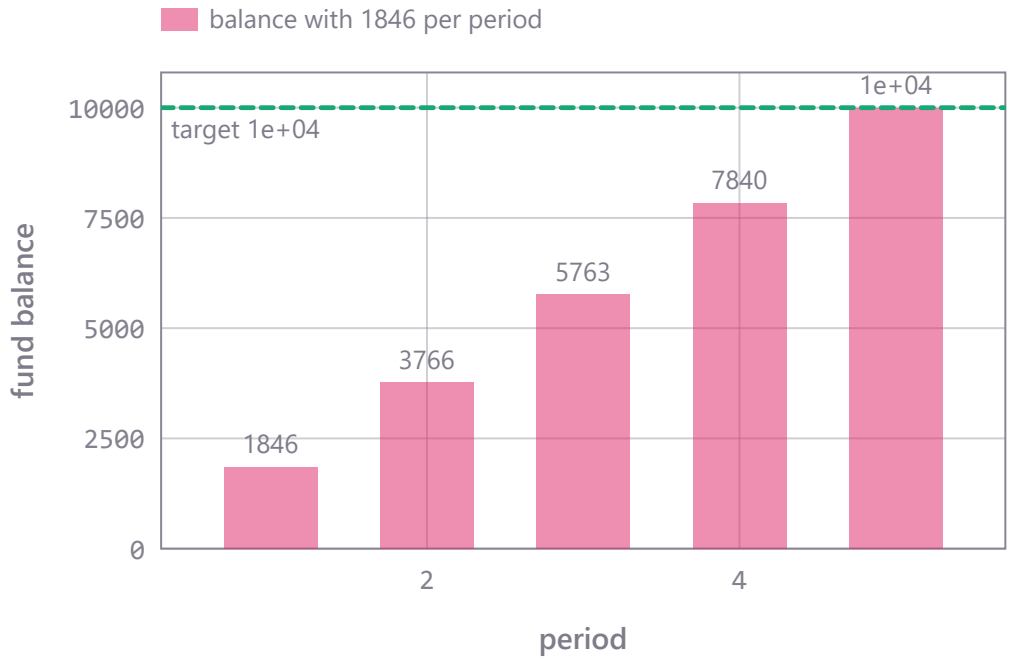
The equal payment per period that accumulates to a target amount after n periods.

$$PMT = \frac{FV\, r}{(1+r)^n - 1}$$

input	meaning	condition
`future_value`	target amount	
`rate`	interest rate per period (decimal)	
`periods`	number of payments	
`timing`	'ordinary' (end of each period, default) or 'due' (beginning)	

output	meaning
`payment`	payment per period

- **Method:** PMT = FV r / ((1 + r)^n - 1), divided by $(1+r)$ for payments at the beginning
- **Assumes:** future_value > 0; -1 < rate < 1; periods a whole number >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A fund that must reach 10000 after 5 periods at 4 %: the package returns the payment per period, 1846.27; the balance after each payment (future-value-of-annuity runs with that payment) reaches the target exactly.

```
economics_finance_ai999x.solve('sinking_fund', future_value=10000, rate=0.04, periods=5)
# -> payment = 1846.27   (SOLVED)
```

```
economics_finance_ai999x.solve('sinking_fund', future_value=1200, rate=0.0, periods=12)
# -> payment = 100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the sinking fund payment with future value = 10000, rate = 4%, periods = 5.	SOLVED	payment = 1846.27
Find the sinking fund payment with target = 10000, rate = 4%, periods = 5, timing = due.	SOLVED	payment = 1775.26

Number of payments to repay a loan — `number_of_periods`

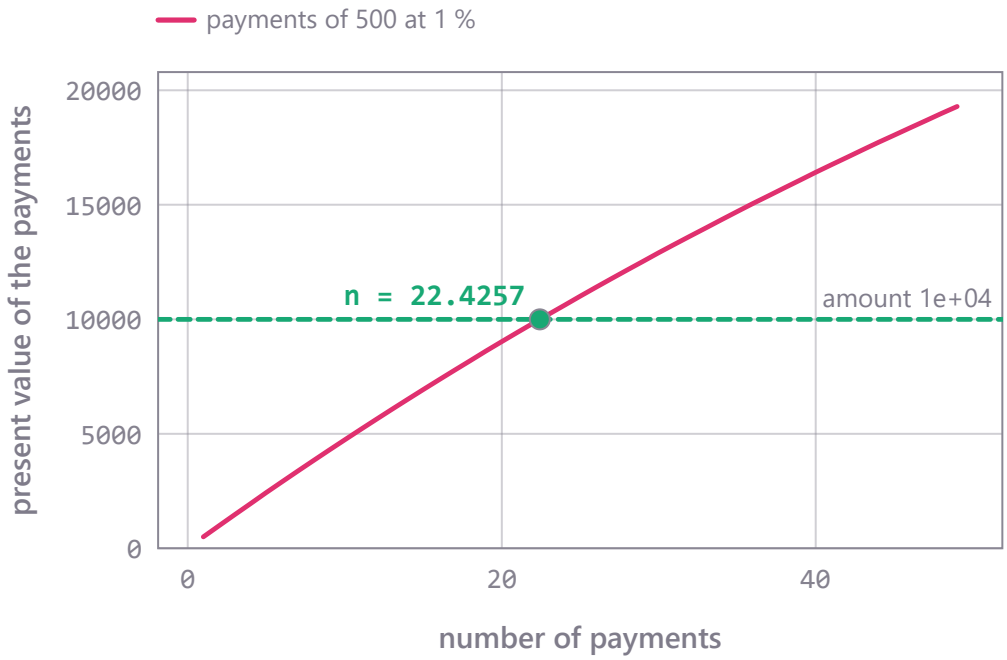
How many periodic payments repay a present value at a rate per period (a fractional answer means the last payment is smaller).

$$n = \frac{-\ln\left(1 - \frac{PV\,r}{PMT}\right)}{\ln(1+r)}$$

input	meaning	condition
`present_value`	amount borrowed (present value)	
`payment`	payment per period	
`rate`	interest rate per period (decimal)	
`timing`	'ordinary' (end of each period, default) or 'due' (beginning)	

output	meaning
`periods`	number of periods n (may be fractional)

- **Method:** $n = -ln(1 - PV$ r / PMT) / ln(1 + r) (PMT $(1 + r)$ for an annuity due)
- **Assumes:** present_value > 0; payment > 0; 0 < rate < 1; payment larger than the interest of one period
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The present value of 1, 2, ... 49 payments of 500 at 1 % per period (runs of the package's present value of an annuity); it reaches the stated 10000 after the number of periods the package returns, n = 22.4257.

```
economics_finance_ai999x.solve('number_of_periods', present_value=10000, payment=500, rate=0.01)
# -> periods = 22.4257   (SOLVED)
```

```
economics_finance_ai999x.solve('number_of_periods', present_value=10000, payment=500, rate=0.01,
timing='due')
# -> periods = 22.1773   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the number of payments with present value = 10000, payment = 500, rate = 1%.	SOLVED	periods = 22.4257
How many payments with loan amount = 10000, payment = 50, rate = 1%?	INVALID_INPUT	Number of payments to repay a loan: the payment does not even cover the interest: the loan is never repaid.

Interest rate of an annuity — `annuity_rate`

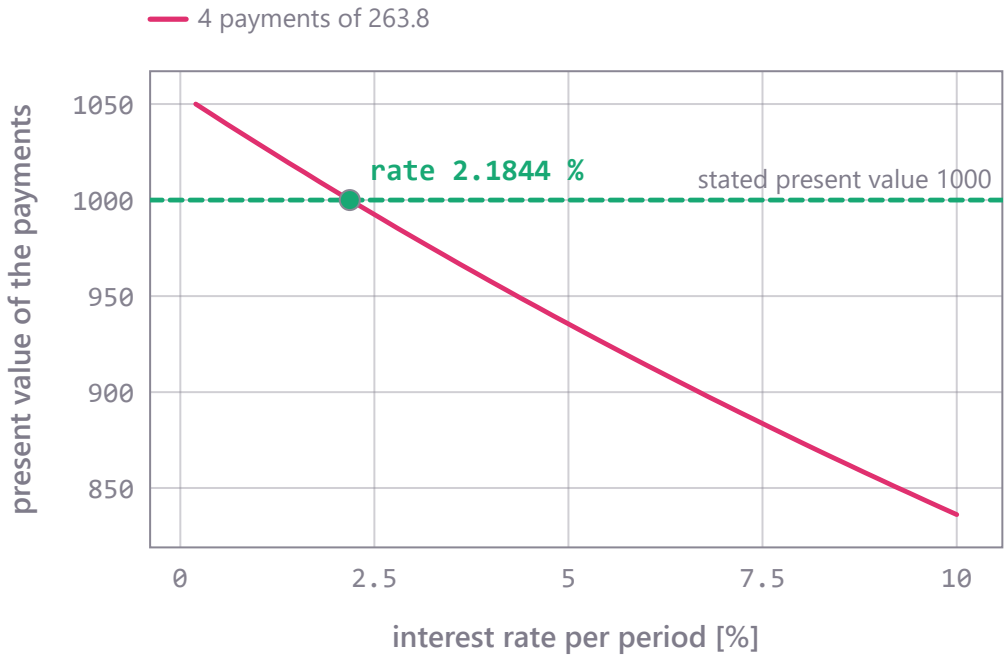
The rate per period at which n equal payments are worth a given present value (bisection; the rate is unique because the present value falls as the rate rises).

$$PV = PMT\,\frac{1-(1+r)^{-n}}{r}$$

input	meaning	condition
`present_value`	present value (amount borrowed or price paid)	
`payment`	payment per period	
`periods`	number of payments	
`timing`	'ordinary' (end of each period, default) or 'due' (beginning)	

output	meaning
`rate`	interest rate per period

- **Method:** bisection on PMT $(1-(1+r)^{-n})/r = PV$
- **Assumes:** present_value > 0; payment > 0; periods a whole number >= 1; the rate lies between -99% and 99%
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The present value of 4 payments of 263.8 falls as the rate rises (50 runs of the package's present value of an annuity); the rate that makes it equal the stated 1000 is the one the package returns, 2.1844 %. Marked: that rate.

```
economics_finance_ai999x.solve('annuity_rate', present_value=1000, payment=263.8, periods=4)
# -> rate = 0.021844   (SOLVED)
```

```
economics_finance_ai999x.solve('annuity_rate', present_value=386.0867, payment=100, periods=5,
timing='ordinary')
# -> rate = 0.0928691   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the interest rate of an annuity with present value = 1000, payment = 263.80, periods = 4.	SOLVED	rate = 0.021844
Find the interest rate of an annuity with present value = 1000, payment = 1, periods = 1.	INVALID_INPUT	Interest rate of an annuity: no rate between -99% and 99% per period gives this present value.

Present value of a perpetuity — `perpetuity`

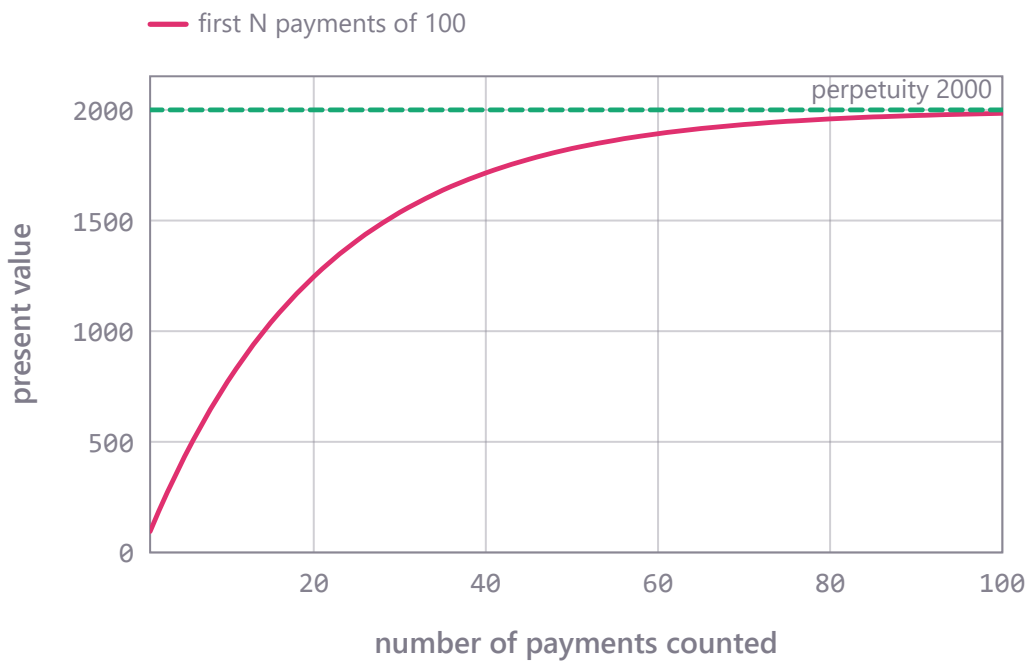
The value today of an equal payment every period forever: $PV = C/r$ (C / r + C if payments start now).

$$PV = \frac{C}{r}$$

input	meaning	condition
`payment`	payment per period	
`rate`	discount rate per period (decimal)	
`timing`	'ordinary' (first payment in one period, default) or 'due' (first payment now)	

output	meaning
`present_value`	value today

- **Method:** $PV = C/r$, times $(1 + r)$ for a perpetuity due
- **Assumes:** 0 < rate < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The present value at 5 % of the first N payments of 100 (runs of the package's present value of an annuity, N up to 100) approaches the value of payments that never stop, which the package returns: 2000.



present_value (value today) against rate (discount rate per period (decimal)) from 0.025 to 0.075. One curve for each value of payment (payment per period): 100, 50, 200. 31 points per curve computed by economics-finance-ai999x; marked: the worked example, rate = 0.05, present_value = 2000.

```
economics_finance_ai999x.solve('perpetuity', payment=100, rate=0.05)
# -> present_value = 2000   (SOLVED)
```

```
economics_finance_ai999x.solve('perpetuity', payment=100, rate=0.05, timing='due')
# -> present_value = 2100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the present value of a perpetuity with payment = 100, rate = 5%.	SOLVED	present_value = 2000
Find the present value of a perpetuity due with payment = 100, rate = 5%, timing = due.	SOLVED	present_value = 2100
Find the present value of a perpetuity with payment = 100, rate = 0.	INVALID_INPUT	Present value of a perpetuity: rate must be a decimal rate between 0 and 1 (write 5% or 0.05).

Present value of a growing perpetuity — `growing_perpetuity`

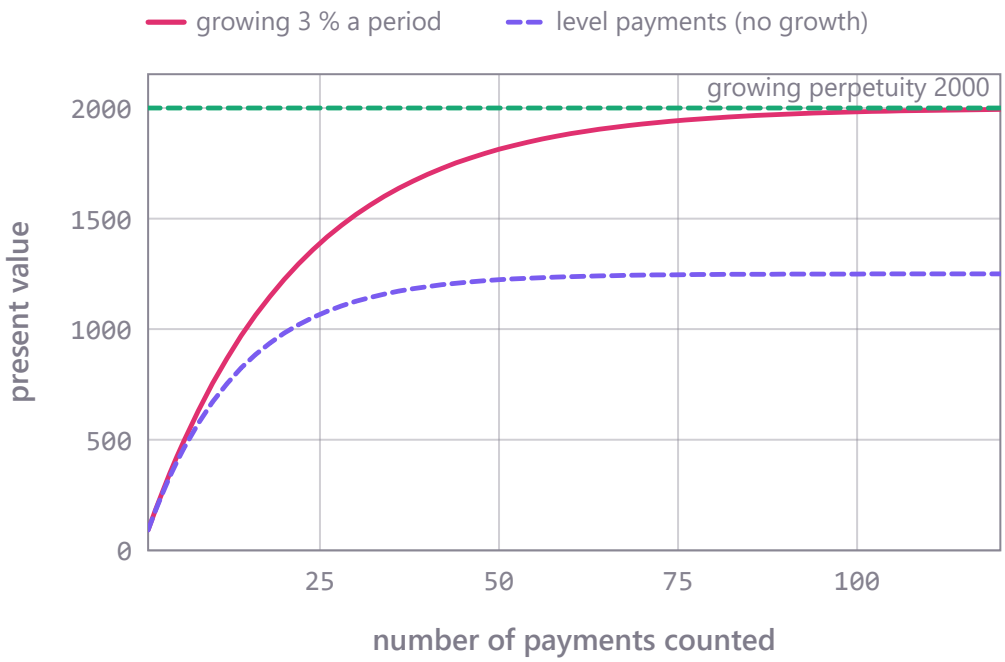
The value today of payments that start at C one period from now and grow at g per period forever: C / (r - g).

$$PV = \frac{C_1}{r - g}$$

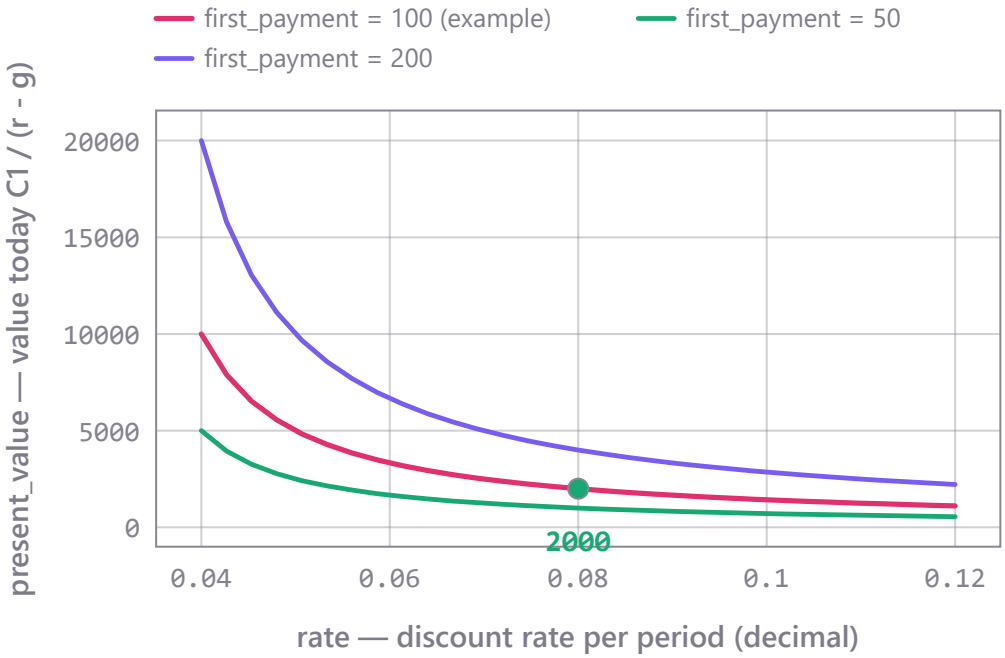
input	meaning	condition
`first_payment`	payment one period from now	
`rate`	discount rate per period (decimal)	
`growth`	growth rate per period (decimal)	

output	meaning
`present_value`	value today $C_1/(r-g)$

- **Method:** $PV = C_1/(r-g)$
- **Assumes:** 0 < rate < 1; growth < rate
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Payments starting at 100 and growing 3 % a period, discounted at 8 %: the present value of the first N (growing-annuity runs of the package) approaches the growing perpetuity's value the package returns, 2000; dashed: the same first payment without growth.



present_value (value today C1 / (r - g)) against rate (discount rate per period (decimal)) from 0.04 to 0.12. One curve for each value of first_payment (payment one period from now): 100, 50, 200. Other inputs as in the worked example: growth = 0.03. 31 points per curve computed by economics-finance-ai999x; marked: the worked example, rate = 0.08, present_value = 2000.

```
economics_finance_ai999x.solve('growing_perpetuity', first_payment=100, rate=0.08, growth=0.03)
# -> present_value = 2000   (SOLVED)
```

```
economics_finance_ai999x.solve('growing_perpetuity', first_payment=50, rate=0.1, growth=-0.02)
# -> present_value = 416.667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the present value of a growing perpetuity with first payment = 100, rate = 8%, growth = 3%.	SOLVED	present_value = 2000
Find the present value of a growing perpetuity with first payment = 100, rate = 3%, growth = 8%.	INVALID_INPUT	Present value of a growing perpetuity: the discount rate must exceed the growth rate (otherwise the value is infinite).

Present and future value of a growing annuity — `growing_annuity`

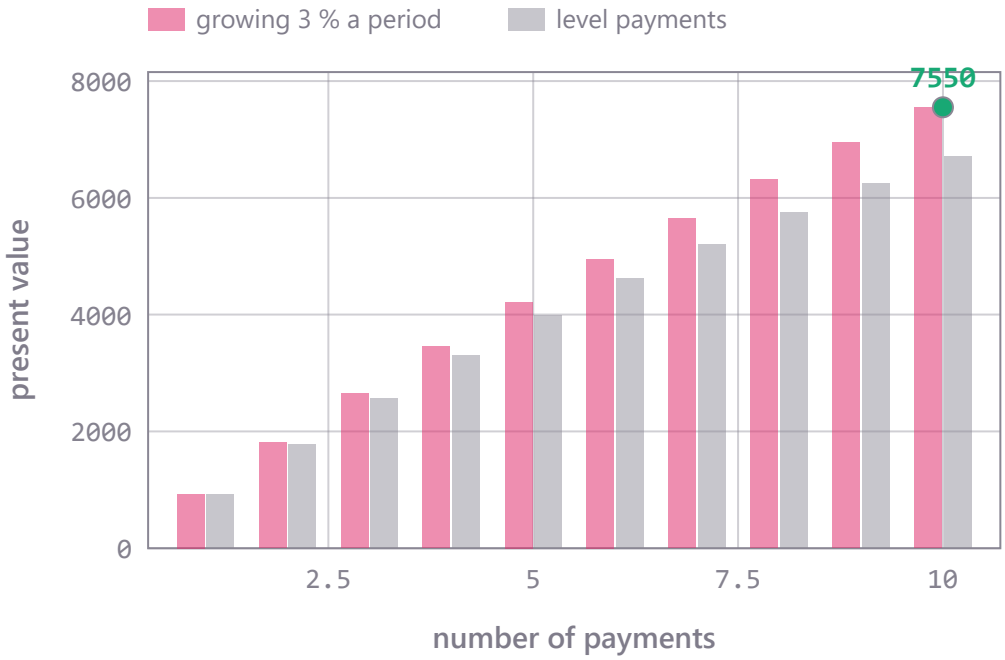
n payments at the end of each period, the first C1, each growing at g per period.

$$PV = \frac{C_1}{r-g}\left[1-\left(\frac{1+g}{1+r}\right)^n\right]$$

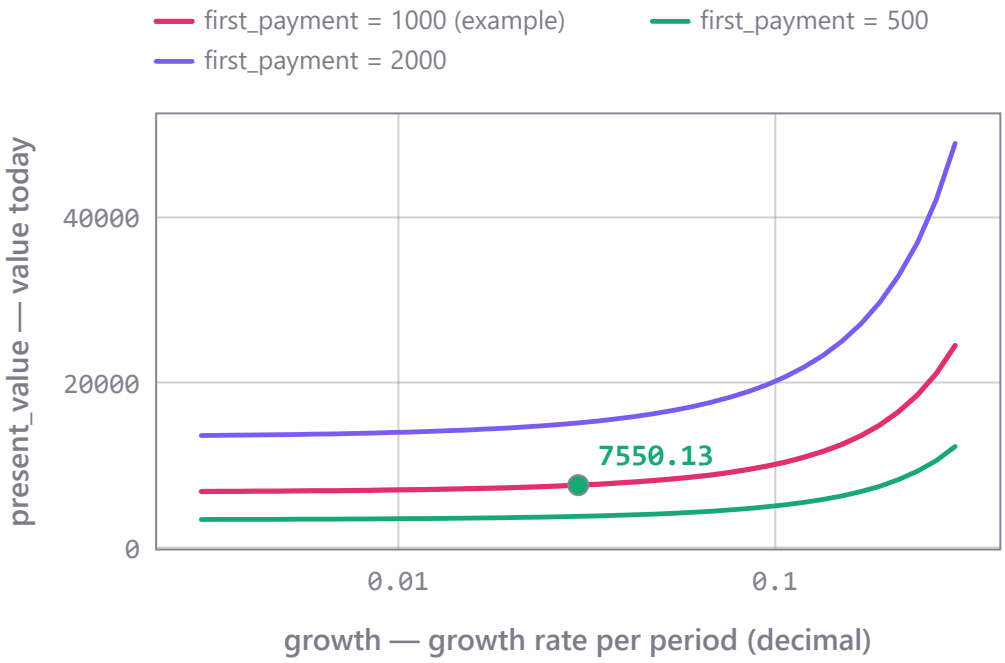
input	meaning	condition
`first_payment`	first payment (end of period 1)	
`rate`	discount rate per period (decimal)	
`growth`	growth rate per period (decimal)	
`periods`	number of payments	

output	meaning
`present_value`	value today
`future_value`	value at the end of period n

- **Method:** $PV = C_1/(r-g)$ (1 - (($1+g$)/($1+r$))^n) (n $C_1/(1+r)$ when r = g); $FV = PV\,(1+r)^n$
- **Assumes:** -1 < rate < 1; -1 < growth < 1; periods a whole number >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Present value at 8 % of the first k payments of an annuity starting at 1000 and growing 3 % a period, beside level payments of 1000 (both runs of the package for k = 1 … 10). Marked: all 10 payments, 7550.13 (future value 16300.2).



present_value (value today) against growth (growth rate per period (decimal)) from 0.003 to 0.3 on a logarithmic axis. One curve for each value of first_payment (first payment (end of period 1)): 1000, 500, 2000. Other inputs as in the worked example: rate = 0.08, periods = 10. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, growth = 0.03, present_value = 7550.13.

```
economics_finance_ai999x.solve('growing_annuity', first_payment=1000, rate=0.08, growth=0.03,
periods=10)
# -> present_value = 7550.13, future_value = 16300.2   (SOLVED)
```

```
economics_finance_ai999x.solve('growing_annuity', first_payment=100, rate=0.05, growth=0.05,
periods=3)
# -> present_value = 285.714, future_value = 330.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the present value of a growing annuity with first payment = 1000, rate = 8%, growth = 3%, periods = 10.	SOLVED	present_value = 7550.13, future_value = 16300.2

Doubling time — `doubling_time`

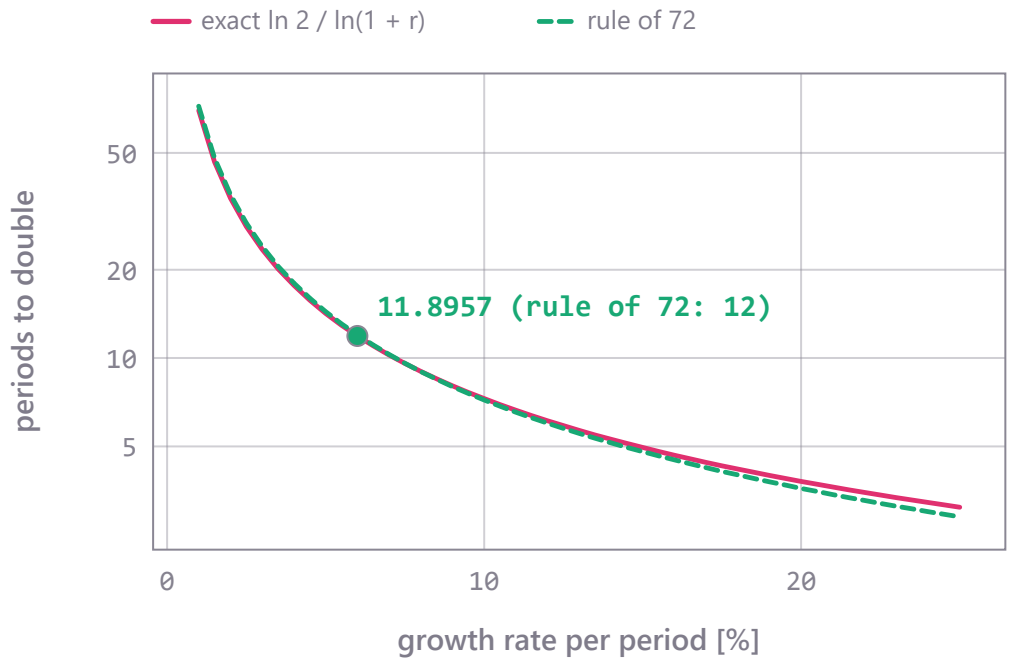
How many periods an amount compounding at r per period takes to double, with the rule-of-72 estimate.

$$n = \frac{\ln 2}{\ln(1+r)}$$

input	meaning	condition
`rate`	interest or growth rate per period (decimal)	

output	meaning
`periods`	exact doubling time ln $2/ln$(1 + r)
`rule_of_72`	rule-of-72 estimate 72 / (100 r)

- **Method:** $n = ln\,2/ln$(1 + r)
- **Assumes:** 0 < rate < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Periods for an amount to double at growth rates from 1 % to 25 % per period: the exact doubling time and the rule of 72, both returned by the package at 49 rates. Marked: the worked example, 6 %: 11.8957 periods exactly, 12 by the rule.

```
economics_finance_ai999x.solve('doubling_time', rate=0.06)
# -> periods = 11.8957, rule_of_72 = 12   (SOLVED)
```

```
economics_finance_ai999x.solve('doubling_time', rate=0.1111111111111111)
# -> periods = 6.57881, rule_of_72 = 6.48   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the doubling time with rate = 6%.	SOLVED	periods = 11.8957, rule_of_72 = 12
Find the doubling time with rate = -3%.	INVALID_INPUT	Doubling time: rate must be a decimal rate between 0 and 1 (write 5% or 0.05).

Compound annual growth rate — `cagr`

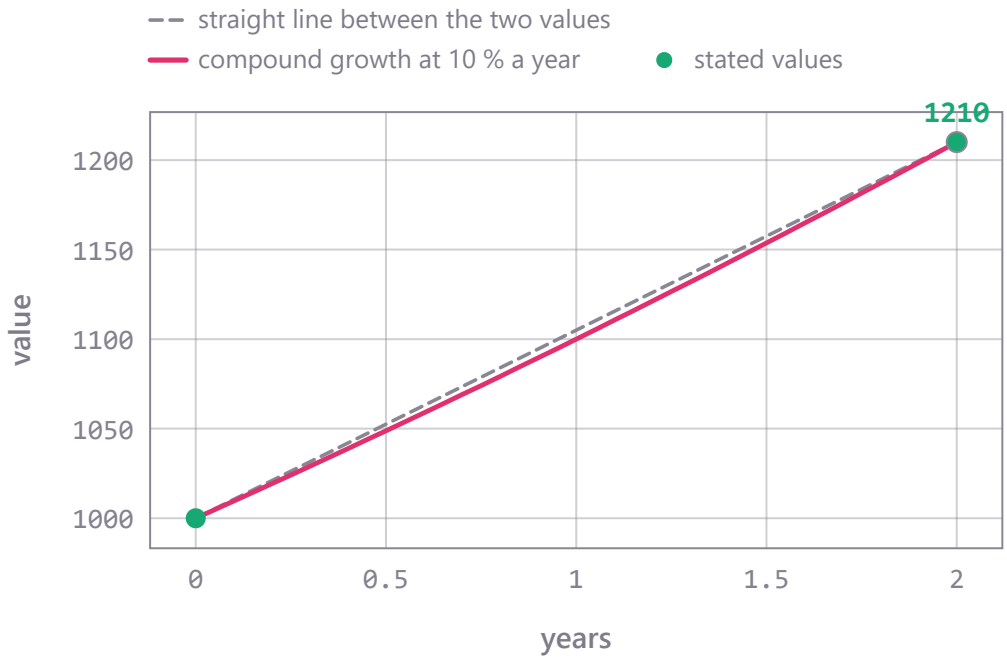
The constant annual growth rate that takes a beginning value to an ending value over a number of years.

$$\mathrm{CAGR} = \left(\frac{V_{end}}{V_{begin}}\right)^{1/t} - 1$$

input	meaning	condition
`begin_value`	value at the start	
`end_value`	value at the end	
`years`	number of years	

output	meaning
`cagr`	compound annual growth rate (end/begin)^(1/t) - 1
`total_growth`	end/begin - 1

- **Method:** CAGR = (end/begin)^(1/t) - 1
- **Assumes:** begin_value > 0; end_value > 0; years > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A value growing from 1000 to 1210 in 2 years: the compound annual growth rate the package returns, 10 % (total growth 21 %), compounds 1000 into 1210 (curve: compound-interest runs at that rate); dashed: a straight line between the stated values.

```
economics_finance_ai999x.solve('cagr', begin_value=1000, end_value=1210, years=2)
# -> cagr = 0.1, total_growth = 0.21   (SOLVED)
```

```
economics_finance_ai999x.solve('cagr', begin_value=1000, end_value=2000, years=5)
# -> cagr = 0.148698, total_growth = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the compound annual growth rate with begin value = 1000, end value = 2000, years = 5.	SOLVED	cagr = 0.148698, total_growth = 1
Find the CAGR with initial value = 1000, final value = 1210, years = 2.	SOLVED	cagr = 0.1, total_growth = 0.21
Find the CAGR with initial value = 0, final value = 1210, years = 2.	INVALID_INPUT	Compound annual growth rate: begin_value must be positive.

Remaining balance of a loan — `loan_balance`

The balance still owed on an amortizing loan after k level payments.

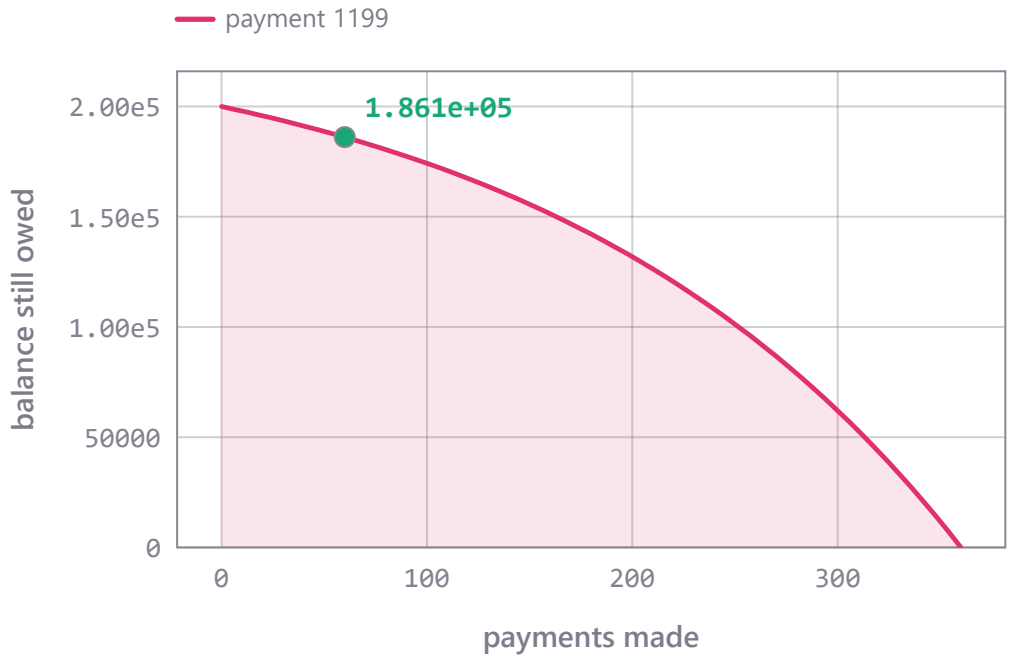
$$B_k = P(1+i)^k - PMT\,\frac{(1+i)^k - 1}{i}$$

input	meaning	condition
`principal`	amount borrowed	
`annual_rate`	nominal annual interest rate (decimal), compounded at the payment frequency	
`years`	term of the loan in years	
`payments_made`	number of payments already made	

input	meaning	condition
`payments_per_year`	payments per year (12 = monthly, default)	

output	meaning
`balance`	balance owed after k payments
`payment`	level payment per period

- **Method:** $B_k = P$ (1 + i)^k - PMT ((1 + i)^k - 1)/i
- **Assumes:** principal > 0; 0 <= annual_rate < 1; 0 <= payments_made <= number of payments
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The balance of a 200000 loan at 6 % over 30 years after each number of payments (every 6 payments, runs of the package): it falls slowly at first, while most of each payment is interest. Marked: after 60 payments, 186109.

```
economics_finance_ai999x.solve('loan_balance', principal=200000, annual_rate=0.06, years=30,
payments_made=60)
# -> balance = 186109, payment = 1199.1   (SOLVED)
```

```
economics_finance_ai999x.solve('loan_balance', principal=200000, annual_rate=0.06, years=30,
payments_made=360)
# -> balance = 0, payment = 1199.1   (SOLVED)
```

```
economics_finance_ai999x.solve('loan_balance', principal=12000, annual_rate=0.0, years=1,
payments_made=3)
# -> balance = 9000, payment = 1000   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the remaining balance with principal = 200000, annual rate = 6%, years = 30, payments made = 60.	SOLVED	balance = 186109, payment = 1199.1

question	outcome	result
Find the remaining balance with principal = 200000, annual rate = 6%, years = 30, payments made = 400.	INVALID_INPUT	Remaining balance of a loan: payments_made must be at most 360.

Interest and principal in a loan payment — `loan_payment_split`

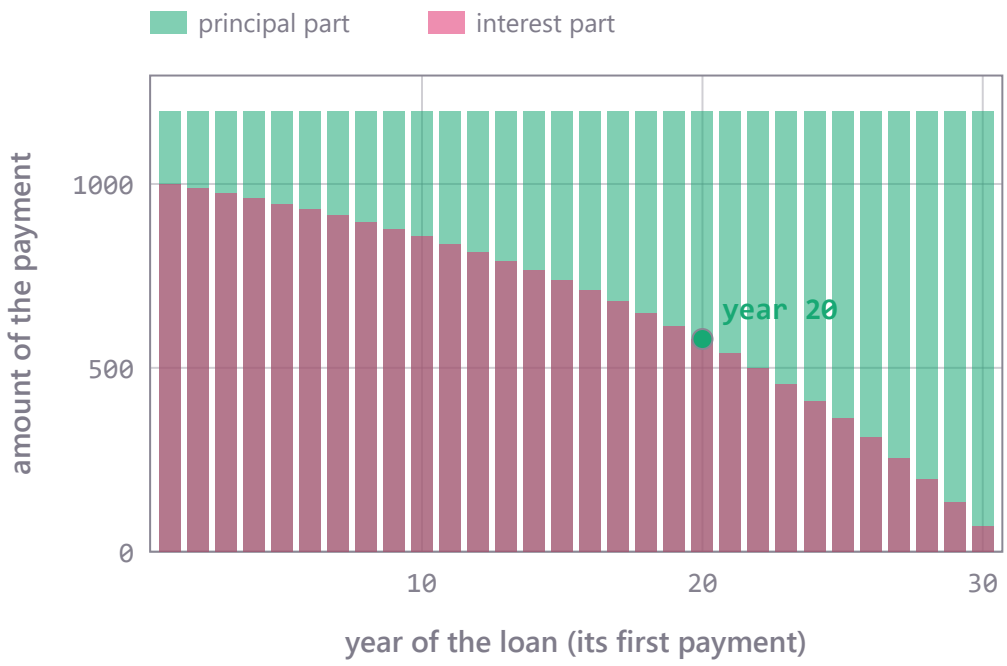
How payment number k of an amortizing loan splits into interest and principal repayment.

$$I_k = i\,B_{k-1}, \quad P_k = PMT - I_k$$

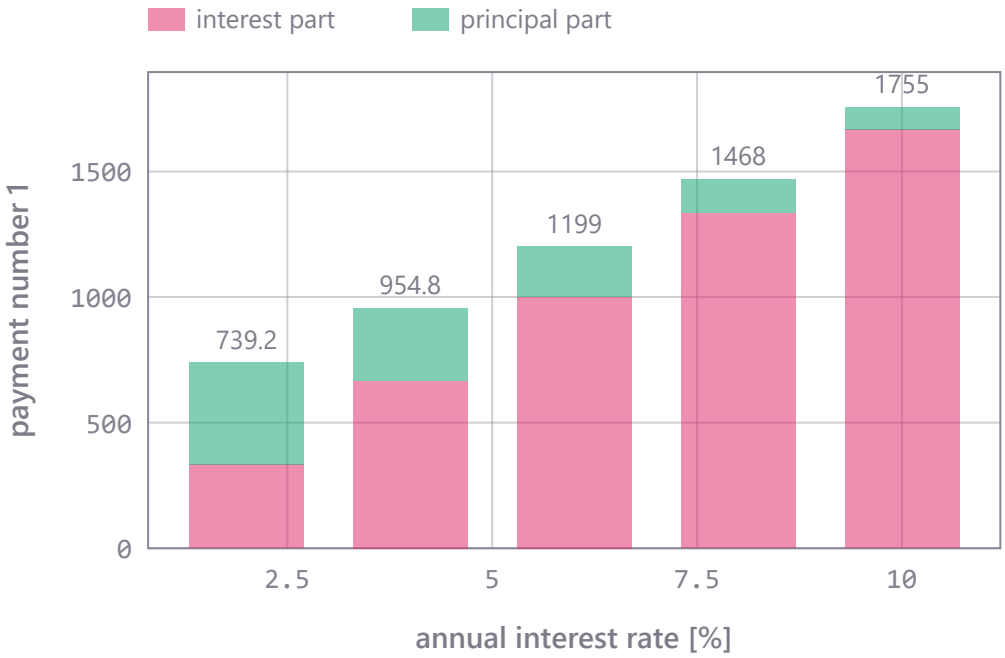
input	meaning	condition
`principal`	amount borrowed	
`annual_rate`	nominal annual interest rate (decimal), compounded at the payment frequency	
`years`	term of the loan in years	
`payment_number`	which payment (1 = the first)	
`payments_per_year`	payments per year (12 = monthly, default)	

output	meaning
`interest_part`	interest in payment k
`principal_part`	principal repaid by payment k
`payment`	level payment

- **Method:** interest = i B_(k-1), principal = PMT - interest
- **Assumes:** principal > 0; 0 <= annual_rate < 1; 1 <= payment_number <= number of payments
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



How the monthly payment of 1199.1 on a loan of 200000 at 6 % over 30 years splits into interest (lower part) and principal (upper part): the first payment of each year, as computed by the package. Year 1: interest 1000, principal 199.101; principal is the larger part from year 20 (interest 578.324). Total interest over the loan: 231676.



How payment number 1 on a loan of 200000 over 30 years splits into interest and principal at annual rates from 2 % to 10 % (each a run of the package; the worked example uses 6 %). The higher the rate, the larger the payment and the more of it is interest.

```
economics_finance_ai999x.solve('loan_payment_split', principal=200000, annual_rate=0.06, years=30,
payment_number=1)
# -> interest_part = 1000, principal_part = 199.101, payment = 1199.1   (SOLVED)
```

```
economics_finance_ai999x.solve('loan_payment_split', principal=200000, annual_rate=0.06, years=30,
payment_number=12)
# -> interest_part = 988.772, principal_part = 210.33, payment = 1199.1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the interest and principal in payment number = 12 with principal = 200000, annual rate = 6%, years = 30.	SOLVED	interest_part = 988.772, principal_part = 210.33, payment = 1199.1
Find the interest portion with principal = 200000, annual rate = 6%, years = 30, payment number = 1.	SOLVED	interest_part = 1000, principal_part = 199.101, payment = 1199.1
Find the interest portion with principal = 200000, annual rate = 6%, years = 30, payment number = 0.	INVALID_INPUT	payment_number: must be at least 1, got 0.

Cumulative interest of a loan — `loan_cumulative_interest`

Total interest paid and principal repaid over the first k payments of an amortizing loan.

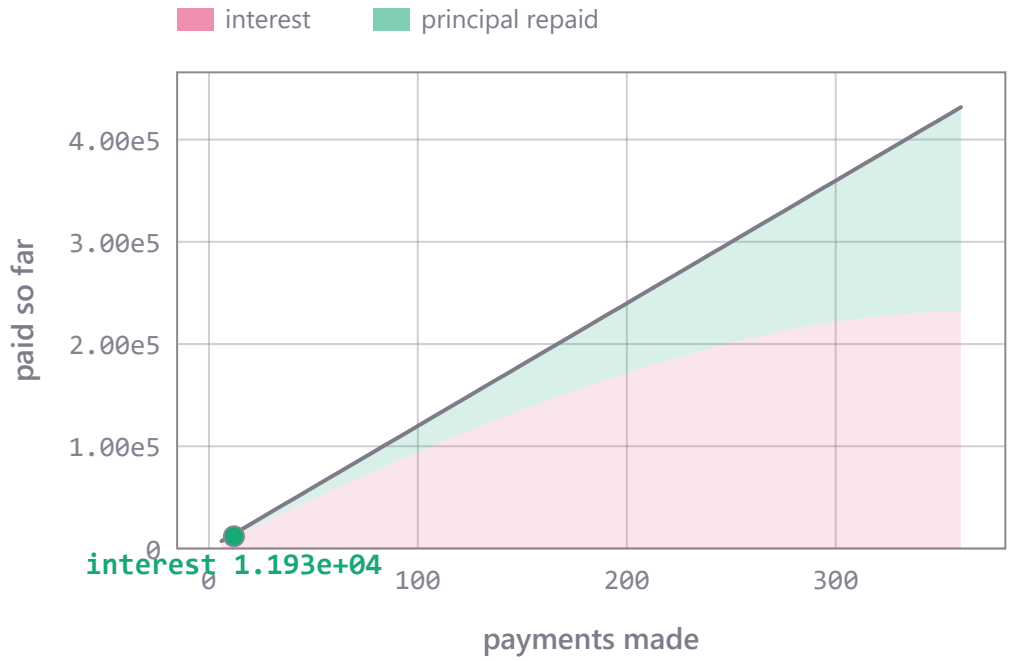
$$\sum_{j=1}^{k} I_j = k\,PMT - (P - B_k)$$

input	meaning	condition
`principal`	amount borrowed	

input	meaning	condition
`annual_rate`	nominal annual interest rate (decimal), compounded at the payment frequency	
`years`	term of the loan in years	
`payments_made`	number of payments counted (the first k)	
`payments_per_year`	payments per year (12 = monthly, default)	

output	meaning
`cumulative_interest`	interest paid in payments 1..k
`principal_repaid`	principal repaid in payments 1..k

- **Method:** principal repaid = P - B_k; interest = k PMT - (P - B_k)
- **Assumes:** principal > 0; 0 <= annual_rate < 1; 1 <= payments_made <= number of payments
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



What has been paid after each number of payments on a 200000 loan at 6 % over 30 years, split into interest (lower) and principal (upper), both returned by the package every 6 payments. Marked: after 12 payments, interest 11933.2, principal 2456.02.

```
economics_finance_ai999x.solve('loan_cumulative_interest', principal=200000, annual_rate=0.06,
years=30, payments_made=12)
# -> cumulative_interest = 11933.2, principal_repaid = 2456.02   (SOLVED)
```

```
economics_finance_ai999x.solve('loan_cumulative_interest', principal=200000, annual_rate=0.06,
years=30, payments_made=1)
# -> cumulative_interest = 1000, principal_repaid = 199.101   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the cumulative interest with principal = 200000, annual rate = 6%, years = 30, payments made = 12.	SOLVED	cumulative_interest = 11933.2, principal_repaid = 2456.02

Net present value — `npv`

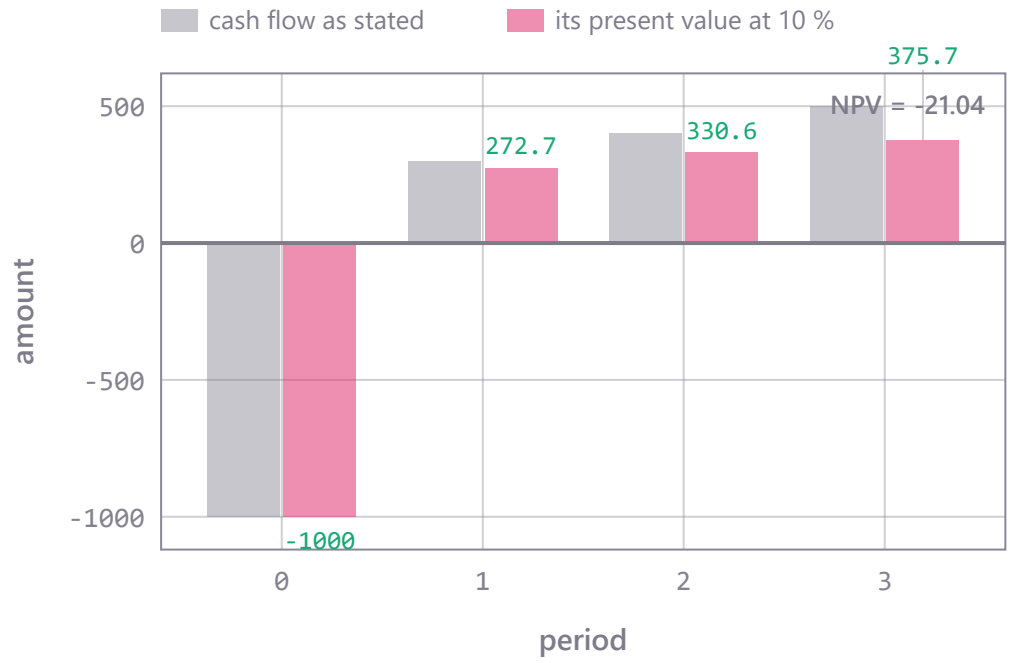
NPV of cash flows [CF0, CF1, ..., CFn] (CF0 now, CFt at the end of period t) at a discount rate per period.

$$NPV = \sum_{t=0}^{n} \frac{CF_t}{(1+r)^t}$$

input	meaning	condition
`rate`	discount rate per period (decimal)	
`cash_flows`	cash flows [CF0, CF1, ..., CFn], CF0 at time 0	

output	meaning
`npv`	net present value

- **Method:** NPV = sum CFt / (1 + r)^t, $t = 0..n$
- **Assumes:** -1 < rate < 1; at least two cash flows
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cash flows -1000, 300, 400, 500 (period 0 first) beside their present values at the discount rate 10 %, each a run of the package's present value; the net present value it returns, NPV = -21.0368, is their sum. Marked: each present value.

```
economics_finance_ai999x.solve('npv', rate=0.1, cash_flows='-1000, 300, 400, 500')
# -> npv = -21.0368   (SOLVED)
```

```
economics_finance_ai999x.solve('npv', rate=0.0, cash_flows=[-100, 50, 60])
# -> npv = 10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the net present value with rate = 10%, cash flows = [-1000, 300, 400, 500].	SOLVED	npv = -21.0368
What is the NPV with discount rate = 0%, cash flows = [-100, 50, 60]?	SOLVED	npv = 10
Find the NPV with rate = 10%, cash flows = [-1000].	INVALID_INPUT	cash_flows: needs at least 2 entries, got 1.
Find the NPV with rate = 10%, cash flows = -1000, 300, 400.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: economics_finance_ai999x.template
Find the NPV with cash flows = [-1000, 300, 400, 500].	NEEDS_INPUT	npv needs rate (discount rate per period (decimal)). Example: solve('npv', rate=0.1, cash_flows='-1000, 300, 400, 500').

Internal rate of return — `irr`

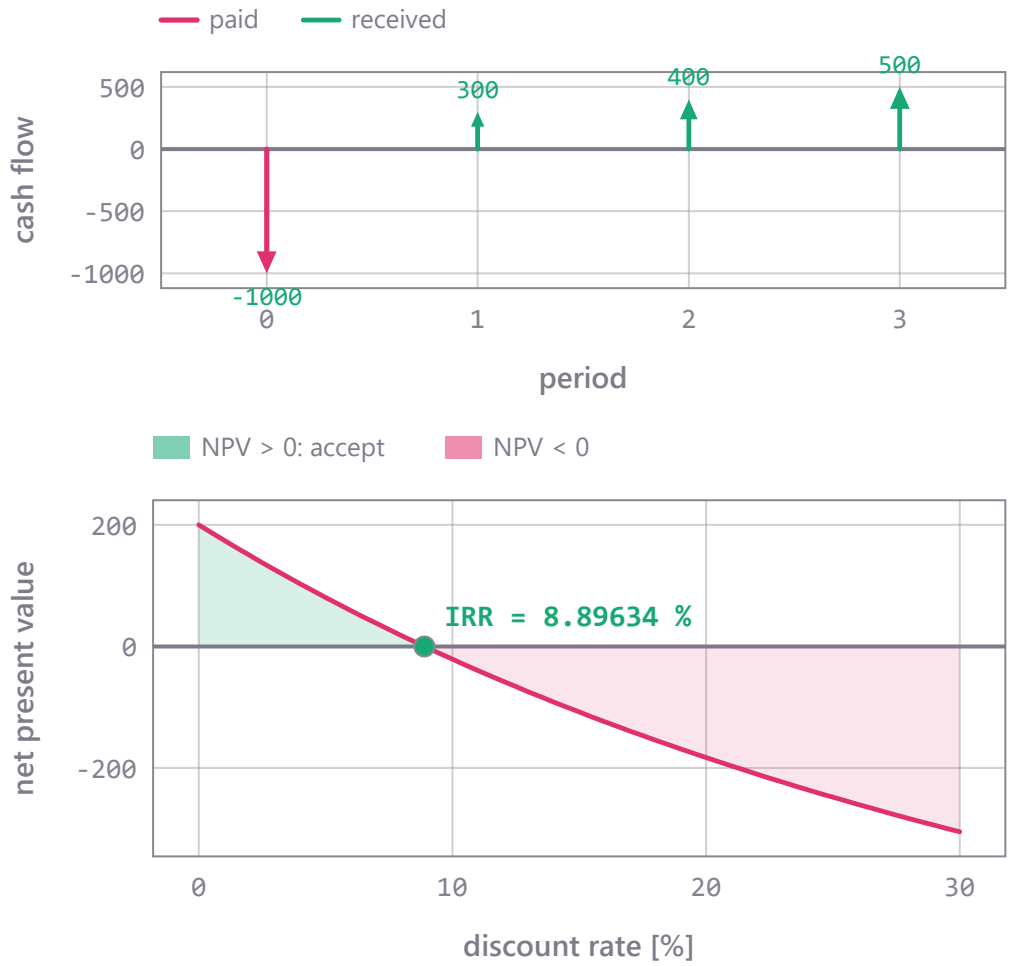
The discount rate at which the NPV of [CF0, ..., CFn] is zero; given only when it is unique (the cash flows change sign exactly once).

$$\sum_{t=0}^{n} \frac{CF_t}{(1+\mathrm{IRR})^t} = 0$$

input	meaning	condition
`cash_flows`	cash flows [CF0, CF1, ..., CFn], CF0 at time 0	

output	meaning
`irr`	internal rate of return per period

- **Method:** bisection on $v = 1/(1+r)$ for the unique positive root of sum CFt v^t, Newton polishing
- **Assumes:** the cash flows change sign exactly once (otherwise the IRR is not unique and none is given)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: the cash flows -1000, 300, 400, 500 as a timeline (arrows down: paid; up: received). Bottom: their net present value at 61 discount rates from 0 to 30 %, each a run of the package's NPV. The NPV is zero at the internal rate of return the package returns; marked: IRR = 8.89634 %.

```
economics_finance_ai999x.solve('irr', cash_flows='-1000, 300, 400, 500')
# -> irr = 0.0889634   (SOLVED)
```

```
economics_finance_ai999x.solve('irr', cash_flows=[-100, 110])
# -> irr = 0.1   (SOLVED)
```

```
economics_finance_ai999x.solve('irr', cash_flows=[-1000, 0, 1210])
# -> irr = 0.1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the internal rate of return with cash flows = [-1000, 300, 400, 500].	SOLVED	irr = 0.0889634
What is the IRR of the cash flows = [-100, 110]?	SOLVED	irr = 0.1
Find the IRR with cash flows = [-100, 230, -132].	INVALID_INPUT	Internal rate of return: the cash flows change sign 2 times: the internal rate of return is not unique, so none is given
Find the IRR with cash flows = [100, 200, 300].	INVALID_INPUT	Internal rate of return: the cash flows never change sign: there is no internal rate of return.

Modified internal rate of return — `mirr`

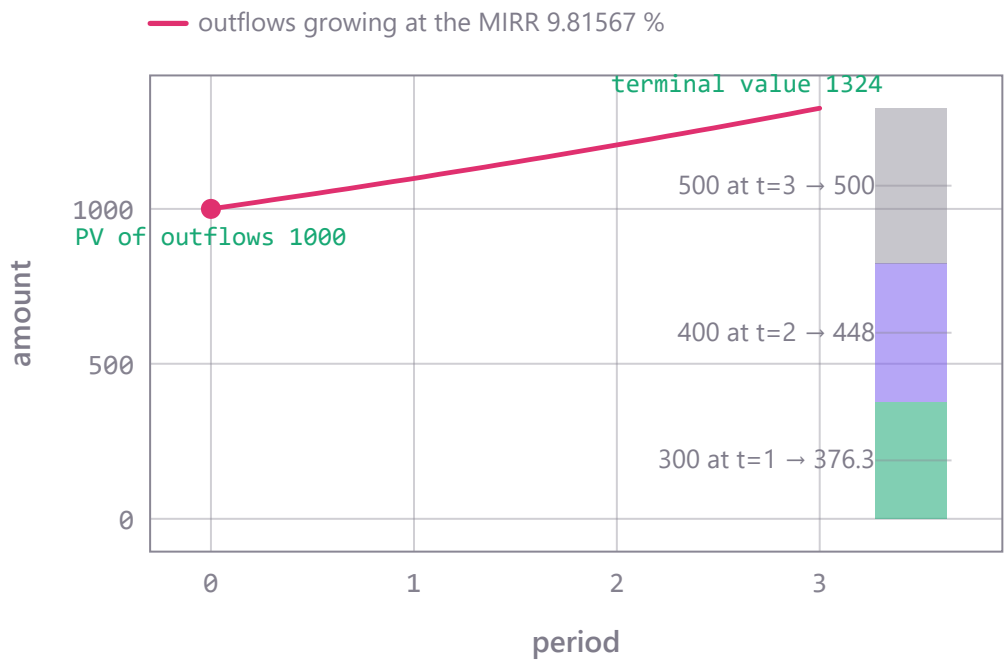
MIRR: outflows discounted at the finance rate, inflows compounded to the end at the reinvestment rate.

$$\mathrm{MIRR} = \left(\frac{FV_{+}}{PV_{-}}\right)^{1/n} - 1$$

input	meaning	condition
`cash_flows`	cash flows [CF0, ..., CFn], CF0 at time 0	
`finance_rate`	finance rate for outflows (decimal)	
`reinvest_rate`	reinvestment rate for inflows (decimal)	

output	meaning
`mirr`	modified IRR per period
`terminal_value`	inflows compounded to period n
`pv_outflows`	present value of the outflows

- **Method:** MIRR = (FV of inflows at the reinvestment rate / PV of outflows at the finance rate)^(1/n) - 1
- **Assumes:** at least one inflow and one outflow; -1 < rates < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Modified IRR of the flows -1000, 300, 400, 500: each inflow is carried forward to period 3 at the reinvestment rate 12 % (stacked bar, each a compound-interest run of the package; they sum to the returned terminal value 1324.32), and the present value of the outflows 1000 (finance rate 10 %) grows at the returned MIRR = 9.81567 % to exactly that terminal value (the curve, compound-interest runs).



mirr (modified IRR per period) against reinvest_rate (reinvestment rate for inflows (decimal)) from 0.06 to 0.18. Other inputs as in the worked example: cash_flows = -1000, 300, 400, 500, finance_rate = 0.1. 31 points computed by economics-finance-ai999x; marked: the worked example, reinvest_rate = 0.12, mirr = 0.0981567.

```
economics_finance_ai999x.solve('mirr', cash_flows='-1000, 300, 400, 500', finance_rate=0.1,
reinvest_rate=0.12)
# -> mirr = 0.0981567, terminal_value = 1324.32, pv_outflows = 1000   (SOLVED)
```

```
economics_finance_ai999x.solve('mirr', cash_flows=[-100, 0, 121], finance_rate=0.05,
reinvest_rate=0.05)
# -> mirr = 0.1, terminal_value = 121, pv_outflows = 100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the modified internal rate of return with cash flows = [-1000, 300, 400, 500], finance rate = 10%, reinvestment rate = 12%.	SOLVED	mirr = 0.0981567, terminal_value = 1324.32, pv_outflows = 1000
Find the MIRR with cash flows = [100, 200], finance rate = 10%, reinvestment rate = 12%.	INVALID_INPUT	Modified internal rate of return: the cash flows need at least one inflow and one outflow.

Payback period — `payback_period`

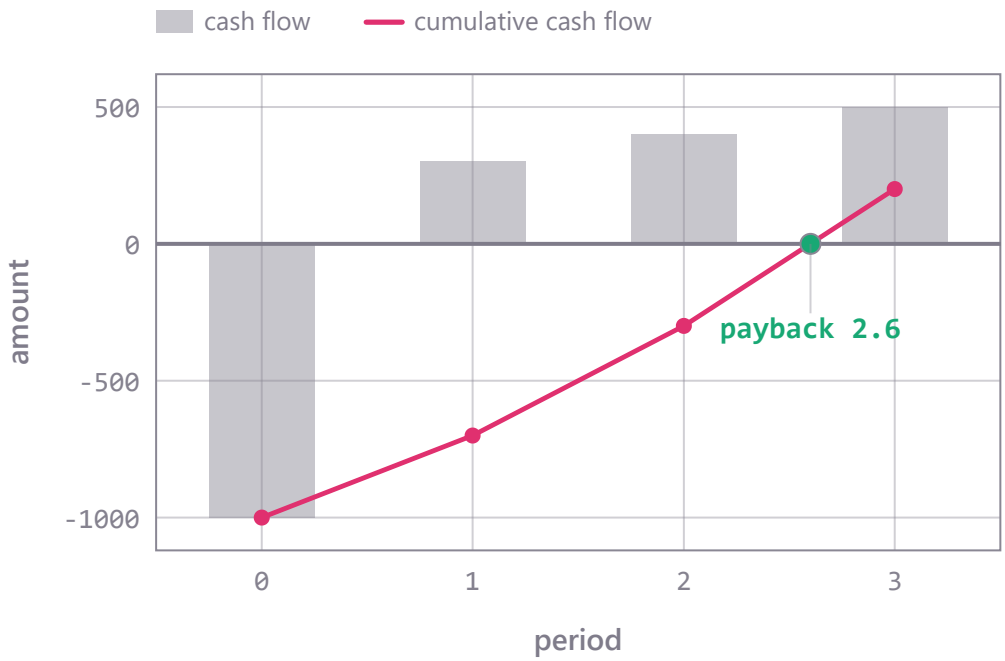
Time until the cumulative cash flow turns non-negative, interpolating linearly within the period (cash arriving evenly during the period).

$$\text{payback} = (t-1) + \frac{-\sum_{s<t} CF_s}{CF_t}$$

input	meaning	condition
`cash_flows`	cash flows [$CF0 < 0$, CF1, ..., CFn > 0]	

output	meaning
`payback`	payback period in periods

- **Method:** first t with cumulative flow >= 0; payback = (t - 1) + (unrecovered at t - 1) / CFt
- **Assumes:** $CF0 < 0$; every later cash flow > 0; the investment is recovered within the flows
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cash flows -1000, 300, 400, 500 as bars and their running sum (the cumulative cash flow, summed here from the stated flows) as a line: the investment is recovered where the line crosses zero. Marked: the payback period the package returns, 2.6 periods (flows assumed spread evenly within a period).

```
economics_finance_ai999x.solve('payback_period', cash_flows='-1000, 300, 400, 500')
# -> payback = 2.6   (SOLVED)
```

```
economics_finance_ai999x.solve('payback_period', cash_flows=[-100, 50, 50, 50])
# -> payback = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the payback period with cash flows = [-1000, 300, 400, 500].	SOLVED	payback = 2.6
Find the payback period with cash flows = [-1000, 300, 400].	INVALID_INPUT	Payback period: the investment is not recovered within the cash flows given.

Discounted payback period — `discounted_payback`

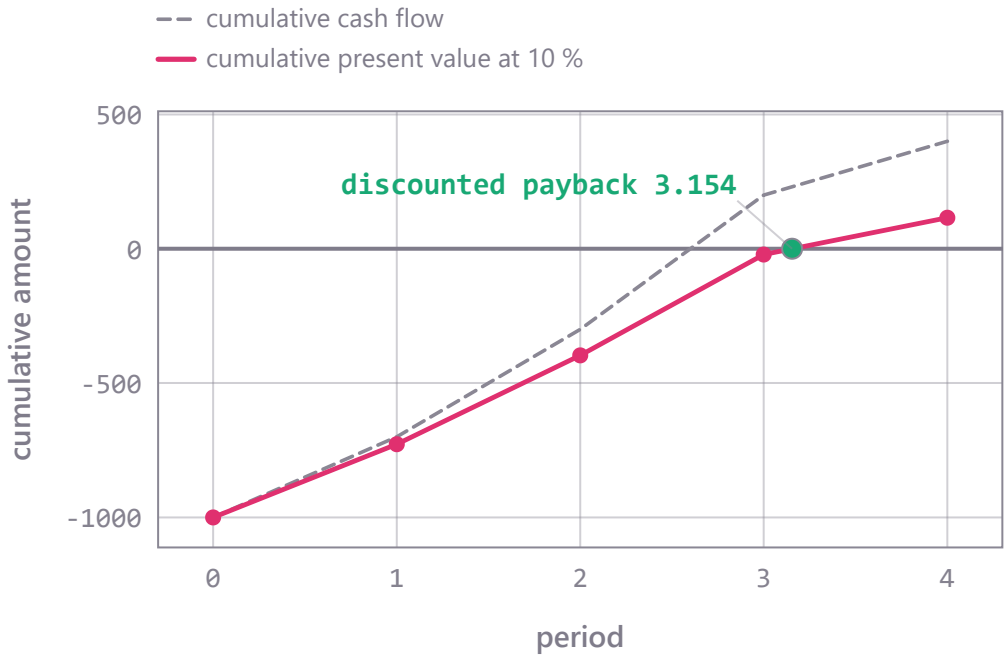
Time until the cumulative DISCOUNTED cash flow turns non-negative (linear within the period).

$$\sum_{s \le t} \frac{CF_s}{(1+r)^s} \ge 0$$

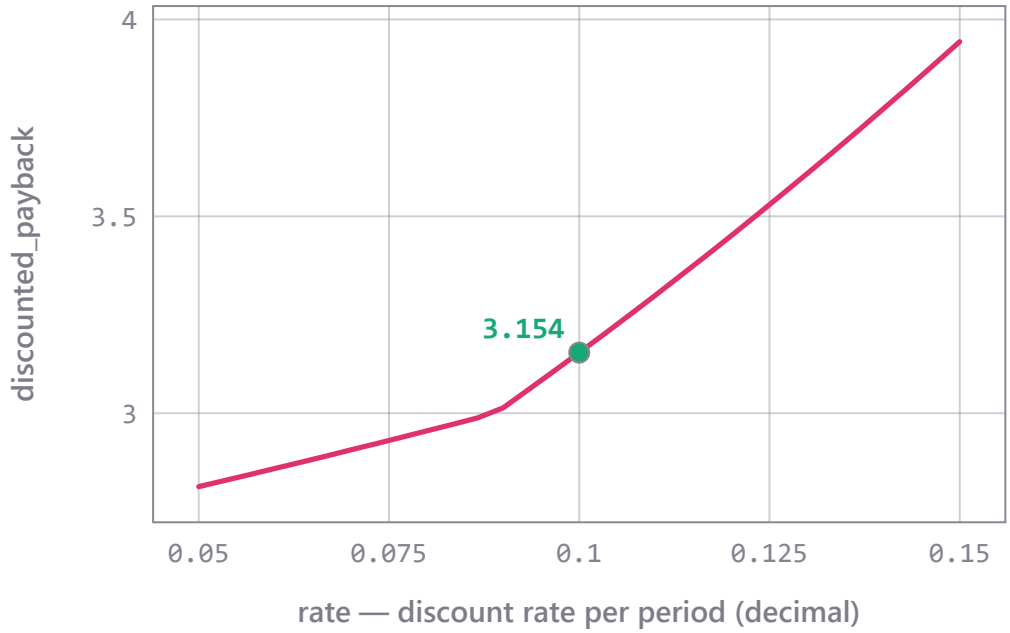
input	meaning	condition
`rate`	discount rate per period (decimal)	
`cash_flows`	cash flows [$CF0 < 0$, CF1, ..., CFn > 0]	

output	meaning
`discounted_payback`	discounted payback period in periods

- **Method:** payback of the discounted flows CFt / (1 + r)^t
- **Assumes:** $CF0 < 0$; every later cash flow > 0; recovered within the flows; -1 < rate < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cash flows -1000, 300, 400, 500, 200: their running sum (dashed) and the running sum of their present values at 10 % (each present value a run of the package; the sums are added here). Discounting delays recovery; marked: the discounted payback period the package returns, 3.154 periods.



discounted_payback (discounted payback period in periods) against rate (discount rate per period (decimal)) from 0.05 to 0.15. Other inputs as in the worked example: cash_flows = -1000, 300, 400, 500, 200. 31 points computed by economics-finance-ai999x; marked: the worked example, rate = 0.1, discounted_payback = 3.154.

```
economics_finance_ai999x.solve('discounted_payback', rate=0.1, cash_flows='-1000, 300, 400, 500,
200')
# -> discounted_payback = 3.154   (SOLVED)
```

```
economics_finance_ai999x.solve('discounted_payback', rate=0.0, cash_flows=[-100, 50, 50, 50])
# -> discounted_payback = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the discounted payback period with rate = 10%, cash flows = [-1000, 300, 400, 500, 200].	SOLVED	discounted_payback = 3.154
Find the discounted payback period with rate = 10%, cash flows = [-1000, 300, 400, 500].	INVALID_INPUT	Discounted payback period: the investment is not recovered within the cash flows given.

Profitability index — `profitability_index`

PV of the future cash flows divided by the initial investment (benefit-cost ratio).

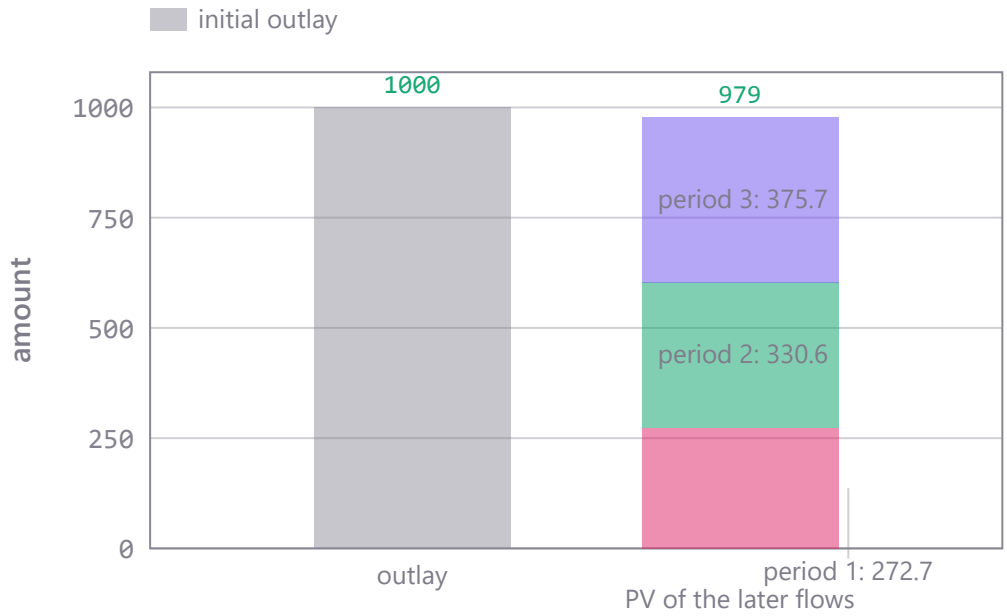
$$PI = \frac{\sum_{t=1}^{n} CF_t (1+r)^{-t}}{-CF_0}$$

input	meaning	condition
`rate`	discount rate per period (decimal)	
`cash_flows`	cash flows [$CF0 < 0$, CF1, ..., CFn]	

output	meaning
`profitability_index`	PV(CF1..CFn) / |CF0|

output	meaning
`pv_future`	PV of CF1..CFn
`npv`	net present value

- **Method:** PI = sum_{$t \geq 1$} CFt/(1+r)^t / (-CF0)
- **Assumes:** $CF0 < 0$; -1 < rate < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The initial outlay 1000 beside the present values at 10 % of the later flows 300, 400, 500 (stacked, each a run of the package; they sum to the returned 978.963). Profitability index = PV ÷ outlay = 0.978963, NPV = -21.0368, as returned.

```
economics_finance_ai999x.solve('profitability_index', rate=0.1, cash_flows='-1000, 300, 400, 500')
# -> profitability_index = 0.978963, pv_future = 978.963, npv = -21.0368   (SOLVED)
```

```
economics_finance_ai999x.solve('profitability_index', rate=0.0, cash_flows=[-100, 60, 60])
# -> profitability_index = 1.2, pv_future = 120, npv = 20   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the profitability index with rate = 10%, cash flows = [-1000, 300, 400, 500].	SOLVED	profitability_index = 0.978963, pv_future = 978.963, npv = -21.0368
Find the profitability index with rate = 10%, cash flows = [1000, 300, 400, 500].	INVALID_INPUT	Profitability index: the first cash flow (the investment at time 0) must be negative.

Equivalent annual annuity — `equivalent_annual_annuity`

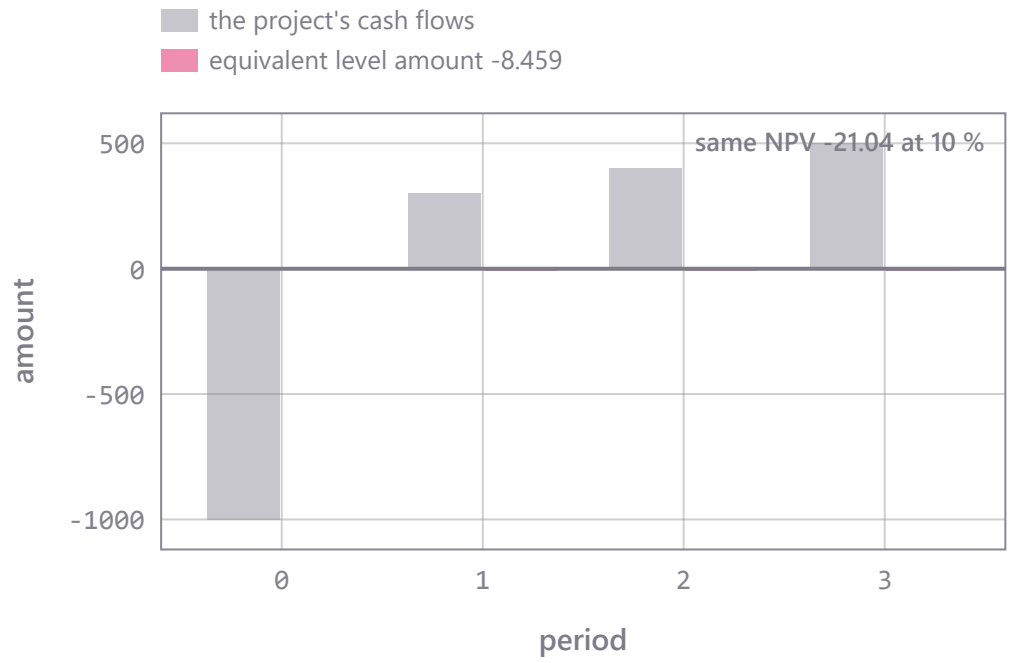
The level payment over the project's life with the same NPV (for comparing projects of unequal lives).

$$EAA = \frac{NPV\, r}{1-(1+r)^{-n}}$$

input	meaning	condition
`rate`	discount rate per period (decimal)	
`cash_flows`	cash flows [CF0, CF1, ..., CFn]	

output	meaning
`eaa`	equivalent annual annuity NPV $r/(1-(1+r)^{-n})$
`npv`	net present value

- **Method:** EAA = NPV $r/(1-(1+r)^{-n})$
- **Assumes:** -1 < rate < 1; at least two cash flows
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The cash flows -1000, 300, 400, 500 and the level amount per period with the same net present value at 10 %: the package returns NPV = -21.0368 and the equivalent annual annuity -8.45921 per period over 3 periods.

```
economics_finance_ai999x.solve('equivalent_annual_annuity', rate=0.1, cash_flows='-1000, 300, 400, 500')
# -> eaa = -8.45921, npv = -21.0368   (SOLVED)
```

```
economics_finance_ai999x.solve('equivalent_annual_annuity', rate=0.0, cash_flows=[-100, 70, 70])
# -> eaa = 20, npv = 40   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equivalent annual annuity with rate = 10%, cash flows = [-1000, 300, 400, 500].	SOLVED	eaa = -8.45921, npv = -21.0368

NPV crossover rate of two projects — `crossover_rate`

The discount rate at which two projects have equal NPV (the IRR of the difference A - B); given only when it is unique.

$$\sum_t \frac{CF_t^A - CF_t^B}{(1+r^*)^t} = 0$$

input	meaning	condition
`cash_flows_a`	cash flows of project A [CF0, ..., CFn]	
`cash_flows_b`	cash flows of project B, same periods	

output	meaning
`crossover_rate`	rate at which NPV(A) = NPV(B)
`npv_at_crossover`	the common NPV at that rate

- **Method:** IRR of the incremental cash flows A - B
- **Assumes:** equal lengths; the difference A - B changes sign exactly once
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



NPV profiles of two projects (A: -1000, 100, 1200; B: -1000, 700, 500) at 61 discount rates, each a run of the package's NPV. Below the crossover rate one project is worth more, above it the other; marked: the crossover rate the package returns, 16.6667 %, where both NPVs equal -32.6531. Dots on the axis: each project's IRR, also from the package.

```
economics_finance_ai999x.solve('crossover_rate', cash_flows_a='-1000, 100, 1200',
cash_flows_b='-1000, 700, 500')
# -> crossover_rate = 0.166667, npv_at_crossover = -32.6531   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the crossover rate with cash flows a = [-1000, 100, 1200], cash flows b = [-1000, 700, 500].	SOLVED	crossover_rate = 0.166667, npv_at_crossover = -32.6531
Find the crossover rate with project a = [-1000, 100, 1200], project b = [-1000, 700].	INVALID_INPUT	NPV crossover rate of two projects: the two projects need cash flows over the same periods (lists of equal length).

Bond price from its yield — `bond_price`

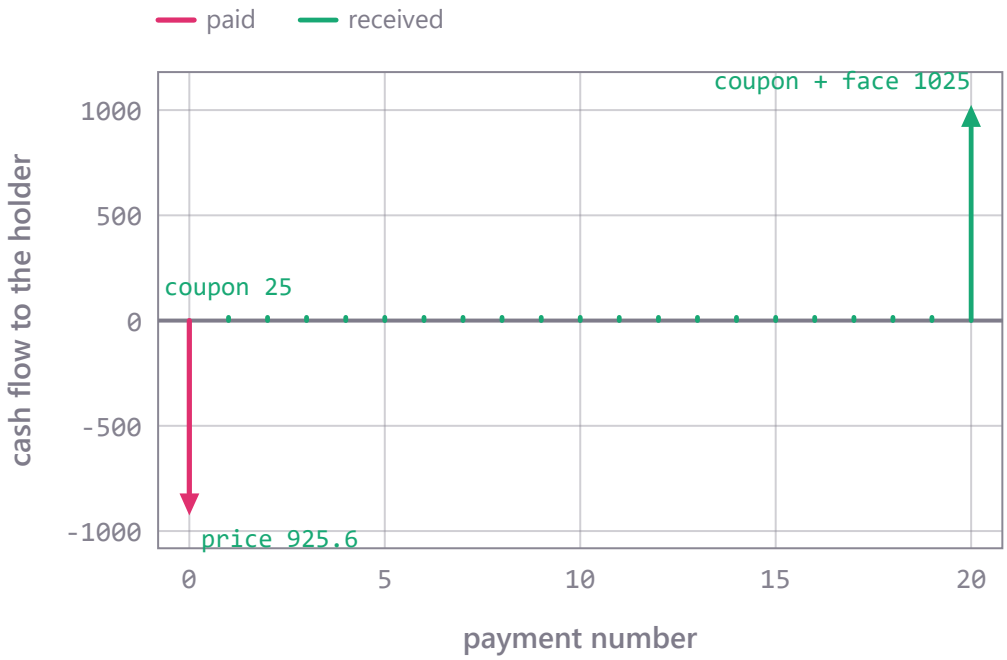
Price of a fixed-coupon bond on a coupon date from its yield to maturity, with the current yield.

$$P = \sum_{k=1}^{N} \frac{C}{(1+y/m)^k} + \frac{F}{(1+y/m)^N}$$

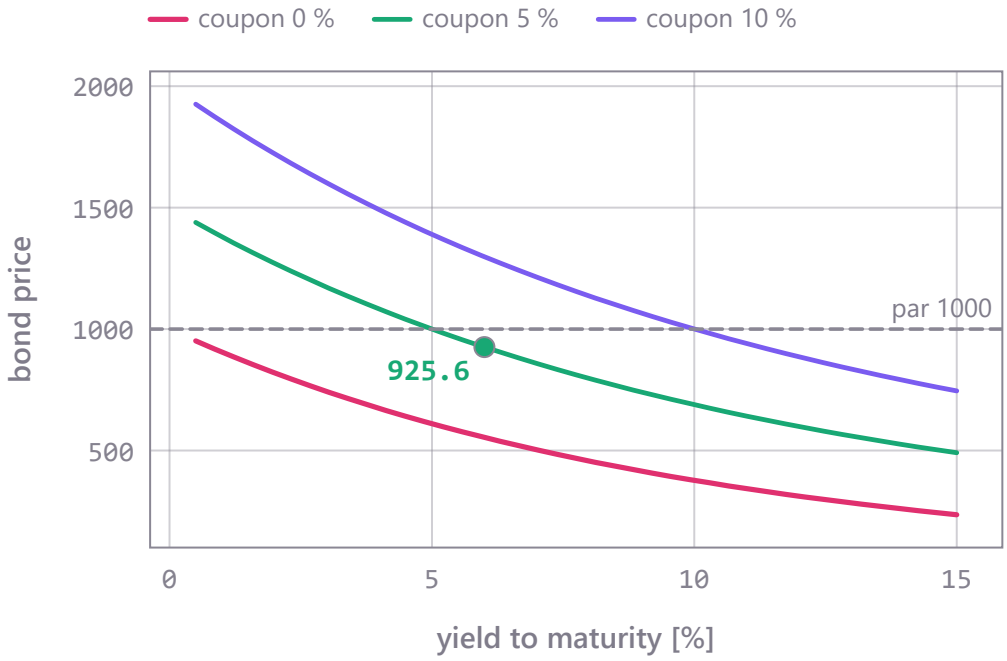
input	meaning	condition
`face`	face (par) value repaid at maturity	
`coupon_rate`	annual coupon rate (decimal)	
`years`	years to maturity (a whole number of coupon periods)	
`ytm`	yield to maturity, nominal annual, compounded at the coupon frequency	
`frequency`	coupons per year (2 = semiannual, default)	

output	meaning
`price`	bond price
`current_yield`	annual coupon / price
`coupon`	coupon per period

- **Method:** $P = C\,(1-(1+y/m)^{-N})/(y/m) + F\,(1+y/m)^{-N}$, $C = F$ c / m, N = years x m
- **Assumes:** face > 0; 0 <= coupon_rate < 1; -1 < ytm < 1; years x frequency a whole number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The cash flows of a 10-year bond with face 1000 and a 5 % coupon paid 2 time(s) a year, bought at the price the package returns for a yield of 6 %: 925.613 paid, then 20 coupons of 25 (returned) and the face at maturity. Current yield 5.40183 %.



Prices of 10-year bonds (face 1000, 2 payment(s) a year) with coupons 0, 5 % and 10 % at yields from 0.5 % to 15 % (41 runs of the package each): a bond trades at par where its yield equals its coupon, and the zero-coupon bond is the most sensitive. Marked: the worked example, yield 6 %, price 925.613.

```
economics_finance_ai999x.solve('bond_price', face=1000, coupon_rate=0.05, years=10, ytm=0.06,
frequency=2)
# -> price = 925.613, current_yield = 0.0540183, coupon = 25   (SOLVED)
```

```
economics_finance_ai999x.solve('bond_price', face=1000, coupon_rate=0.06, years=5, ytm=0.06,
frequency=1)
# -> price = 1000, current_yield = 0.06, coupon = 60   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the bond price with face = 1000, coupon rate = 5%, years = 10, ytm = 6%, frequency = 2.	SOLVED	price = 925.613, current_yield = 0.0540183, coupon = 25
What is the price of a bond with par value = 1000, coupon rate = 6%, maturity = 5, yield = 6%, frequency = 1?	SOLVED	price = 1000, current_yield = 0.06, coupon = 60
Find the bond price with face = 1000, coupon rate = 5%, years = 10.25, ytm = 6%, frequency = 2.	INVALID_INPUT	Bond price from its yield: years times the number of payments per year must be a whole number of periods (at least 1).
Find the bond price with face = 1000, coupon rate = 5%, years = 10.	NEEDS_INPUT	bond_price needs ytm (yield to maturity, nominal annual, compounded at the coupon frequency). Example: solve('bond_price

Yield to maturity of a bond — `bond_ytm`

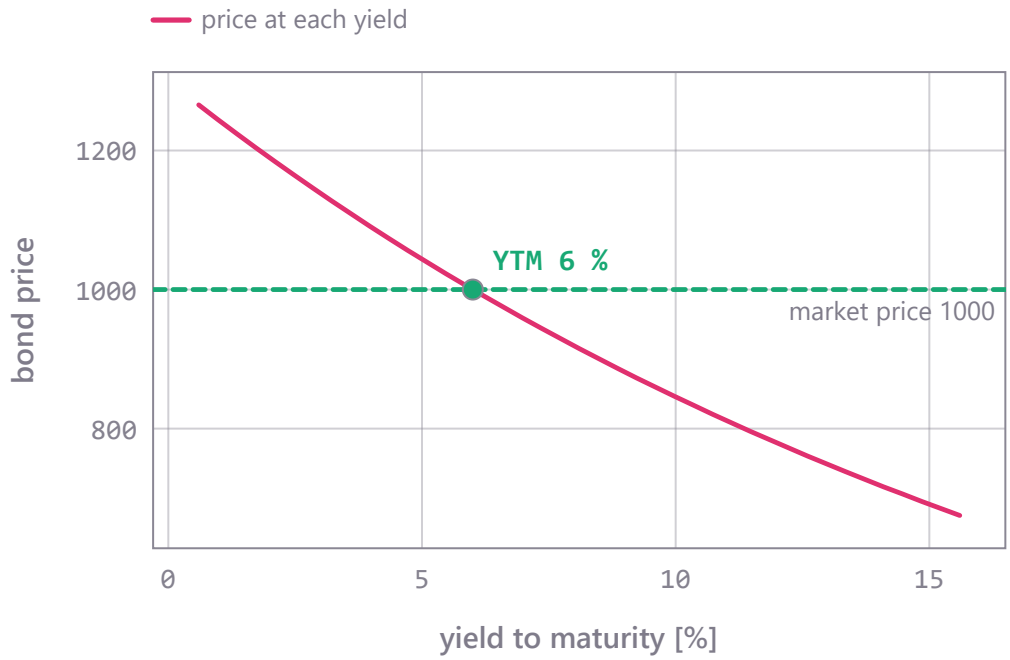
The yield to maturity (nominal annual, compounded at the coupon frequency) that equates the bond's price to the present value of its coupons and face value.

$$\text{price} = \sum_{k=1}^{N} \frac{C}{(1+y/m)^k} + \frac{F}{(1+y/m)^N}$$

input	meaning	condition
`price`	bond price (clean = dirty on a coupon date)	
`face`	face (par) value repaid at maturity	
`coupon_rate`	annual coupon rate (decimal)	
`years`	years to maturity (a whole number of coupon periods)	
`frequency`	coupons per year (2 = semiannual, default)	

output	meaning
`ytm`	yield to maturity, nominal annual
`current_yield`	annual coupon / price

- **Method:** bisection on P(y) = price (P falls as y rises, so the yield is unique)
- **Assumes:** price > 0; face > 0; 0 <= coupon_rate < 1; years x frequency a whole number; a yield between -50% and 100% per period
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The price of a 5-year bond (face 1000, coupon 6 %) at 41 yields, each a run of the package's bond price; the yield to maturity is where that curve meets the stated market price 1000. Marked: the yield the package returns, 6 % (current yield 6 %).

```
economics_finance_ai999x.solve('bond_ytm', price=1000, face=1000, coupon_rate=0.06, years=5,
frequency=2)
# -> ytm = 0.06, current_yield = 0.06   (SOLVED)
```

```
economics_finance_ai999x.solve('bond_ytm', price=925.61, face=1000, coupon_rate=0.05, years=10,
frequency=2)
# -> ytm = 0.0600004, current_yield = 0.0540184   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the yield to maturity with price = 925.61, face = 1000, coupon rate = 5%, years = 10, frequency = 2.	SOLVED	ytm = 0.0600004, current_yield = 0.0540184
What is the YTM with price = 1000, par = 1000, coupon = 6%, maturity = 5?	SOLVED	ytm = 0.06, current_yield = 0.06
Find the yield to maturity with price = 10, face = 1000, coupon rate = 5%, years = 10.	INVALID_INPUT	Yield to maturity of a bond: no yield between -50% and 100% per period gives this price.

Macaulay and modified duration of a bond — `bond_duration`

The Macaulay duration (PV-weighted average time to the cash flows, years) and the modified duration (the price's sensitivity -dP/dy / P) of a fixed-coupon bond.

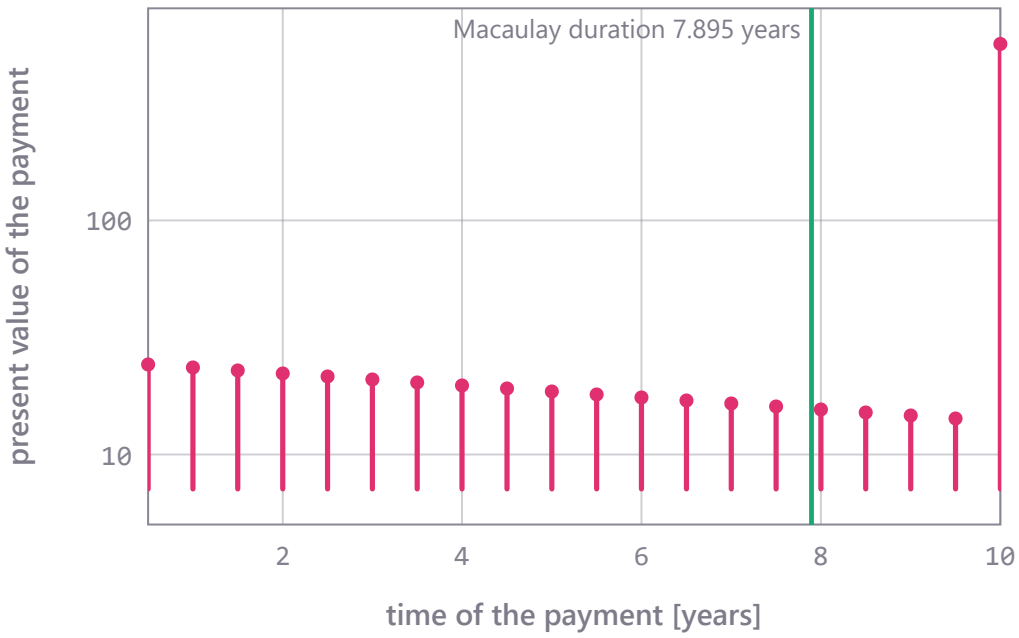
$$D_{mac} = \frac{1}{P}\sum_{k=1}^{N}\frac{k}{m}\frac{CF_k}{(1+y/m)^k}, \quad D_{mod} = \frac{D_{mac}}{1+y/m}$$

input	meaning	condition
`face`	face (par) value repaid at maturity	

input	meaning	condition
`coupon_rate`	annual coupon rate (decimal)	
`years`	years to maturity (a whole number of coupon periods)	
`ytm`	yield to maturity, nominal annual, compounded at the coupon frequency	
`frequency`	coupons per year (2 = semiannual, default)	

output	meaning
`macaulay_duration`	Macaulay duration in years
`modified_duration`	modified duration = Macaulay / (1 + y/m)
`price`	bond price

- **Method:** D_mac = sum (k/m) PV_k / P; D_mod = D_mac / (1 + y/m)
- **Assumes:** face > 0; 0 <= coupon_rate < 1; -1 < ytm < 1; years x frequency a whole number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Each payment of a 10-year bond (coupon 25 per period, face 1000) at its present value at the yield 6 % (runs of the package's present value; they sum to the returned price 925.613). The Macaulay duration the package returns, 7.895 years, is the present-value-weighted average time of the payments, the balance point of these stems (modified duration 7.66505). Logarithmic value axis.



macaulay_duration (Macaulay duration in years) against ytm (yield to maturity, nominal annual, compounded at the coupon frequency) from 0.006 to 0.6 on a logarithmic axis. One curve for each value of years (years to maturity (a whole number of coupon periods)): 10, 5, 20. Other inputs as in the worked example: face = 1000, coupon_rate = 0.05, frequency = 2. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, ytm = 0.06, macaulay_duration = 7.895.

```
economics_finance_ai999x.solve('bond_duration', face=1000, coupon_rate=0.05, years=10, ytm=0.06,
frequency=2)
# -> macaulay_duration = 7.895, modified_duration = 7.66505, price = 925.613   (SOLVED)
```

```
economics_finance_ai999x.solve('bond_duration', face=1000, coupon_rate=0.0, years=7, ytm=0.05,
frequency=1)
# -> macaulay_duration = 7, modified_duration = 6.66667, price = 710.681   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Macaulay duration with face = 1000, coupon rate = 5%, years = 10, ytm = 6%, frequency = 2.	SOLVED	macaulay_duration = 7.895, modified_duration = 7.66505, price = 925.613
Find the modified duration with face = 1000, coupon rate = 0%, years = 7, yield = 5%, frequency = 1.	SOLVED	macaulay_duration = 7, modified_duration = 6.66667, price = 710.681

Convexity of a bond — `bond_convexity`

The convexity $(1/P)\, d^2P/dy^2$ of a fixed-coupon bond, in years squared.

$$C = \frac{1}{P\,m^2} \sum_{k=1}^{N} \frac{k(k+1)\,CF_k}{(1+y/m)^{k+2}}$$

input	meaning	condition
`face`	face (par) value repaid at maturity	
`coupon_rate`	annual coupon rate (decimal)	

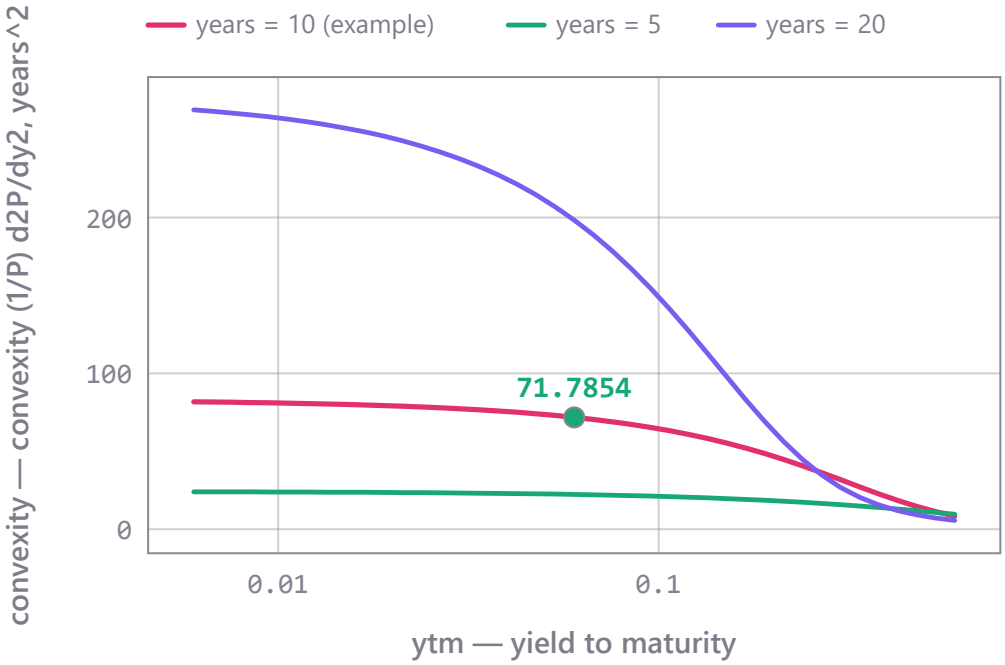
input	meaning	condition
`years`	years to maturity (a whole number of coupon periods)	
`ytm`	yield to maturity, nominal annual, compounded at the coupon frequency	
`frequency`	coupons per year (2 = semiannual, default)	

output	meaning
`convexity`	convexity $(1/P)\ d2P/dy2$, years^2
`price`	bond price

- **Method:** C = sum CF_k k $(k+1)\ (1+y/m)$^-(k+2) / (m^2 P)
- **Assumes:** face > 0; 0 <= coupon_rate < 1; -1 < ytm < 1; years x frequency a whole number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Convexity of bonds yielding 6 % against maturity, for coupons 0, 5 % and 10 % (30 runs of the package each): longer and lower-coupon bonds curve more. Marked: the worked example, 10 years, convexity 71.7854, price 925.613.



convexity (convexity (1/P) d2P/dy2, years^2) against ytm (yield to maturity, nominal annual, compounded at the coupon frequency) from 0.006 to 0.6 on a logarithmic axis. One curve for each value of years (years to maturity (a whole number of coupon periods)): 10, 5, 20. Other inputs as in the worked example: face = 1000, coupon_rate = 0.05, frequency = 2. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, ytm = 0.06, convexity = 71.7854.

```
economics_finance_ai999x.solve('bond_convexity', face=1000, coupon_rate=0.05, years=10, ytm=0.06, frequency=2)
# -> convexity = 71.7854, price = 925.613   (SOLVED)
```

```
economics_finance_ai999x.solve('bond_convexity', face=100, coupon_rate=0.0, years=5, ytm=0.05, frequency=1)
# -> convexity = 27.2109, price = 78.3526   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the convexity with face = 1000, coupon rate = 5%, years = 10, ytm = 6%, frequency = 2.	SOLVED	convexity = 71.7854, price = 925.613

Price of a zero-coupon bond — `zero_coupon_bond`

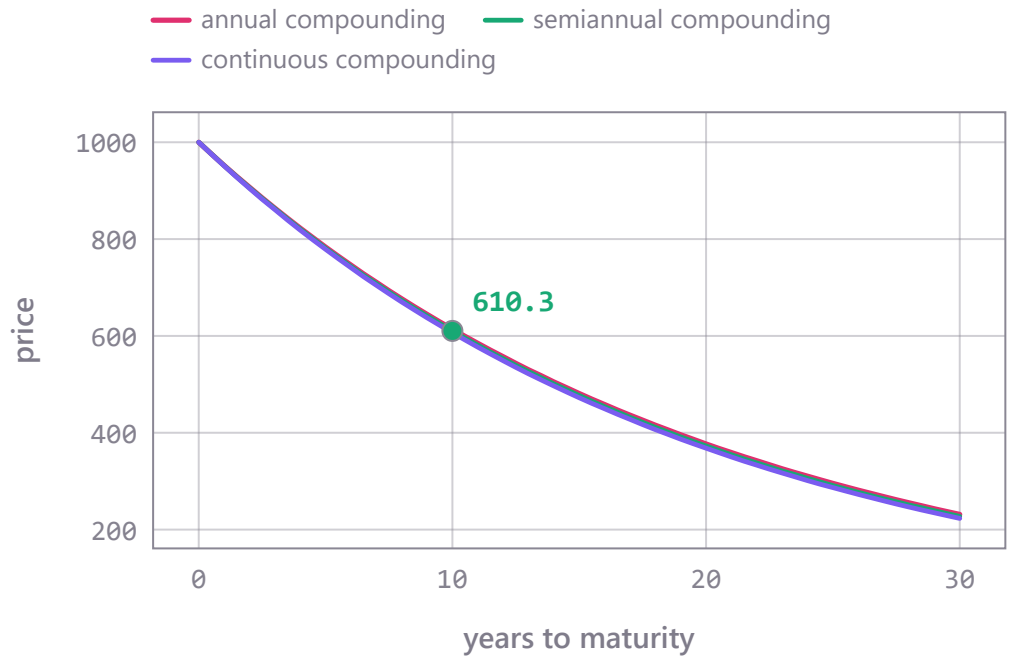
Price of a bond that pays only its face value at maturity, at a yield with a stated compounding.

$$P = \frac{F}{(1 + y/m)^{mt}}$$

input	meaning	condition
`face`	face value paid at maturity	
`ytm`	yield (decimal), with the stated compounding	
`years`	years to maturity	
`compounding`	annual (default), semiannual, quarterly, monthly or continuous	

output	meaning
`price`	price F x discount factor
`discount_factor`	(1 + y/m)^(-m t) or e^(-y t)

- **Method:** $P = F$ (1 + y/m)^(-m t), or F e^(-y t) when continuous
- **Assumes:** face > 0; -1 < ytm < 1; years >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Price of a zero-coupon bond with face 1000 at a yield of 5 % against its maturity, with the yield compounded annual, semiannual, continuous (61 runs of the package each). Marked: the worked example, 10 years, price 610.271 (discount factor 0.610271).

```
economics_finance_ai999x.solve('zero_coupon_bond', face=1000, ytm=0.05, years=10,
compounding='semiannual')
# -> price = 610.271, discount_factor = 0.610271   (SOLVED)
```

```
economics_finance_ai999x.solve('zero_coupon_bond', face=1000, ytm=0.05, years=2)
# -> price = 907.029, discount_factor = 0.907029   (SOLVED)
```

```
economics_finance_ai999x.solve('zero_coupon_bond', face=1000, ytm=0.05, years=10,
compounding='continuous')
# -> price = 606.531, discount_factor = 0.606531   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the price of a zero-coupon bond with face = 1000, ytm = 5%, years = 10, compounding = semiannual.	SOLVED	price = 610.271, discount_factor = 0.610271
Find the price of a zero coupon bond with face = 1000, yield = 5%, years = 10, compounding = continuously.	SOLVED	price = 606.531, discount_factor = 0.606531

question	outcome	result
Find the price of a zero coupon bond with face = 1000, yield = 5%, years = 10, compounding = weekly.	INVALID_INPUT	compounding: one of annual, semiannual, quarterly, monthly, continuous; got 'weekly'.

Bond price change from duration and convexity — `bond_price_change`

The price change of a bond for a change in its yield: duration estimate, duration-plus-convexity estimate and the exact change by repricing.

$$\Delta P \approx -D_{mod}\, P\, \Delta y + \tfrac{1}{2}\, C\, P\, (\Delta y)^2$$

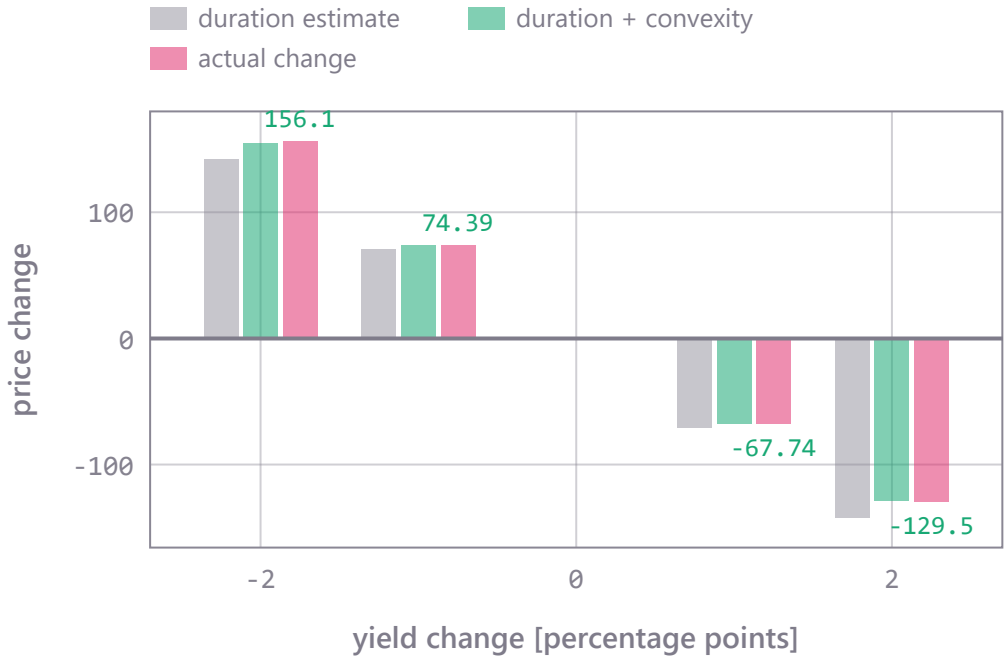
input	meaning	condition
`face`	face (par) value repaid at maturity	
`coupon_rate`	annual coupon rate (decimal)	
`years`	years to maturity (a whole number of coupon periods)	
`ytm`	current yield to maturity, nominal annual	
`yield_change`	change in the yield (decimal; +1% = 0.01, 25 basis points = 0.0025)	
`frequency`	coupons per year (2 = semiannual, default)	

output	meaning
`duration_estimate`	-D_mod P dy
`duration_convexity_estimate`	-D_mod P dy + C P $dy^2/2$
`actual_change`	exact change by repricing
`new_price`	price at the new yield
`price`	price now

- **Method:** dP ~ -D_mod P $dy + (1/2)$ C P dy^2; exact change by repricing at y + dy
- **Assumes:** face > 0; 0 <= coupon_rate < 1; the old and new yields between -100% and 100%; |yield_change| < 50%
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Price of a 10-year bond (face 1000, coupon 5 %, 2 payments a year) against its yield, with the duration (tangent) and duration-plus-convexity approximations, all computed by the package for yield changes of ±4 points. Modified duration 7.66505; marked: yield = 6 %, price = 925.613.



Price change of a 10-year bond (face 1000, coupon 5 %, yield 6 %) for yield moves of −2, −1, +1 and +2 points: the duration estimate, the duration-plus-convexity estimate and the actual change, all returned by the package. Duration alone overstates a fall and understates a rise; convexity closes most of the gap.

```
economics_finance_ai999x.solve('bond_price_change', face=1000, coupon_rate=0.05, years=10,
ytm=0.06, yield_change=0.01, frequency=2)
# -> duration_estimate = -70.9486, duration_convexity_estimate = -67.6264, actual_change =
-67.7367, new_price = 857.876, price = 925.613   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the bond price change with face = 1000, coupon rate = 5%, years = 10, ytm = 6%, yield change = 1%, frequency = 2.	SOLVED	duration_estimate = -70.9486, duration_convexity_estimate = -67.6264, actual_change = -67.7367, new_price = 857.876, price = 925.613

Implied forward rate from spot rates — `forward_rate`

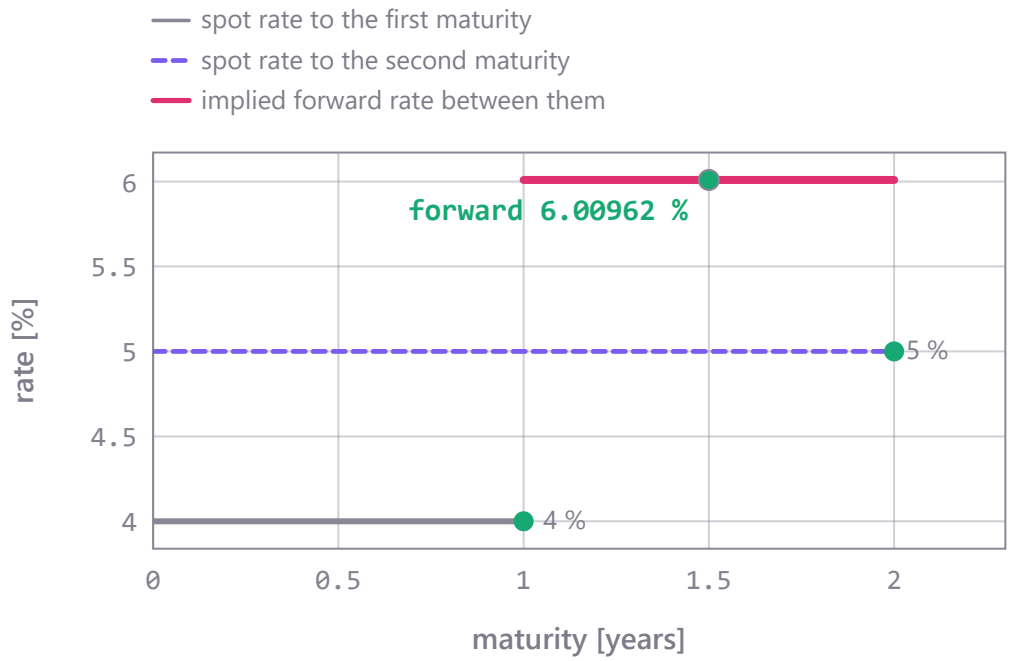
The annually compounded forward rate between t1 and t2 implied by the spot rates for t1 and t2.

$$f_{t_1,t_2} = \left[\frac{(1+s_2)^{t_2}}{(1+s_1)^{t_1}}\right]^{1/(t_2-t_1)} - 1$$

input	meaning	condition
`spot_rate_1`	spot rate to maturity t1 (annual compounding)	
`maturity_1`	first maturity t1 in years	
`spot_rate_2`	spot rate to maturity t2 (annual compounding)	
`maturity_2`	second maturity $t_2 > t_1$ in years	

output	meaning
`forward_rate`	forward rate f(t1, t2), annual compounding

- **Method:** $(1+s_2)^{t_2} = (1+s_1)^{t_1}\,(1+f)^{t_2-t_1}$
- **Assumes:** -1 < spot rates < 1; $0 \le t_1 < t_2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Spot rates 4 % for 1 year(s) and 5 % for 2 years (stated): investing at the first and rolling over at the forward rate must equal investing at the second, which gives the forward rate the package returns for the period from 1 to 2 years, 6.00962 %.

```
economics_finance_ai999x.solve('forward_rate', spot_rate_1=0.04, maturity_1=1, spot_rate_2=0.05,
maturity_2=2)
# -> forward_rate = 0.0600962   (SOLVED)
```

```
economics_finance_ai999x.solve('forward_rate', spot_rate_1=0.05, maturity_1=2, spot_rate_2=0.05,
maturity_2=5)
# -> forward_rate = 0.05   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the forward rate with s1 = 4%, t1 = 1, s2 = 5%, t2 = 2.	SOLVED	forward_rate = 0.0600962
Find the forward rate with s1 = 4%, t1 = 2, s2 = 5%, t2 = 1.	INVALID_INPUT	Implied forward rate from spot rates: maturity_2 must be later than maturity_1.

Dividend discount model (Gordon growth) — `gordon_growth`

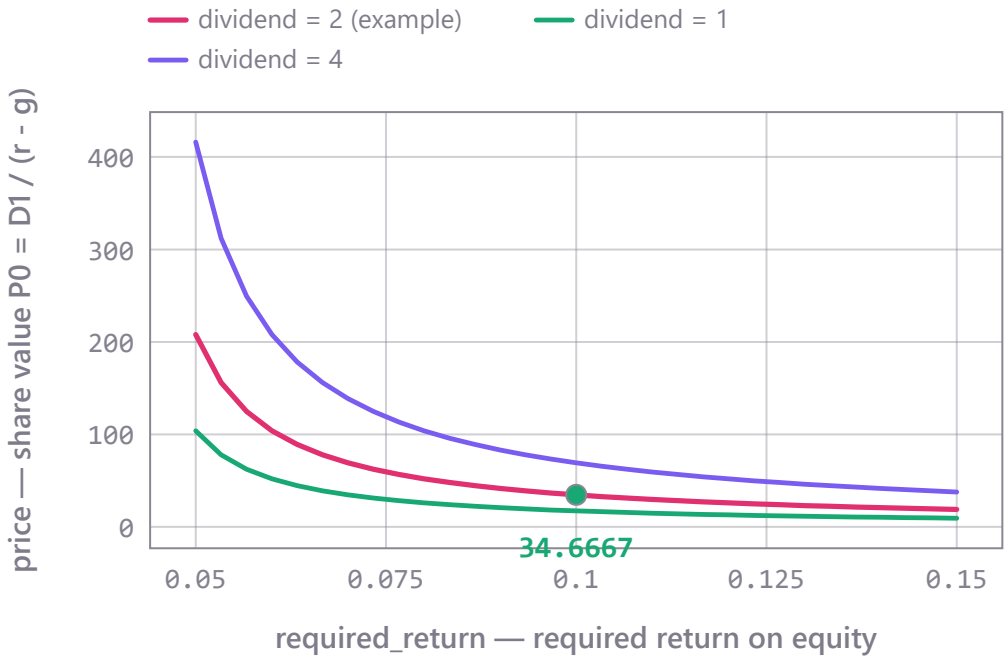
Value of a share whose dividend grows at a constant rate forever: $P_0 = D_1/(r-g)$.

$$P_0 = \frac{D_1}{r-g} = \frac{D_0(1+g)}{r-g}$$

input	meaning	condition
`required_return`	required return on equity (decimal)	
`growth`	constant dividend growth rate (decimal)	
`dividend`	dividend just paid D0 (give this OR next_dividend)	
`next_dividend`	dividend expected one year from now D1 (give this OR dividend)	

output	meaning
`price`	share value $P_0 = D_1/(r-g)$
`next_dividend`	$D_1 = D_0\ (1+g)$
`dividend_yield`	$D_1/P_0 = r-g$

- **Method:** $P_0 = D_0\ (1+g)/(r-g)$
- **Assumes:** 0 < required_return < 1; growth < required_return; exactly one of dividend (D0), next_dividend (D1)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



price (share value P0 = D1 / (r - g)) against required_return (required return on equity (decimal)) from 0.05 to 0.15. One curve for each value of dividend (dividend just paid D0 (give this OR next_dividend)): 2, 1, 4. Other inputs as in the worked example: growth = 0.04. 31 points per curve computed by economics-finance-ai999x; marked: the worked example, required_return = 0.1, price = 34.6667.

```
economics_finance_ai999x.solve('gordon_growth', dividend=2, required_return=0.1, growth=0.04)
# -> price = 34.6667, next_dividend = 2.08, dividend_yield = 0.06   (SOLVED)
```

```
economics_finance_ai999x.solve('gordon_growth', next_dividend=3, required_return=0.12, growth=0.02)
# -> price = 30, next_dividend = 3, dividend_yield = 0.1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the share price with the Gordon growth model with dividend = 2, required return = 10%, growth = 4%.	SOLVED	price = 34.6667, next_dividend = 2.08, dividend_yield = 0.06
Use the dividend discount model with next dividend = 3, required return = 12%, growth = 2%.	SOLVED	price = 30, next_dividend = 3, dividend_yield = 0.1
Use the dividend discount model with next dividend = 3, required return = 2%, growth = 12%.	INVALID_INPUT	Dividend discount model (Gordon growth): the required return must exceed the growth rate (otherwise the value is infinit
Use the dividend discount model with dividend = 2, next dividend = 3, required return = 12%, growth = 2%.	INVALID_INPUT	Dividend discount model (Gordon growth): give either the dividend just paid (D0) or the next dividend (D1), not both.

Two-stage dividend discount model — `two_stage_ddm`

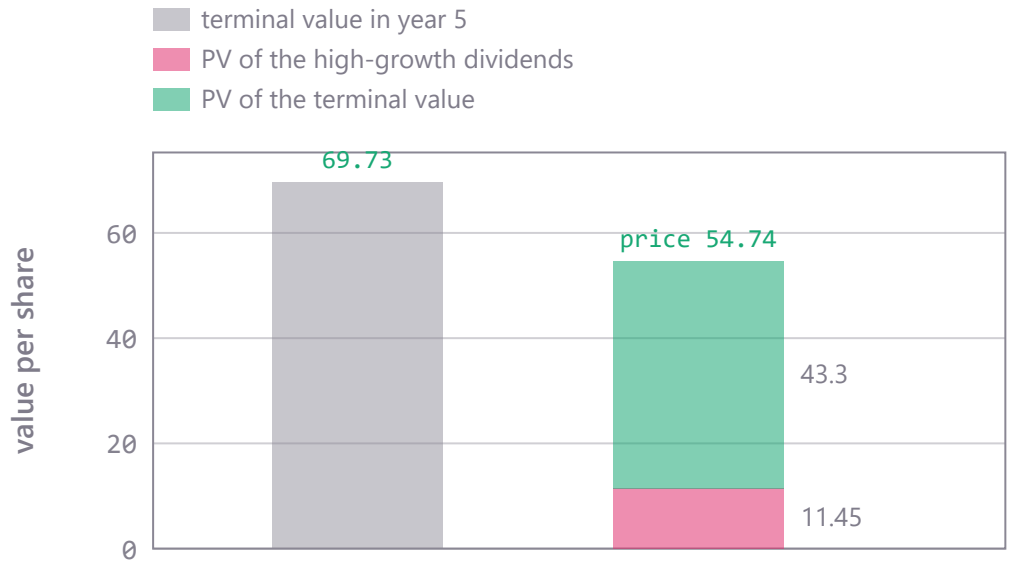
Share value when dividends grow at a high rate for n years and at a stable rate forever after.

$$P_0 = \sum_{k=1}^{n} \frac{D_0(1+g_1)^k}{(1+r)^k} + \frac{D_n(1+g_2)}{(r-g_2)(1+r)^n}$$

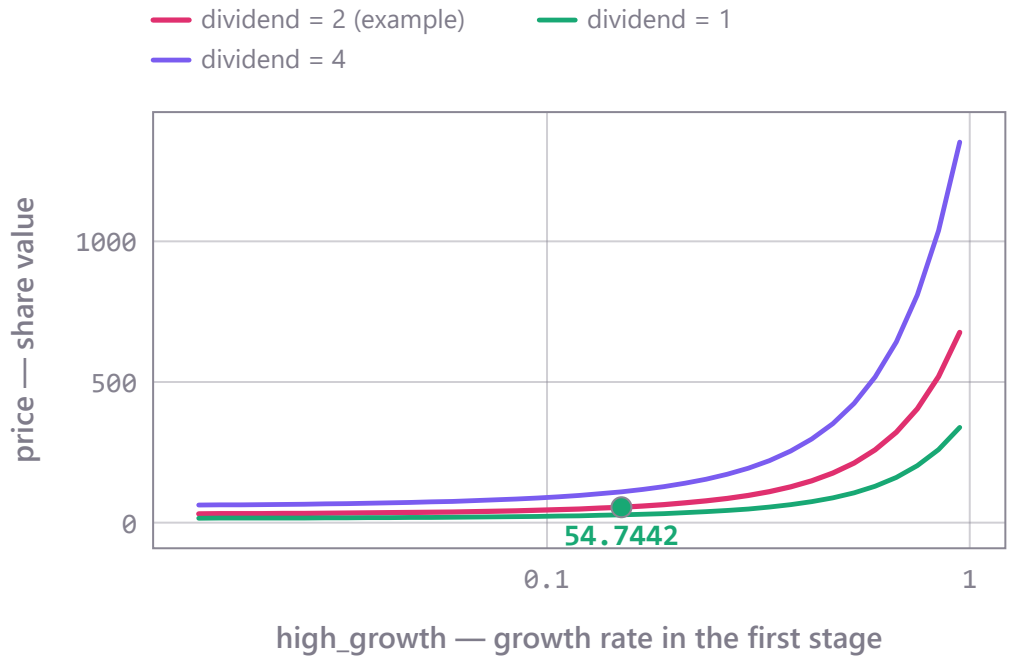
input	meaning	condition
`dividend`	dividend just paid D0	
`high_growth`	growth rate in the first stage (decimal)	
`high_growth_years`	number of years of high growth	
`stable_growth`	growth rate forever after (decimal)	
`required_return`	required return on equity (decimal)	

output	meaning
`price`	share value
`terminal_value`	value at year n of the dividends after n
`pv_terminal`	present value of the terminal value
`pv_high_growth_dividends`	present value of D1..Dn

- **Method:** P0 = sum_{$k = 1$}^{n} D0 $(1+g_1)^k/(1+r)^k$ + [D_n $(1+g_2)/(r-g_2)$] / $(1+r)^n$
- **Assumes:** dividend > 0; stable_growth < required_return; high_growth_years a whole number >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A share paying 2 now, growing 15 % for 5 years and 4 % after, at a required return of 10 %: the package returns the terminal value in year 5, 69.727, its present value 43.295, the present value of the high-growth dividends 11.4492 and their sum, the price 54.7442.



price (share value) against high_growth (growth rate in the first stage (decimal)) from 0.015 to 1.5 on a logarithmic axis. One curve for each value of dividend (dividend just paid D0): 2, 1, 4. Other inputs as in the worked example: high_growth_years = 5, stable_growth = 0.04, required_return = 0.1. 37 points per curve computed by economics-finance-ai999x; marked: the worked example, high_growth = 0.15, price = 54.7442.

```
economics_finance_ai999x.solve('two_stage_ddm', dividend=2, high_growth=0.15, high_growth_years=5,
stable_growth=0.04, required_return=0.1)
# -> price = 54.7442, terminal_value = 69.727, pv_terminal = 43.295, pv_high_growth_dividends =
11.4492   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Value the share with the two-stage dividend discount model with dividend = 2, high growth = 15%, high growth years = 5, stable growth = 4%, required return = 10%.	SOLVED	price = 54.7442, terminal_value = 69.727, pv_terminal = 43.295, pv_high_growth_dividends = 11.4492
Value the share with the two-stage DDM with dividend = 2, g1 = 15%, n = 5, g2 = 12%, r = 10%.	INVALID_INPUT	Two-stage dividend discount model: the required return must exceed the stable growth rate.

Required return implied by the Gordon model — `gordon_implied_return`

The return investors require given a share price and a constant dividend growth rate: $r = D_1/P_0 + g$.

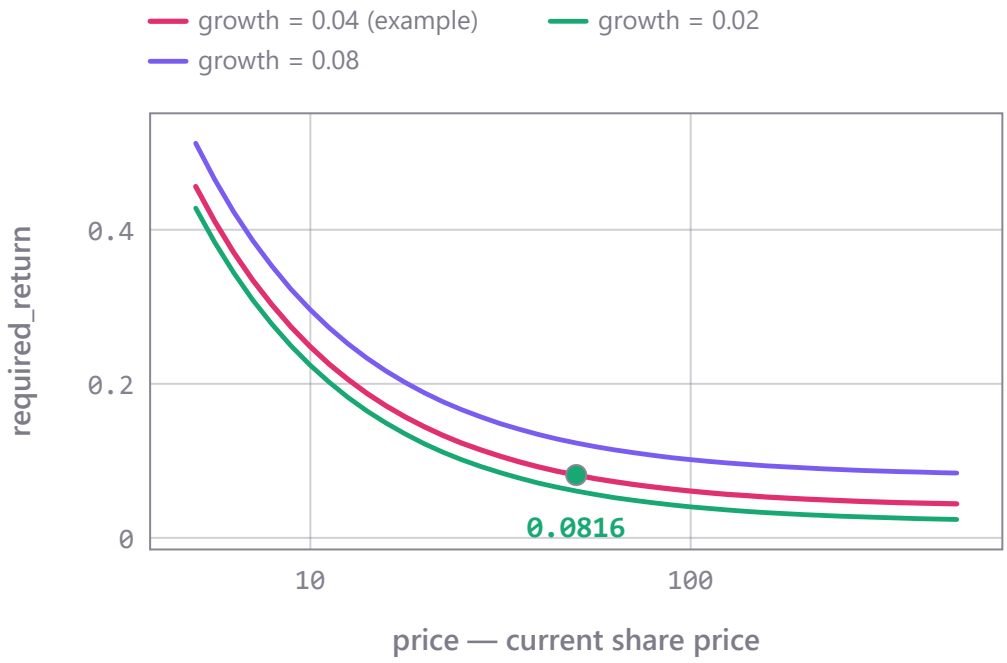
$$r = \frac{D_1}{P_0} + g$$

input	meaning	condition
`price`	current share price	
`growth`	constant dividend growth rate (decimal)	
`dividend`	dividend just paid D0 (give this OR next_dividend)	

input	meaning	condition
`next_dividend`	dividend expected one year from now D1 (give this OR dividend)	

output	meaning
`required_return`	implied required return $D_1/P_0 + g$
`dividend_yield`	D_1/P_0
`next_dividend`	D1

- **Method:** $r = D_1/P_0 + g$
- **Assumes:** price > 0; exactly one of dividend (D0), next_dividend (D1); dividend > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



required_return (implied required return D1/P0 + g) against price (current share price) from 5 to 500 on a logarithmic axis. One curve for each value of growth (constant dividend growth rate (decimal)): 0.04, 0.02, 0.08. Other inputs as in the worked example: dividend = 2. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, price = 50, required_return = 0.0816.

```
economics_finance_ai999x.solve('gordon_implied_return', price=50, dividend=2, growth=0.04)
# -> required_return = 0.0816, dividend_yield = 0.0416, next_dividend = 2.08   (SOLVED)
```

```
economics_finance_ai999x.solve('gordon_implied_return', price=40, next_dividend=2, growth=0.05)
# -> required_return = 0.1, dividend_yield = 0.05, next_dividend = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the implied required return with price = 50, dividend = 2, growth = 4%.	SOLVED	required_return = 0.0816, dividend_yield = 0.0416, next_dividend = 2.08
Find the implied cost of equity with share price = 40, next dividend = 2, growth = 5%.	SOLVED	required_return = 0.1, dividend_yield = 0.05, next_dividend = 2

Capital asset pricing model — `capm`

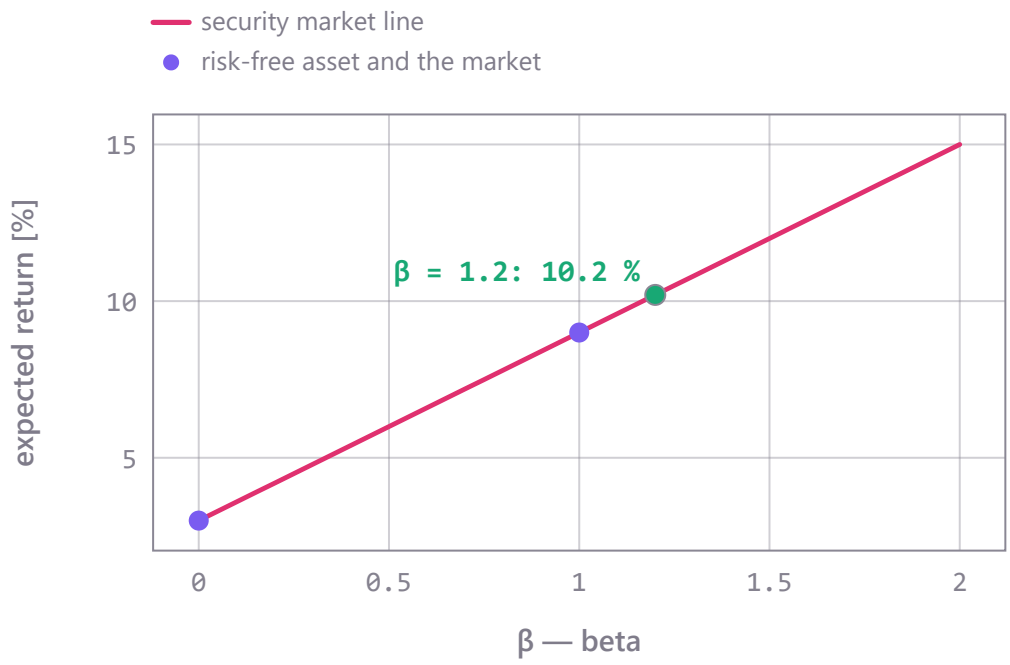
Expected return of an asset from its beta: E[R] = rf + beta (E[Rm] - rf) (the security market line).

$$E[R_i] = r_f + \beta_i\,(E[R_m] - r_f)$$

input	meaning	condition
`risk_free`	risk-free rate (decimal)	
`beta`	beta of the asset	
`market_return`	expected market return (decimal)	

output	meaning
`expected_return`	expected (required) return
`market_risk_premium`	rm - rf
`risk_premium`	beta (rm - rf)

- **Method:** E[R] = rf + beta (rm - rf)
- **Assumes:** -1 < risk_free < 1; -1 < market_return < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The security market line for a risk-free rate of 3 % and a market return of 9 %: the expected return the package returns at 41 betas from 0 to 2. Marked: β = 1.2, expected return 10.2 % (risk premium 7.2 %, market premium 6 %).

```
economics_finance_ai999x.solve('capm', risk_free=0.03, beta=1.2, market_return=0.09)
# -> expected_return = 0.102, market_risk_premium = 0.06, risk_premium = 0.072   (SOLVED)
```

```
economics_finance_ai999x.solve('capm', risk_free=0.02, beta=0.0, market_return=0.1)
# -> expected_return = 0.02, market_risk_premium = 0.08, risk_premium = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the expected return with CAPM with risk-free rate = 3%, beta = 1.2, market return = 9%.	SOLVED	expected_return = 0.102, market_risk_premium = 0.06, risk_premium = 0.072
Use the capital asset pricing model with rf = 2%, beta = 0.8.	NEEDS_INPUT	capm needs market_return (expected market return (decimal)). Example: solve('capm', risk_free=0.03, beta=1.2, market_ret

Weighted average cost of capital — `wacc`

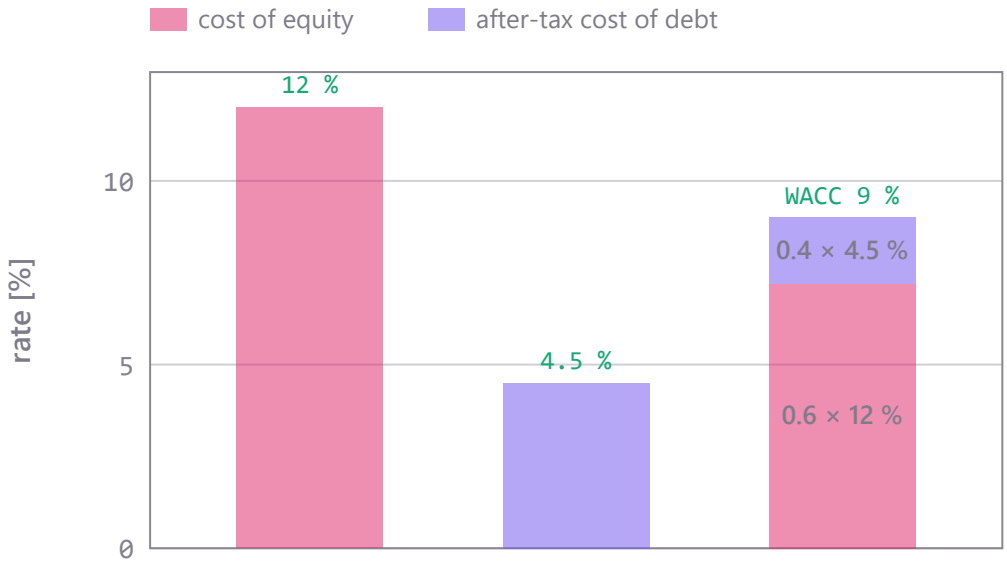
WACC = E/V re + D/V rd $(1 - t)$: the after-tax cost of a firm's capital at market-value weights.

$$\mathrm{WACC} = \frac{E}{V} r_e + \frac{D}{V} r_d (1 - t)$$

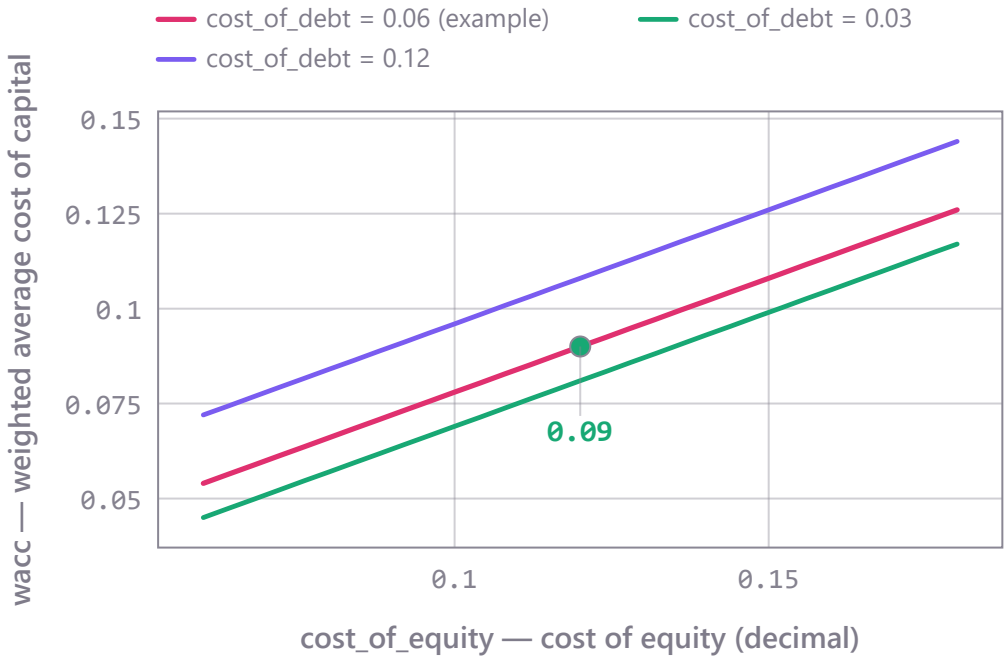
input	meaning	condition
`equity_value`	market value of equity	
`debt_value`	market value of debt	
`cost_of_equity`	cost of equity (decimal)	
`cost_of_debt`	pre-tax cost of debt (decimal)	
`tax_rate`	corporate tax rate (decimal)	

output	meaning
`wacc`	weighted average cost of capital
`after_tax_cost_of_debt`	rd $(1 - t)$
`equity_weight`	E / V
`debt_weight`	D / V

- **Method:** WACC = E/V re + D/V rd $(1 - t)$
- **Assumes:** equity_value, debt_value >= 0 and not both 0; 0 <= tax_rate < 0.99
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Equity 600 at 12 % and debt 400 at 6 % before tax (tax rate 25 %): the package returns the after-tax cost of debt 4.5 %, the weights 0.6 and 0.4 and the weighted average cost of capital 9 %, drawn as the sum of each weight × cost (right bar; the products are computed here).



wacc (weighted average cost of capital) against cost_of_equity (cost of equity (decimal)) from 0.06 to 0.18. One curve for each value of cost_of_debt (pre-tax cost of debt (decimal)): 0.06, 0.03, 0.12. Other inputs as in the worked example: equity_value = 600, debt_value = 400, tax_rate = 0.25. 31 points per curve computed by economics-finance-ai999x; marked: the worked example, cost_of_equity = 0.12, wacc = 0.09.

```
economics_finance_ai999x.solve('wacc', equity_value=600, debt_value=400, cost_of_equity=0.12,
cost_of_debt=0.06, tax_rate=0.25)
# -> wacc = 0.09, after_tax_cost_of_debt = 0.045, equity_weight = 0.6, debt_weight = 0.4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the WACC with equity value = 600, debt value = 400, cost of equity = 12%, cost of debt = 6%, tax rate = 25%.	SOLVED	wacc = 0.09, after_tax_cost_of_debt = 0.045, equity_weight = 0.6, debt_weight = 0.4
Find the WACC with equity value = 0, debt value = 0, cost of equity = 12%, cost of debt = 6%, tax rate = 25%.	INVALID_INPUT	Weighted average cost of capital: the firm needs a positive total value.

Beta of an asset from return series — `beta_regression`

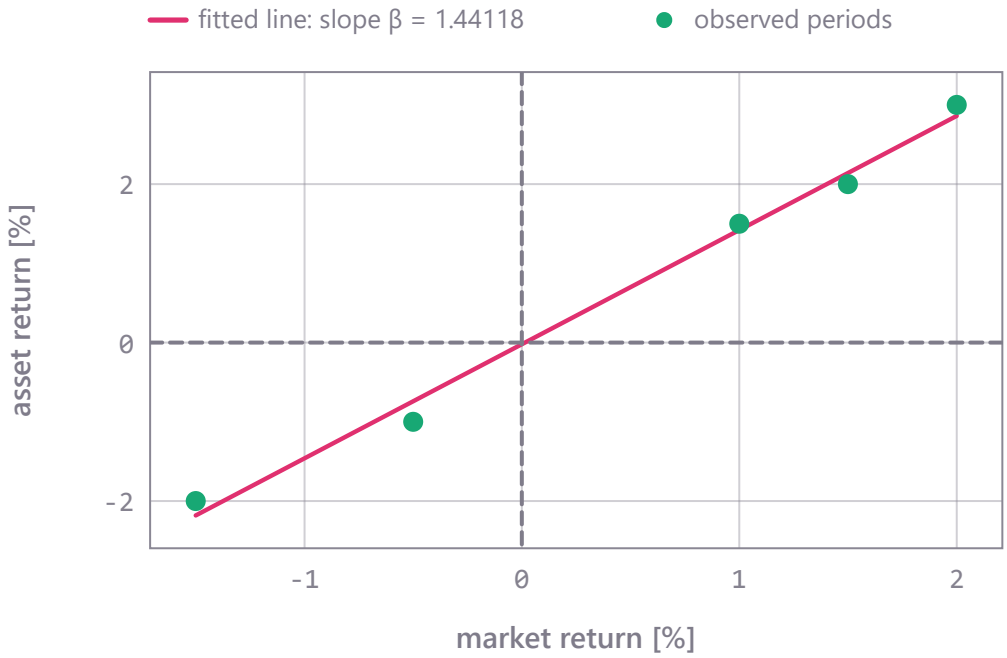
Beta (and alpha, correlation, R^2) of an asset from paired asset and market returns: the slope of the ordinary least-squares regression of asset on market returns.

$$\beta = \frac{\mathrm{Cov}(R_i, R_m)}{\mathrm{Var}(R_m)}$$

input	meaning	condition
`asset_returns`	returns of the asset (decimals), one per period	
`market_returns`	returns of the market in the same periods	

output	meaning
`beta`	cov(asset, market) / var(market)
`alpha`	intercept mean(asset) - beta mean(market)
`correlation`	correlation of asset and market
`r_squared`	share of variance explained

- **Method:** $\beta = S_xy / S_xx$ (sample covariance over sample variance)
- **Assumes:** same number of observations, at least 3; returns between -100% and 100%; neither series constant
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Asset returns 0.02, -0.01, 0.03, 0.015, -0.02 against market returns 0.015, -0.005, 0.02, 0.01, -0.015: the package returns the regression slope β = 1.44118 and intercept α = -0.000205882 (the line), correlation 0.995902, R^2 = 0.991821.

```
economics_finance_ai999x.solve('beta_regression', asset_returns='0.02, -0.01, 0.03, 0.015, -0.02',
market_returns='0.015, -0.005, 0.02, 0.01, -0.015')
# -> beta = 1.44118, alpha = -0.000205882, correlation = 0.995902, r_squared = 0.991821   (SOLVED)
```

```
economics_finance_ai999x.solve('beta_regression', asset_returns=[0.02, 0.04, 0.06], market_returns=
[0.01, 0.02, 0.03])
# -> beta = 2, alpha = 0, correlation = 1, r_squared = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Estimate the beta with asset returns = [0.02, -0.01, 0.03, 0.015, -0.02], market returns = [0.015, -0.005, 0.02, 0.01, -0.015].	SOLVED	beta = 1.44118, alpha = -0.000205882, correlation = 0.995902, r_squared = 0.991821
Estimate the beta with asset returns = [2%, 4%, 6%], market returns = [1%, 2%, 3%].	SOLVED	beta = 2, alpha = 0, correlation = 1, r_squared = 1
Estimate the beta with asset returns = [0.02, 0.04, 0.06], market returns = [0.01, 0.01, 0.01].	INVALID_INPUT	Beta of an asset from return series: the market returns must not all be equal.
Estimate the beta with asset returns = [0.02, 0.04, 0.06], market returns = [0.01, 0.02].	INVALID_INPUT	market_returns: needs at least 3 entries, got 2.

Sharpe ratio — `sharpe_ratio`

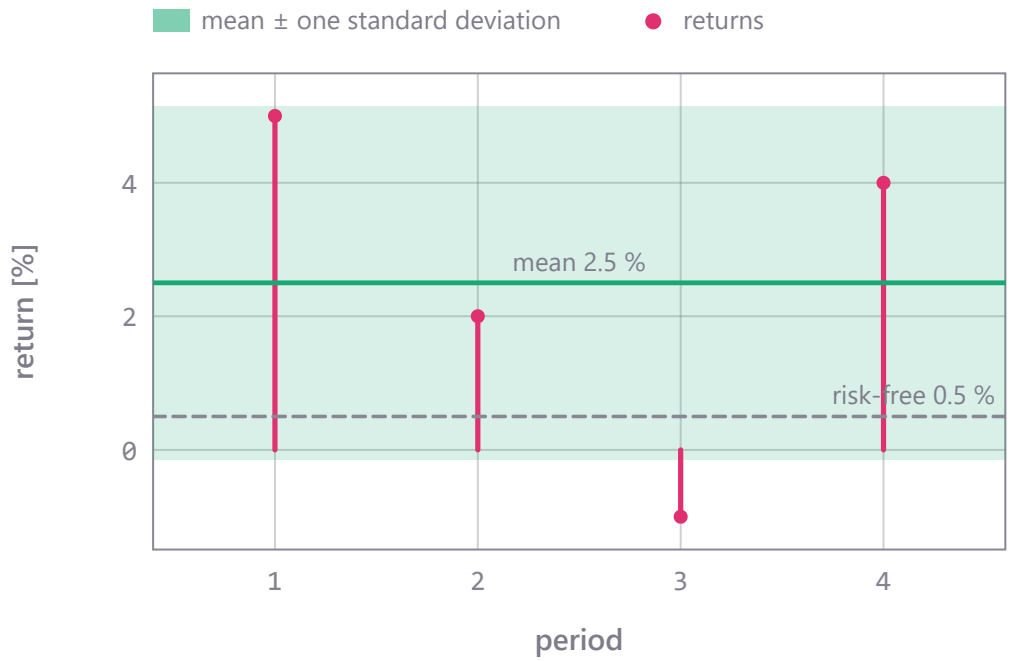
Mean excess return per unit of total risk, from a return series (sample standard deviation, n - 1); annualized by sqrt(periods per year).

$$S = \frac{\bar{R} - r_f}{s_R}$$

input	meaning	condition
`returns`	periodic returns (decimals)	
`risk_free`	risk-free rate per period (decimal, default 0)	
`periods_per_year`	periods per year for annualizing (12 monthly, 252 daily; default 1)	

output	meaning
`sharpe`	(mean - rf) / sd per period
`annualized_sharpe`	sharpe x sqrt(periods per year)
`mean_excess_return`	mean - rf
`std_dev`	sample standard deviation

- **Method:** S = (mean(R) - rf) / s(R), s with $n - 1$
- **Assumes:** at least 2 returns, not all equal; returns between -100% and 100%
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Returns 0.05, 0.02, -0.01, 0.04 against a risk-free rate of 0.5 %: the package returns the mean excess return 2 % and the standard deviation 2.64575 % (the band around the mean), whose ratio is the Sharpe ratio 0.755929 (annualised 0.755929).

```
economics_finance_ai999x.solve('sharpe_ratio', returns='0.05, 0.02, -0.01, 0.04', risk_free=0.005)
# -> sharpe = 0.755929, annualized_sharpe = 0.755929, mean_excess_return = 0.02, std_dev =
0.0264575   (SOLVED)
```

```
economics_finance_ai999x.solve('sharpe_ratio', returns=[0.01, 0.03], risk_free=0.0,
periods_per_year=12)
# -> sharpe = 1.41421, annualized_sharpe = 4.89898, mean_excess_return = 0.02, std_dev = 0.0141421
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Sharpe ratio with returns = [0.05, 0.02, -0.01, 0.04], risk-free rate = 0.5%.	SOLVED	sharpe = 0.755929, annualized_sharpe = 0.755929, mean_excess_return = 0.02, std_dev = 0.0264575
Find the Sharpe ratio with returns = [0.02, 0.02, 0.02].	INVALID_INPUT	Sharpe ratio: the returns must not all be equal (zero standard deviation).

Sortino ratio — `sortino_ratio`

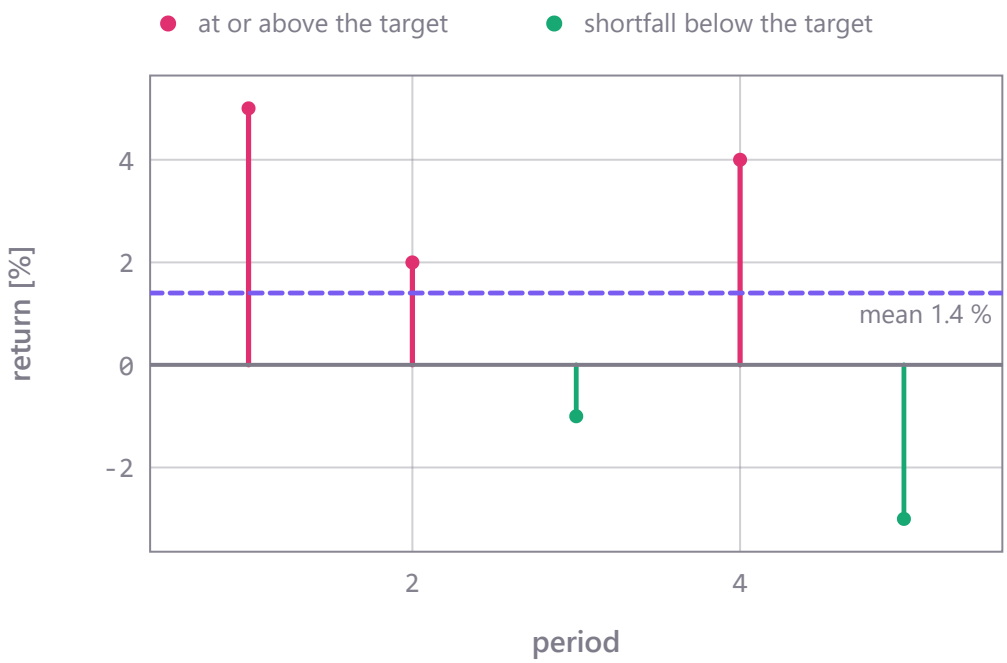
Mean return above a target per unit of downside deviation sqrt(mean(min(0, R - T)^2)) over ALL observations.

$$\text{Sortino} = \frac{\bar{R} - T}{\sqrt{\frac{1}{n}\sum_i \min(0, R_i - T)^2}}$$

input	meaning	condition
`returns`	periodic returns (decimals)	
`target`	target (minimum acceptable) return per period (default 0)	

output	meaning
`sortino`	(mean - T) / downside deviation
`downside_deviation`	sqrt(sum min(0, R - T)^2 / n)
`mean_return`	mean return

- **Method:** Sortino = (mean(R) - T) / sqrt($(1/n)$ sum min(0, R - T)^2)
- **Assumes:** at least 2 returns; at least one return below the target
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Returns 0.05, 0.02, -0.01, 0.04, -0.03 measured from the target 0 %: only the shortfalls (highlighted) count as risk; the package returns their downside deviation 1.41421 %, the mean return 1.4 % and the Sortino ratio 0.989949.

```
economics_finance_ai999x.solve('sortino_ratio', returns='0.05, 0.02, -0.01, 0.04, -0.03',
target=0.0)
# -> sortino = 0.989949, downside_deviation = 0.0141421, mean_return = 0.014   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Sortino ratio with returns = [0.05, 0.02, -0.01, 0.04, -0.03], target = 0.	SOLVED	sortino = 0.989949, downside_deviation = 0.0141421, mean_return = 0.014
Find the Sortino ratio with returns = [0.05, 0.02, 0.04].	INVALID_INPUT	Sortino ratio: no return is below the target: the downside deviation is zero.

Treynor ratio and Jensen's alpha — `treynor_jensen`

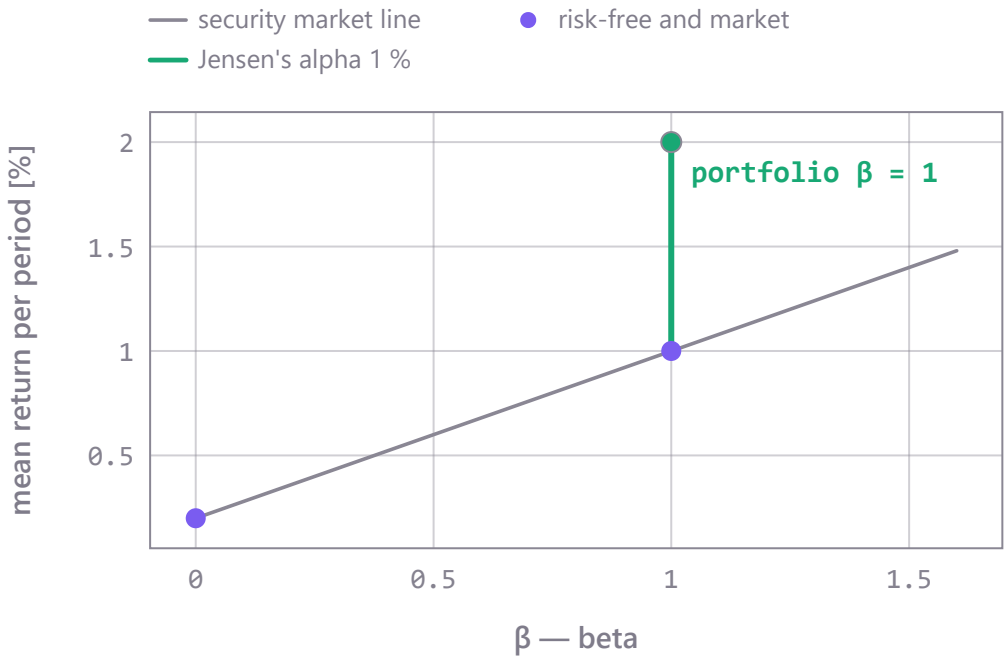
Excess return per unit of beta (Treynor) and the return above the CAPM prediction (Jensen's alpha), from paired portfolio and market returns and a constant risk-free rate per period.

$$T = \frac{\bar{R}_p - r_f}{\beta_p}, \quad \alpha_J = \bar{R}_p - [r_f + \beta_p(\bar{R}_m - r_f)]$$

input	meaning	condition
`portfolio_returns`	portfolio returns (decimals) per period	
`market_returns`	market returns in the same periods	
`risk_free`	risk-free rate per period (decimal, default 0)	

output	meaning
`beta`	beta of the portfolio
`treynor`	(mean Rp - rf) / beta
`jensen_alpha`	mean Rp - [rf + beta (mean Rm - rf)]
`mean_portfolio_return`	mean Rp
`mean_market_return`	mean Rm

- **Method:** $\beta = S_pm/S_mm$; Treynor = (mean Rp - rf)/beta; alpha = mean Rp - rf - beta (mean Rm - rf)
- **Assumes:** same number of observations, at least 3; market returns not constant; beta not 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The portfolio (returns 0.03, -0.01, 0.04, 0.02) against the market (returns 0.02, -0.02, 0.03, 0.01) with a risk-free rate of 0.2 %: the package returns beta 1, mean returns 2 % (portfolio) and 1 % (market). The line through the risk-free and market points is the security market line; the portfolio's height above it is the returned Jensen's alpha 1 %; Treynor ratio 0.018.

```
economics_finance_ai999x.solve('treynor_jensen', portfolio_returns='0.03, -0.01, 0.04, 0.02',
market_returns='0.02, -0.02, 0.03, 0.01', risk_free=0.002)
# -> beta = 1, treynor = 0.018, jensen_alpha = 0.01, mean_portfolio_return = 0.02,
mean_market_return = 0.01   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Treynor ratio and Jensen's alpha with portfolio returns = [0.03, -0.01, 0.04, 0.02], market returns = [0.02, -0.02, 0.03, 0.01], risk-free rate = 0.2%.	SOLVED	beta = 1, treynor = 0.018, jensen_alpha = 0.01, mean_portfolio_return = 0.02, mean_market_return = 0.01

Information ratio and tracking error — `information_ratio`

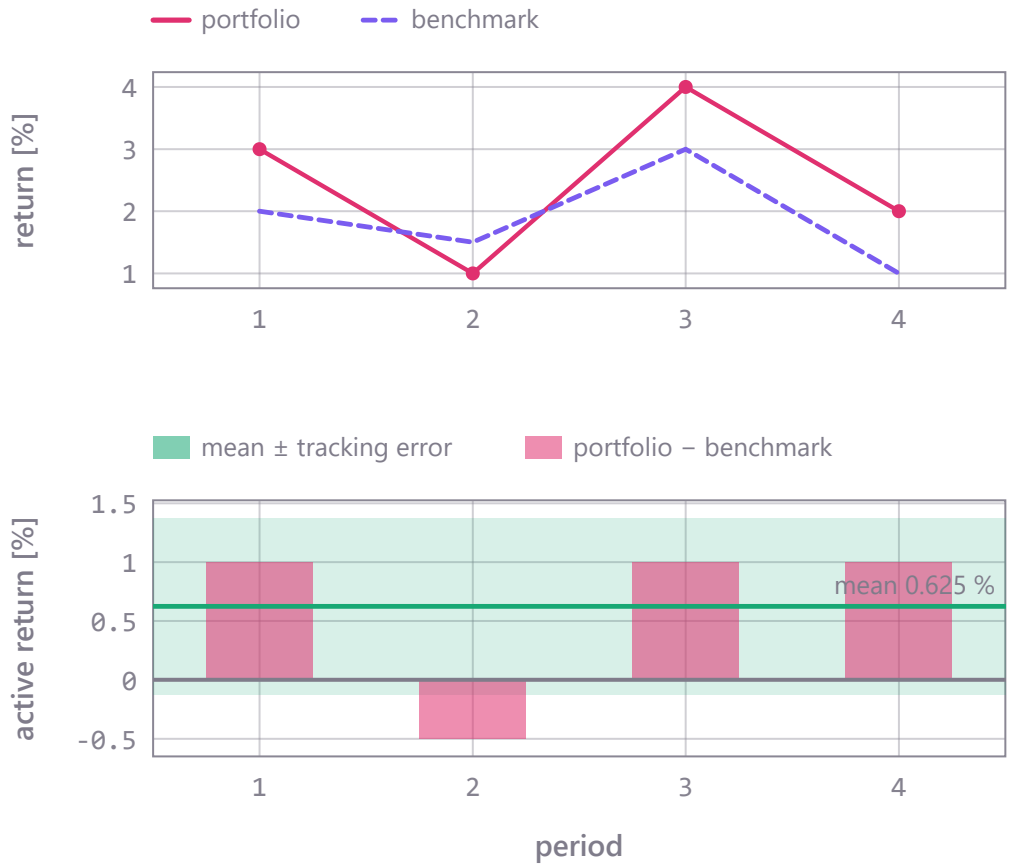
Mean active return (portfolio - benchmark) per unit of tracking error (sample sd of active returns).

$$IR = \frac{\overline{R_p - R_b}}{s(R_p - R_b)}$$

input	meaning	condition
`portfolio_returns`	portfolio returns (decimals)	
`benchmark_returns`	benchmark returns in the same periods	

output	meaning
`information_ratio`	mean active / tracking error
`tracking_error`	sample sd of active returns
`mean_active_return`	mean of portfolio - benchmark

- **Method:** IR = mean(Rp - Rb) / s(Rp - Rb)
- **Assumes:** same number of observations, at least 2; active returns not constant
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: portfolio returns 0.03, 0.01, 0.04, 0.02 and benchmark returns 0.02, 0.015, 0.03, 0.01. Bottom: their difference each period (computed here) with the mean active return 0.625 % and the tracking error 0.75 % returned by the package; information ratio = 0.833333.

```
economics_finance_ai999x.solve('information_ratio', portfolio_returns='0.03, 0.01, 0.04, 0.02',
benchmark_returns='0.02, 0.015, 0.03, 0.01')
# -> information_ratio = 0.833333, tracking_error = 0.0075, mean_active_return = 0.00625   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the information ratio with portfolio returns = [0.03, 0.01, 0.04, 0.02], benchmark returns = [0.02, 0.015, 0.03, 0.01].	SOLVED	information_ratio = 0.833333, tracking_error = 0.0075, mean_active_return = 0.00625

Portfolio expected return and risk — `portfolio_return_risk`

Expected return w'mu, variance w'Sigma w and standard deviation of a portfolio from its weights, expected returns and covariance matrix.

$$E[R_p] = w^\top \mu, \quad \sigma_p^2 = w^\top \Sigma\, w$$

input	meaning	condition
`weights`	portfolio weights, adding up to 1 (negative = short)	
`expected_returns`	expected return of each asset (decimals)	
`covariance`	covariance matrix of the returns [[s11, s12], [s21, s22]]	

output	meaning
`expected_return`	w' mu
`variance`	w' Sigma w
`std_dev`	sqrt(variance)

- **Method:** E = sum w_i mu_i; Var = sum_i sum_j w_i w_j sigma_ij
- **Assumes:** one weight, return and row per asset; weights add up to 1; covariance symmetric, positive semidefinite
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Expected return and standard deviation of portfolios of two assets (expected returns 0.1, 0.15, covariance [[0.04, 0.006], [0.006, 0.09]]) at 51 weights, each a run of the package. Marked: weights 0.6, 0.4: return 12 %, σ 17.7989 % (variance 0.03168).

```
economics_finance_ai999x.solve('portfolio_return_risk', weights='0.6, 0.4', expected_returns='0.1, 0.15', covariance='[[0.04, 0.006], [0.006, 0.09]]')
# -> expected_return = 0.12, variance = 0.03168, std_dev = 0.177989   (SOLVED)
```

```
economics_finance_ai999x.solve('portfolio_return_risk', weights=[1.0], expected_returns=[0.08], covariance=[[0.04]])
# -> expected_return = 0.08, variance = 0.04, std_dev = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the portfolio variance with weights = [0.6, 0.4], expected returns = [0.1, 0.15], covariance = [[0.04, 0.006], [0.006, 0.09]].	SOLVED	expected_return = 0.12, variance = 0.03168, std_dev = 0.177989

question	outcome	result
Find the portfolio variance with weights = [0.6, 0.6], expected returns = [0.1, 0.15], covariance = [[0.04, 0.006], [0.006, 0.09]].	INVALID_INPUT	Portfolio expected return and risk: the weights must add up to 1.
Find the portfolio variance with weights = [0.6, 0.4], expected returns = [0.1, 0.15], covariance = [[0.04, 0.006], [0.007, 0.09]].	INVALID_INPUT	Portfolio expected return and risk: covariance must be symmetric.

Two-asset portfolio return and risk — `two_asset_portfolio`

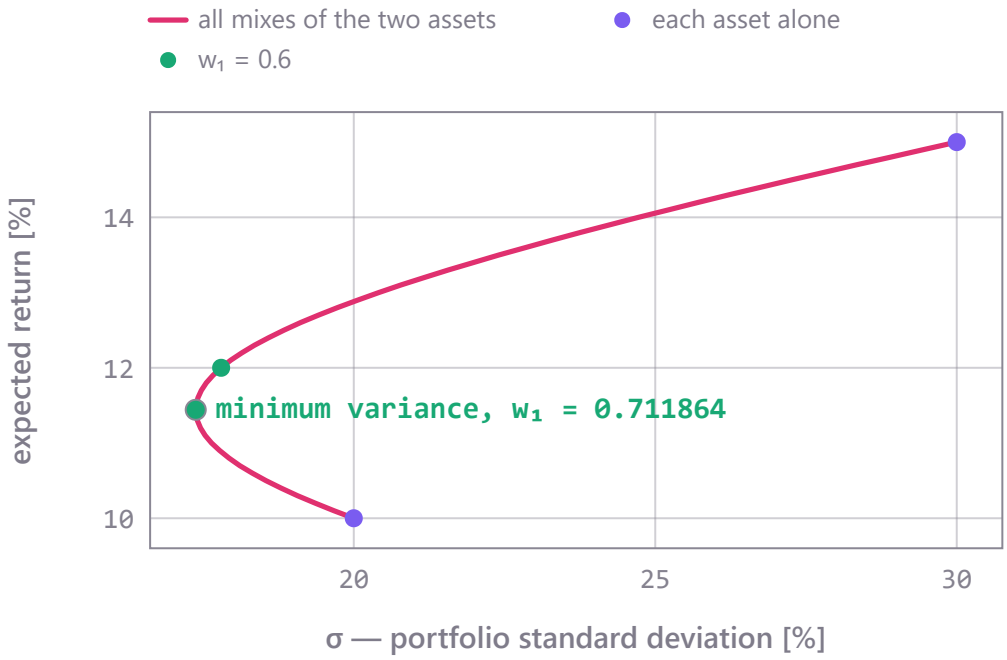
Expected return and standard deviation of a portfolio of two assets from the weight, means, standard deviations and correlation.

$$\sigma_p^2 = w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + 2w_1w_2\rho\,\sigma_1\sigma_2$$

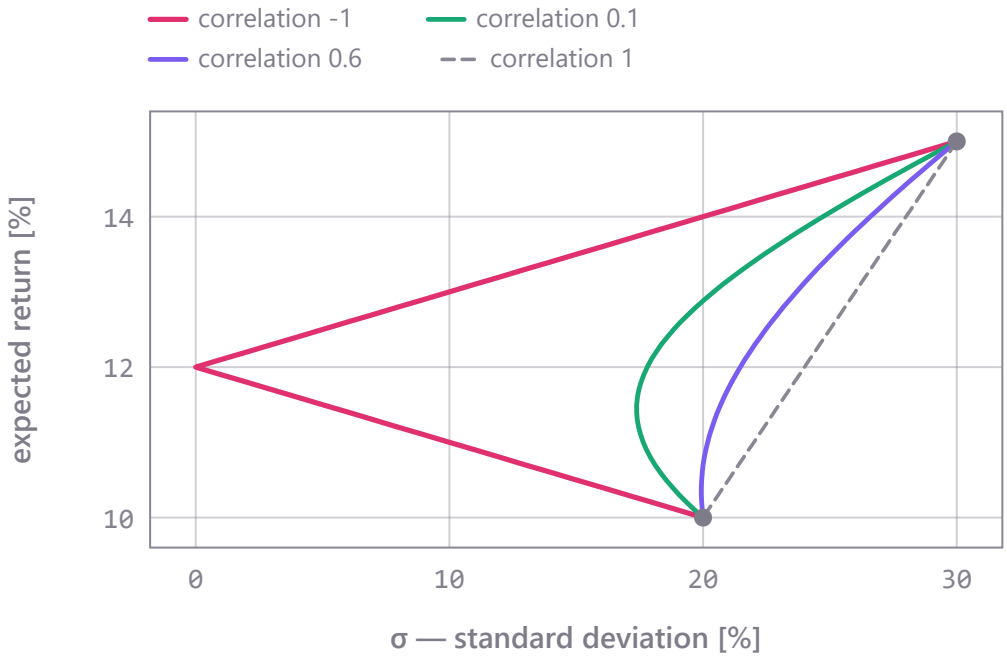
input	meaning	condition
`weight_1`	weight of asset 1 (asset 2 gets 1 - w1)	
`return_1`	expected return of asset 1 (decimal)	
`return_2`	expected return of asset 2 (decimal)	
`sd_1`	standard deviation of asset 1 (decimal)	
`sd_2`	standard deviation of asset 2 (decimal)	
`correlation`	correlation between the two assets (-1..1)	

output	meaning
`expected_return`	w1 $r_1 + w_2$ r2
`std_dev`	portfolio standard deviation
`variance`	portfolio variance
`weight_2`	$1 - w_1$

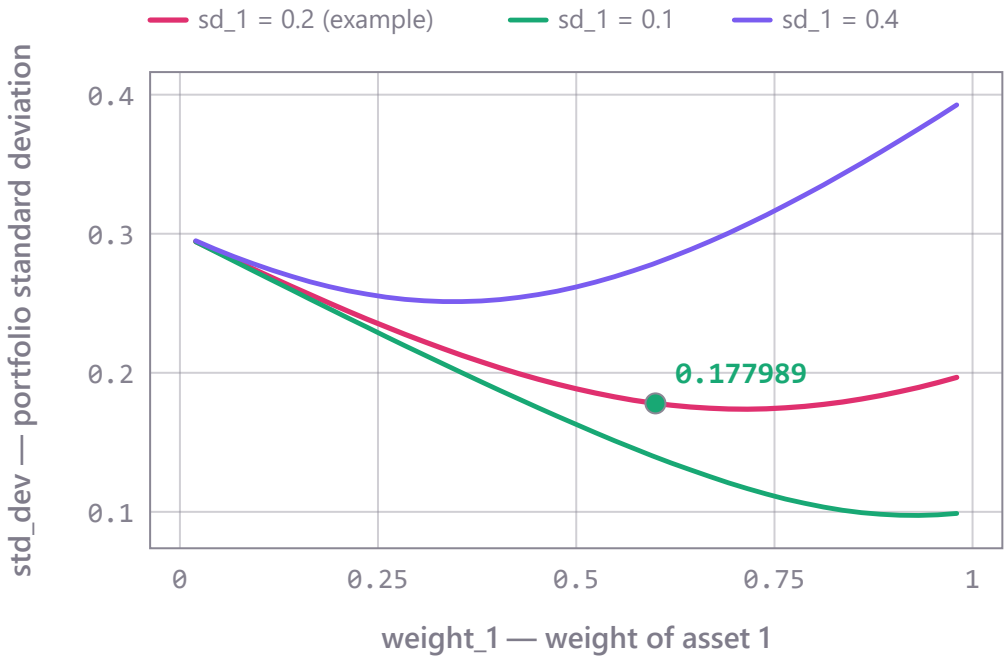
- **Method:** var = w1^2 $s_1^2 + w_2^2\ s_2^2 + 2$ w1 w2 rho s1 s2
- **Assumes:** -1 <= correlation <= 1; standard deviations >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Risk and return of every mix of two assets (returns 10 % and 15 %, standard deviations 20 % and 30 %, correlation 0.1), computed by the package at 51 weights. The mix w_1 = 0.6 gives 12 % at σ = 17.7989 %; marked: the minimum-variance mix w_1 = 0.711864, σ = 17.3791 %, return 11.4407 %.



All mixes of two assets (returns 10 % and 15 %, standard deviations 20 % and 30 %) for correlations -1, 0.1, 0.6, 1 (51 weights each, runs of the package): the lower the correlation, the more risk diversification removes; at −1 a riskless mix exists, at +1 the mixes lie on a straight line. Dots: each asset alone.



std_dev (portfolio standard deviation) against weight_1 (weight of asset 1 (asset 2 gets 1 - w1)) from 0.02 to 0.98. One curve for each value of sd_1 (standard deviation of asset 1 (decimal)): 0.2, 0.1, 0.4. Other inputs as in the worked example: return_1 = 0.1, return_2 = 0.15, sd_2 = 0.3, correlation = 0.1. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, weight_1 = 0.6, std_dev = 0.177989.

```
economics_finance_ai999x.solve('two_asset_portfolio', weight_1=0.6, return_1=0.1, return_2=0.15,
sd_1=0.2, sd_2=0.3, correlation=0.1)
# -> expected_return = 0.12, std_dev = 0.177989, variance = 0.03168, weight_2 = 0.4   (SOLVED)
```

```
economics_finance_ai999x.solve('two_asset_portfolio', weight_1=0.5, return_1=0.1, return_2=0.1,
sd_1=0.2, sd_2=0.2, correlation=-1.0)
# -> expected_return = 0.1, std_dev = 0, variance = 0, weight_2 = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the two-asset portfolio return and risk with w1 = 0.6, r1 = 10%, r2 = 15%, sd1 = 20%, sd2 = 30%, correlation = 0.1.	SOLVED	expected_return = 0.12, std_dev = 0.177989, variance = 0.03168, weight_2 = 0.4
Find the two-asset portfolio return and risk with w1 = 0.6, r1 = 10%, r2 = 15%, sd1 = 20%, sd2 = 30%, correlation = 1.5.	INVALID_INPUT	Two-asset portfolio return and risk: correlation must lie between -1 and 1.

Minimum-variance portfolio of two assets — `min_variance_two_asset`

The weights of two assets that minimise the portfolio variance (short sales allowed), and that variance's sd.

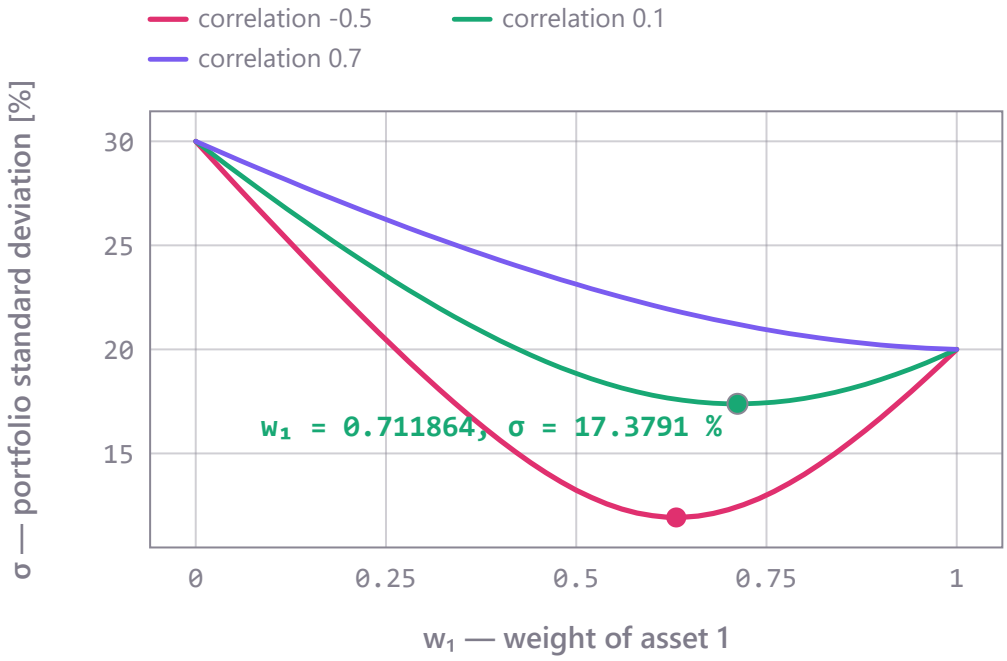
$$w_1^* = \frac{\sigma_2^2 - \rho\sigma_1\sigma_2}{\sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2}$$

input	meaning	condition
`sd_1`	standard deviation of asset 1 (decimal)	
`sd_2`	standard deviation of asset 2 (decimal)	

input	meaning	condition
`correlation`	correlation between the assets (-1..1)	

output	meaning
`weight_1`	w1 = (s2^2 - rho s1 s_2)/($s_1^2 + s_2^2 - 2$ rho s1 s2)
`weight_2`	$1 - w_1$
`std_dev`	standard deviation of the minimum-variance portfolio

- **Method:** w1 = (s2^2 - rho s1 s_2)/($s_1^2 + s_2^2 - 2$ rho s1 s2)
- **Assumes:** sd_1, $sd_2 > 0$; -1 <= correlation <= 1; not (correlation = 1 and sd_1 = sd_2)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Portfolio standard deviation of two assets (σ_1 = 20 %, σ_2 = 30 %) against the weight of asset 1, for correlations -0.5, 0.1, 0.7 (two-asset portfolio runs of the package; expected returns set to 0, they do not affect σ). Dots: the minimum-variance weights the package returns; marked: the worked example, correlation 0.1: w_1 = 0.711864, σ = 17.3791 %.

```
economics_finance_ai999x.solve('min_variance_two_asset', sd_1=0.2, sd_2=0.3, correlation=0.1)
# -> weight_1 = 0.711864, weight_2 = 0.288136, std_dev = 0.173791   (SOLVED)
```

```
economics_finance_ai999x.solve('min_variance_two_asset', sd_1=0.2, sd_2=0.2, correlation=0.3)
# -> weight_1 = 0.5, weight_2 = 0.5, std_dev = 0.161245   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the minimum variance weights with sd1 = 20%, sd2 = 30%, correlation = 0.1.	SOLVED	weight_1 = 0.711864, weight_2 = 0.288136, std_dev = 0.173791
Find the minimum variance weights with sd1 = 20%, sd2 = 20%, correlation = 1.	INVALID_INPUT	Minimum-variance portfolio of two assets: the two assets are perfectly correlated with equal risk: every mix has the sam

Global minimum-variance portfolio — `global_min_variance`

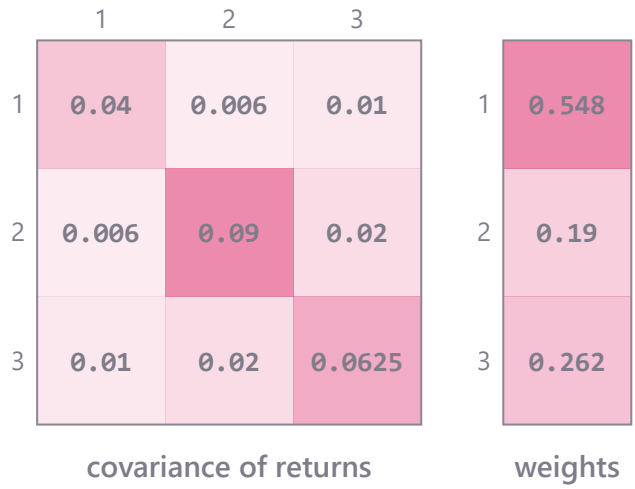
Weights (adding up to 1, short sales allowed) of the portfolio with the smallest variance, from a covariance matrix: $w = Sigma^{-1}$ 1 / (1' $Sigma^{-1}$ 1).

$$w^* = \frac{\Sigma^{-1}\mathbf{1}}{\mathbf{1}^\top\Sigma^{-1}\mathbf{1}}$$

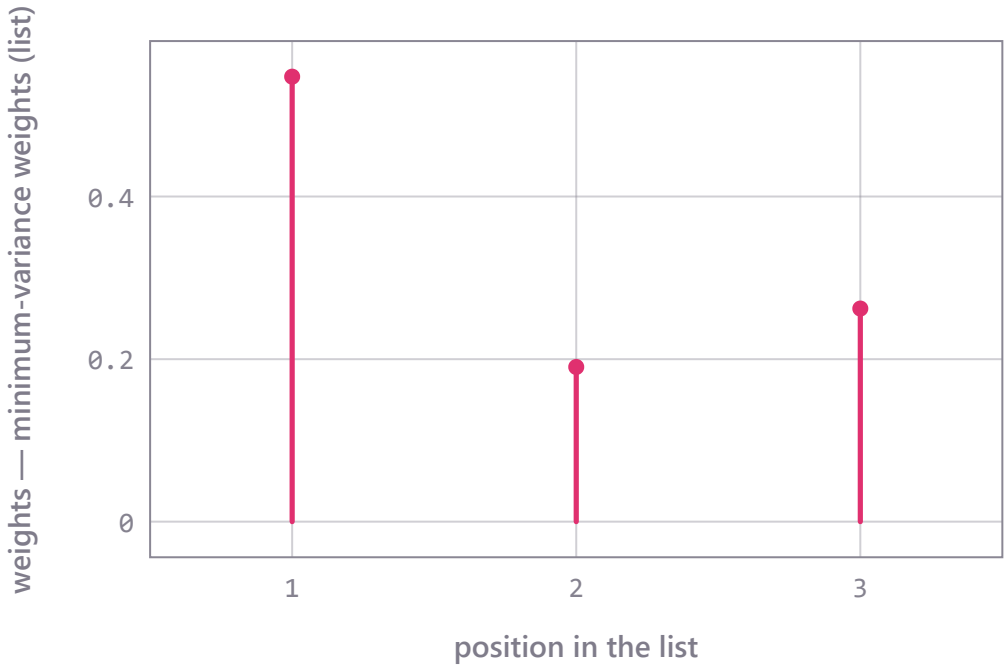
input	meaning	condition
`covariance`	covariance matrix of the asset returns	

output	meaning
`weights`	minimum-variance weights (list)
`variance`	portfolio variance
`std_dev`	portfolio sd

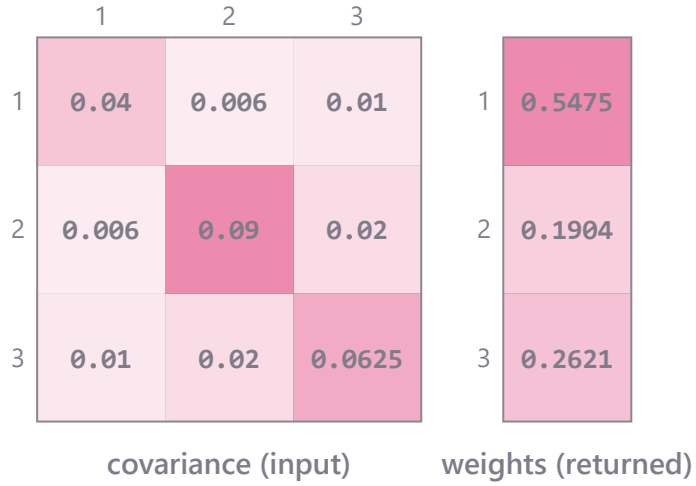
- **Method:** Cholesky solve of Sigma $x = 1$, w = x / sum(x)
- **Assumes:** covariance symmetric and positive definite
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The global minimum-variance portfolio of three assets with the covariance matrix shown (left): the package returns weights [0.547506, 0.190409, 0.262085] (right), a portfolio standard deviation of 0.160198, below every single asset's (0.2, 0.3, 0.25, each also computed by the package).



minimum-variance weights (list) (weights) returned by economics-finance-ai999x for the worked example, one value for each position in the list (3 values, first 0.547506, last 0.262085, largest 0.547506). Inputs: covariance = [[0.04, 0.006, 0.01], [0.006, 0.09, 0.02], [0....



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): covariance (input); weights (returned). Returned with them by economics-finance-ai999x: variance = 0.0256635, std_dev = 0.160198.

```
economics_finance_ai999x.solve('global_min_variance', covariance='[[0.04, 0.006, 0.01], [0.006, 0.09, 0.02], [0.01, 0.02, 0.0625]]')
# -> weights = [0.5475060905244262, 0.19040902679830743, 0.2620848826772663], variance = 0.0256635, std_dev = 0.160198   (SOLVED)
```

```
economics_finance_ai999x.solve('global_min_variance', covariance=[[0.04, 0.0], [0.0, 0.04]])
# -> weights = [0.5, 0.5], variance = 0.02, std_dev = 0.141421   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the global minimum variance portfolio with covariance = [[0.04, 0.006, 0.01], [0.006, 0.09, 0.02], [0.01, 0.02, 0.0625]].	SOLVED	weights = [0.5475060905244262, 0.19040902679830743, 0.2620848826772663], variance = 0.0256635, std_dev = 0.160198
Find the global minimum variance portfolio with covariance = [[0.04, 0.04], [0.04, 0.04]].	INVALID_INPUT	Global minimum-variance portfolio: the covariance matrix must be positive definite (no riskless combination of the asset

Tangency (maximum Sharpe ratio) portfolio — `tangency_portfolio`

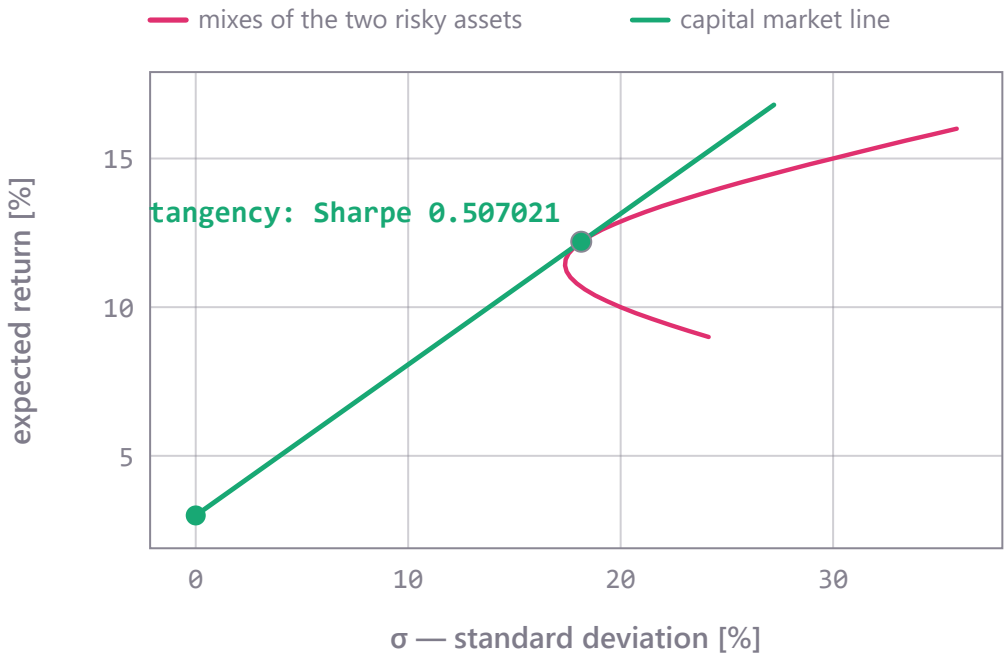
Weights of the risky portfolio with the highest Sharpe ratio (short sales allowed): w proportional to $Sigma^{-1}$ (mu - rf).

$$w^* = \frac{\Sigma^{-1}(\mu - r_f \mathbf{1})}{\mathbf{1}^\top \Sigma^{-1}(\mu - r_f \mathbf{1})}$$

input	meaning	condition
`expected_returns`	expected returns of the assets (decimals)	
`covariance`	covariance matrix of the returns	
`risk_free`	risk-free rate (decimal)	

output	meaning
`weights`	tangency weights (list, add up to 1)
`expected_return`	portfolio expected return
`std_dev`	portfolio sd
`sharpe`	(E - rf) / sd

- **Method:** $z = Sigma^{-1}$ (mu - rf) by Cholesky, w = z / sum(z)
- **Assumes:** covariance positive definite; 1' $Sigma^{-1}\ (mu - rf) > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Risk and return of mixes of two assets (expected returns 0.1, 0.15, covariance [[0.04, 0.006], [0.006, 0.09]]; portfolio runs of the package for weights −0.2 … 1.2) and the line from the risk-free rate 3 % through the tangency portfolio the package returns (weights 0.560241, 0.439759; return 12.1988 %, σ 18.1428 %, Sharpe ratio 0.507021); the line is drawn through those two returned points.

```
economics_finance_ai999x.solve('tangency_portfolio', expected_returns='0.1, 0.15',
covariance='[[0.04, 0.006], [0.006, 0.09]]', risk_free=0.03)
# -> weights = [0.5602409638554217, 0.4397590361445783], expected_return = 0.121988, std_dev =
0.181428, sharpe = 0.507021   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the tangency portfolio with expected returns = [0.1, 0.15], covariance = [[0.04, 0.006], [0.006, 0.09]], risk-free rate = 3%.	SOLVED	weights = [0.5602409638554217, 0.4397590361445783], expected_return = 0.121988, std_dev = 0.181428, sharpe = 0.507021
Find the tangency portfolio with expected returns = [0.01, 0.02], covariance = [[0.04, 0.006], [0.006, 0.09]], risk-free rate = 5%.	INVALID_INPUT	Tangency (maximum Sharpe ratio) portfolio: no portfolio of positive total weight has the highest Sharpe ratio (the exces

Arithmetic and geometric mean return — `mean_returns`

Arithmetic mean, geometric (compound) mean and cumulative return of a series of periodic returns.

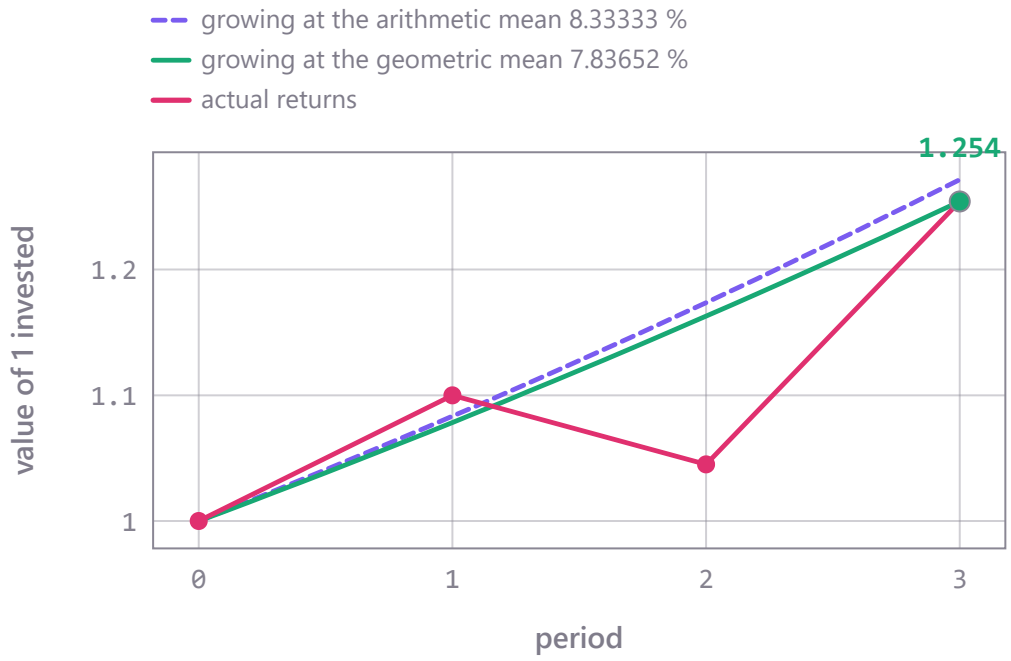
$$\bar{R}_G = \left[\prod_{t=1}^{n}(1+R_t)\right]^{1/n} - 1$$

input	meaning	condition
`returns`	periodic returns (decimals)	

output	meaning
`arithmetic_mean`	sum R / n

output	meaning
`geometric_mean`	(prod $(1+R)$)^$(1/n)-1$
`cumulative_return`	prod $(1+R)-1$

- **Method:** geometric mean = exp(mean ln(1 + R)) - 1
- **Assumes:** each return between -100% and 100%
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



1 invested through returns 0.1, -0.05, 0.2: the value after each period is 1 plus the cumulative return the package returns for the returns so far. Growth at the returned geometric mean 7.83652 % ends at the same value; growth at the arithmetic mean 8.33333 % overstates it (both compound-interest runs).

```
economics_finance_ai999x.solve('mean_returns', returns='0.1, -0.05, 0.2')
# -> arithmetic_mean = 0.0833333, geometric_mean = 0.0783652, cumulative_return = 0.254   (SOLVED)
```

```
economics_finance_ai999x.solve('mean_returns', returns=[0.5, -0.5])
# -> arithmetic_mean = 0, geometric_mean = -0.133975, cumulative_return = -0.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the geometric mean return with returns = [10%, -5%, 20%].	SOLVED	arithmetic_mean = 0.0833333, geometric_mean = 0.0783652, cumulative_return = 0.254
Find the geometric mean return with returns = [10%, -150%, 20%].	INVALID_INPUT	Arithmetic and geometric mean return: returns: each return is a decimal between -100% and 100% (write 5% or 0.05).

Expected return and risk from scenarios — `scenario_return`

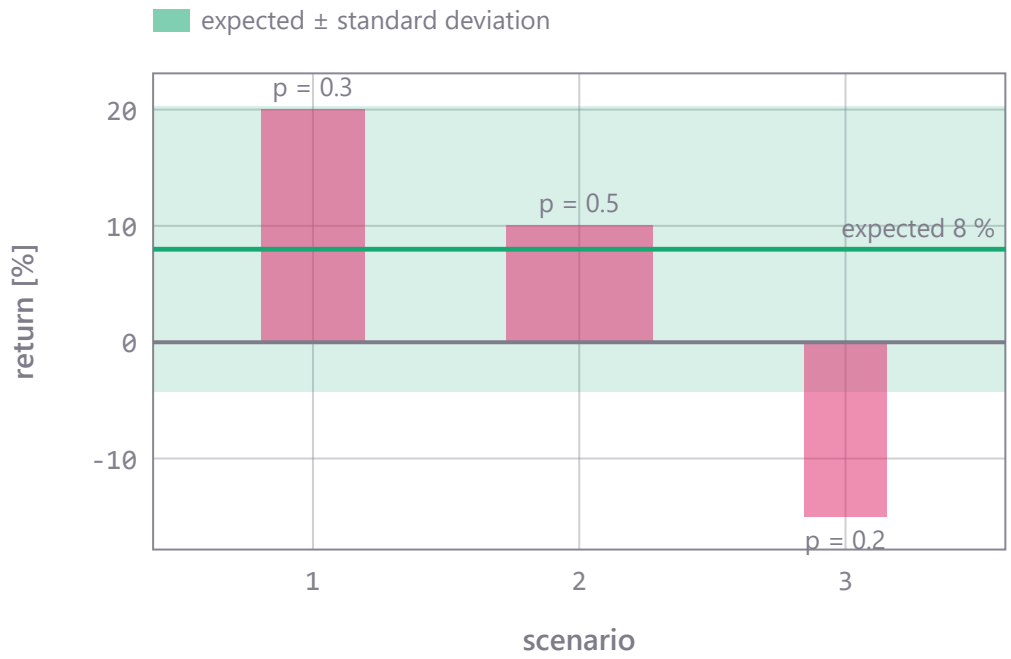
Expected return, variance and standard deviation of an asset whose return depends on scenarios with given probabilities.

$$E[R] = \sum_i p_i R_i, \quad \sigma^2 = \sum_i p_i (R_i - E[R])^2$$

input	meaning	condition
`probabilities`	probability of each scenario (adding up to 1)	
`returns`	return in each scenario (decimals)	

output	meaning
`expected_return`	sum p R
`variance`	sum p $(R - E)^2$
`std_dev`	sqrt(variance)

- **Method:** E = sum p_i R_i; Var = sum p_i $(R_i - E)^2$
- **Assumes:** one probability per return; probabilities >= 0 adding up to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Returns 20 %, 10 %, -15 % in 3 scenarios with probabilities 0.3, 0.5, 0.2 (bar width grows with the probability): the package returns the expected return 8 % and the standard deviation 12.2882 % (band).

```
economics_finance_ai999x.solve('scenario_return', probabilities='0.3, 0.5, 0.2', returns='0.2, 0.1, -0.15')
# -> expected_return = 0.08, variance = 0.0151, std_dev = 0.122882   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the expected return from scenarios with probabilities = [0.3, 0.5, 0.2], returns = [0.2, 0.1, -0.15].	SOLVED	expected_return = 0.08, variance = 0.0151, std_dev = 0.122882
Find the expected return from scenarios with probabilities = [0.3, 0.5, 0.3], returns = [0.2, 0.1,	INVALID_INPUT	Expected return and risk from scenarios: the probabilities must be non-negative and add up to 1.

question	outcome	result
-0.15].		

Historical value at risk and expected shortfall — `historical_var`

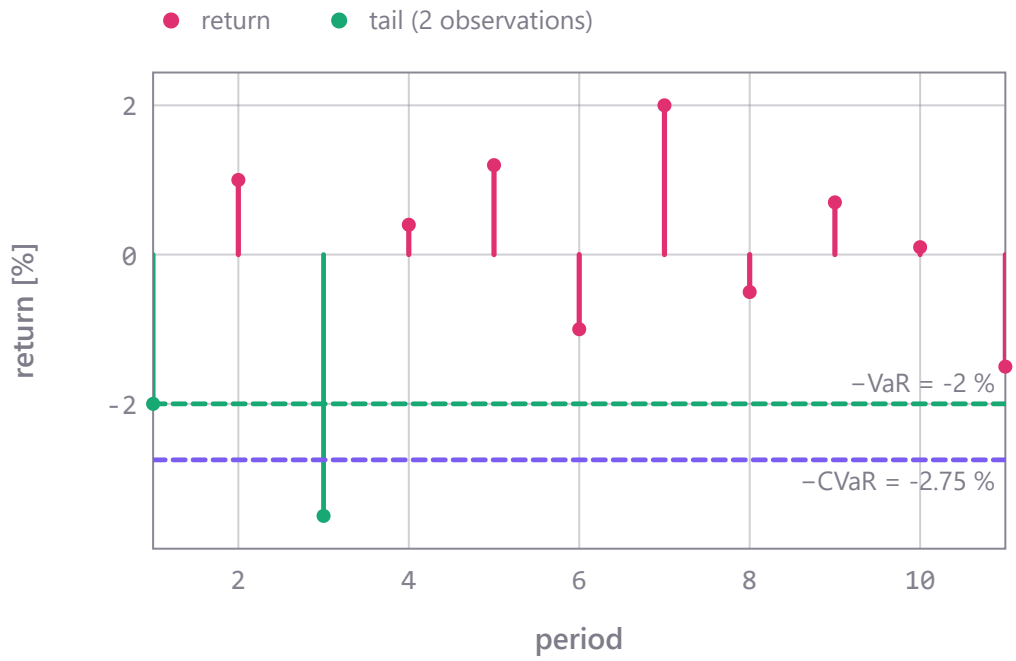
VaR and CVaR (expected shortfall) by historical simulation: with k = ceil(n (1 - confidence)), VaR is minus the k-th smallest return and CVaR minus the mean of the k smallest, times the portfolio value.

$$\mathrm{VaR}_c = -R_{(k)}, \quad \mathrm{CVaR}_c = -\frac{1}{k}\sum_{i=1}^{k} R_{(i)}, \quad k = \lceil n(1-c) \rceil$$

input	meaning	condition
`returns`	historical returns (decimals)	
`confidence`	confidence level (0.95 = 95%)	
`portfolio_value`	portfolio value (default 1: VaR as a fraction)	

output	meaning
`var`	value at risk (a positive number is a loss)
`cvar`	expected shortfall (mean loss beyond VaR)
`tail_observations`	k, the number of returns in the tail

- **Method:** k = ceil(n $(1 - c)$); VaR = -R_(k); CVaR = -$(1/k)$ sum_{$i \leq k$} R_(i)
- **Assumes:** at least 2 returns; 0.5 < confidence < 1; portfolio_value > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The historical returns -0.02, 0.01, -0.035, 0.004, 0.012, -0.01, 0.02, -0.005, 0.007, 0.001, -0.015 as stems: the package returns the 90 % value at risk, a loss of 2 %, and the average loss beyond it (CVaR) 2.75 %; the 2 returns in the tail are highlighted.

```
economics_finance_ai999x.solve('historical_var', returns='-0.02, 0.01, -0.035, 0.004, 0.012, -0.01, 0.02, -0.005, 0.007, 0.001, -0.015', confidence=0.9)
# -> var = 0.02, cvar = 0.0275, tail_observations = 2   (SOLVED)
```

```
economics_finance_ai999x.solve('historical_var', returns=[0.01, -0.02, 0.03, -0.04, 0.05, -0.01, 0.02, 0.0, 0.01, -0.03], confidence=0.9, portfolio_value=1000)
# -> var = 40, cvar = 40, tail_observations = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the historical VaR with returns = [-0.02, 0.01, -0.035, 0.004, 0.012, -0.01, 0.02, -0.005, 0.007, 0.001, -0.015], confidence = 90%.	SOLVED	var = 0.02, cvar = 0.0275, tail_observations = 2
Find the historical VaR with returns = [-0.02, 0.01, -0.035], confidence = 40%.	INVALID_INPUT	Historical value at risk and expected shortfall: confidence is a decimal between 0.5 and 1 (95% = 0.95).

Parametric (normal) value at risk and expected shortfall — `parametric_var`

VaR and CVaR of normally distributed returns with a mean and standard deviation per period, scaled to a horizon by mean x h and sd x sqrt(h).

$$\mathrm{VaR} = V(z_c\,\sigma\sqrt{h} - \mu h), \quad \mathrm{ES} = V\left(\frac{\sigma\sqrt{h}\,\varphi(z_c)}{1-c} - \mu h\right)$$

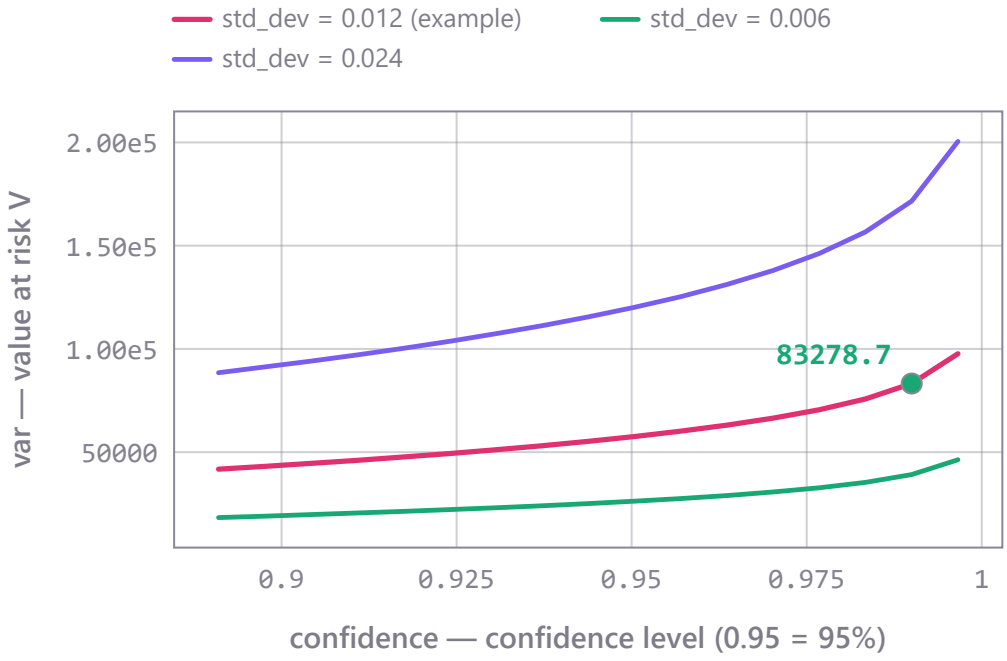
input	meaning	condition
`mean`	mean return per period (decimal)	
`std_dev`	standard deviation of returns per period (decimal)	
`confidence`	confidence level (0.95 = 95%)	
`horizon`	horizon in periods (default 1)	
`portfolio_value`	portfolio value (default 1)	

output	meaning
`var`	value at risk V (z sd sqrt(h) - mean h)
`cvar`	expected shortfall V (sd sqrt(h) phi(z)/(1 - c) - mean h)
`z`	standard normal quantile at the confidence level

- **Method:** VaR = V (z_c sigma sqrt(h) - mu h); $ES = V$ (sigma sqrt(h) phi(z_c) / (1 - c) - mu h)
- **Assumes:** 0 < std_dev < 5; 0.5 < confidence < 1; horizon > 0; returns normally distributed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The distribution of the loss on a 1000000 portfolio over 10 period(s) (mean return 0.0005, standard deviation 0.012 per period), above its median: the density is drawn from the VaR the package returns at 120 confidence levels between 0.5 and 0.9999 (probability differences divided by loss differences). Shaded: losses beyond the 99 % VaR = 83278.7; the expected loss in that tail, CVaR = 96137.8, is also returned.



var (value at risk V (z sd sqrt(h) - mean h)) against confidence (confidence level (0.95 = 95%)) from 0.891 to 1.089. One curve for each value of std_dev (standard deviation of returns per period (decimal)): 0.012, 0.006, 0.024. Other inputs as in the worked example: mean = 0.0005, horizon = 10, portfolio_value = 1000000. 17 points per curve computed by economics-finance-ai999x; marked: the worked example, confidence = 0.99, var = 83278.7.

```
economics_finance_ai999x.solve('parametric_var', mean=0.0005, std_dev=0.012, confidence=0.99,
horizon=10, portfolio_value=1000000)
# -> var = 83278.7, cvar = 96137.8, z = 2.32635   (SOLVED)
```

```
economics_finance_ai999x.solve('parametric_var', mean=0.0, std_dev=0.02, confidence=0.95)
# -> var = 0.0328971, cvar = 0.0412543, z = 1.64485   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the parametric VaR with mean = 0.05%, std dev = 1.2%, confidence = 99%, horizon = 10, portfolio value = 1000000.	SOLVED	var = 83278.7, cvar = 96137.8, z = 2.32635
Find the parametric VaR with mean = 0, sigma = 2%, confidence = 95%.	SOLVED	var = 0.0328971, cvar = 0.0412543, z = 1.64485
Find the parametric VaR with mean = 0, sigma = 20, confidence = 95%.	INVALID_INPUT	Parametric (normal) value at risk and expected shortfall: std_dev is a decimal (20% = 0.2).

Maximum drawdown — `max_drawdown`

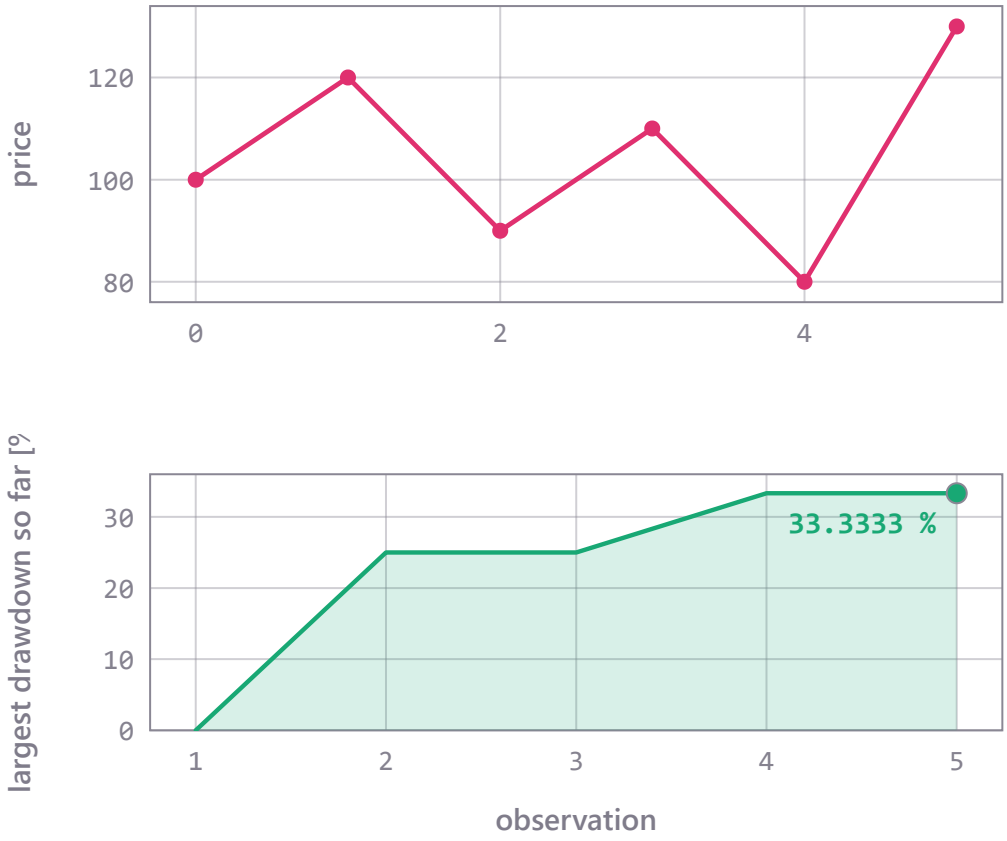
The largest fall from a running peak to a later trough, as a fraction of the peak.

$$\mathrm{MDD} = \max_{t} \left(1 - \frac{P_t}{\max_{s \le t} P_s} \right)$$

input	meaning	condition
`prices`	price or portfolio value series in time order	

output	meaning
`max_drawdown`	max over t of 1 - P_t / max_{$s \le t$} P_s

- **Method:** running peak; drawdown 1 - P/peak; maximum over time
- **Assumes:** prices > 0; at least 2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: the price series 100, 120, 90, 110, 80, 130. Bottom: the largest fall from a previous peak seen so far, a run of the package on the prices up to each observation; marked: the maximum drawdown of the whole series, 33.3333 %.

```
economics_finance_ai999x.solve('max_drawdown', prices='100, 120, 90, 110, 80, 130')
# -> max_drawdown = 0.333333   (SOLVED)
```

```
economics_finance_ai999x.solve('max_drawdown', prices=[100, 101, 102])
# -> max_drawdown = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the maximum drawdown with prices = [100, 120, 90, 110, 80, 130].	SOLVED	max_drawdown = 0.333333
Find the maximum drawdown with prices = [100, -5, 90].	INVALID_INPUT	Maximum drawdown: prices must be positive.

Portfolio value at risk (variance-covariance) — `portfolio_var`

Zero-mean normal VaR of a portfolio from its weights and the covariance matrix of periodic returns, with the undiversified VaR (sum of the assets' own VaRs) and the diversification benefit.

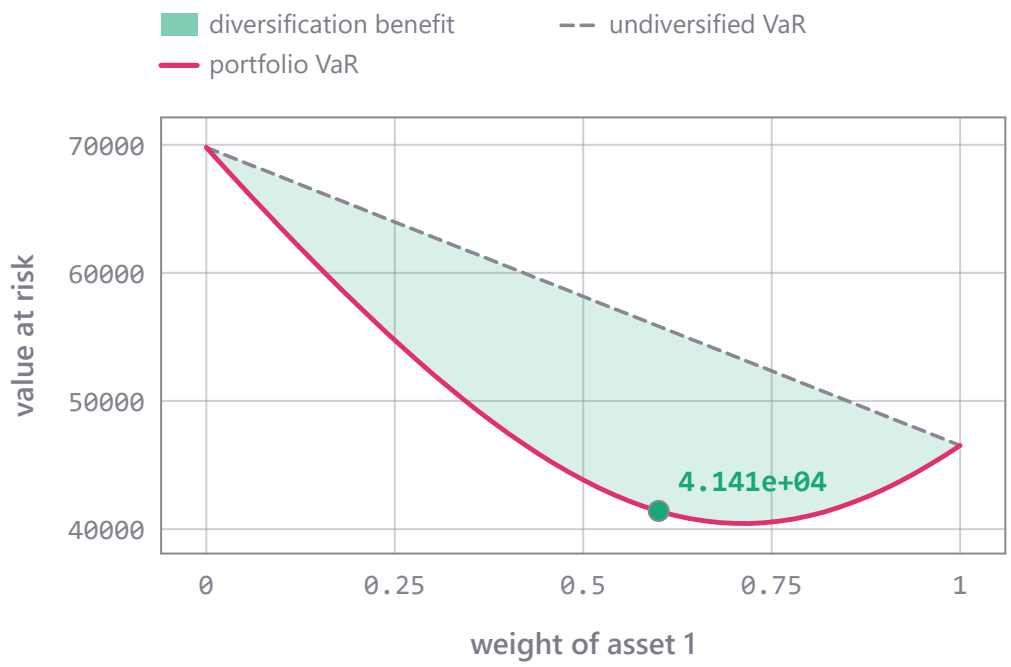
$$\mathrm{VaR} = z_c \sqrt{w^\top \Sigma w}\,\sqrt{h}\,V$$

input	meaning	condition
`weights`	portfolio weights (fractions of the portfolio value)	

input	meaning	condition
`covariance`	covariance matrix of periodic returns	
`confidence`	confidence level (0.99 = 99%)	
`portfolio_value`	portfolio value (default 1)	
`horizon`	horizon in periods (default 1)	

output	meaning
`var`	z sqrt(w'Sigma w) sqrt(h) V
`portfolio_sd`	sqrt(w'Sigma w) per period
`undiversified_var`	z sqrt(h) V sum |w_i| sigma_i
`diversification_benefit`	undiversified - portfolio VaR

- **Method:** VaR = z_c sqrt(w' Sigma w) sqrt(h) V (mean return taken as 0)
- **Assumes:** one weight per row of the covariance; covariance symmetric positive semidefinite; 0.5 < confidence < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Value at risk at 99 % confidence of a 1000000 two-asset portfolio (covariance [[0.0004, 0.00006], [0.00006, 0.0009]]) as the weight of asset 1 runs from 0 to 1: the portfolio VaR and the undiversified VaR (the sum of the stand-alone VaRs), both returned by the package at 51 weights; the shaded gap is the diversification benefit. Marked: weights 0.6, 0.4, VaR 41406.4, benefit 14426.

```
economics_finance_ai999x.solve('portfolio_var', weights='0.6, 0.4', covariance='[[0.0004, 0.00006], [0.00006, 0.0009]]', confidence=0.99, portfolio_value=1000000)
# -> var = 41406.4, portfolio_sd = 0.0177989, undiversified_var = 55832.3, diversification_benefit = 14426   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the portfolio VaR with weights = [0.6, 0.4], covariance = [[0.0004, 0.00006], [0.00006, 0.0009]], confidence = 99%,	SOLVED	var = 41406.4, portfolio_sd = 0.0177989, undiversified_var = 55832.3, diversification_benefit

question	outcome	result
portfolio value = 1000000.		= 14426

Black-Scholes option prices — `black_scholes`

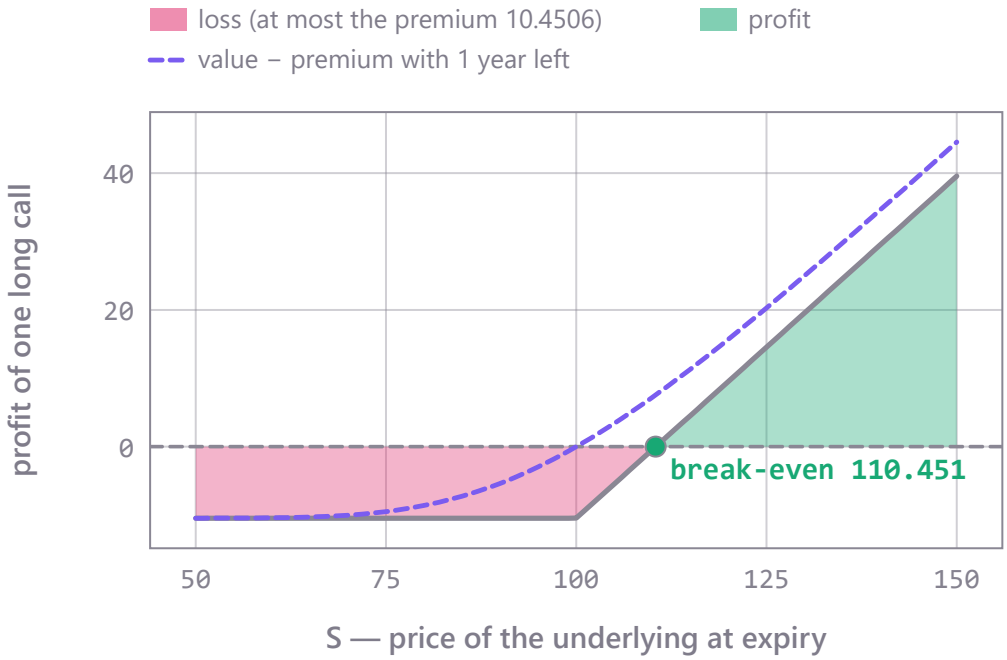
Prices of European call and put options on an asset with a continuous dividend yield (Black-Scholes-Merton).

$$C = Se^{-qT}N(d_1) - Ke^{-rT}N(d_2), \quad d_1 = \frac{\ln(S/K) + (r - q + \sigma^2/2)T}{\sigma\sqrt{T}}$$

input	meaning	condition
`spot`	price of the underlying now	
`strike`	strike (exercise) price	
`time`	time to expiry in years	
`rate`	risk-free rate, continuously compounded (decimal)	
`volatility`	volatility of the underlying, annual (decimal)	
`dividend_yield`	continuous dividend yield (decimal, default 0)	

output	meaning
`call`	European call price
`put`	European put price
`d1`	d1
`d2`	$d_2 = d_1 - \sigma$ sqrt(T)

- **Method:** $C = S\,e^{-qT}$ N(d1) - K e^{-rT} N(d2); $P = K\,e^{-rT}$ N(-d2) - S e^{-qT} N(-d1)
- **Assumes:** spot, strike, time > 0; 0 < volatility < 5; European exercise; lognormal prices
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Profit and loss at expiry of a European call (strike 100) bought for the Black–Scholes price the package returns, 10.4506 (rate 5 %, volatility 20 %, 1 year): the payoff is the package's value with 10^{-6} year left, at 101 prices, minus the premium. The loss (shaded) is capped at the premium; profit starts above the break-even price S ≈ 110.451 (strike + premium = 110.451). Dashed: value before expiry minus the premium.



Black–Scholes values of a European call and put (strike 100, rate 5 %, volatility 20 %, 1 year to expiry) at 101 underlying prices, each a run of the package; dashed: the values with 10^{-6} year left (the payoff at expiry). The gap is the time value. Marked: S = 100, call 10.4506, put 5.57353.

```
economics_finance_ai999x.solve('black_scholes', spot=100, strike=100, time=1, rate=0.05,
volatility=0.2)
# -> call = 10.4506, put = 5.57353, d1 = 0.35, d2 = 0.15   (SOLVED)
```

```
economics_finance_ai999x.solve('black_scholes', spot=50, strike=55, time=0.5, rate=0.03,
volatility=0.3, dividend_yield=0.02)
# -> call = 2.42891, put = 7.10757, d1 = -0.31966, d2 = -0.531792   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Price the option with Black-Scholes with spot = 100, strike = 100, time = 1, rate = 5%, volatility = 20%.	SOLVED	call = 10.4506, put = 5.57353, d1 = 0.35, d2 = 0.15
Find the Black-Scholes price with s = 50, k = 55, t = 0.5, r = 3%, sigma = 30%, q = 2%.	SOLVED	call = 2.42891, put = 7.10757, d1 = -0.31966, d2 = -0.531792
Find the Black-Scholes price with s = 50, k = 55, t = 0, r = 3%, sigma = 30%.	INVALID_INPUT	Black-Scholes option prices: time must be positive.
Find the Black-Scholes price with s = 50, k = 55, t = 0.5, r = 3%.	NEEDS_INPUT	black_scholes needs volatility (volatility of the underlying, annual (decimal)). Example: solve('black_scholes', spot=10

Black-Scholes Greeks — `bs_greeks`

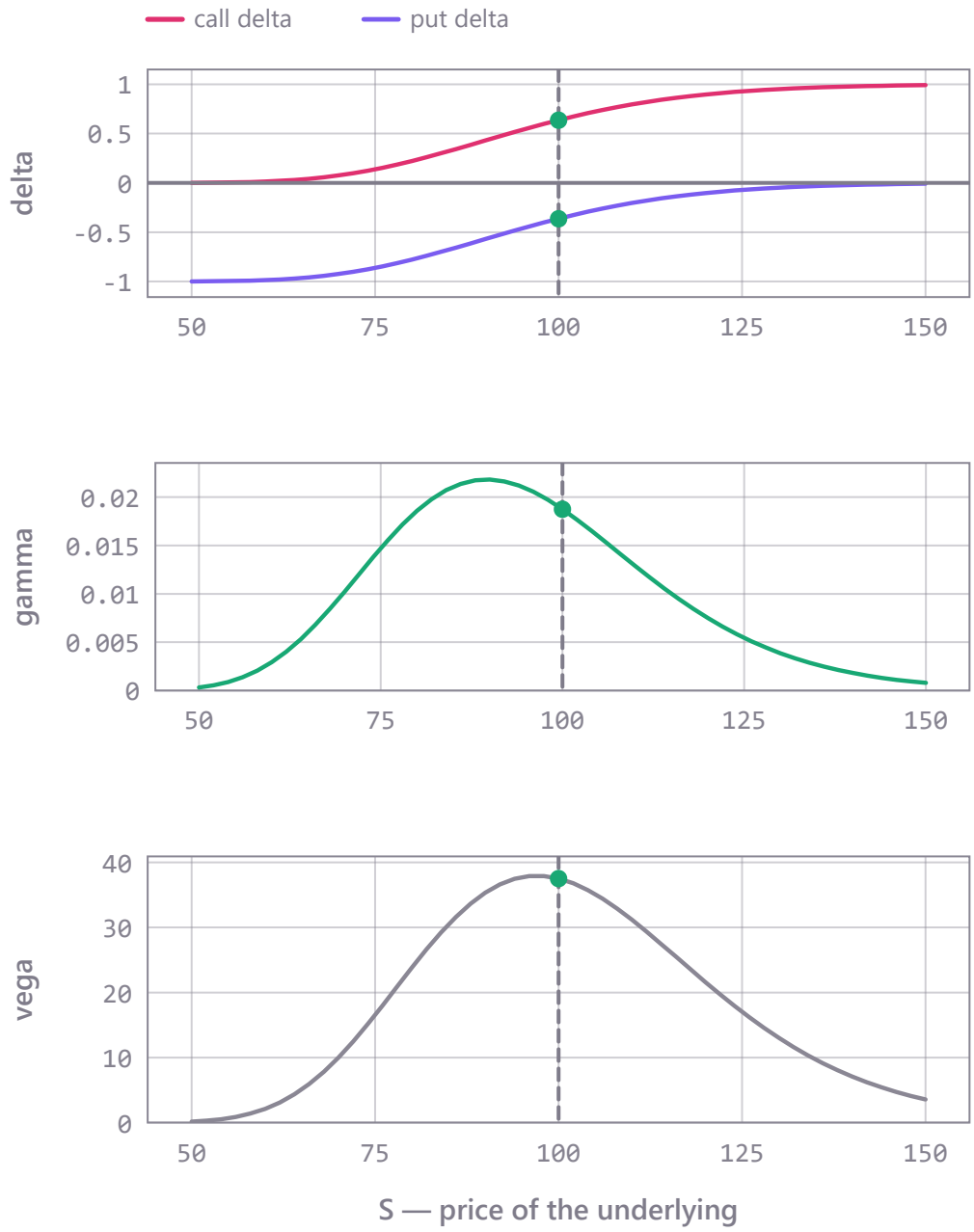
Delta, gamma, vega, theta and rho of European calls and puts (Black-Scholes-Merton, continuous dividend yield). Vega per 1.00 of volatility, theta per year, rho per 1.00 of rate.

$$\Delta_C = e^{-qT}N(d_1),\ \Gamma = \frac{e^{-qT}\varphi(d_1)}{S\sigma\sqrt{T}},\ \nu = Se^{-qT}\varphi(d_1)\sqrt{T}$$

input	meaning	condition
`spot`	price of the underlying now	
`strike`	strike (exercise) price	
`time`	time to expiry in years	
`rate`	risk-free rate, continuously compounded (decimal)	
`volatility`	volatility of the underlying, annual (decimal)	
`dividend_yield`	continuous dividend yield (decimal, default 0)	

output	meaning
`call_delta`	dC/dS
`put_delta`	dP/dS
`gamma`	$d2V/dS2$ (call = put)
`vega`	dV/dsigma (call = put)
`call_theta`	-dC/dT per year
`put_theta`	-dP/dT per year
`call_rho`	dC/dr
`put_rho`	dP/dr

- **Method:** closed-form Greeks of the Black-Scholes-Merton formula
- **Assumes:** spot, strike, time > 0; 0 < volatility < 5
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Delta (call and put), gamma and vega of a European option (strike 100, 1 year, rate 5 %, volatility 20 %) at 51 underlying prices, each a run of the package; the vertical line is the strike. Gamma and vega peak near the money. Marked at S = 100: call delta 0.636831, put delta -0.363169, gamma 0.018762, vega 37.524.

```
economics_finance_ai999x.solve('bs_greeks', spot=100, strike=100, time=1, rate=0.05,
volatility=0.2)
# -> call_delta = 0.636831, put_delta = -0.363169, gamma = 0.018762, vega = 37.524, call_theta =
-6.41403, put_theta = -1.65788, call_rho = 53.2325, put_rho = -41.8905   (SOLVED)
```

```
economics_finance_ai999x.solve('bs_greeks', spot=50, strike=55, time=0.5, rate=0.03,
volatility=0.3, dividend_yield=0.02)
# -> call_delta = 0.370885, put_delta = -0.619164, gamma = 0.0353836, vega = 13.2689, call_theta =
-4.09323, put_theta = -3.45785, call_rho = 8.05768, put_rho = -19.0329   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the option greeks with spot = 100, strike = 100, time = 1, rate = 5%, volatility = 20%.	SOLVED	call_delta = 0.636831, put_delta = -0.363169, gamma = 0.018762, vega = 37.524, call_theta = -6.41403, put_theta = -1.65788, call_rho = 53.2325, put_rho = -41.8905

Implied volatility — `implied_volatility`

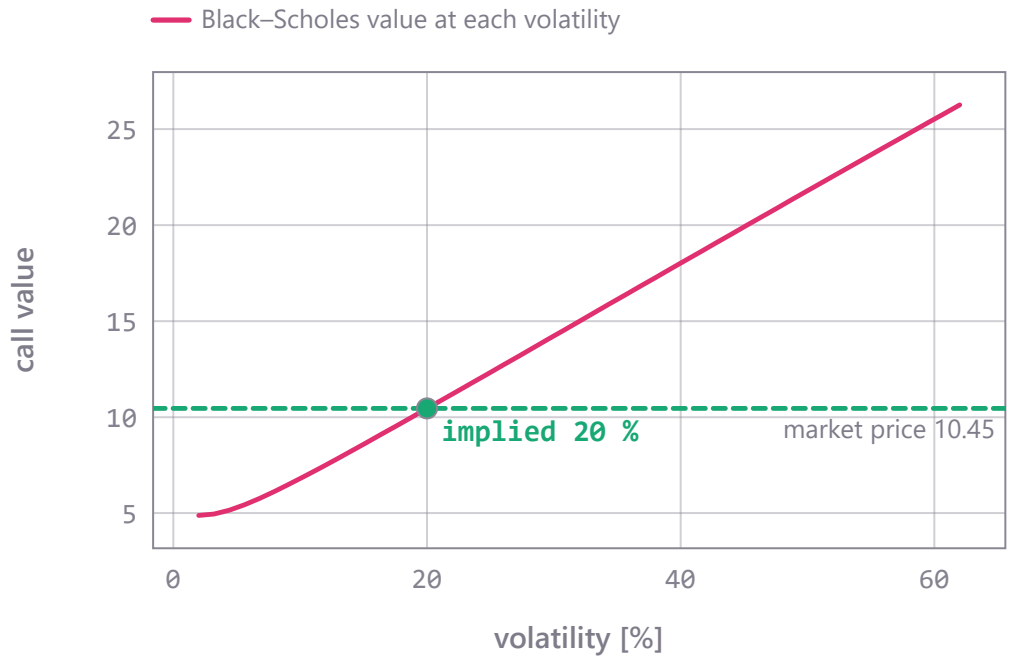
The volatility at which the Black-Scholes price of a European call or put equals its market price.

$$C_{BS}(\sigma_{imp}) = C_{market}$$

input	meaning	condition
`option_price`	market price of the option	
`spot`	price of the underlying now	
`strike`	strike (exercise) price	
`time`	time to expiry in years	
`rate`	risk-free rate, continuously compounded (decimal)	
`dividend_yield`	continuous dividend yield (decimal, default 0)	
`option_type`	call (default) or put	

output	meaning
`volatility`	implied volatility, annual
`vega`	vega at that volatility

- **Method:** bisection on sigma (the price rises with sigma, so the implied volatility is unique)
- **Assumes:** price within the no-arbitrage bounds; implied volatility between 0.01% and 500%; not too far from the money
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Black–Scholes value of a call (spot 100, strike 100, 1 year, rate 5 %) at 51 volatilities, each a run of the package; the implied volatility is where it equals the market price 10.4506. Marked: the volatility the package returns, 20 % (vega 37.524, the slope there).

```
economics_finance_ai999x.solve('implied_volatility', option_price=10.450583572185565, spot=100,
strike=100, time=1, rate=0.05)
# -> volatility = 0.2, vega = 37.524   (SOLVED)
```

```
economics_finance_ai999x.solve('implied_volatility', option_price=6.5, spot=50, strike=55,
time=0.5, rate=0.03, option_type='put')
# -> volatility = 0.277741, vega = 13.44   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the implied volatility with option price = 10.45, spot = 100, strike = 100, time = 1, rate = 5%.	SOLVED	volatility = 0.199984, vega = 37.5239
Find the implied volatility with option price = 6.5, s = 50, k = 55, t = 0.5, r = 3%, type = put.	SOLVED	volatility = 0.277741, vega = 13.44
Find the implied volatility with option price = 120, spot = 100, strike = 100, time = 1, rate = 5%.	INVALID_INPUT	Implied volatility: the price is outside the range Black-Scholes can produce (volatility 0.01% to 500%): it violates the

Put-call parity — `put_call_parity`

The price of a European put from the call with the same strike and expiry: $P = C - S\,e^{-qT} + K\,e^{-rT}$.

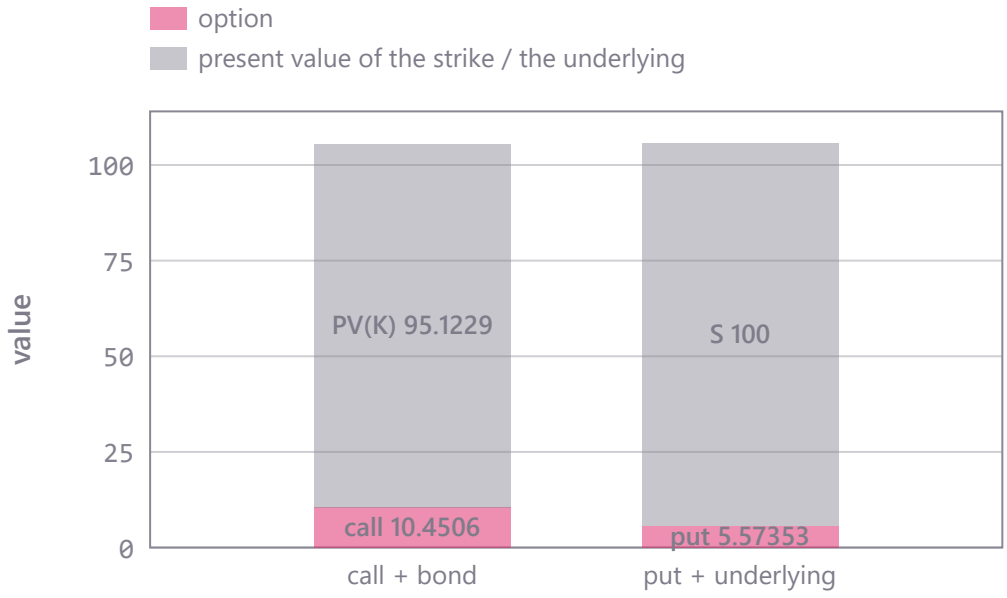
$$C + Ke^{-rT} = P + Se^{-qT}$$

input	meaning	condition
`call_price`	price of the European call	

input	meaning	condition
`spot`	price of the underlying now	
`strike`	strike (exercise) price	
`time`	time to expiry in years	
`rate`	risk-free rate, continuously compounded (decimal)	
`dividend_yield`	continuous dividend yield (decimal, default 0)	

output	meaning
`put_price`	European put price
`pv_strike`	K e^{-rT}
`pv_spot`	S e^{-qT}

- **Method:** $P = C - S\,e^{-qT} + K\,e^{-rT}$
- **Assumes:** European options, same strike and expiry; call within the no-arbitrage bounds
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Put–call parity: a call priced 10.4506 plus the present value of the strike (95.1229, returned) is worth the same as the put plus the underlying (5.57353 + 100, both returned), for strike 100, 1 year at 5 %; the two bars have equal height.

```
economics_finance_ai999x.solve('put_call_parity', call_price=10.450583572185565, spot=100,
strike=100, time=1, rate=0.05)
# -> put_price = 5.57353, pv_strike = 95.1229, pv_spot = 100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use put-call parity with call price = 10.45, spot = 100, strike = 100, time = 1, rate = 5%.	SOLVED	put_price = 5.57294, pv_strike = 95.1229, pv_spot = 100
Use put-call parity with call price = 150, spot = 100, strike = 100, time = 1, rate = 5%.	INVALID_INPUT	Put-call parity: the call price violates the no-arbitrage bounds max(0, S e^-qT - K e^-rT) < C < S e^-qT.

Binomial option price (Cox-Ross-Rubinstein) — `binomial_option`

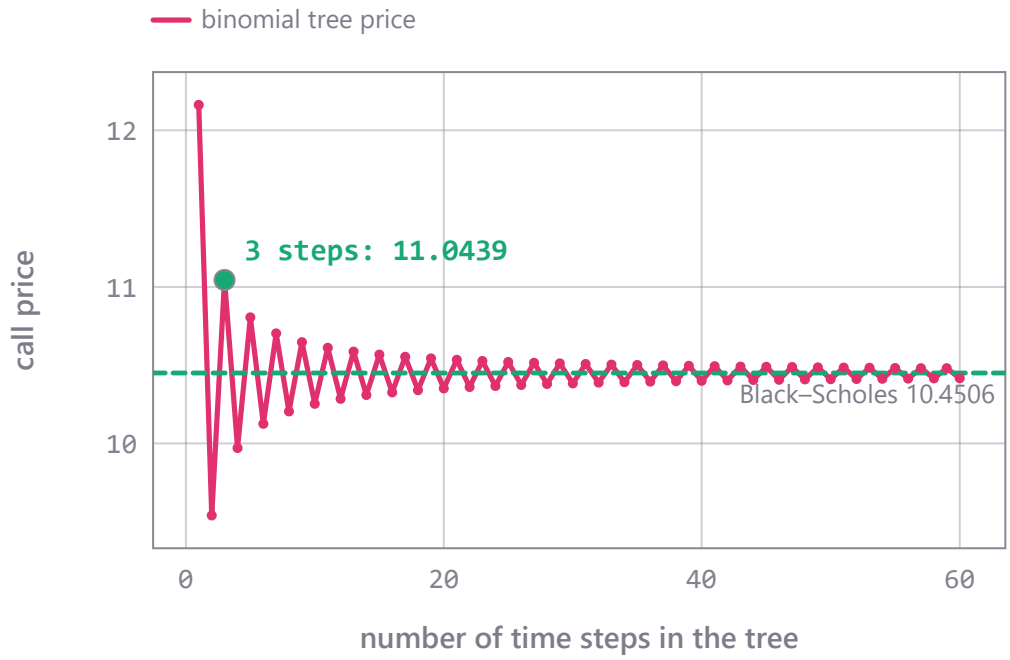
European call or put price on an n-step Cox-Ross-Rubinstein tree (u = e^(sigma sqrt(dt)), d = 1/u), with the delta of the first step.

$$u = e^{\sigma\sqrt{\Delta t}},\ d = 1/u,\ p = \frac{e^{(r-q)\Delta t} - d}{u - d},\ V = e^{-r\Delta t}[pV_u + (1-p)V_d]$$

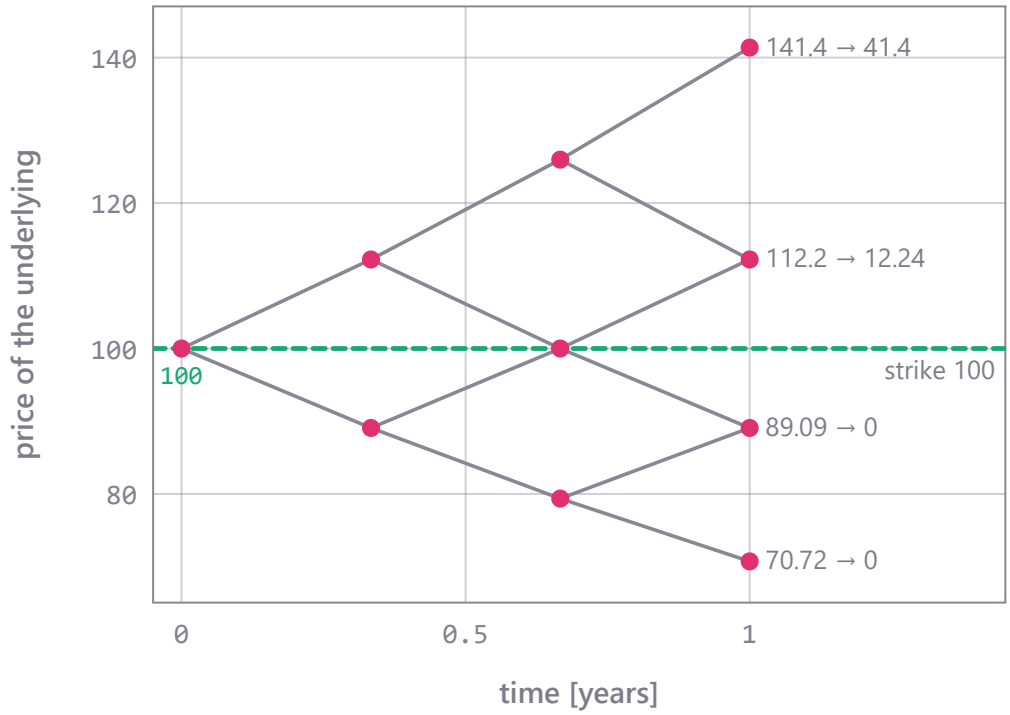
input	meaning	condition
`spot`	price of the underlying now	
`strike`	strike (exercise) price	
`time`	time to expiry in years	
`rate`	risk-free rate, continuously compounded (decimal)	
`volatility`	volatility of the underlying, annual (decimal)	
`steps`	number of time steps of the tree	
`option_type`	call (default) or put	
`dividend_yield`	continuous dividend yield (decimal, default 0)	

output	meaning
`price`	option price at time 0
`delta`	(V_u - V_d) / (S u - S d)
`up_factor`	u
`probability_up`	risk-neutral p = (e^((r-q)dt) - d)/(u - d)

- **Method:** backward induction V = e^(-r dt) (p $V_u p + (1-p)$ V_down) from the payoffs at expiry
- **Assumes:** European exercise; 1 <= steps <= 1000; $0 < p < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Price of a European call (spot 100, strike 100, 1 year, rate 5 %, volatility 20 %) from binomial trees of 1 to 60 steps, each a run of the package; the prices oscillate towards the Black–Scholes value the package returns (dashed). Marked: the worked example's 3 steps, 11.0439.



The 3-step price tree behind the binomial call (spot 100, strike 100, 1 year): each step multiplies the price by the up factor the package returns, u = 1.1224, or by 1/u (node prices computed here from u); the labels at expiry give each price and the payoff max(S − K, 0) for a call (max(K − S, 0) for a put). Risk-neutral up probability 0.543777; option price 11.0439, delta 0.614086, as returned.

```
economics_finance_ai999x.solve('binomial_option', spot=100, strike=100, time=1, rate=0.05,
volatility=0.2, steps=3)
# -> price = 11.0439, delta = 0.614086, up_factor = 1.1224, probability_up = 0.543777   (SOLVED)
```

```
economics_finance_ai999x.solve('binomial_option', spot=100, strike=100, time=1, rate=0.05,
volatility=0.2, steps=200, option_type='put')
# -> price = 5.56353, delta = -0.363329, up_factor = 1.01424, probability_up = 0.505304   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Price the option with a binomial tree with spot = 100, strike = 100, time = 1, rate = 5%, volatility = 20%, steps = 3.	SOLVED	price = 11.0439, delta = 0.614086, up_factor = 1.1224, probability_up = 0.543777
Price the option with a binomial tree with spot = 100, strike = 100, time = 1, rate = 5%, volatility = 20%, steps = 2.5.	INVALID_INPUT	steps: expected a whole number written exactly, got 2.5.

Forward price with cost of carry — `forward_price`

No-arbitrage forward price of an asset with a continuous income yield q and storage cost u: S e^((r + u - q) T).

$$F_0 = S_0\, e^{(r+u-q)T}$$

input	meaning	condition
`spot`	spot price now	
`rate`	risk-free rate, continuously compounded (decimal)	
`time`	time to delivery in years	
`dividend_yield`	continuous income or convenience yield (decimal, default 0)	
`storage_cost`	continuous storage cost rate (decimal, default 0)	

output	meaning
`forward`	forward price
`cost_of_carry`	r + u - q

- **Method:** $F = S$ exp((r + u - q) T)
- **Assumes:** spot > 0; time >= 0; rates as continuously compounded decimals
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Forward price of an asset with spot 100 at a rate of 5 % for deliveries up to 3 years, with dividend yields 0 %, 2 %, 7 % (60 runs of the package each): the forward rises above spot when the rate exceeds the yield and falls below it otherwise. Marked: the worked example, 0.5 year, forward 101.511 (cost of carry 3 %).

```
economics_finance_ai999x.solve('forward_price', spot=100, rate=0.05, time=0.5, dividend_yield=0.02)
# -> forward = 101.511, cost_of_carry = 0.03   (SOLVED)
```

```
economics_finance_ai999x.solve('forward_price', spot=100, rate=0.05, time=0)
# -> forward = 100, cost_of_carry = 0.05   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the forward price with spot = 100, rate = 5%, time = 0.5, dividend yield = 2%.	SOLVED	forward = 101.511, cost_of_carry = 0.03
Find the futures price with spot = 1800, r = 4%, t = 1, storage cost = 1%.	SOLVED	forward = 1892.29, cost_of_carry = 0.05

Forward price with known dividends — `forward_price_dividends`

Forward price of a stock paying known cash dividends before delivery: F = (S - PV(dividends)) e^{rT}.

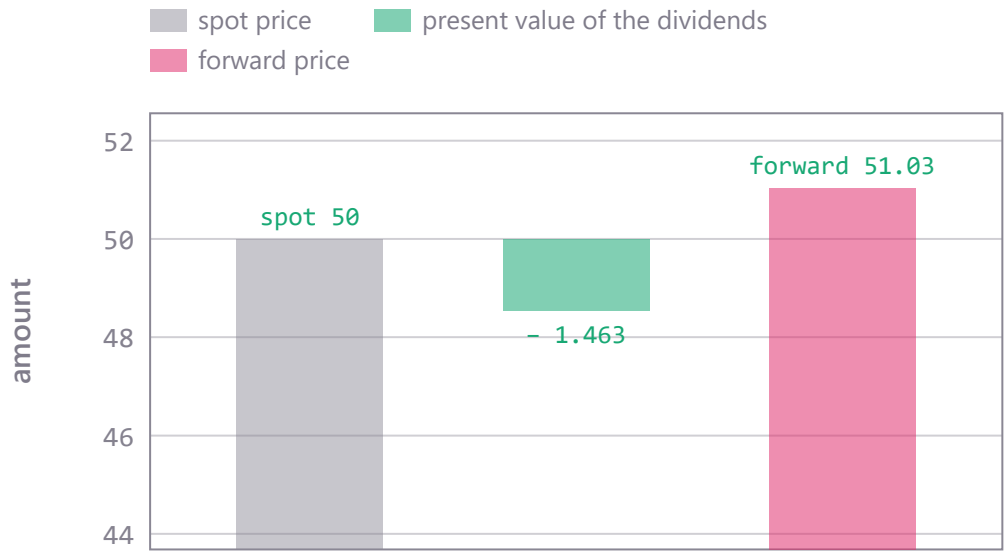
$$F_0 = \left(S_0 - \sum_i D_i e^{-r t_i}\right) e^{rT}$$

input	meaning	condition
`spot`	stock price now	
`rate`	risk-free rate, continuously compounded (decimal)	
`time`	time to delivery in years	
`dividends`	cash dividends (list)	

input	meaning	condition
`dividend_times`	time of each dividend in years (list)	

output	meaning
`forward`	forward price
`pv_dividends`	present value of the dividends

- **Method:** F = (S - sum D_i e^(-r t_i)) e^{rT}
- **Assumes:** 0 < each dividend time <= time; PV(dividends) < spot
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Forward price for delivery in 1 year(s) of an asset priced 50 that pays dividends 0.75, 0.75 at times 0.25, 0.75: the package returns the present value of the dividends, 1.46308, which is taken off the spot, and the forward price 51.0255, the remainder carried at 5 %. The value axis does not start at zero.

```
economics_finance_ai999x.solve('forward_price_dividends', spot=50, rate=0.05, time=1,
dividends='0.75, 0.75', dividend_times='0.25, 0.75')
# -> forward = 51.0255, pv_dividends = 1.46308   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the forward price with dividends with spot = 50, rate = 5%, time = 1, dividends = [0.75, 0.75], dividend times = [0.25, 0.75].	SOLVED	forward = 51.0255, pv_dividends = 1.46308
Find the forward price with dividends with spot = 50, rate = 5%, time = 1, dividends = [0.75], dividend times = [1.5].	INVALID_INPUT	Forward price with known dividends: each dividend time must lie in (0, time].

Survival and death probabilities from a life table — `survival_probability`

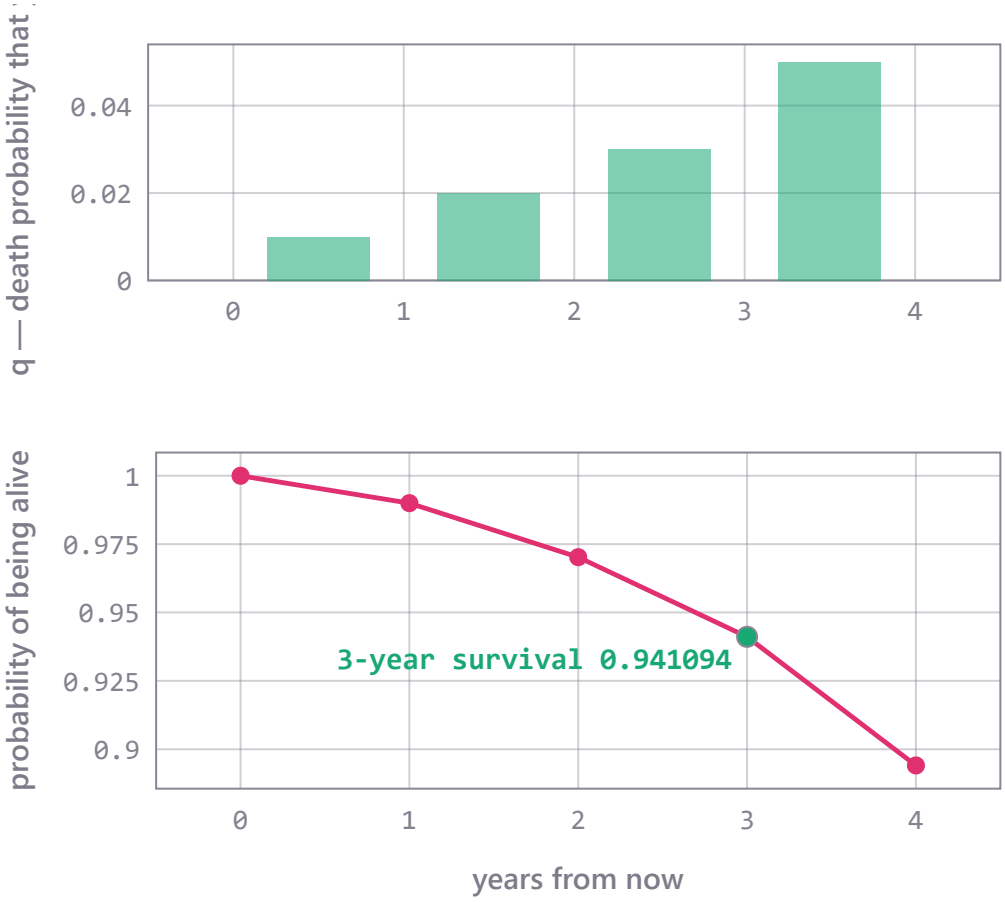
t p_x (survive t years), t q_x (die within t years) and t|u q_x (survive t years, then die within the next u years) from one-year mortality rates.

$$ {}_tp_x = \prod_{k=0}^{t-1}(1 - q_{x+k}), \quad {}_{t|u}q_x = {}_tp_x - {}_{t+u}p_x $$

input	meaning	condition
`mortality_rates`	one-year death probabilities q_x, $q_x + 1$, ... from the current age	
`years`	number of years t	
`deferred_years`	u, the length of the death window after t (default 1)	

output	meaning
`survival`	t p_x = prod $(1 - q)$
`death_probability`	t $q_x = 1 - t$ p_x
`deferred_death_probability`	t|u $q_x = t$ p_x - (t+u) p_x

- **Method:** t p_x = prod_{$k < t$} (1 - q_{x+k})
- **Assumes:** $0 \leq q \leq 1$; t + u <= number of rates
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: the one-year death probabilities 0.01, 0.02, 0.03, 0.05 for each year from the current age (stated). Bottom: the probability of surviving t years, a run of the package for t = 0 ... 3 (the last point is the survival to year 3 less the returned probability of dying in that year). Marked: t = 3: survival 0.941094, death within 3 years 0.058906.

```
economics_finance_ai999x.solve('survival_probability', mortality_rates='0.01, 0.02, 0.03, 0.05',
years=3)
# -> survival = 0.941094, death_probability = 0.058906, deferred_death_probability = 0.0470547
(SOLVED)
```

```
economics_finance_ai999x.solve('survival_probability', mortality_rates=[0.1, 0.2], years=1,
deferred_years=1)
# -> survival = 0.9, death_probability = 0.1, deferred_death_probability = 0.18   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the survival probability with mortality rates = [0.01, 0.02, 0.03, 0.05], years = 3.	SOLVED	survival = 0.941094, death_probability = 0.058906, deferred_death_probability = 0.0470547
Find the survival probability with q = [0.01, 0.02], t = 5.	INVALID_INPUT	Survival and death probabilities from a life table: years must be at most 2.

Life expectancy from a life table — `life_expectancy`

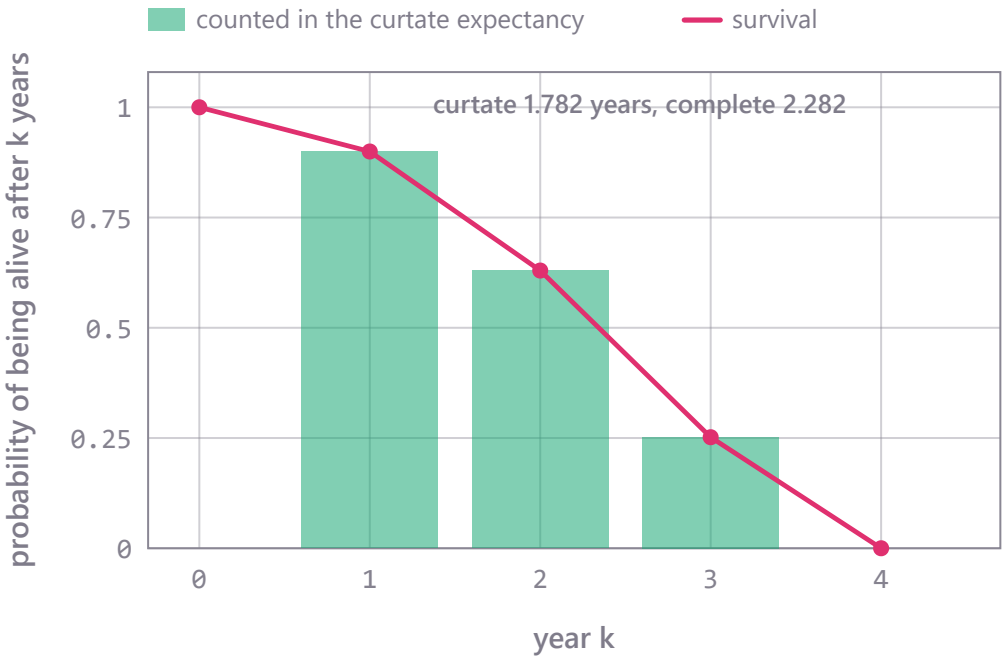
Curtate expectation of life e_x = sum_{$k \geq 1$} k p_x (whole years lived) and the complete expectation $e_x + 1/2$ (deaths uniform within each year).

$$e_x = \sum_{k=1}^{\infty} {}_k p_x$$

input	meaning	condition
`mortality_rates`	one-year death probabilities q_x, $q_x + 1$, ... from the current age	

output	meaning
`curtate`	curtate expectation of life e_x
`complete`	complete expectation $e_x + 0.5$ (UDD)

- **Method:** e_x = sum_{$k = 1$}^{n} k p_x
- **Assumes:** the table ends with $q = 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Survival for the one-year death probabilities 0.1, 0.3, 0.6, 1 (each value a run of the package's survival probability). The curtate life expectancy the package returns, 1.782 years, is the sum of the bars (whole years lived); the complete expectancy 2.282 adds half a year.

```
economics_finance_ai999x.solve('life_expectancy', mortality_rates='0.1, 0.3, 0.6, 1')
# -> curtate = 1.782, complete = 2.282   (SOLVED)
```

```
economics_finance_ai999x.solve('life_expectancy', mortality_rates=[1.0])
# -> curtate = 0, complete = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the life expectancy with mortality rates = [0.1, 0.3, 0.6, 1].	SOLVED	curtate = 1.782, complete = 2.282
Find the life expectancy with mortality rates = [0.1, 0.3, 0.6].	INVALID_INPUT	Life expectancy from a life table: the table must end with q = 1 (no one survives past its last age).

Whole life insurance (net single premium) — `whole_life_insurance`

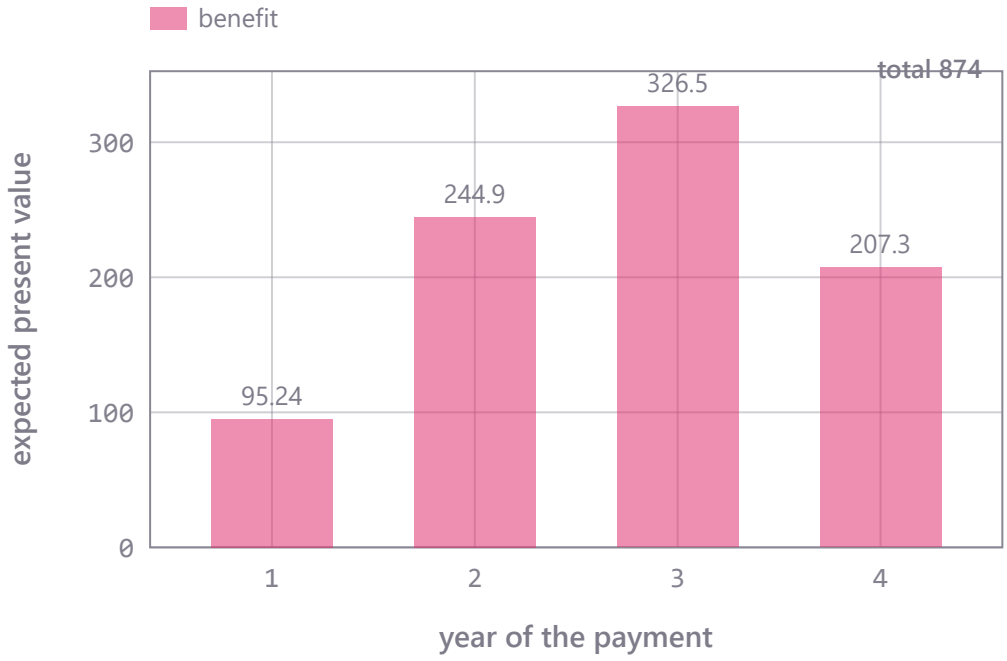
Actuarial present value A_x of a benefit paid at the end of the year of death, whenever it occurs.

$$A_x = \sum_{k=0}^{\infty} v^{k+1}\, {}_kp_x\, q_{x+k}$$

input	meaning	condition
`mortality_rates`	one-year death probabilities q_x, q_x + 1, ... from the current age	
`interest_rate`	annual effective interest rate (decimal)	
`benefit`	benefit amount (default 1)	

output	meaning
apv	B $A_x = B$ sum v^{k+1} k p_x q_x+k

- **Method:** A_x = sum_k v^{k+1} k p_x q_(x+k)
- **Assumes:** the table ends with $q = 1$; -1 < interest_rate < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A benefit of 1000 paid at the end of the year of death, for death probabilities 0.1, 0.3, 0.6, 1 and interest 5 %. Each bar is the amount × the probability of payment that year (from the package's survival probabilities) × the discount factor (from the package's present value); the bars sum to the returned actuarial present value 873.988.



apv (B A_x = B sum v^(k+1) k p_x q_x+k) against interest_rate (annual effective interest rate (decimal)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of benefit (benefit amount (default 1)): 1000, 500, 2000. Other inputs as in the worked example: mortality_rates = 0.1, 0.3, 0.6, 1. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, interest_rate = 0.05, apv = 873.988.

```
economics_finance_ai999x.solve('whole_life_insurance', mortality_rates='0.1, 0.3, 0.6, 1',
interest_rate=0.05, benefit=1000)
# -> apv = 873.988   (SOLVED)
```

```
economics_finance_ai999x.solve('whole_life_insurance', mortality_rates=[1.0], interest_rate=0.25)
# -> apv = 0.8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the whole life insurance value with mortality rates = [0.1, 0.3, 0.6, 1], interest rate = 5%, benefit = 1000.	SOLVED	apv = 873.988

Term life insurance (net single premium) — `term_insurance`

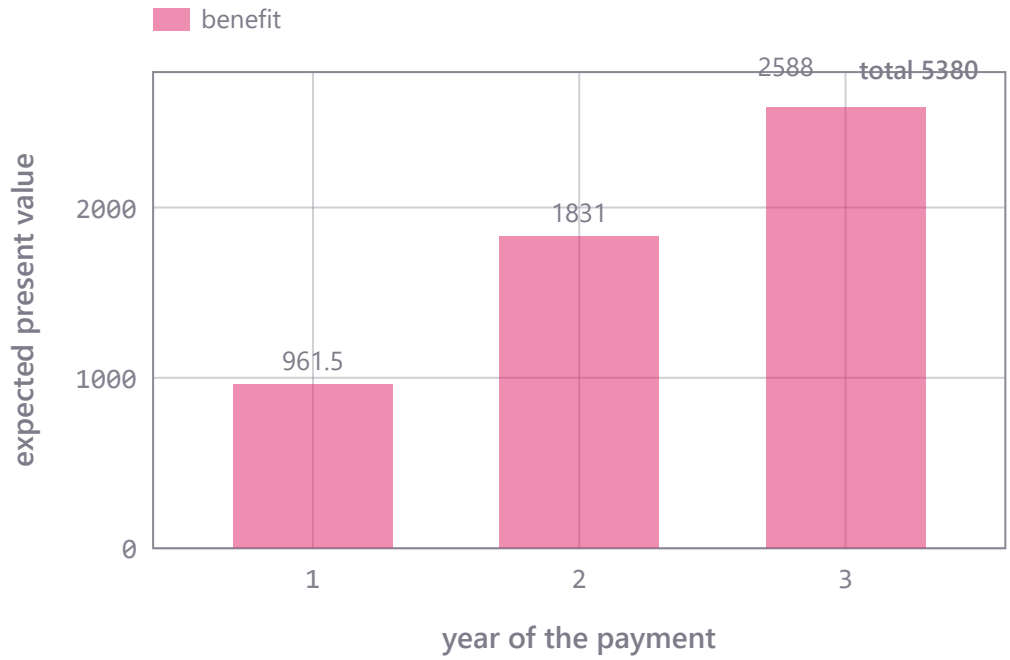
Actuarial present value of a benefit paid at the end of the year of death if death occurs within n years.

$$A^{1}_{x:\overline{n}|} = \sum_{k=0}^{n-1} v^{k+1}\ {}_{k}p_{x}\ q_{x+k}$$

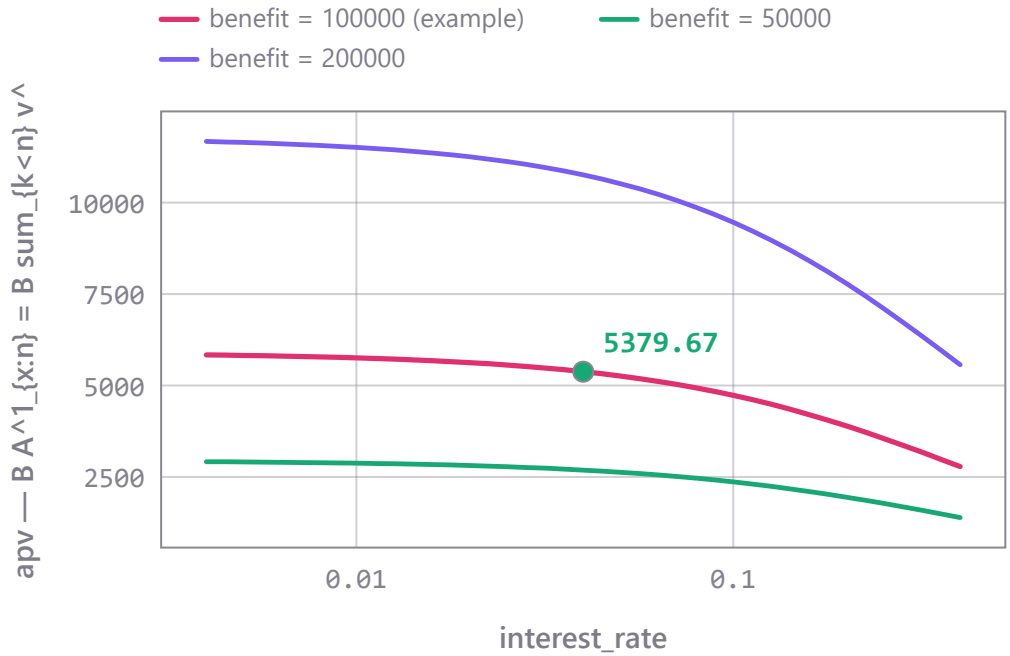
input	meaning	condition
`mortality_rates`	one-year death probabilities q_x, $q_x + 1$, ... from the current age	
`interest_rate`	annual effective interest rate (decimal)	
`term`	term n in years	
`benefit`	benefit amount (default 1)	

output	meaning
`apv`	B Missing superscript or subscript argument{x:n} = B sum_{$k < n$} v^{k+1} k p_x q_x+k

- **Method:** Missing superscript or subscript argument{x:n} = sum_{$k = 0$}^{$n - 1$} v^{k+1} k p_x q_(x+k)
- **Assumes:** 1 <= term <= number of rates
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A benefit of 100000 paid at the end of the year of death within 3 years, for death probabilities 0.01, 0.02, 0.03 and interest 4 %. Each bar is the amount × the probability of payment that year (from the package's survival probabilities) × the discount factor (from the package's present value); the bars sum to the returned actuarial present value 5379.67.



apv (B A^1_{x:n} = B sum_{k<n} v^(k+1) k p_x q_x+k) against interest_rate (annual effective interest rate (decimal)) from 0.004 to 0.4 on a logarithmic axis. One curve for each value of benefit (benefit amount (default 1)): 100000, 50000, 200000. Other inputs as in the worked example: mortality_rates = 0.01, 0.02, 0.03, term = 3. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, interest_rate = 0.04, apv = 5379.67.

```
economics_finance_ai999x.solve('term_insurance', mortality_rates='0.01, 0.02, 0.03',
interest_rate=0.04, term=3, benefit=100000)
# -> apv = 5379.67   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the term insurance value with mortality rates = [0.01, 0.02, 0.03], interest rate = 4%, term = 3, benefit = 100000.	SOLVED	apv = 5379.67
Find the term insurance value with q = [0.01, 0.02], i = 4%, n = 3.	INVALID_INPUT	Term life insurance (net single premium): term must be at most 2.

Whole life annuity-due — `life_annuity_due`

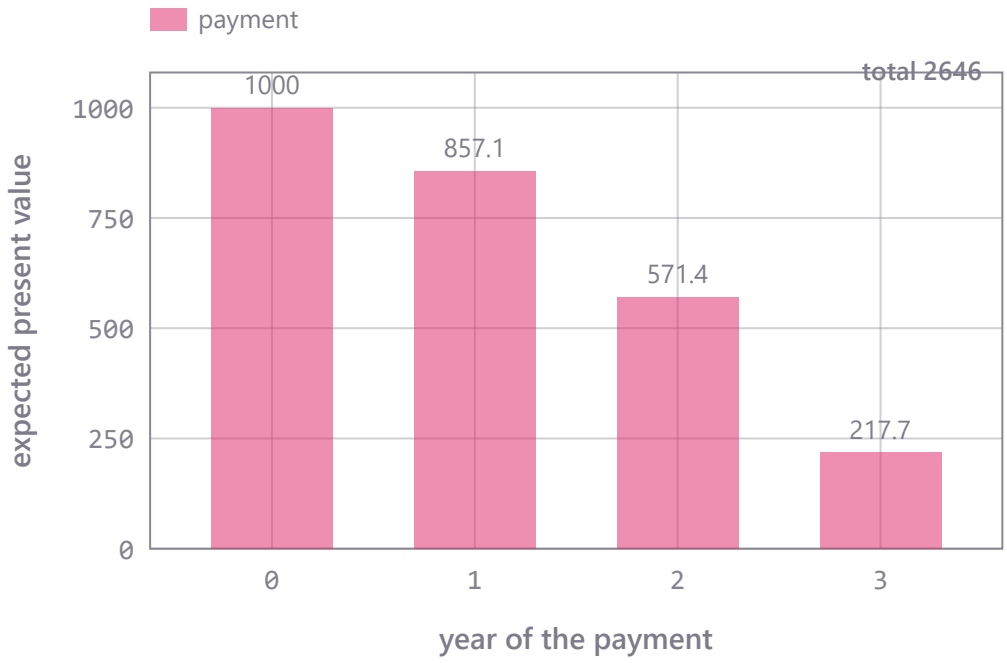
Actuarial present value of a payment at the start of every year while the person is alive.

$$\ddot{a}_x = \sum_{k=0}^{\infty} v^k \, {}_kp_x$$

input	meaning	condition
`mortality_rates`	one-year death probabilities q_x, $q_x + 1$, ... from the current age	
`interest_rate`	annual effective interest rate (decimal)	
`payment`	annual payment (default 1)	

output	meaning
`apv`	P a-due_x = P sum v^k k p_x

- **Method:** a-due_x = sum_k v^k k p_x
- **Assumes:** the table ends with $q = 1$; -1 < interest_rate < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Payments of 1000 at the start of each year while alive, for death probabilities 0.1, 0.3, 0.6, 1 and interest 5 %. Each bar is the amount × the probability of payment that year (from the package's survival probabilities) × the discount factor (from the package's present value); the bars sum to the returned actuarial present value 2646.26.



apv (P a-due_x = P sum v^k k p_x) against interest_rate (annual effective interest rate (decimal)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of payment (annual payment (default 1)): 1000, 500, 2000. Other inputs as in the worked example: mortality_rates = 0.1, 0.3, 0.6, 1. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, interest_rate = 0.05, apv = 2646.26.

```
economics_finance_ai999x.solve('life_annuity_due', mortality_rates='0.1, 0.3, 0.6, 1',
interest_rate=0.05, payment=1000)
# -> apv = 2646.26   (SOLVED)
```

```
economics_finance_ai999x.solve('life_annuity_due', mortality_rates=[1.0], interest_rate=0.05)
# -> apv = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the life annuity-due value with mortality rates = [0.1, 0.3, 0.6, 1], interest rate = 5%, payment = 1000.	SOLVED	apv = 2646.26

Temporary life annuity-due — `temporary_annuity_due`

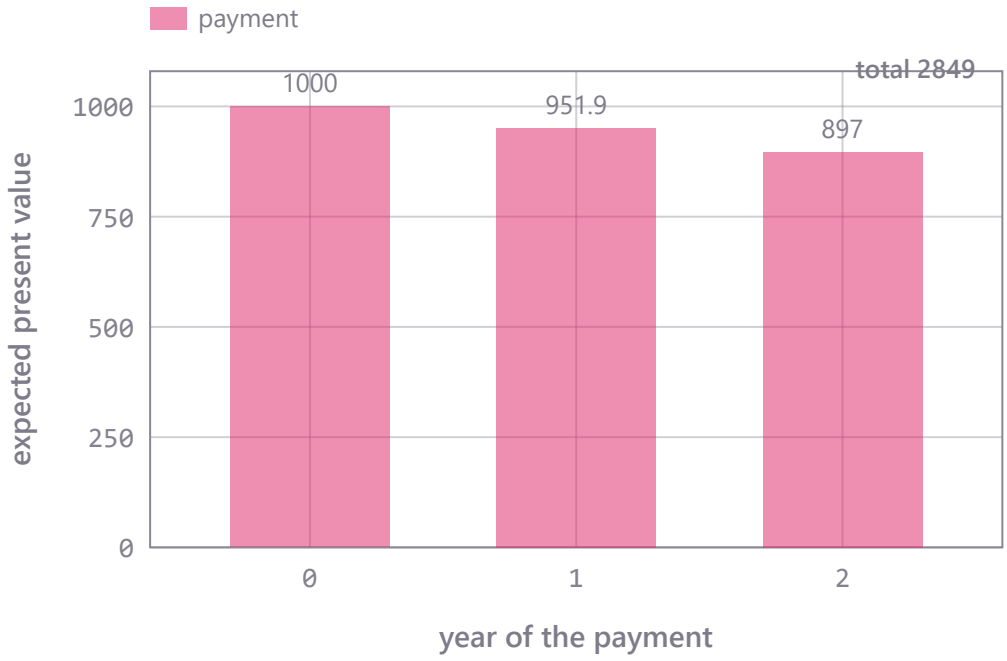
Actuarial present value of up to n annual payments at the start of each year while alive.

$$\ddot{a}_{x:\overline{n}|} = \sum_{k=0}^{n-1} v^k \, {}_kp_x$$

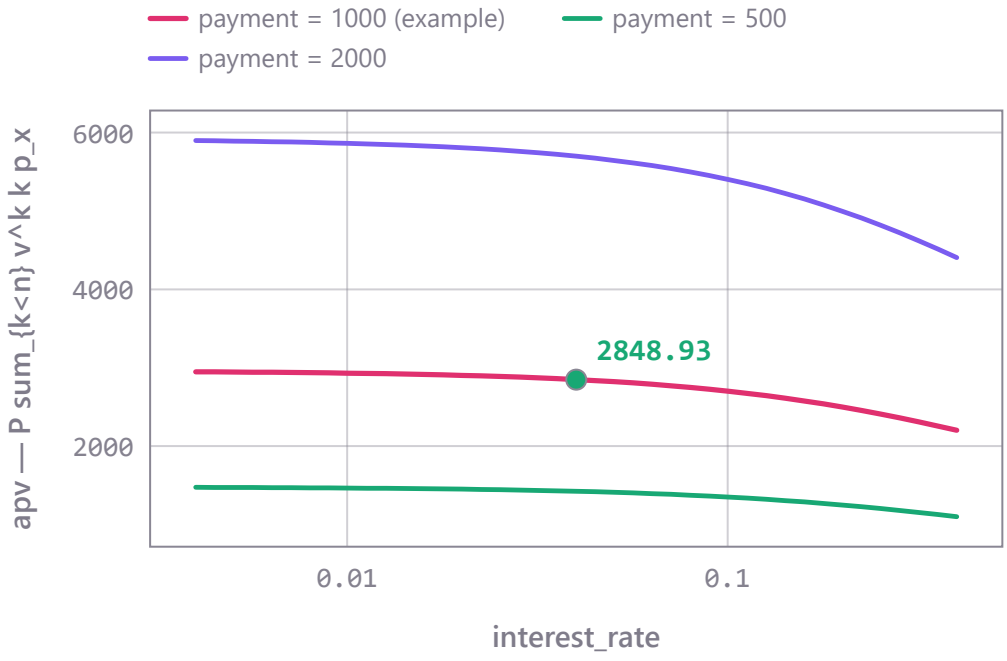
input	meaning	condition
`mortality_rates`	one-year death probabilities q_x, $q_x + 1$, ... from the current age	
`interest_rate`	annual effective interest rate (decimal)	
`term`	term n in years	
`payment`	annual payment (default 1)	

output	meaning
apv	P sum_{$k < n$} v^k k p_x

- **Method:** a-due_{x:n} = sum_{$k = 0$}^{$n - 1$} v^k k p_x
- **Assumes:** 1 <= term <= number of rates
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Payments of 1000 at the start of each of 3 years while alive, for death probabilities 0.01, 0.02, 0.03 and interest 4 %. Each bar is the amount × the probability of payment that year (from the package's survival probabilities) × the discount factor (from the package's present value); the bars sum to the returned actuarial present value 2848.93.



apv (P sum_{k<n} v^k k p_x) against interest_rate (annual effective interest rate (decimal)) from 0.004 to 0.4 on a logarithmic axis. One curve for each value of payment (annual payment (default 1)): 1000, 500, 2000. Other inputs as in the worked example: mortality_rates = 0.01, 0.02, 0.03, term = 3. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, interest_rate = 0.04, apv = 2848.93.

```
economics_finance_ai999x.solve('temporary_annuity_due', mortality_rates='0.01, 0.02, 0.03',
interest_rate=0.04, term=3, payment=1000)
# -> apv = 2848.93   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the temporary annuity value with mortality rates = [0.01, 0.02, 0.03], interest rate = 4%, term = 3, payment = 1000.	SOLVED	apv = 2848.93

Net annual premium for whole life insurance — `net_premium_whole_life`

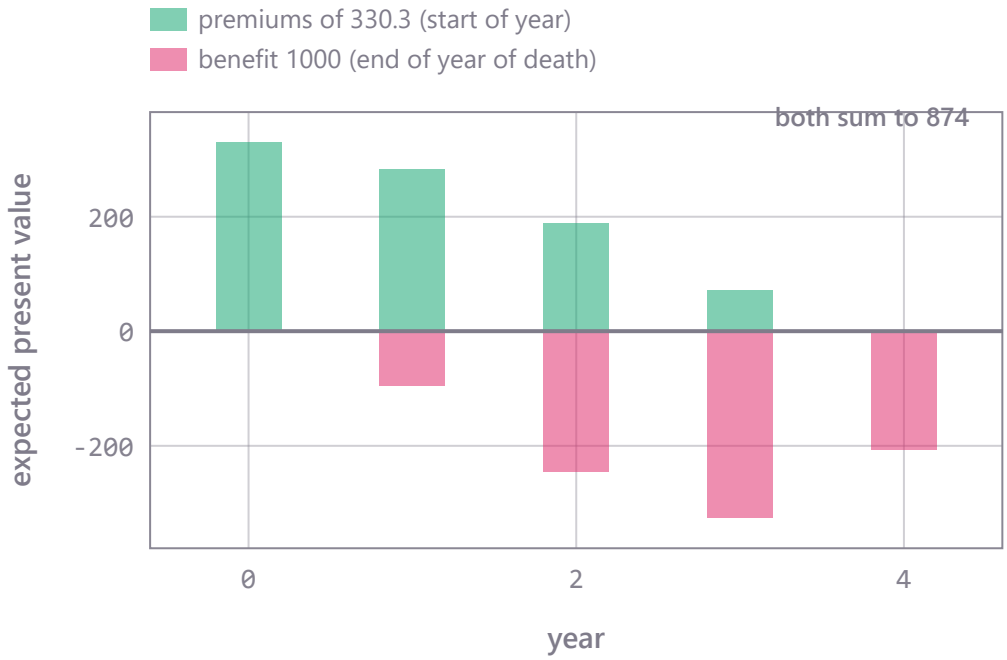
The level annual net premium, paid at the start of each year while alive, for a whole life benefit (equivalence principle): $P = B$ A_x / a-due_x.

$$P = B\,\frac{A_x}{\ddot{a}_x}$$

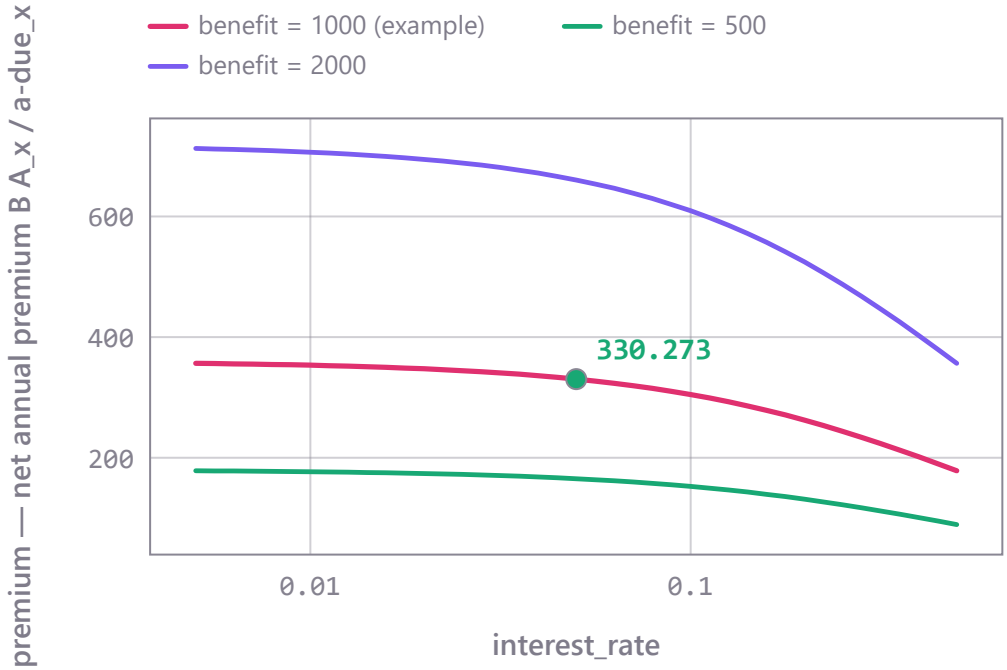
input	meaning	condition
`mortality_rates`	one-year death probabilities q_x, $q_x + 1$, ... from the current age	
`interest_rate`	annual effective interest rate (decimal)	
`benefit`	benefit amount (default 1)	

output	meaning
`premium`	net annual premium B A_x / a-due_x
`insurance_apv`	B A_x
`annuity_due`	a-due_x

- **Method:** equivalence principle: P a-due_x = B A_x
- **Assumes:** the table ends with $q = 1$; 0 < interest_rate < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Whole life insurance of 1000 (death probabilities 0.1, 0.3, 0.6, 1, interest 5 %) paid for by the net annual premium the package returns, 330.273. Up: expected present value of each premium (premium × survival × discount); down: of each year's benefit (benefit × death probability × discount); survival and discount factors are runs of the package. Both sum to the returned insurance value 873.988 (= premium × annuity-due 2.64626).



premium (net annual premium B A_x / a-due_x) against interest_rate (annual effective interest rate (decimal)) from 0.005 to 0.5 on a logarithmic axis. One curve for each value of benefit (benefit amount (default 1)): 1000, 500, 2000. Other inputs as in the worked example: mortality_rates = 0.1, 0.3, 0.6, 1. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, interest_rate = 0.05, premium = 330.273.

```
economics_finance_ai999x.solve('net_premium_whole_life', mortality_rates='0.1, 0.3, 0.6, 1',
interest_rate=0.05, benefit=1000)
# -> premium = 330.273, insurance_apv = 873.988, annuity_due = 2.64626   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the net annual premium with mortality rates = [0.1, 0.3, 0.6, 1], interest rate = 5%, benefit = 1000.	SOLVED	premium = 330.273, insurance_apv = 873.988, annuity_due = 2.64626

Endowment insurance — `endowment_insurance`

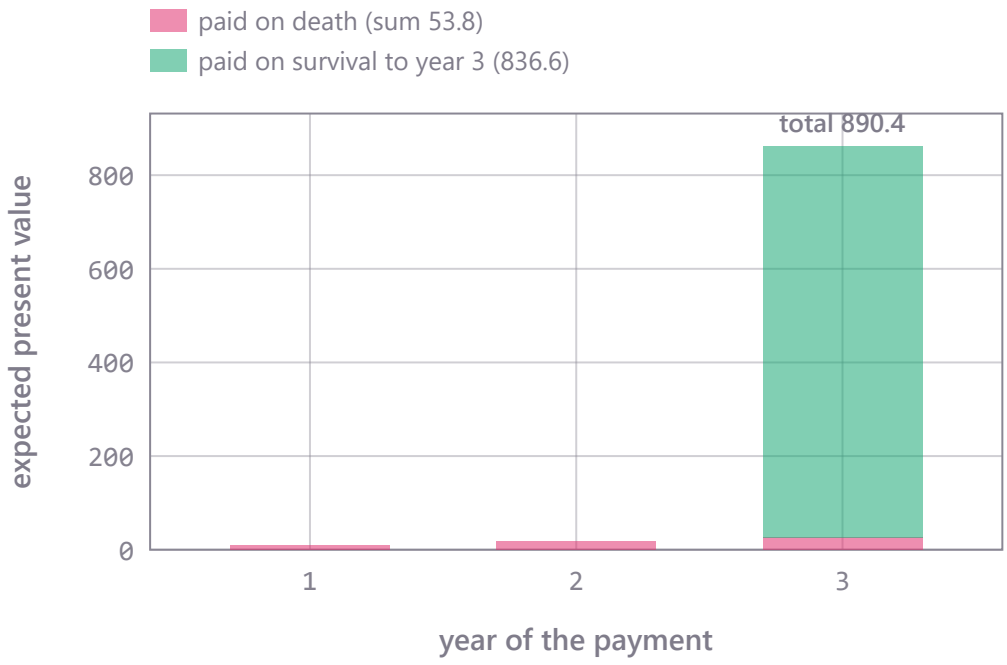
Actuarial present value of a benefit paid at the end of the year of death within n years, or at year n if alive then (term insurance + pure endowment).

$$A_{x:\overline{n}|} = A^{1}_{x:\overline{n}|} + v^{n}\,{}_{n}p_{x}$$

input	meaning	condition
`mortality_rates`	one-year death probabilities q_x, $q_x + 1$, ... from the current age	
`interest_rate`	annual effective interest rate (decimal)	
`term`	term n in years	
`benefit`	benefit amount (default 1)	

output	meaning
`apv`	B (A^1_{x:n} + nE_x)
`term_part`	B Missing superscript or subscript argument{x:n}
`pure_endowment`	B v^n n p_x

- **Method:** A_{x:n} = Missing superscript or subscript argument{x:n} + v^n n p_x
- **Assumes:** 1 <= term <= number of rates
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



An endowment of 1000 paid at the end of the year of death within 3 years, or at year 3 on survival (death probabilities 0.01, 0.02, 0.03, interest 4 %). The death bars are benefit × death probability that year (differences of the package's survival probabilities) × discount factor and sum to the returned term part 53.7967; stacked on the last year: the returned pure endowment 836.629. Total 890.426, as returned.



apv (B (A^1_{x:n} + nE_x)) against interest_rate (annual effective interest rate (decimal)) from 0.004 to 0.4 on a logarithmic axis. One curve for each value of benefit (benefit amount (default 1)): 1000, 500, 2000. Other inputs as in the worked example: mortality_rates = 0.01, 0.02, 0.03, term = 3. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, interest_rate = 0.04, apv = 890.426.

```
economics_finance_ai999x.solve('endowment_insurance', mortality_rates='0.01, 0.02, 0.03',
interest_rate=0.04, term=3, benefit=1000)
# -> apv = 890.426, term_part = 53.7967, pure_endowment = 836.629   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the endowment insurance value with mortality rates = [0.01, 0.02, 0.03], interest rate = 4%, term = 3, benefit = 1000.	SOLVED	apv = 890.426, term_part = 53.7967, pure_endowment = 836.629

Price elasticity of demand (midpoint) — `arc_price_elasticity`

Arc (midpoint) price elasticity of demand between two price-quantity points, and whether demand is elastic, inelastic or unit elastic there.

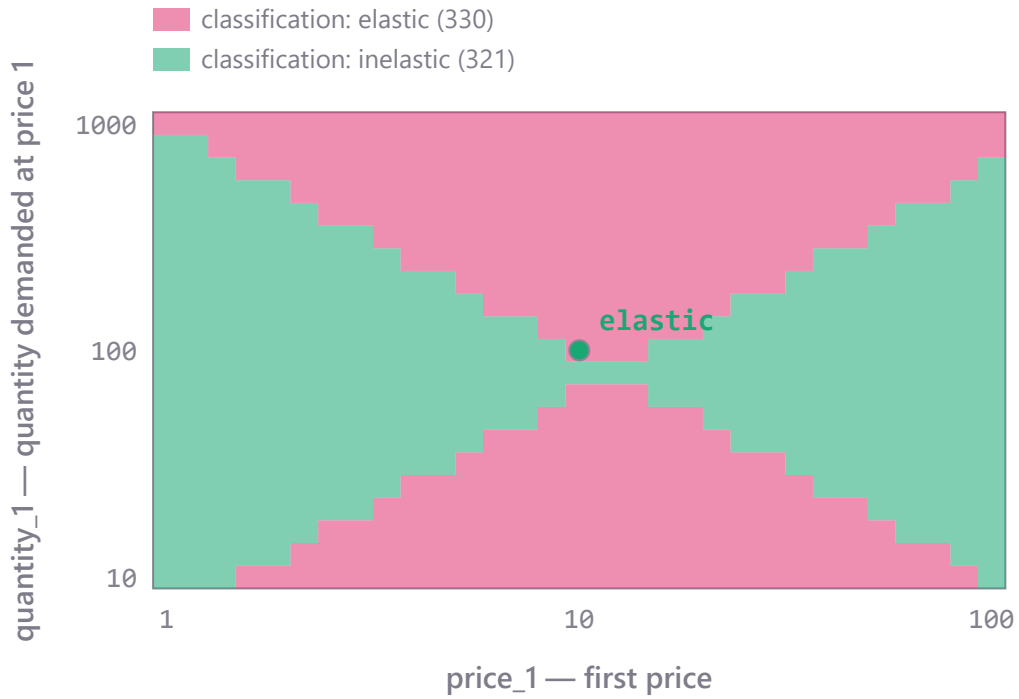
$$E = \frac{(Q_2 - Q_1)/[(Q_1 + Q_2)/2]}{(P_2 - P_1)/[(P_1 + P_2)/2]}$$

input	meaning	condition
`price_1`	first price	
`quantity_1`	quantity demanded at price 1	
`price_2`	second price	
`quantity_2`	quantity demanded at price 2	

output	meaning
`elasticity`	(dQ / mean Q) / (dP / mean P)

output	meaning
`abs_elasticity`	|elasticity|
`classification`	elastic, inelastic, unit elastic or perfectly inelastic

- **Method:** midpoint formula: percentage changes relative to the averages
- **Assumes:** prices and quantities > 0; the two prices differ
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



elastic, inelastic, unit elastic or perfectly inelastic (classification) over price_1 (first price) from 1 to 100 and quantity_1 (quantity demanded at price 1) from 10 to 1000 (logarithmic axes): each of the 31 × 21 cells is one run of economics-finance-ai999x, coloured by the verdict it returned — elastic: 330 cells, inelastic: 321 cells. Other inputs as in the worked example: price_2 = 12, quantity_2 = 80. Marked: the worked example, price_1 = 10, quantity_1 = 100 → elastic.



elasticity ((dQ / mean Q) / (dP / mean P)) against price_1 (first price) from 1 to 11.2202 on a logarithmic axis. One curve for each value of quantity_1 (quantity demanded at price 1): 100, 50, 200. Other inputs as in the worked example: price_2 = 12, quantity_2 = 80. 22 points per curve computed by economics-finance-ai999x; marked: the worked example, price_1 = 10, elasticity = -1.22222.

```
economics_finance_ai999x.solve('arc_price_elasticity', price_1=10, quantity_1=100, price_2=12, quantity_2=80)
# -> elasticity = -1.22222, abs_elasticity = 1.22222, classification = elastic   (SOLVED)
```

```
economics_finance_ai999x.solve('arc_price_elasticity', price_1=10, quantity_1=12, price_2=12, quantity_2=10)
# -> elasticity = -1, abs_elasticity = 1, classification = unit elastic   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the price elasticity of demand with p1 = 10, q1 = 100, p2 = 12, q2 = 80.	SOLVED	elasticity = -1.22222, abs_elasticity = 1.22222, classification = elastic
Find the price elasticity of demand with p1 = 10, q1 = 100, p2 = 10, q2 = 80.	INVALID_INPUT	Price elasticity of demand (midpoint): the two prices must differ.

Income elasticity of demand (midpoint) — `income_elasticity`

Midpoint income elasticity of demand and the good's type (inferior, necessity, luxury).

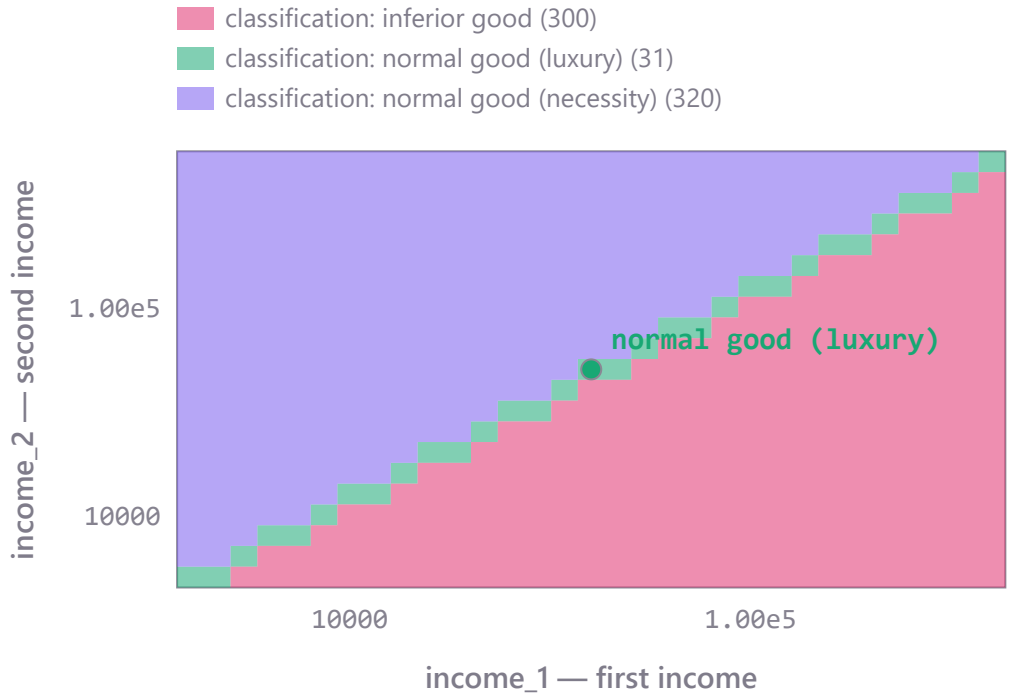
$$E_I = \frac{(Q_2 - Q_1)/\bar{Q}}{(I_2 - I_1)/\bar{I}}$$

input	meaning	condition
`income_1`	first income	
`quantity_1`	quantity at income 1	

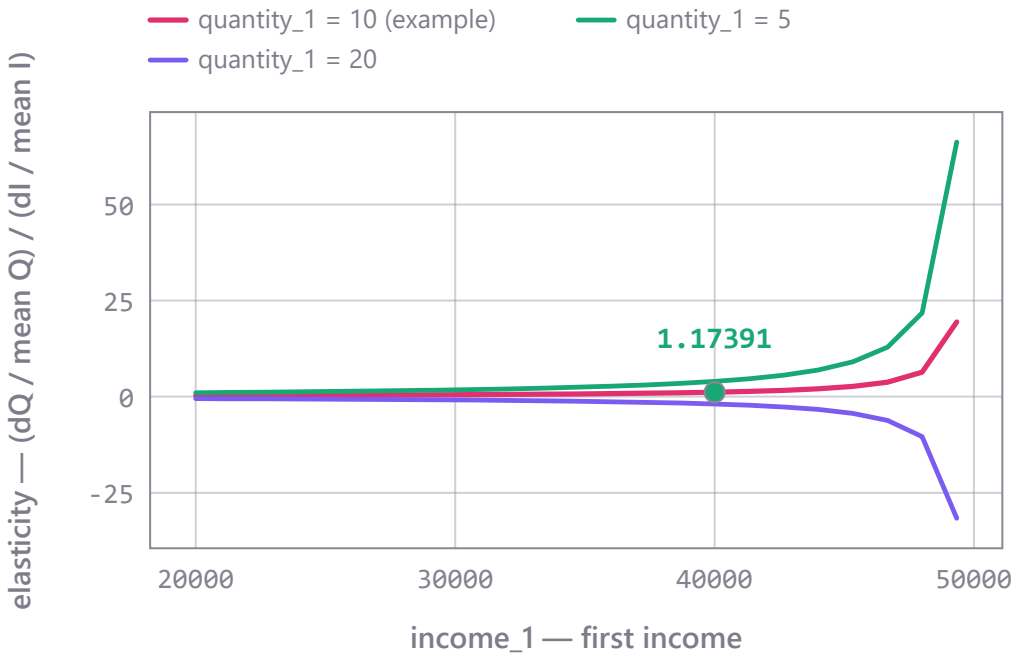
input	meaning	condition
`income_2`	second income	
`quantity_2`	quantity at income 2	

output	meaning
`elasticity`	(dQ / mean Q) / (dI / mean I)
`classification`	inferior, necessity or luxury

- **Method:** midpoint formula
- **Assumes:** incomes and quantities > 0; the incomes differ
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



inferior, necessity or luxury (classification) over income_1 (first income) from 4000 to 400000 and income_2 (second income) from 5000 to 500000 (logarithmic axes): each of the 31 × 21 cells is one run of economics-finance-ai999x, coloured by the verdict it returned — inferior good: 300 cells, normal good (luxury): 31 cells, normal good (necessity): 320 cells. Other inputs as in the worked example: quantity_1 = 10, quantity_2 = 13. Marked: the worked example, income_1 = 40000, income_2 = 50000 → normal good (luxury).



elasticity ((dQ / mean Q) / (dI / mean I)) against income_1 (first income) from 20000 to 49333.3. One curve for each value of quantity_1 (quantity at income 1): 10, 5, 20. Other inputs as in the worked example: income_2 = 50000, quantity_2 = 13. 23 points per curve computed by economics-finance-ai999x; marked: the worked example, income_1 = 40000, elasticity = 1.17391.

```
economics_finance_ai999x.solve('income_elasticity', income_1=40000, quantity_1=10, income_2=50000, quantity_2=13)
# -> elasticity = 1.17391, classification = normal good (luxury)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the income elasticity with income 1 = 40000, q1 = 10, income 2 = 50000, q2 = 13.	SOLVED	elasticity = 1.17391, classification = normal good (luxury)

Cross-price elasticity of demand (midpoint) — `cross_price_elasticity`

Midpoint cross-price elasticity: response of the quantity of good A to the price of good B; positive = substitutes, negative = complements.

$$E_{AB} = \frac{\Delta Q_A / \bar{Q}_A}{\Delta P_B / \bar{P}_B}$$

input	meaning	condition
`price_other_1`	first price of the other good	
`quantity_1`	quantity of this good at that price	
`price_other_2`	second price of the other good	
`quantity_2`	quantity of this good at the second price	

output	meaning
`elasticity`	(dQa / mean Qa) / (dPb / mean Pb)
`relationship`	substitutes, complements or independent

- **Method:** midpoint formula
- **Assumes:** prices and quantities > 0; the prices differ
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



substitutes, complements or independent (relationship) over price_other_1 (first price of the other good) from 0.2 to 20 and quantity_1 (quantity of this good at that price) from 10 to 1000 (logarithmic axes): each of the 31 × 21 cells is one run of economics-finance-ai999x, coloured by the verdict it returned — complements: 324 cells, substitutes: 327 cells. Other inputs as in the worked example: price_other_2 = 2.5, quantity_2 = 120. Marked: the worked example, price_other_1 = 2, quantity_1 = 100 → substitutes.



elasticity ((dQa / mean Qa) / (dPb / mean Pb)) against price_other_1 (first price of the other good) from 1 to 2.46667. One curve for each value of quantity_1 (quantity of this good at that price): 100, 50, 200. Other inputs as in the worked example: price_other_2 = 2.5, quantity_2 = 120. 23 points per curve computed by economics-finance-ai999x; marked: the worked example, price_other_1 = 2, elasticity = 0.818182.

```
economics_finance_ai999x.solve('cross_price_elasticity', price_other_1=2, quantity_1=100,
price_other_2=2.5, quantity_2=120)
# -> elasticity = 0.818182, relationship = substitutes   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the cross-price elasticity with p1 = 2, q1 = 100, p2 = 2.5, q2 = 120.	SOLVED	elasticity = 0.818182, relationship = substitutes
Find the cross-price elasticity of demand with price 1 = 2, q1 = 100, price 2 = 2.5, q2 = 80.	SOLVED	elasticity = -1, relationship = complements

Point elasticity of a linear demand curve — `point_elasticity`

Quantity, point price elasticity and revenue on the linear demand curve $Q = a - b$ P at a price, and the revenue-maximising price.

$$E = \frac{dQ}{dP}\frac{P}{Q} = \frac{-bP}{a - bP}$$

input	meaning	condition
`demand_intercept`	a in $Q = a - bP$	
`demand_slope`	b in $Q = a - bP$ $(b > 0)$	
`price`	price at which to evaluate	

output	meaning
`quantity`	$Q = a - bP$
`elasticity`	-b P / Q
`revenue`	P Q
`revenue_maximizing_price`	$a/(2b)$

- **Method:** $E = (dQ/dP)(P/Q) = -b$ P / (a - bP)
- **Assumes:** a, b, $P > 0$; $Q = a - bP > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Along the demand line Q = 100 − 2P: revenue (top) and the point price elasticity (bottom) at 48 prices, each a run of the package. Revenue peaks where demand is unit elastic, at the returned price 25. Marked: P = 20: elasticity -0.666667, quantity 60, revenue 1200.

```
economics_finance_ai999x.solve('point_elasticity', demand_intercept=100, demand_slope=2, price=20)
# -> quantity = 60, elasticity = -0.666667, revenue = 1200, revenue_maximizing_price = 25
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the point elasticity with demand intercept = 100, demand slope = 2, price = 20.	SOLVED	quantity = 60, elasticity = -0.666667, revenue = 1200, revenue_maximizing_price = 25
Find the point elasticity with a = 100, b = 2, price = 60.	INVALID_INPUT	Point elasticity of a linear demand curve: the price is at or above the choke price a/b: nothing is demanded.

Market equilibrium and surplus (linear curves) — `market_equilibrium`

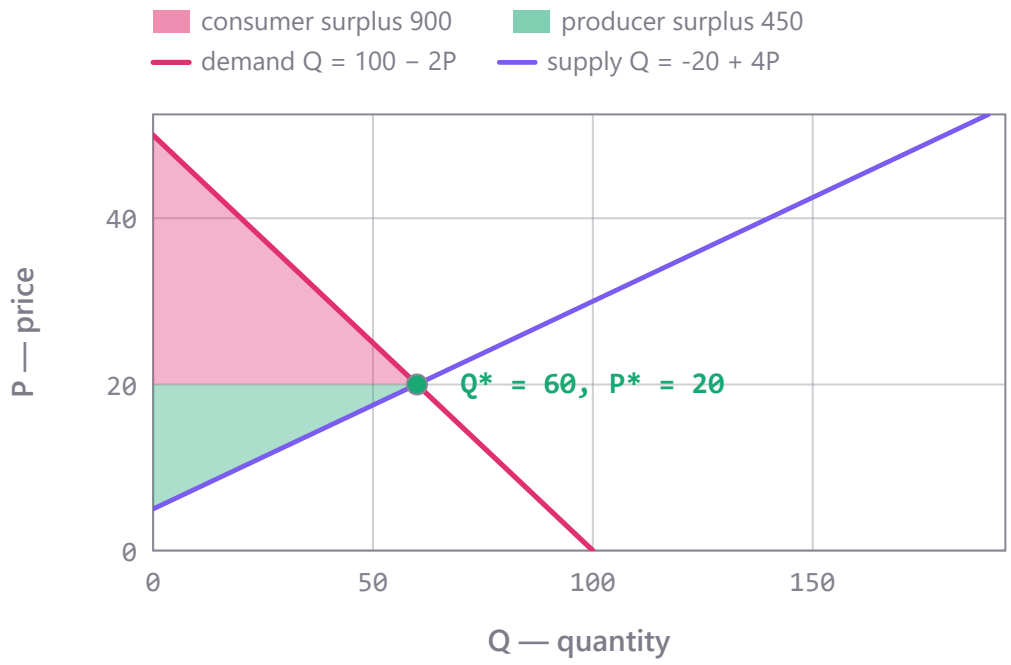
Equilibrium price and quantity of $Qd = a - bP$ and $Qs = c + dP$, with consumer, producer and total surplus (producer surplus counted at prices >= 0).

$$P^* = \frac{a-c}{b+d}, \quad CS = \frac{Q^{*2}}{2b}$$

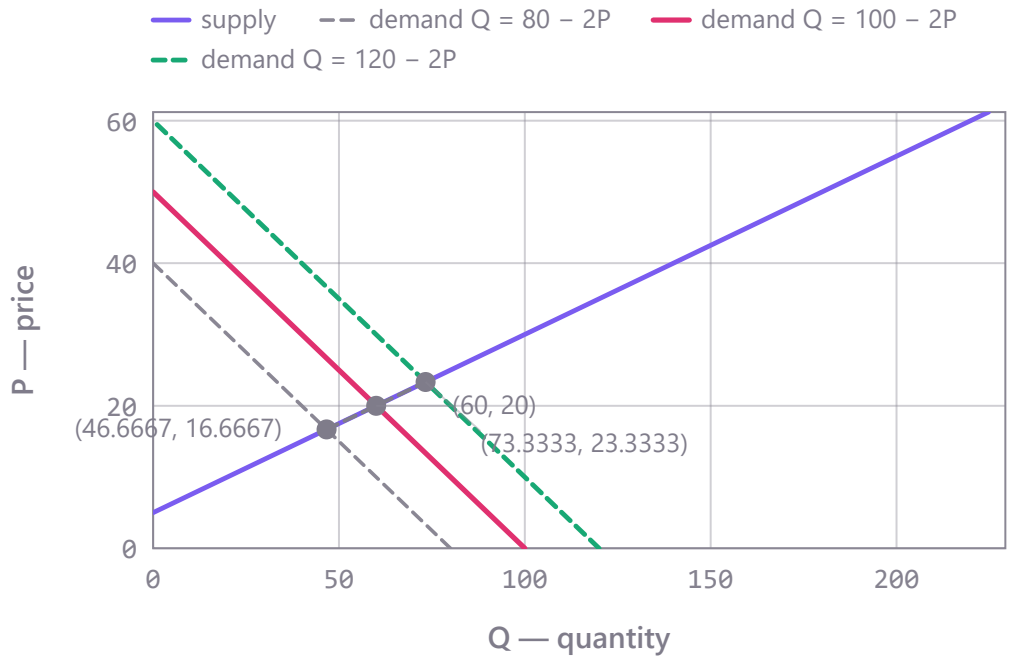
input	meaning	condition
`demand_intercept`	a in $Qd = a - bP$	
`demand_slope`	b in $Qd = a - bP$ $(b > 0)$	
`supply_intercept`	c in $Qs = c + dP$	
`supply_slope`	d in $Qs = c + dP$ $(d > 0)$	

output	meaning
`price`	equilibrium price (a - c)/(b + d)
`quantity`	equilibrium quantity
`consumer_surplus`	$Q^2/(2b)$
`producer_surplus`	area above supply, below P
`total_surplus`	CS + PS

- **Method:** $a - bP = c + dP$; surpluses are triangle/trapezoid areas
- **Assumes:** b, $d > 0$; positive equilibrium price and quantity
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Linear demand Q = 100 − 2P and supply Q = -20 + 4P: the package returns the equilibrium P* = 20, Q* = 60, consumer surplus 900 and producer surplus 450 (the shaded triangles; total 1350). Marked: Q = 60, P = 20.



Supply Q = -20 + 4P with demand shifted to intercepts 80, 100 and 120 (slope 2): the equilibrium the package returns for each, (Q*, P*) = (46.6667, 16.6667); (60, 20); (73.3333, 23.3333). A rise in demand raises both price and quantity, moving along the supply line.

```
economics_finance_ai999x.solve('market_equilibrium', demand_intercept=100, demand_slope=2,
supply_intercept=-20, supply_slope=4)
# -> price = 20, quantity = 60, consumer_surplus = 900, producer_surplus = 450, total_surplus =
1350   (SOLVED)
```

```
economics_finance_ai999x.solve('market_equilibrium', demand_intercept=100, demand_slope=2,
supply_intercept=10, supply_slope=1)
# -> price = 30, quantity = 40, consumer_surplus = 400, producer_surplus = 750, total_surplus =
1150   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the market equilibrium with demand intercept = 100, demand slope = 2, supply intercept = -20, supply slope = 4.	SOLVED	price = 20, quantity = 60, consumer_surplus = 900, producer_surplus = 450, total_surplus = 1350
Find the equilibrium price with a = 100, b = 2, c = 200, d = 4.	INVALID_INPUT	Market equilibrium and surplus (linear curves): the curves do not cross at a positive price and quantity.

Per-unit tax incidence (linear curves) — `per_unit_tax`

Effect of a per-unit tax on $Qd = a - bP$, $Qs = c + dP$: quantity, price paid by buyers and kept by sellers, tax revenue, deadweight loss and the buyers' share of the burden.

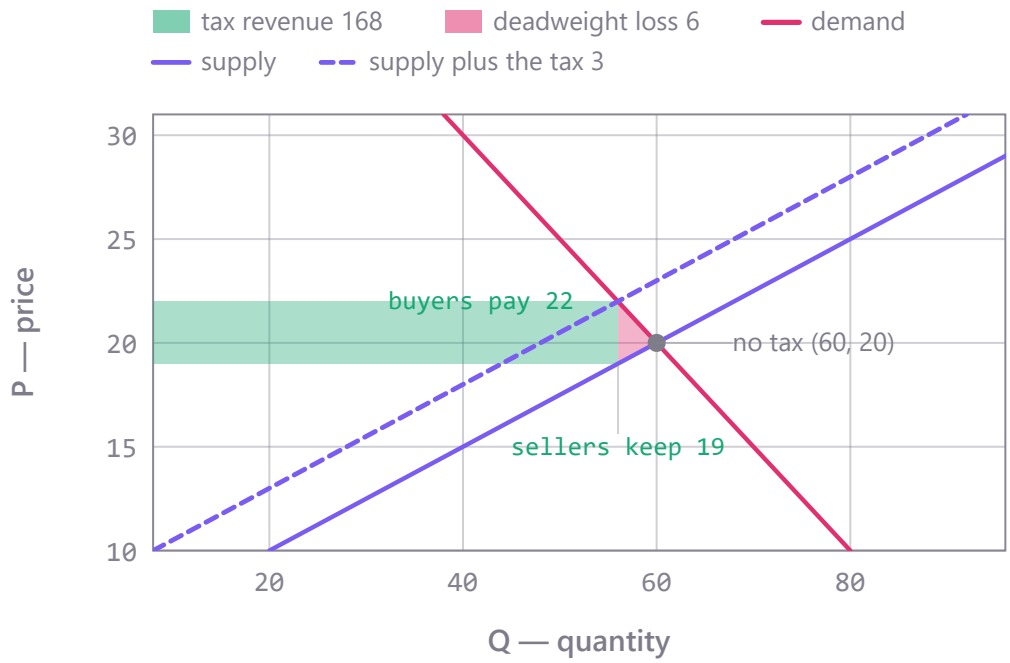
$$P_b = \frac{a - c + dt}{b + d}, \quad DWL = \tfrac{1}{2}\, t\,(Q_0 - Q_t)$$

input	meaning	condition
`demand_intercept`	a in $Qd = a - bP$	

input	meaning	condition
`demand_slope`	b in $Qd = a - bP$ $(b > 0)$	
`supply_intercept`	c in $Qs = c + dP$	
`supply_slope`	d in $Qs = c + dP$ $(d > 0)$	
`tax`	tax per unit	

output	meaning
`quantity`	quantity with the tax
`buyer_price`	price paid by buyers
`seller_price`	buyer price - tax
`tax_revenue`	t Q
`deadweight_loss`	t $(Q_0 - Qt)/2$
`buyer_share`	share of the tax borne by buyers d/(b+d)

- **Method:** a - b $Pb = c + d$ (Pb - t)
- **Assumes:** b, $d > 0$; tax > 0; positive quantity and seller price with the tax
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Demand Q = 100 − 2P and supply Q = -20 + 4P with a tax of 3 per unit (lines from the stated inputs; the dashed supply is shifted up by the tax). The package returns the taxed quantity 56, the buyers' price 22 and the sellers' price 19 (buyers bear 66.6667 % of the tax), the tax revenue 168 (rectangle) and the deadweight loss 6 (triangle up to the untaxed equilibrium 60, 20, also returned).

```
economics_finance_ai999x.solve('per_unit_tax', demand_intercept=100, demand_slope=2,
supply_intercept=-20, supply_slope=4, tax=3)
# -> quantity = 56, buyer_price = 22, seller_price = 19, tax_revenue = 168, deadweight_loss = 6,
buyer_share = 0.666667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the tax incidence with demand intercept = 100, demand slope = 2, supply intercept = -20, supply slope = 4, tax = 3.	SOLVED	quantity = 56, buyer_price = 22, seller_price = 19, tax_revenue = 168, deadweight_loss = 6, buyer_share = 0.666667
Find the tax incidence with a = 100, b = 2, c = -20, d = 4, tax = 50.	INVALID_INPUT	Per-unit tax incidence (linear curves): the tax is so large that the market closes (no positive quantity or seller price

Monopoly profit maximization — `monopoly`

Profit-maximising quantity and price of a monopoly facing $P = A - BQ$ with cost $F + cQ + eQ^2$, its profit, Lerner index and the deadweight loss against marginal-cost pricing.

$$Q^* = \frac{A - c}{2(B + e)}, \quad P^* = A - BQ^*$$

input	meaning	condition
`price_intercept`	A in the inverse demand $P = A - B$ Q	
`price_slope`	B in $P = A - B$ Q $(B > 0)$	
`marginal_cost`	c, the constant part of marginal cost	
`fixed_cost`	fixed cost F (default 0)	
`cost_quadratic`	e in the cost $F + cQ + eQ^2$ (default 0)	

output	meaning
`quantity`	Q* where $MR = MC$
`price`	A - B Q*
`profit`	P Q - cost
`lerner_index`	(P - MC)/P
`deadweight_loss`	loss against $P = MC$

- **Method:** $MR = A - 2BQ = MC = c + 2eQ$
- **Assumes:** $A > c \geq 0$; $B > 0$; F, $e \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A monopolist facing demand P = 100 − 2Q with marginal cost 20 (lines from the stated inputs; marginal revenue 100 − 4Q). The package returns the output 20 where marginal revenue meets marginal cost, the price 60, the profit 800 (rectangle), the deadweight loss 400 (triangle out to the competitive quantity 40) and the Lerner index 0.666667.

```
economics_finance_ai999x.solve('monopoly', price_intercept=100, price_slope=2, marginal_cost=20)
# -> quantity = 20, price = 60, profit = 800, lerner_index = 0.666667, deadweight_loss = 400
(SOLVED)
```

```
economics_finance_ai999x.solve('monopoly', price_intercept=100, price_slope=1, marginal_cost=10,
fixed_cost=200, cost_quadratic=0.5)
# -> quantity = 30, price = 70, profit = 1150, lerner_index = 0.428571, deadweight_loss = 225
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the monopoly price and quantity with price intercept = 100, price slope = 2, marginal cost = 20.	SOLVED	quantity = 20, price = 60, profit = 800, lerner_index = 0.666667, deadweight_loss = 400
Find the monopoly price and quantity with a = 100, b = 2, mc = 120.	INVALID_INPUT	Monopoly profit maximization: the demand intercept must exceed marginal cost (otherwise nothing is produced).

Cournot duopoly equilibrium — `cournot`

Quantities, price and profits of two firms choosing quantities simultaneously against $P = A - B$(q1 + q2) with constant marginal costs.

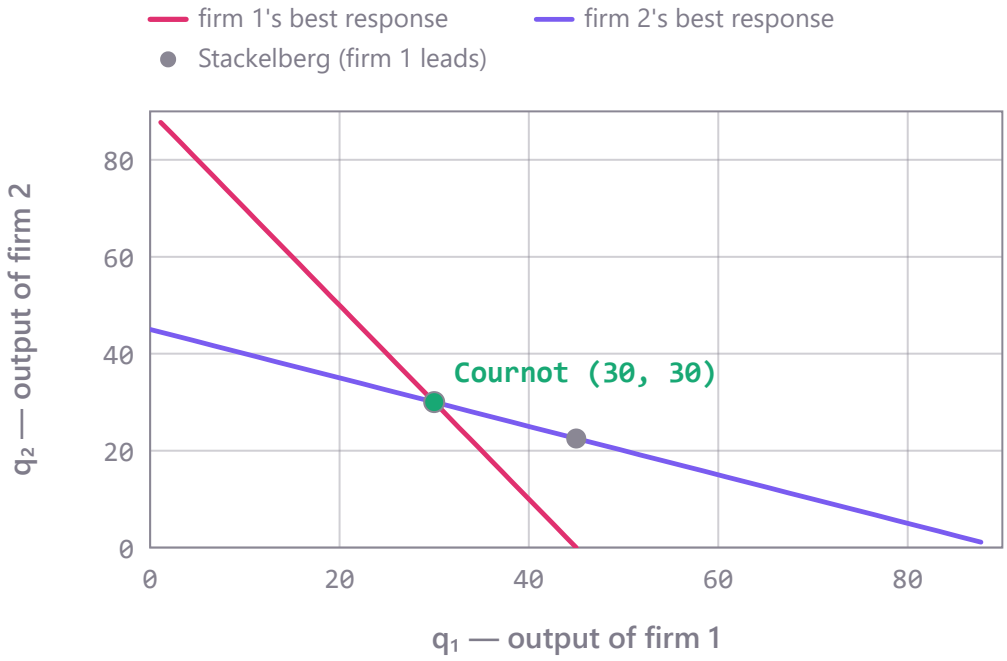
$$q_i^* = \frac{A - 2c_i + c_j}{3B}$$

input	meaning	condition
`price_intercept`	A in the inverse demand $P = A - B$ Q	

input	meaning	condition
`price_slope`	B in $P = A - B$ Q $(B > 0)$	
`cost_1`	marginal cost of firm 1	
`cost_2`	marginal cost of firm 2	

output	meaning
`q1`	$(A - 2c1 + c_2)/(3B)$
`q2`	$(A - 2c2 + c_1)/(3B)$
`price`	A - B(q1 + q2)
`profit_1`	$(P - c_1)$ q1
`profit_2`	$(P - c_2)$ q2

- **Method:** intersection of the two best-response functions
- **Assumes:** $B > 0$; both equilibrium quantities positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Two firms with inverse demand P = 100 − 1(q_1 + q_2) and marginal costs 10 and 10. Each best-response line gives one firm's profit-maximising output for every output of the other (40 runs of the package's monopoly output against the demand left over). They cross at the Cournot equilibrium the package returns, q_1 = 30, q_2 = 30, price 40, profits 900 and 900. Dot: the Stackelberg outcome with firm 1 leading (45, 22.5), also returned.

```
economics_finance_ai999x.solve('cournot', price_intercept=100, price_slope=1, cost_1=10, cost_2=10)
# -> q1 = 30, q2 = 30, price = 40, profit_1 = 900, profit_2 = 900   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Cournot equilibrium with price intercept = 100, price slope = 1, c1 = 10, c2 = 10.	SOLVED	q1 = 30, q2 = 30, price = 40, profit_1 = 900, profit_2 = 900

question	outcome	result
Find the Cournot equilibrium with a = 100, b = 1, c1 = 10, c2 = 90.	INVALID_INPUT	Cournot duopoly equilibrium: one firm would not produce: the costs are too different for an interior Cournot equilibrium

Stackelberg duopoly equilibrium — `stackelberg`

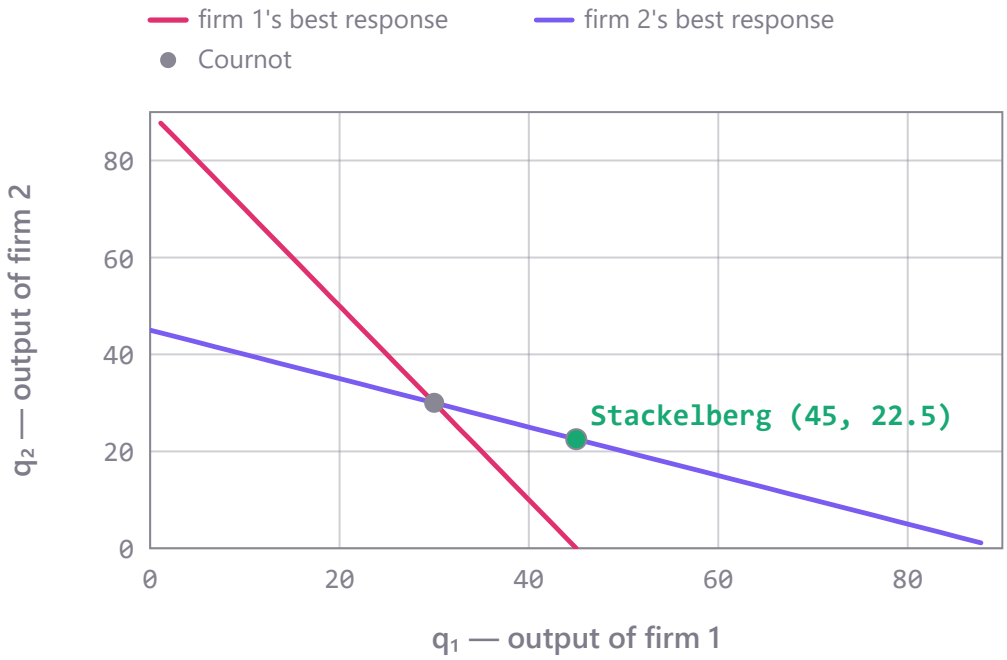
Quantities, price and profits when a leader chooses its quantity first and the follower best-responds, against $P = A - B$(qL + qF) with constant marginal costs.

$$q_L^* = \frac{A - 2c_L + c_F}{2B}, \quad q_F^* = \frac{A - c_F - Bq_L^*}{2B}$$

input	meaning	condition
`price_intercept`	A in the inverse demand $P = A - B$ Q	
`price_slope`	B in $P = A - B$ Q $(B > 0)$	
`cost_leader`	marginal cost of the leader	
`cost_follower`	marginal cost of the follower	

output	meaning
`q_leader`	$(A - 2cL + cF)/(2B)$
`q_follower`	(A - cF - B qL)/(2B)
`price`	market price
`profit_leader`	leader's profit
`profit_follower`	follower's profit

- **Method:** backward induction: follower's best response, then the leader's optimum
- **Assumes:** $B > 0$; both quantities positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A leader (firm 1, cost 10) and a follower (cost 10) facing P = 100 − 1Q. The follower always plays its best response (lines: 40 runs of the package's monopoly output against the residual demand); the leader picks the point on that line it likes best. The package returns q_leader = 45, q_follower = 22.5, price 32.5, profits 1012.5 and 506.25. Dot: the simultaneous (Cournot) outcome (30, 30) for comparison.

```
economics_finance_ai999x.solve('stackelberg', price_intercept=100, price_slope=1, cost_leader=10, cost_follower=10)
# -> q_leader = 45, q_follower = 22.5, price = 32.5, profit_leader = 1012.5, profit_follower = 506.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Stackelberg equilibrium with price intercept = 100, price slope = 1, cost leader = 10, cost follower = 10.	SOLVED	q_leader = 45, q_follower = 22.5, price = 32.5, profit_leader = 1012.5, profit_follower = 506.25

Break-even and target-profit volume — `break_even`

Units to sell to cover fixed costs (and reach a target profit): Q = (F + target) / (p - v).

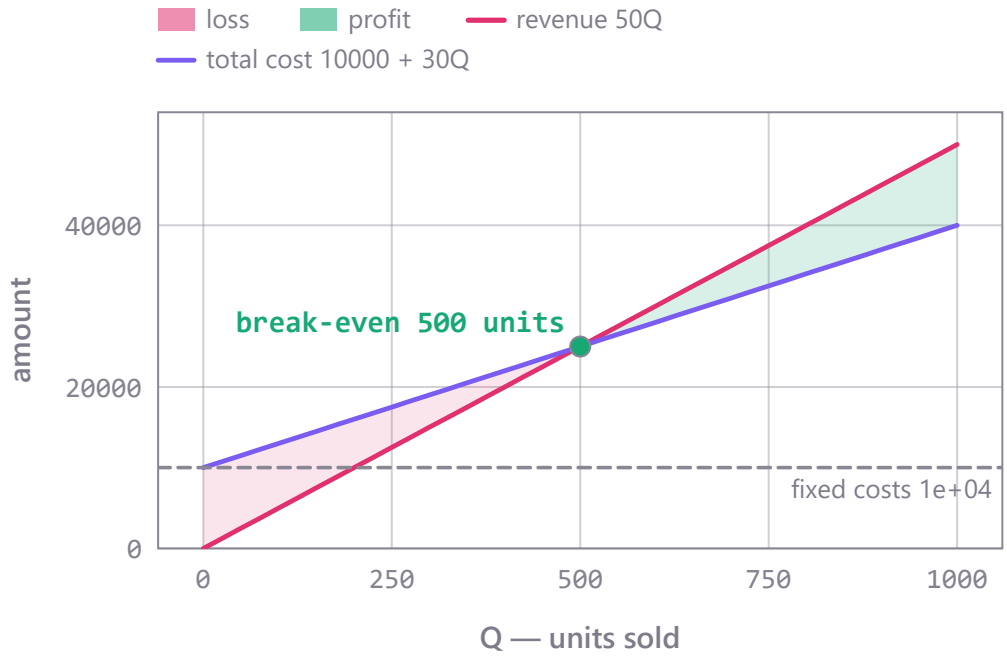
$$Q_{BE} = \frac{F + \pi_{target}}{p - v}$$

input	meaning	condition
`fixed_costs`	total fixed costs	
`price`	selling price per unit	
`variable_cost`	variable cost per unit	
`target_profit`	target profit (default 0: break-even)	

output	meaning
`quantity`	units (F + target)/(p - v)

output	meaning
`revenue`	p Q
`contribution_margin`	p - v per unit
`contribution_margin_ratio`	(p - v)/p

- **Method:** Q = (F + target) / (p - v)
- **Assumes:** price > variable cost; fixed costs >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Revenue at a price of 50 and total cost with fixed costs 10000 and a variable cost of 30 per unit (lines from the stated inputs): they cross at the break-even quantity the package returns, 500 units (revenue 25000, contribution margin 20 per unit, ratio 0.4).

```
economics_finance_ai999x.solve('break_even', fixed_costs=10000, price=50, variable_cost=30)
# -> quantity = 500, revenue = 25000, contribution_margin = 20, contribution_margin_ratio = 0.4
(SOLVED)
```

```
economics_finance_ai999x.solve('break_even', fixed_costs=10000, price=50, variable_cost=30,
target_profit=4000)
# -> quantity = 700, revenue = 35000, contribution_margin = 20, contribution_margin_ratio = 0.4
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the break-even quantity with fixed costs = 10000, price = 50, variable cost = 30.	SOLVED	quantity = 500, revenue = 25000, contribution_margin = 20, contribution_margin_ratio = 0.4
Find the break-even quantity with fixed costs = 10000, price = 30, variable cost = 50.	INVALID_INPUT	Break-even and target-profit volume: the price must exceed the variable cost per unit (no positive contribution margin).

Utility-maximising demand (Cobb-Douglas) — `cobb_douglas_demand`

The bundle that maximises $U = x^{\alpha}\, y^{\beta}$ subject to px x + py $y = M$.

$$x^{*} = \frac{\alpha}{\alpha+\beta}\frac{M}{p_x}, \quad y^{*} = \frac{\beta}{\alpha+\beta}\frac{M}{p_y}$$

input	meaning	condition
`alpha`	exponent of good x	
`beta`	exponent of good y	
`income`	income (budget)	
`price_x`	price of good x	
`price_y`	price of good y	

output	meaning
`x`	alpha/(alpha+beta) M/px
`y`	beta/(alpha+beta) M/py
`utility`	$x^{\alpha}\, y^{\beta}$
`share_x`	budget share of x

- **Method:** tangency MRS = px/py with the budget line
- **Assumes:** alpha, beta, income, prices > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cobb–Douglas preferences U = x^0.3 y^0.7 with income 1000 and prices 10 and 20: the budget line from the stated inputs and the indifference curve through the optimum the package returns, x = 30, y = 35, utility 33.4183 (curves drawn from that utility and ±25 %); the budget share of x is 0.3.



x (alpha/(alpha+beta) M/px) against price_x (price of good x) from 5 to 15. One curve for each value of income (income (budget)): 1000, 500, 2000. Other inputs as in the worked example: alpha = 0.3, beta = 0.7, price_y = 20. 31 points per curve computed by economics-finance-ai999x; marked: the worked example, price_x = 10, x = 30.

```
economics_finance_ai999x.solve('cobb_douglas_demand', alpha=0.3, beta=0.7, income=1000, price_x=10, price_y=20)
# -> x = 30, y = 35, utility = 33.4183, share_x = 0.3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Cobb-Douglas optimal bundle with alpha = 0.3, beta = 0.7, income = 1000, px = 10, py = 20.	SOLVED	x = 30, y = 35, utility = 33.4183, share_x = 0.3
Find the Cobb-Douglas optimal bundle with alpha = 0.3, beta = 0.7, income = 1000, px = 0, py = 20.	INVALID_INPUT	Utility-maximising demand (Cobb-Douglas): price_x must be positive.

Nominal GDP, real GDP and the GDP deflator — `gdp_deflator`

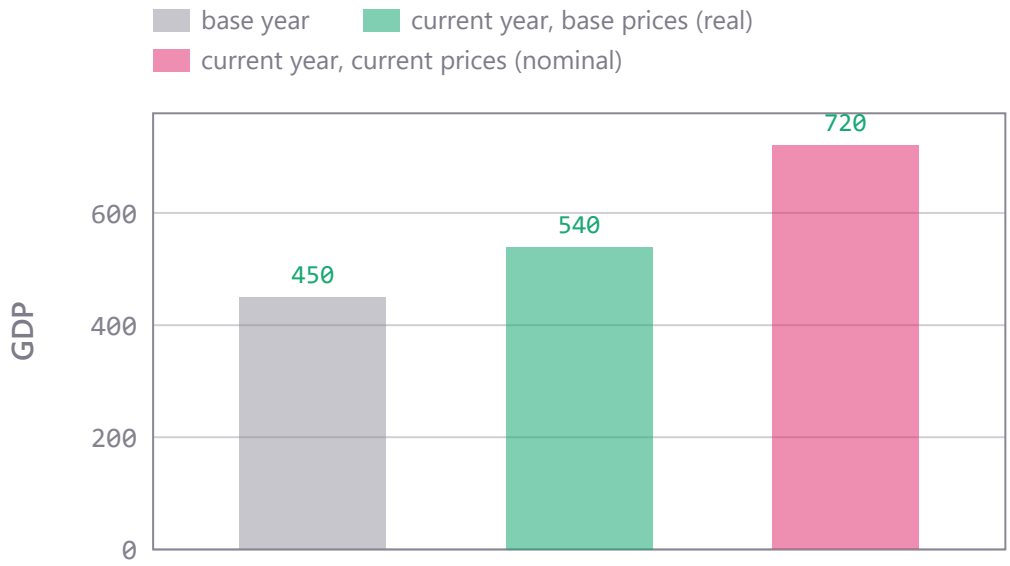
Nominal GDP (current prices), real GDP (base-year prices), the GDP deflator, inflation since the base year and real growth, from the prices and quantities of the goods produced.

$$\text{Deflator} = 100\,\frac{\sum p_1 q_1}{\sum p_0 q_1}$$

input	meaning	condition
`base_prices`	prices in the base year	
`base_quantities`	quantities in the base year	
`current_prices`	prices in the current year	
`current_quantities`	quantities in the current year	

output	meaning
`nominal_gdp`	sum p1 q1
`real_gdp`	sum p0 q1
`deflator`	100 x nominal / real
`inflation`	deflator/100 - 1 (since the base year)
`real_growth`	real GDP / base-year GDP - 1

- **Method:** deflator = 100 sum p1 q1 / sum p0 q1
- **Assumes:** one price and quantity per good; prices > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



GDP of the economy with base prices 2, 5, base quantities 100, 50, current prices 3, 6 and current quantities 120, 60: base-year GDP 450 (a run of the package with the base year as the current one), real GDP 540 and nominal GDP 720 as returned. Real growth 20 %; the deflator 133.333 = 100 × nominal ÷ real (inflation 33.3333 %).

```
economics_finance_ai999x.solve('gdp_deflator', base_prices='2, 5', base_quantities='100, 50',
current_prices='3, 6', current_quantities='120, 60')
# -> nominal_gdp = 720, real_gdp = 540, deflator = 133.333, inflation = 0.333333, real_growth = 0.2
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the GDP deflator with base prices = [2, 5], base quantities = [100, 50], current prices = [3, 6], current quantities = [120, 60].	SOLVED	nominal_gdp = 720, real_gdp = 540, deflator = 133.333, inflation = 0.333333, real_growth = 0.2
Find the GDP deflator with base prices = [2, 5], base quantities = [100, 50], current prices = [3], current quantities = [120, 60].	INVALID_INPUT	Nominal GDP, real GDP and the GDP deflator: one price and quantity per good in each year.

Laspeyres, Paasche and Fisher price indices — `price_indices`

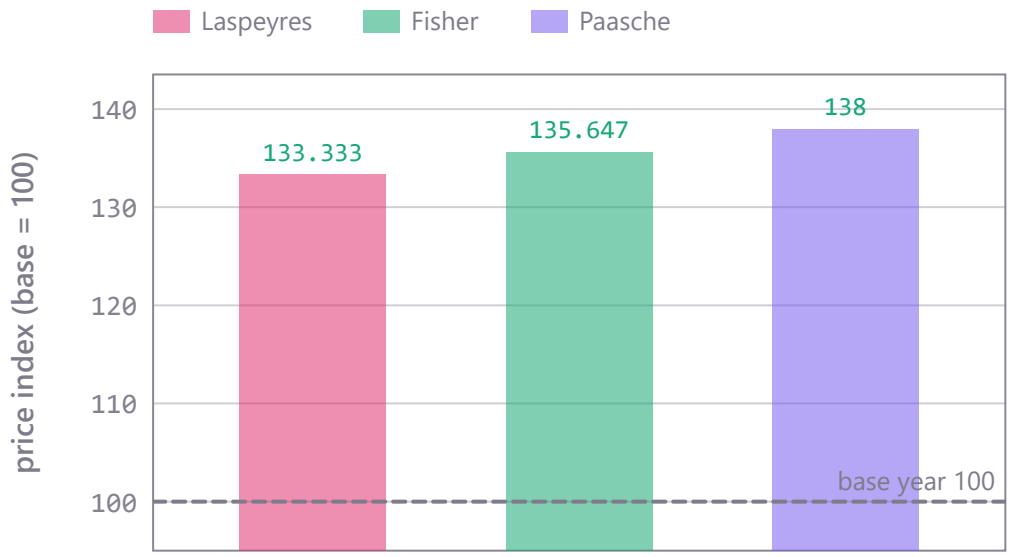
Price indices (base = 100) of a basket: Laspeyres (base-year quantities, the CPI method), Paasche (current quantities) and Fisher (their geometric mean).

$$L = 100\frac{\sum p_1 q_0}{\sum p_0 q_0}, \quad P = 100\frac{\sum p_1 q_1}{\sum p_0 q_1}, \quad F = \sqrt{LP}$$

input	meaning	condition
`base_prices`	prices in the base year	
`base_quantities`	quantities in the base year	
`current_prices`	prices in the current year	
`current_quantities`	quantities in the current year	

output	meaning
`laspeyres`	100 sum p1 q0 / sum p0 q0
`paasche`	100 sum p1 q1 / sum p0 q1
`fisher`	sqrt(L P)

- **Method:** ratios of basket costs
- **Assumes:** one price and quantity per good; prices > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Price indices for base prices 2, 5, base quantities 100, 50, current prices 3, 6 and current quantities 150, 40, as returned by the package: Laspeyres 133.333 (base quantities), Paasche 138 (current quantities) and Fisher 135.647, their geometric mean. The axis starts at 95.

```
economics_finance_ai999x.solve('price_indices', base_prices='2, 5', base_quantities='100, 50',
current_prices='3, 6', current_quantities='150, 40')
# -> laspeyres = 133.333, paasche = 138, fisher = 135.647   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Laspeyres and Paasche price index with base prices = [2, 5], base quantities = [100, 50], current prices = [3, 6], current quantities = [150, 40].	SOLVED	laspeyres = 133.333, paasche = 138, fisher = 135.647

Real interest rate (Fisher equation) — `fisher_real_rate`

The exact real rate $(1+i)/(1+pi)-1$ and the approximation i - pi.

$$1+i=(1+r)(1+\pi)$$

input	meaning	condition
`nominal_rate`	nominal interest rate (decimal)	
`inflation_rate`	inflation rate (decimal)	

output	meaning
`real_rate`	$(1+i)/(1+pi)-1$
`approximate_real_rate`	i - pi

- **Method:** $(1+i)=(1+r)(1+pi)$
- **Assumes:** -1 < rates < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The real interest rate earned at a nominal 8 % as inflation runs from 0 to 15 %: the exact Fisher rate and the approximation i − π, both returned by the package (31 runs each). Marked: inflation 3 %: real rate 4.85437 % (approximation 5 %).

```
economics_finance_ai999x.solve('fisher_real_rate', nominal_rate=0.08, inflation_rate=0.03)
# -> real_rate = 0.0485437, approximate_real_rate = 0.05   (SOLVED)
```

```
economics_finance_ai999x.solve('fisher_real_rate', nominal_rate=0.1, inflation_rate=0.1)
# -> real_rate = 0, approximate_real_rate = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the real interest rate with nominal rate = 8%, inflation = 3%.	SOLVED	real_rate = 0.0485437, approximate_real_rate = 0.05
Find the real interest rate with nominal rate = 8, inflation = 3.	INVALID_INPUT	Real interest rate (Fisher equation): nominal_rate must be a decimal rate between -1 and 1 (write 5% or 0.05).

Inflation from a price index series — `inflation_adjustment`

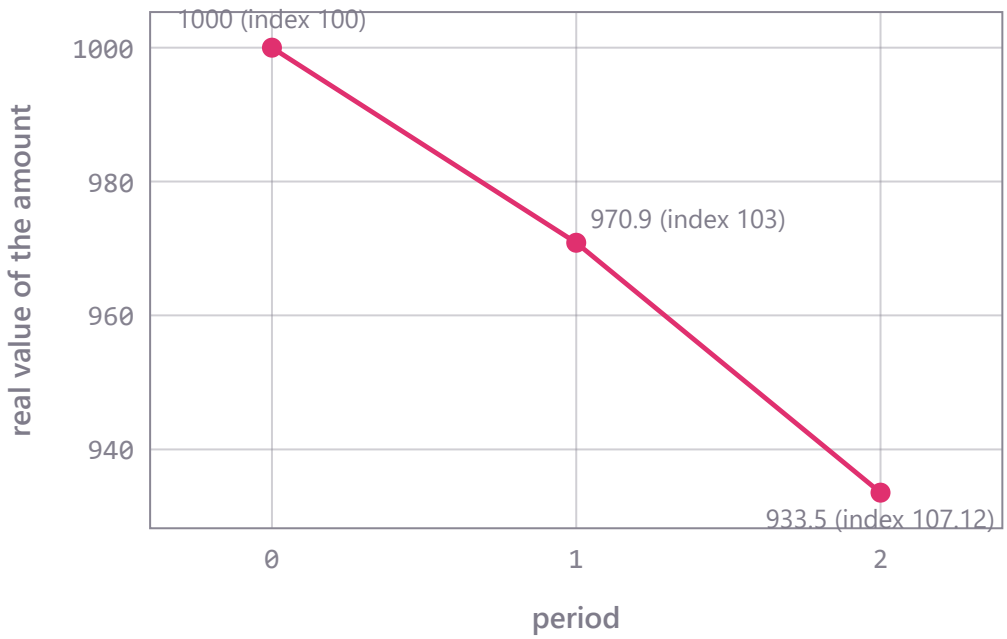
Total and average annual (geometric) inflation from consecutive price levels (CPI), and the value of an amount in the last period's money expressed in first-period money.

$$\bar{\pi} = \left(\frac{P_n}{P_0}\right)^{1/n} - 1$$

input	meaning	condition
`price_levels`	price index levels in consecutive periods (e.g. CPI each year)	
`amount`	an amount in last-period money to deflate (optional)	

output	meaning
`total_inflation`	P_last / P_first - 1
`average_inflation`	(P_last/P_first)^(1/n) - 1
`real_value`	amount x P_first / P_last

- **Method:** geometric mean of the period-to-period price relatives
- **Assumes:** price levels > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The purchasing power of 1000 as the price index moves through 100, 103, 107.12: the real value after each period is a run of the package on the price levels up to that period. Over the whole span: total inflation 7.12 %, average 3.49879 % a period, real value 933.532.

```
economics_finance_ai999x.solve('inflation_adjustment', price_levels='100, 103, 107.12',
amount=1000)
# -> total_inflation = 0.0712, average_inflation = 0.0349879, real_value = 933.532   (SOLVED)
```

```
economics_finance_ai999x.solve('inflation_adjustment', price_levels=[100, 121])
# -> total_inflation = 0.21, average_inflation = 0.21   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the average inflation with cpi = [100, 103, 107.12], amount = 1000.	SOLVED	total_inflation = 0.0712, average_inflation = 0.0349879, real_value = 933.532
Find the average inflation with cpi = [100].	INVALID_INPUT	price_levels: needs at least 2 entries, got 1.

Money multiplier — `money_multiplier`

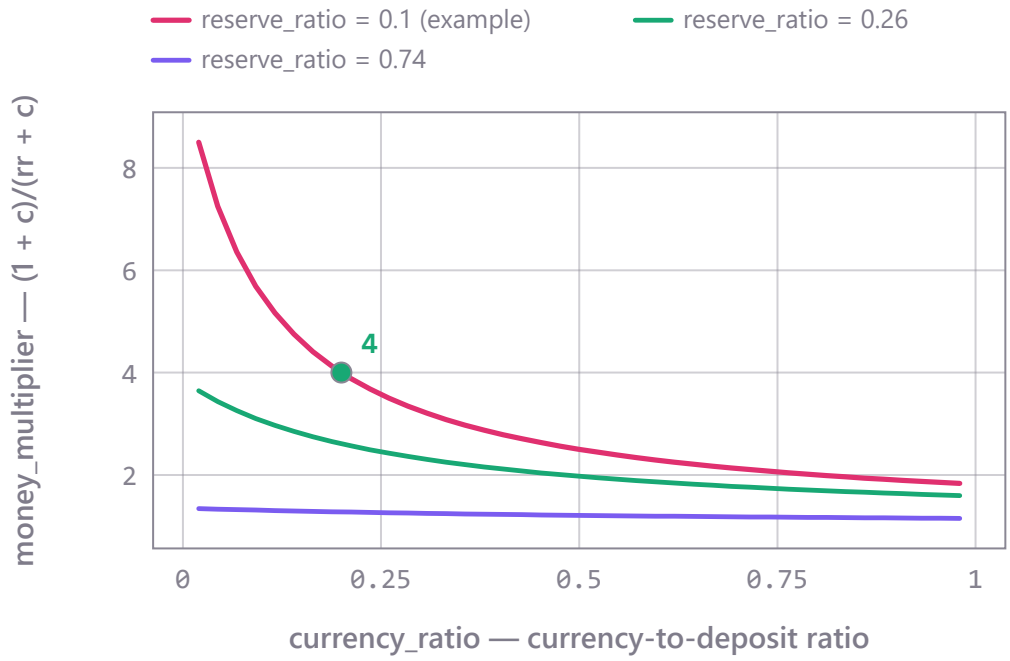
Simple deposit multiplier $1/rr$ and the money multiplier $(1+c)/(rr+c)$ with a currency-deposit ratio c, and the money supply from the monetary base.

$$m = \frac{1+c}{rr+c}$$

input	meaning	condition
`reserve_ratio`	reserve ratio (decimal)	
`currency_ratio`	currency-to-deposit ratio (default 0)	
`monetary_base`	monetary base (optional)	

output	meaning
`simple_multiplier`	$1/rr$
`money_multiplier`	$(1+c)/(rr+c)$
`money_supply`	multiplier x base

- **Method:** $m = (1+c)/(rr+c)$
- **Assumes:** 0 < reserve_ratio <= 1; currency_ratio >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



money_multiplier ((1 + c)/(rr + c)) against currency_ratio (currency-to-deposit ratio (default 0)) from 0.02 to 0.98. One curve for each value of reserve_ratio (reserve ratio (decimal)): 0.1, 0.26, 0.74. Other inputs as in the worked example: monetary_base = 1000. 41 points per curve computed by economics-finance-ai999x; marked: the worked example, currency_ratio = 0.2, money_multiplier = 4.

```
economics_finance_ai999x.solve('money_multiplier', reserve_ratio=0.1, currency_ratio=0.2,
monetary_base=1000)
# -> simple_multiplier = 10, money_multiplier = 4, money_supply = 4000   (SOLVED)
```

```
economics_finance_ai999x.solve('money_multiplier', reserve_ratio=0.2)
# -> simple_multiplier = 5, money_multiplier = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the money multiplier with reserve ratio = 10%, currency ratio = 20%, monetary base = 1000.	SOLVED	simple_multiplier = 10, money_multiplier = 4, money_supply = 4000
Find the money multiplier with reserve ratio = 0.	INVALID_INPUT	Money multiplier: the reserve ratio must be positive.

Keynesian spending multiplier — `spending_multiplier`

Government-spending multiplier 1/(1 - mpc(1 - t) + m), the lump-sum tax multiplier and the change in output from a change in spending.

$$k = \frac{1}{1 - c(1 - t) + m}$$

input	meaning	condition
`mpc`	marginal propensity to consume	
`tax_rate`	proportional income tax rate (default 0)	
`import_propensity`	marginal propensity to import (default 0)	
`spending_change`	change in government spending (optional)	

output	meaning
`spending_multiplier`	1/(1 - mpc(1 - t) + m)
`tax_multiplier`	-mpc x multiplier
`output_change`	multiplier x spending change

- **Method:** k = 1/(1 - mpc(1 - t) + m)
- **Assumes:** 0 < mpc < 1; 0 <= mpc(1 - t) - m < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



spending_multiplier (1/(1 - mpc(1 - t) + m)) against tax_rate (proportional income tax rate (default 0)) from 0.02 to 0.98. One curve for each value of import_propensity (marginal propensity to import (default 0)): 0.1, 0.05, 0.2. Other inputs as in the worked example: mpc = 0.8, spending_change = 100. 36 points per curve computed by economics-finance-ai999x; marked: the worked example, tax_rate = 0.25, spending_multiplier = 2.

```
economics_finance_ai999x.solve('spending_multiplier', mpc=0.8, tax_rate=0.25,
import_propensity=0.1, spending_change=100)
# -> spending_multiplier = 2, tax_multiplier = -1.6, output_change = 200   (SOLVED)
```

```
economics_finance_ai999x.solve('spending_multiplier', mpc=0.75)
# -> spending_multiplier = 4, tax_multiplier = -3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the spending multiplier with mpc = 0.8, tax rate = 25%, mpi = 0.1, spending change = 100.	SOLVED	spending_multiplier = 2, tax_multiplier = -1.6, output_change = 200
Find the spending multiplier with mpc = 1.2.	INVALID_INPUT	Keynesian spending multiplier: the marginal propensity to consume is between 0 and 1.

Solow model steady state — `solow_steady_state`

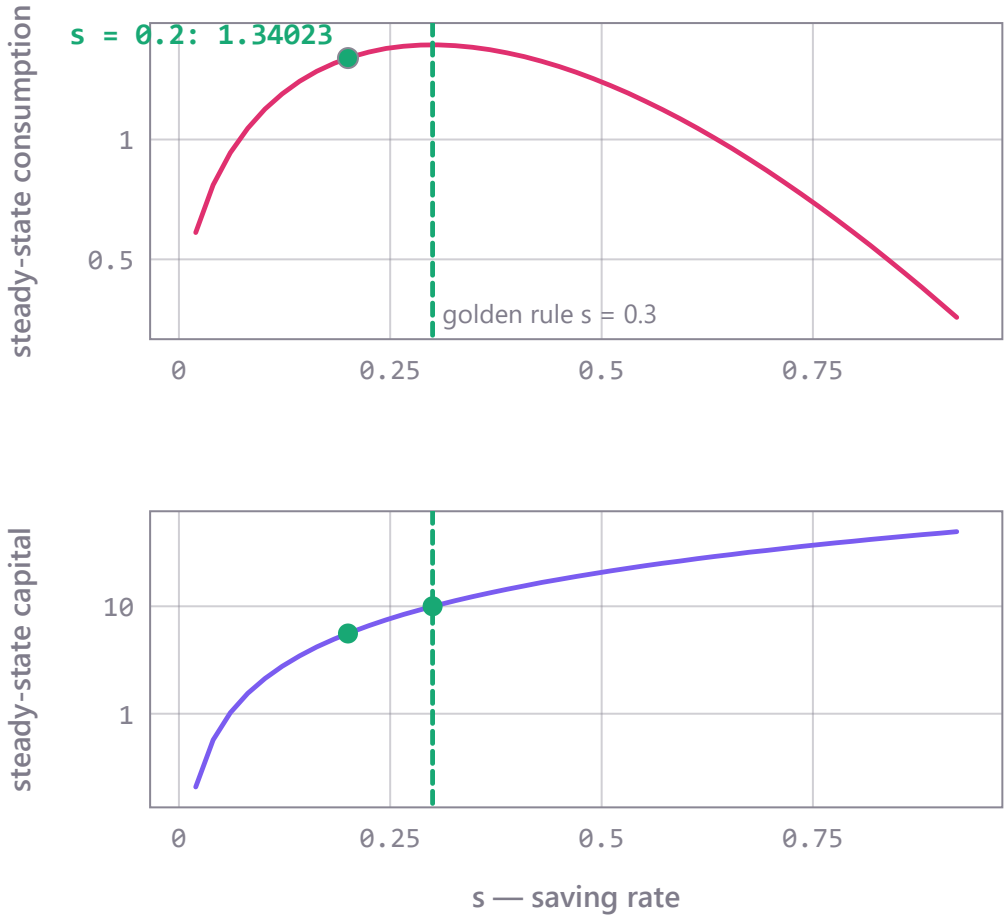
Steady-state capital, output and consumption per effective worker in the Solow model with $y = A\,k^{\alpha}$, and the golden-rule saving rate and capital.

$$k^{*} = \left(\frac{sA}{n+g+\delta}\right)^{\frac{1}{1-\alpha}}$$

input	meaning	condition
`saving_rate`	saving rate s (decimal)	
`population_growth`	population growth n (decimal)	
`depreciation`	depreciation rate (decimal)	
`alpha`	capital share alpha in $y = A\,k^{\alpha}$	
`technology_growth`	technology growth g (default 0)	
`productivity`	total factor productivity A (default 1)	

output	meaning
`capital`	k* = (sA/(n + g + delta))^(1/(1 - alpha))
`output`	y* = A k*^alpha
`consumption`	$(1 - s)$ y*
`golden_rule_saving`	alpha
`golden_rule_capital`	(alpha A/(n + g + delta))^(1/(1 - alpha))

- **Method:** s A $k^{\alpha} = (n + g + \delta)$ k
- **Assumes:** $0 < s, \alpha < 1; n + g + \delta > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Solow model's steady state (population growth 0.01, depreciation 0.05, capital share 0.3) at 45 saving rates, each a run of the package: saving more always raises steady-state capital (bottom, logarithmic) but consumption (top) peaks at the golden-rule saving rate the package returns, 0.3 (capital 9.96618). Marked: s = 0.2: capital 5.58431, output 1.67529, consumption 1.34023.

```
economics_finance_ai999x.solve('solow_steady_state', saving_rate=0.2, population_growth=0.01,
depreciation=0.05, alpha=0.3)
# -> capital = 5.58431, output = 1.67529, consumption = 1.34023, golden_rule_saving = 0.3,
golden_rule_capital = 9.96618   (SOLVED)
```

```
economics_finance_ai999x.solve('solow_steady_state', saving_rate=0.25, population_growth=0.0,
depreciation=0.1, alpha=0.5)
# -> capital = 6.25, output = 2.5, consumption = 1.875, golden_rule_saving = 0.5,
golden_rule_capital = 25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Solow steady state with saving rate = 20%, population growth = 1%, depreciation = 5%, alpha = 0.3.	SOLVED	capital = 5.58431, output = 1.67529, consumption = 1.34023, golden_rule_saving = 0.3, golden_rule_capital = 9.96618
Find the Solow steady state with s = 20%, n = 1%, delta = 5%, alpha = 1.3.	INVALID_INPUT	Solow model steady state: saving_rate and alpha are decimals between 0 and 1.

Simple linear regression (OLS) — `ols_simple`

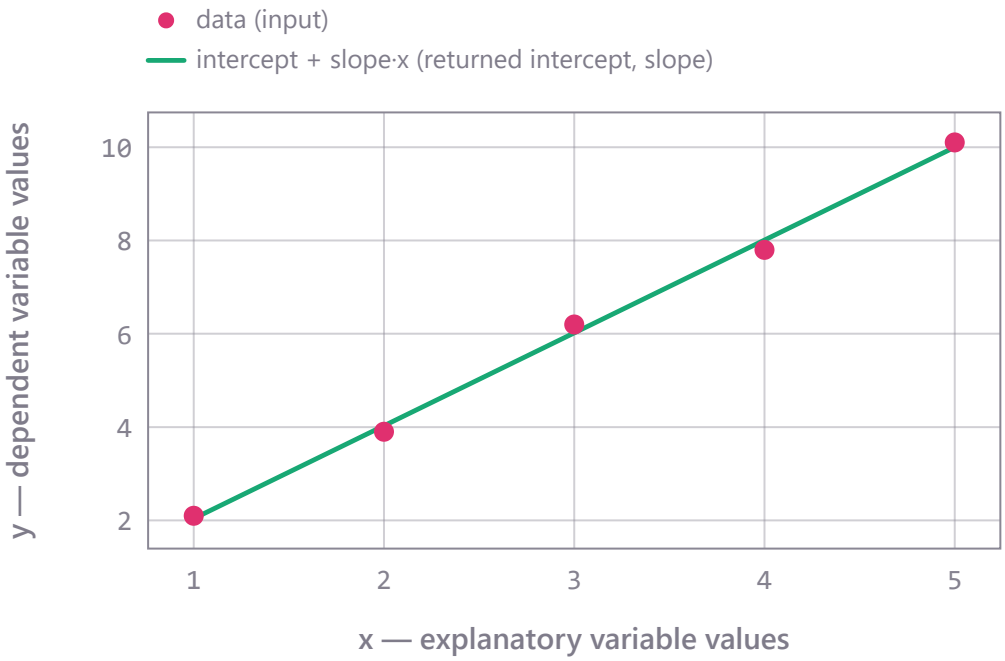
Ordinary least squares fit $y = a + b$ x: slope, intercept, R^2 and the standard error of the slope.

$$\hat{\beta} = \frac{\sum(x-\bar{x})(y-\bar{y})}{\sum(x-\bar{x})^2}, \quad \hat{\alpha} = \bar{y} - \hat{\beta}\bar{x}$$

input	meaning	condition
`x`	explanatory variable values	
`y`	dependent variable values	

output	meaning
`slope`	$b = S_xy/S_xx$
`intercept`	a = mean y - b mean x
`r_squared`	1 - SSR/SST
`slope_std_error`	sqrt(SSR/(n - 2) / S_xx)
`n`	observations

- **Method:** least squares normal equations
- **Assumes:** same number of x and y, at least 3; neither variable constant
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (dependent variable values) against x (explanatory variable values) as given in the worked example (5 points); drawn on it from what economics-finance-ai999x returned: the line y = 0.05 + 1.99·x from the returned intercept and slope. Also returned: r_squared = 0.997305, slope_std_error = 0.0597216.

```
economics_finance_ai999x.solve('ols_simple', x='1, 2, 3, 4, 5', y='2.1, 3.9, 6.2, 7.8, 10.1')
# -> slope = 1.99, intercept = 0.05, r_squared = 0.997305, slope_std_error = 0.0597216, n = 5
(SOLVED)
```

```
economics_finance_ai999x.solve('ols_simple', x=[1, 2, 3], y=[3, 5, 7])
# -> slope = 2, intercept = 1, r_squared = 1, slope_std_error = 0, n = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fit a linear regression with x = [1, 2, 3, 4, 5], y = [2.1, 3.9, 6.2, 7.8, 10.1].	SOLVED	slope = 1.99, intercept = 0.05, r_squared = 0.997305, slope_std_error = 0.0597216, n = 5
Fit a linear regression with x = [1, 1, 1], y = [3, 5, 7].	INVALID_INPUT	Simple linear regression (OLS): x must not be constant.

Multiple linear regression (OLS) — `multiple_regression`

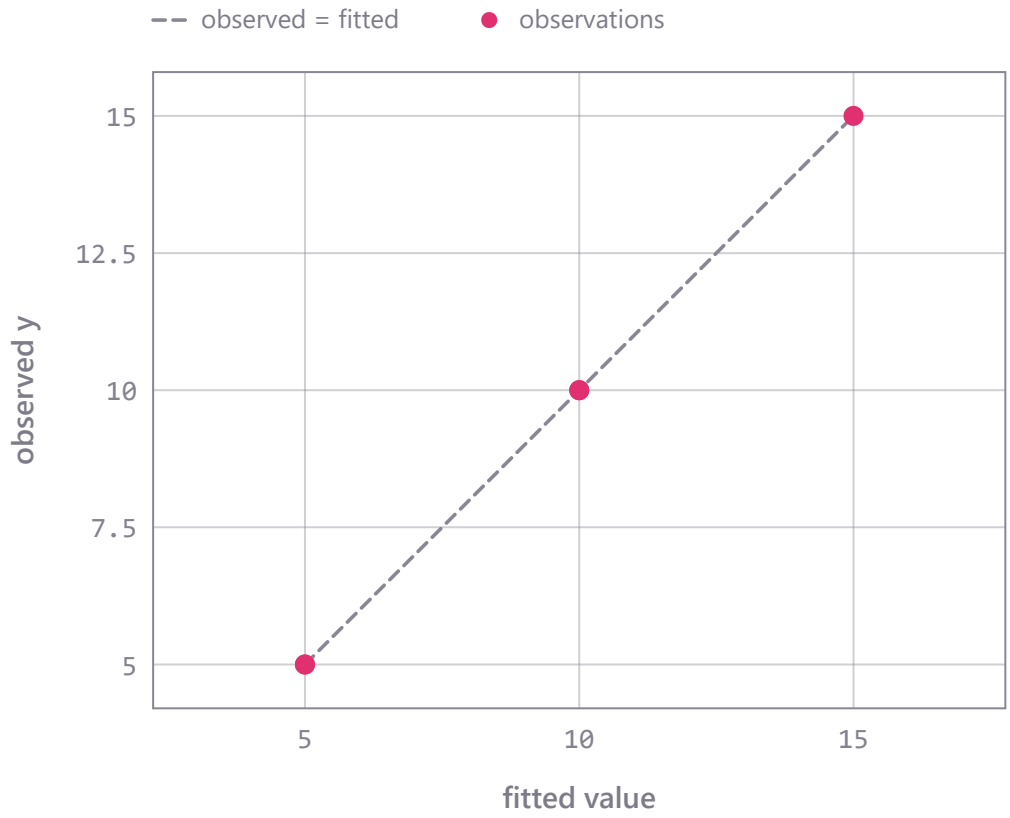
Least-squares coefficients of $y = b_0 + b_1\,x_1 + \ldots + bk$ xk (intercept added), R^2 and adjusted $R^{2.}$

$$\hat{\beta} = (X^\top X)^{-1} X^\top y$$

input	meaning	condition
`X`	regressor matrix, one row per observation, one column per variable (no constant column)	
`y`	dependent variable, one value per row of X	

output	meaning
`coefficients`	[b0 (intercept), b1, ..., bk]
`r_squared`	1 - SSR/SST
`adj_r_squared`	$1 - (1 - R^2)(n-1)/(n-k-1)$

- **Method:** QR decomposition (modified Gram-Schmidt) of [1, X], back substitution
- **Assumes:** one row of X per value of y; $n > k + 1$; columns not collinear; y not constant
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Observed y = 5, 5, 10, 10, 15 against the values fitted from X = [[1, 2], [2, 1], [3, 4], [4, 3], [5, 6]] with the coefficients the package returns (intercept first: 1.25, 1.25, 1.25; fitted values computed here), R^2 = 1, adjusted 1. Points on the dashed line are fitted exactly.

```
economics_finance_ai999x.solve('multiple_regression', X='[[1, 2], [2, 1], [3, 4], [4, 3], [5, 6]]',
y='5, 5, 10, 10, 15')
# -> coefficients = [1.250000000000001, 1.2499999999999973, 1.2500000000000024], r_squared = 1,
adj_r_squared = 1   (SOLVED)
```

```
economics_finance_ai999x.solve('multiple_regression', X=[[1], [2], [3], [4]], y=[3, 5, 7, 9])
# -> coefficients = [1.0, 2.0], r_squared = 1, adj_r_squared = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fit a multiple regression with X = [[1, 2], [2, 1], [3, 4], [4, 3], [5, 6]], y = [5, 5, 10, 10, 15].	SOLVED	coefficients = [1.250000000000001, 1.2499999999999973, 1.2500000000000024], r_squared = 1, adj_r_squared = 1
Fit a multiple regression with X = [[1, 2], [2, 4], [3, 6], [4, 8], [5, 10]], y = [5, 5, 10, 10, 15].	INVALID_INPUT	Multiple linear regression (OLS): the columns of X are (nearly) collinear.

Constant elasticity from a log-log regression — `log_log_elasticity`

The price elasticity estimated as the slope of ln Q on ln P (constant-elasticity demand $Q = A$ P^e).

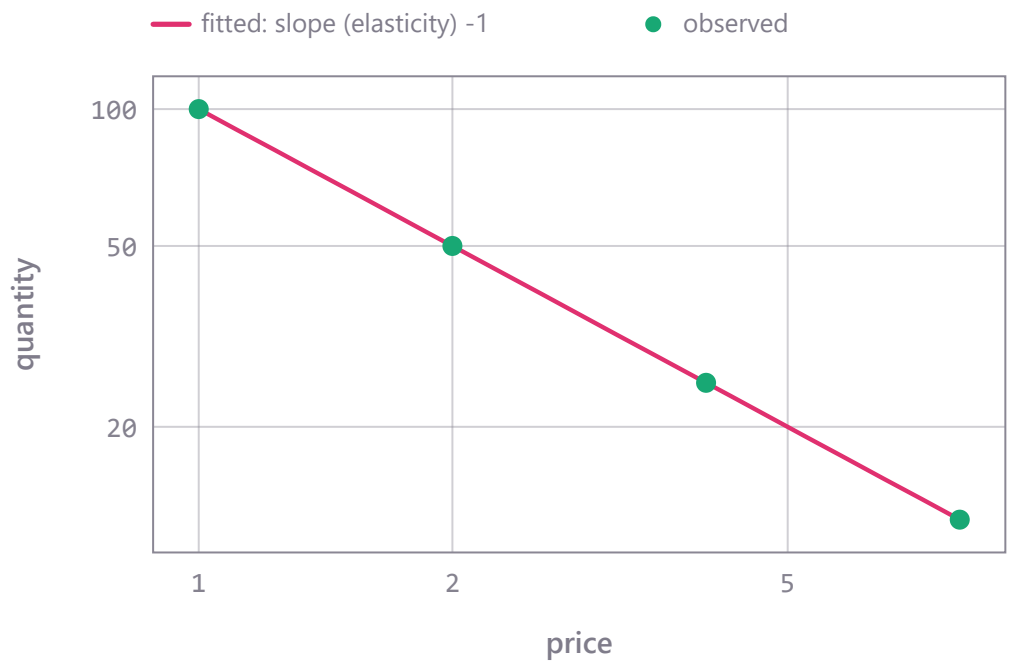
$$\ln Q = \ln A + \varepsilon \ln P$$

input	meaning	condition
`prices`	observed prices	

input	meaning	condition
`quantities`	quantities at those prices	

output	meaning
`elasticity`	slope of ln Q on ln P
`intercept`	ln A
`r_squared`	R^2 of the log-log fit

- **Method:** OLS of ln Q on ln P
- **Assumes:** positive values; prices not all equal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Quantities 100, 50, 25, 12.5 at prices 1, 2, 4, 8 on logarithmic axes: the package fits ln Q = a + e ln P and returns the elasticity e = -1, intercept a = 4.60517 and R^2 = 1; the line Q = exp(a)·P^e is drawn from those returned values.

```
economics_finance_ai999x.solve('log_log_elasticity', prices='1, 2, 4, 8', quantities='100, 50, 25, 12.5')
# -> elasticity = -1, intercept = 4.60517, r_squared = 1   (SOLVED)
```

```
economics_finance_ai999x.solve('log_log_elasticity', prices=[2, 3, 5, 7], quantities=[90, 70, 50, 41])
# -> elasticity = -0.632208, intercept = 4.9386, r_squared = 0.999647   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Estimate the elasticity from data with prices = [1, 2, 4, 8], quantities = [100, 50, 25, 12.5].	SOLVED	elasticity = -1, intercept = 4.60517, r_squared = 1
Estimate the elasticity from data with prices = [1, 2, 4], quantities = [100, 0, 25].	INVALID_INPUT	Constant elasticity from a log-log regression: prices and quantities must be positive (logarithms).

Gini coefficient — `gini_coefficient`

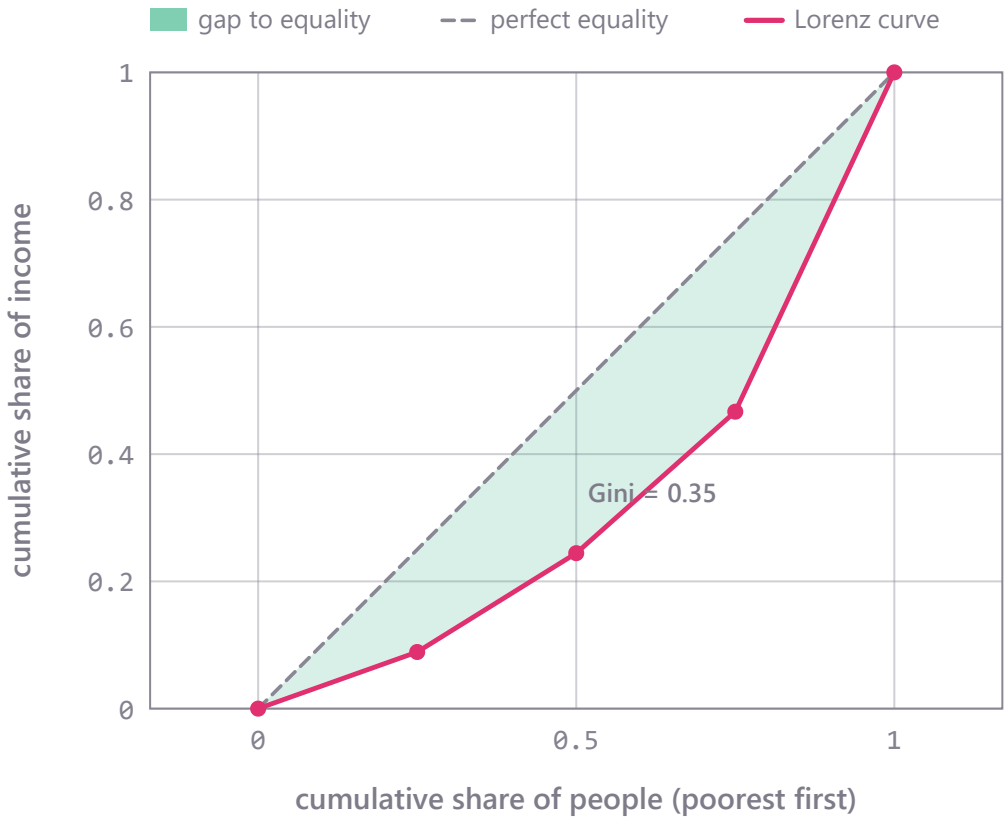
Gini coefficient of a list of incomes (population formula: mean absolute difference / twice the mean; 0 = equality, (n-1)/n = one person has everything).

$$G = \frac{\sum_i \sum_j |x_i - x_j|}{2n^2 \bar{x}}$$

input	meaning	condition
`incomes`	incomes (or wealth) of the people or households	

output	meaning
`gini`	sum $(2i - n - 1)$ x_(i) / (n sum x)

- **Method:** sorted-rank formula
- **Assumes:** incomes >= 0 with a positive total
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Lorenz curve of the incomes 20000, 35000, 50000, 120000: the cumulative income share of the poorest k people (computed here from the sorted incomes). The Gini coefficient the package returns, 0.35, is twice the shaded area between the curve and the line of equality.

```
economics_finance_ai999x.solve('gini_coefficient', incomes='20000, 35000, 50000, 120000')
# -> gini = 0.35   (SOLVED)
```

```
economics_finance_ai999x.solve('gini_coefficient', incomes=[5, 5, 5, 5])
# -> gini = 0   (SOLVED)
```

```
economics_finance_ai999x.solve('gini_coefficient', incomes=[0, 0, 0, 10])
# -> gini = 0.75   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Gini coefficient with incomes = [20000, 35000, 50000, 120000].	SOLVED	gini = 0.35
Find the Gini coefficient with incomes = [0, 0].	INVALID_INPUT	Gini coefficient: incomes must be non-negative with a positive total.

Herfindahl-Hirschman index and concentration ratio — `herfindahl_index`

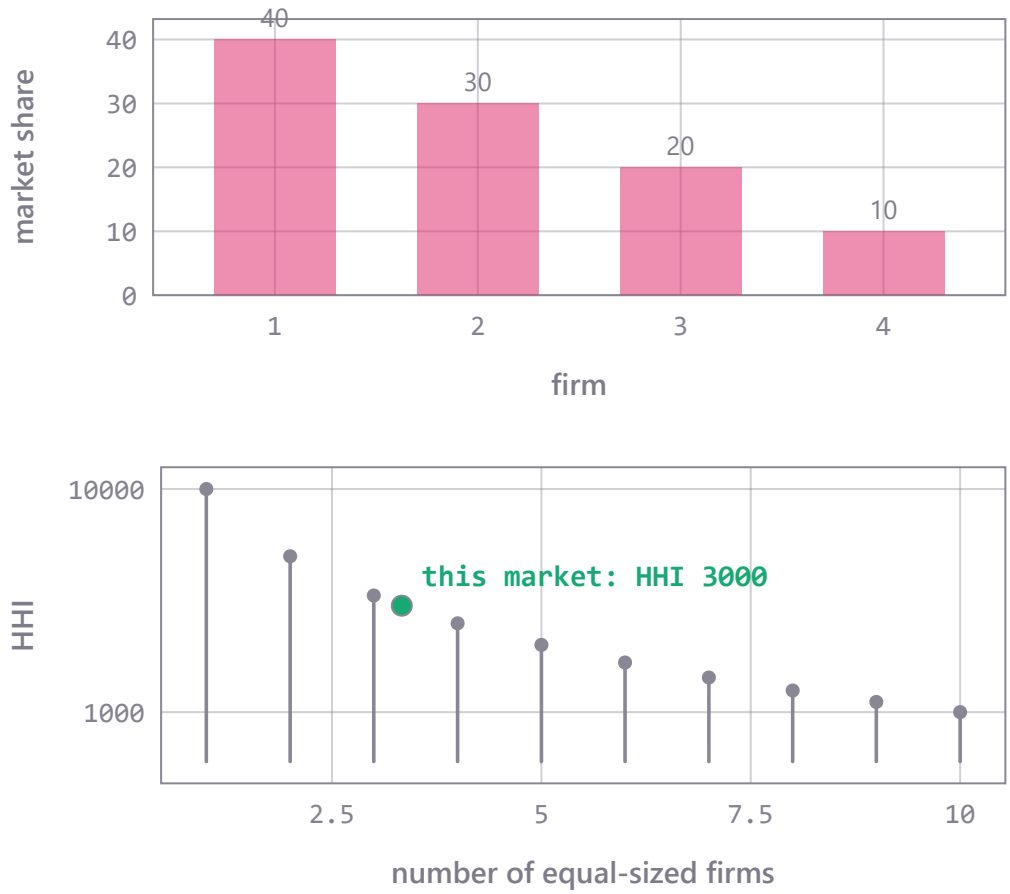
Market concentration from the firms' shares or sales (normalized to shares): HHI on the $0-10{,}000$ scale, the four-firm concentration ratio and the equivalent number of equal-sized firms.

$$HHI = 10\,000 \sum_i s_i^2$$

input	meaning	condition
`market_shares`	market shares (in %) or sales of each firm	

output	meaning
`hhi`	10,000 x sum of squared shares
`hhi_fraction`	sum of squared shares $(0-1)$
`cr4`	combined share of the four largest firms, %
`equivalent_firms`	1 / HHI (fraction)

- **Method:** normalize to shares s_i; HHI = sum s_i^2
- **Assumes:** shares or sales >= 0 with a positive total
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: the market shares 40, 30, 20, 10. Bottom: the Herfindahl–Hirschman index of markets of 1 to 10 equal-sized firms (runs of the package) and this market's index, 3000, which the package also expresses as 3.33333 equivalent equal firms (four-firm concentration 100).

```
economics_finance_ai999x.solve('herfindahl_index', market_shares='40, 30, 20, 10')
# -> hhi = 3000, hhi_fraction = 0.3, cr4 = 100, equivalent_firms = 3.33333   (SOLVED)
```

```
economics_finance_ai999x.solve('herfindahl_index', market_shares=[1, 1, 1, 1, 1])
# -> hhi = 2000, hhi_fraction = 0.2, cr4 = 80, equivalent_firms = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Herfindahl index with market shares = [40, 30, 20, 10].	SOLVED	hhi = 3000, hhi_fraction = 0.3, cr4 = 100, equivalent_firms = 3.33333
Find the HHI with sales = [0, 0, 0].	INVALID_INPUT	Herfindahl-Hirschman index and concentration ratio: shares (or sales) must be non-negative with a positive total.

Durbin-Watson statistic — `durbin_watson`

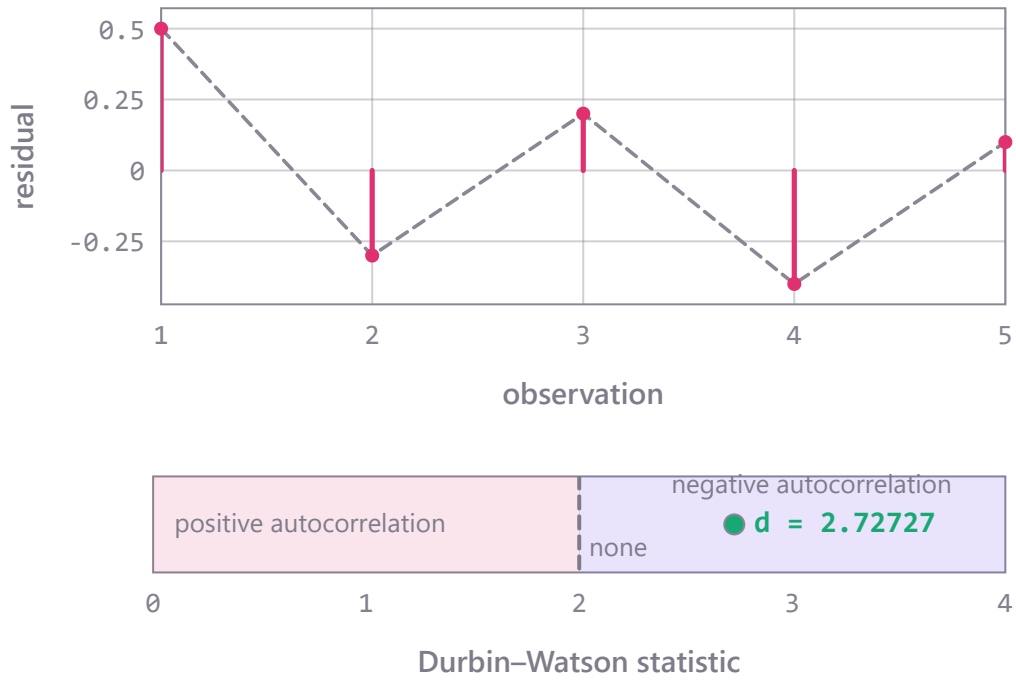
Durbin-Watson statistic of regression residuals in time order (about 2: no first-order autocorrelation; toward 0: positive; toward 4: negative).

$$DW = \frac{\sum_{t=2}^{n}(e_t - e_{t-1})^2}{\sum_{t=1}^{n} e_t^2}$$

input	meaning	condition
`residuals`	regression residuals in time order	

output	meaning
`durbin_watson`	sum (e_t - e_t-1)^2 / sum e_t^2

- **Method:** DW = sum_{$t \geq 2$} (e_t - e_{$t-1$})^2 / sum_t e_t^2
- **Assumes:** residuals not all zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The residuals 0.5, -0.3, 0.2, -0.4, 0.1 in time order (top) and the Durbin–Watson statistic the package returns, d = 2.72727, on its scale from 0 to 4 (bottom): near 2 there is no first-order autocorrelation; residuals that alternate in sign, as these do, push d above 2.

```
economics_finance_ai999x.solve('durbin_watson', residuals='0.5, -0.3, 0.2, -0.4, 0.1')
# -> durbin_watson = 2.72727   (SOLVED)
```

```
economics_finance_ai999x.solve('durbin_watson', residuals=[1, 1])
# -> durbin_watson = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Durbin-Watson statistic with residuals = [0.5, -0.3, 0.2, -0.4, 0.1].	SOLVED	durbin_watson = 2.72727
Find the Durbin-Watson statistic with residuals = [0, 0, 0].	INVALID_INPUT	Durbin-Watson statistic: the residuals are all zero.

Correlation and covariance of two series — `correlation`

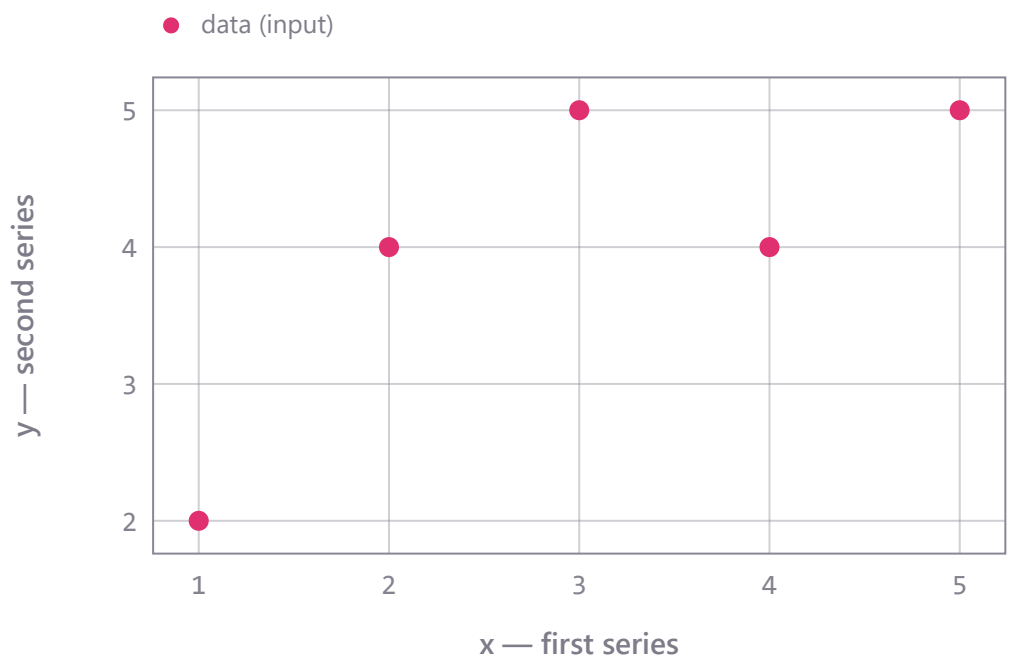
Pearson correlation and sample covariance $(n-1)$ of two paired series, such as the returns of two assets.

$$r = \frac{\sum (x-\bar{x})(y-\bar{y})}{\sqrt{\sum (x-\bar{x})^2 \sum (y-\bar{y})^2}}$$

input	meaning	condition
`x`	first series	
`y`	second series	

output	meaning
`correlation`	Pearson r
`covariance`	sample covariance
`r_squared`	r^2

- **Method:** $r = S_xy/sqrt$(S_xx S_yy)
- **Assumes:** same length, at least 3; neither series constant
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (second series) against x (first series) as given in the worked example (5 points). Also returned: correlation = 0.774597, covariance = 1.5, r_squared = 0.6.

```
economics_finance_ai999x.solve('correlation', x='1, 2, 3, 4, 5', y='2, 4, 5, 4, 5')
# -> correlation = 0.774597, covariance = 1.5, r_squared = 0.6   (SOLVED)
```

```
economics_finance_ai999x.solve('correlation', x=[1, 2, 3], y=[6, 4, 2])
# -> correlation = -1, covariance = -2, r_squared = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the correlation with x = [1, 2, 3, 4, 5], y = [2, 4, 5, 4, 5].	SOLVED	correlation = 0.774597, covariance = 1.5, r_squared = 0.6
Find the correlation with x = [1, 1, 1], y = [6, 4, 2].	INVALID_INPUT	Correlation and covariance of two series: neither variable may be constant.